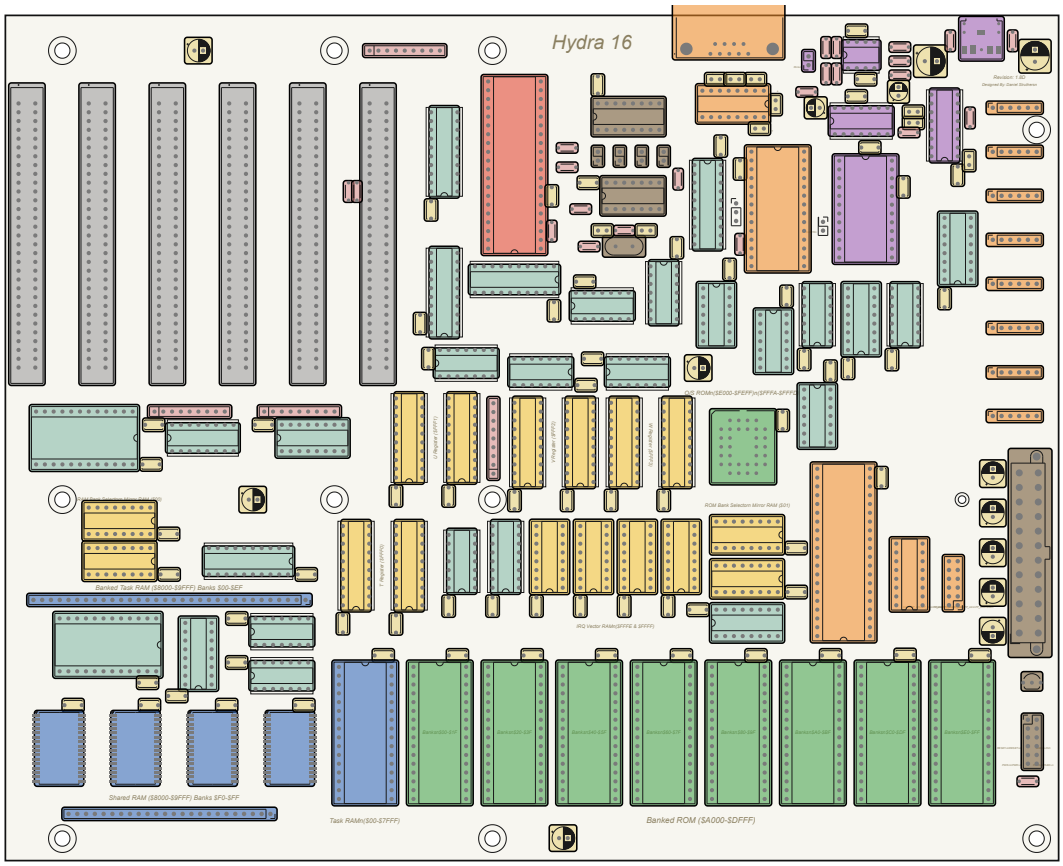
THE HYDRA PROJECT

HYDRA-16

Technical Reference



Every keyword, word, form, command and system call:
its syntax, its exact behaviour, its limits and its errors

HydraOS 1.0 · preview

About This Reference

This is the *Hydra-16 Technical Reference*. Its five chapters cover HyBASIC's keywords, the commands, the system calls, HyForth's words and hylang's forms: every one has an entry.

HyBASIC's keywords, the commands, HyForth's words and hylang's forms and functions were each written out by hand. The system calls are made from the kernel's own specification, the file the kernel itself is built from, with the calls that most need it written out more fully.

The *User's and Builder's Manual* teaches the Hydra, and its appendices list what each language has in a line apiece. This book is where to look when a line isn't enough: when the question is what a keyword does with an empty string, which errors a call can return, or what is left in a variable after a loop ends. It doesn't teach. It assumes the manual has been read, and it says the same thing about every entry, in the same order.

What it holds

CHAPTER	ENTRIES
1. HyBASIC keywords	Every statement, function and operator: 175
2. Commands	Every program in `/bin`, the system's own, and the shell's built-in commands
3. System calls	All 92, with every constant and error code
4. HyForth words	The core and every library, the Hydra's own among them: 294 entries
5. hylang forms	The special forms, the functions and the libraries: 163 entries

The manual's other reference appendices (the 65C02's instructions, the character sets, the memory map) are not repeated here.

How to read an entry

Every entry has the same parts, in this order. A part with nothing to say is left out.

`Syntax`
: Each form the thing can take. Words in `CAPITALS` (or, for a command or a HyForth word, as printed) are typed as they are; *italic* names stand for what you supply; `[ ]` encloses what can be left out; `{ | }` is a choice of one; `...` means the last item can repeat.

`Stack`
: For a HyForth word, in place of Syntax: its stack picture, in the standard's notation, which Chapter 4 explains.

`Arguments`
: Each one: its type, its range, and what it means. For a system call, the registers in.

Result
: What a function returns, a word leaves on the stack, or a call leaves in the registers.

Description
: What happens, in the order it happens: what is evaluated and when, what is changed, where it stops.

Errors
: Each error it can raise, with its number and its message as the Hydra prints it, and what causes it.

Notes
: Limits, edge cases, and behaviour that differs from what another system's user would expect.

See also
: Related entries.

Example
: A transcript, taken from the Hydra. Every one in this book is replayed in the emulator when the book is built, so what is printed here is what the ROM does.

How it was written

Each entry was written from the interpreter's, the program's or the kernel's source, and each statement of behaviour in it was then tried on the system. Where the source and the trial disagreed with what one would expect of the language elsewhere (QuickBASIC, the Forth standard), the entry says what the Hydra does.

Section and chapter numbers such as "section 13.2" and "Appendix G" are the *User's and Builder's Manual*'s. A chapter of this book is always called that: "Chapter 3 of this book".

Contents

About This Reference 2

1 HyBASIC Keywords 21

ABS 21
ABSOLUTE 22
AND 22
APPEND 23
ARG$ 23
AS 24
ASC 24
ASM ... END ASM 25
ATN 27
BANK 28
BASE 29
BASE$ 30
BEEP 30
BINARY 31
BIT 31
BYE 31
CALL 32
CASE 33
CDBL, CSNG 33
CHDIR 34
CHR$ 34
CINT, CLNG 35
CIRCLE 36
CLEAR 37
CLOSE 38
CLS 38
COLOR 39
COMMAND$ 39
COMPLEX 40
CONST 41
CONT 42
COS 43
CSRLIN 43
CVN 44
DATA 45
DATE$ 46
DECLARE 46
DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR 47
DELETE 48
DENOMINATOR 48
DIGITS 49
DIM 50

DIR$ 52
DO ... LOOP 52
DRAW 54
EDIT 55
ELSE, ELSEIF 56
END 56
ENV$, ENVIRON$ 56
ENVIRON 57
EOF 58
EQV 59
ERASE 59
ERL, ERR, ERR$ 60
ERROR 61
EXIT 61
EXP 62
FIB 63
FILES 64
FIX 64
FIXED 65
FOR ... NEXT 66
FRE 68
FREEFILE 68
FUNCTION ... END FUNCTION 69
GCD 70
GET 71
GOSUB 72
GOTO 73
GPRINT 74
HEX$ 75
IF 76
IMAG 77
IMP 78
INCLUDE 78
INKEY$ 79
INPUT 80
INPUT$ 82
INSTR 83
INT 84
IS 85
KILL 85
LBOUND, UBOUND 86
LCASE$, UCASE$ 87
LEFT$, RIGHT$ 87
LEN 88
LET 89
LINE 90
LINE INPUT 91
LIST 92
LOAD 93

LOC, LOF 94
LOCATE 95
LOG 95
LOOP 96
LTRIM$, RTRIM$ 96
MID$ 97
MKDIR, RMDIR 98
MKN$ 99
MOD 100
NAME 101
NEW 102
NEXT 102
NOT 102
NUMERATOR 103
OCT$ 103
ON 103
OPEN 105
OPTION 107
OR 108
OUTPUT 108
PAINT 108
PALETTE 109
PEEK, POKE 110
PI 111
PLAY 112
POINT 113
POS 113
PRESERVE 114
PRESET, PSET 114
PRINT 115
PUT 118
RANDOMIZE 119
RATIONAL 120
READ 121
REAL 122
REDIM 122
REM 123
RESTORE 123
RESUME 124
RETURN 125
RND 125
RREG 126
RUN 127
SAVE 128
SCREEN 128
SEEK 130
SELECT CASE 131
SGN 132
SHARED 133

SHELL 134
SHELL$ 135
SHL, SHR 136
SIN 136
SLEEP 137
SOUND 138
SPACE$ 139
SPC 140
SPRITE 140
SQR 141
STATIC 142
STATUS 143
STEP 143
STOP 143
STR$ 143
STRING$ 144
SUB ... END SUB 145
SWAP 146
SYS 147
SYSTEM 148
TAB 149
TAN 149
THEN 150
TIME$, TIMER 150
TO 150
TYPE ... END TYPE 151
UNTIL 152
USING 152
VAL 152
VIEW 153
WHILE ... WEND 154
WIDTH 155
WINDOW 156
WRITE 156
XOR 158

2 Commands 159

as 159
basic 160
calc 161
cat 162
cmp 163
cp 163
date 165
db 166
df 166
dis 167
du 167
echo 168
ed 169

edit 170
forth 170
free 171
fsck 171
grep 172
head 174
hello 174
hwtest 175
hylang 175
kill 176
label 176
ls 177
mkdir 178
mkfs 179
mods 179
more 180
mv 180
new-window 181
ns 182
play 183
ps 184
pwd 184
rc 185
rm 186
rmdir 187
slay 187
sleep 188
sort 188
tail 189
tee 190
top 191
touch 191
uniq 191
wc 192
xd 193
xmodem 193
The system's own programs 194
The shell's own commands 195

3 System Calls 198
The calls by group 199
BANKS 199
BANKS_ALLOC 200
BANKS_ALLOC_IN 200
BANKS_FREE 201
BIND 202
BREAK 203
CHDIR 203
CLIENT_READ 204
CLIENT_WRITE 205

CLOSE 205
CREATE 206
DBG_KCOPY 207
DBG_PS 208
DBG_SCALL 208
DUP 209
DUP2 210
ENV_DEL 210
ENV_GET 211
ENV_NAME 212
ENV_PUT 213
ERRSTR 214
EXEC 215
EXITS 216
FD2PATH 216
FSTAT 217
FWSTAT 218
GETC 219
GETCWD 219
GETPID 220
GETPPID 220
INIT_RUN 221
IRQ_OWN 222
IRQ_RELEASE 223
KMESG 223
MODINFO 224
MOUNT 225
NOTE 226
NOTE_POST 226
NOTE_QUEUE 227
NOTIFY 228
NSINFO 228
OPEN 229
PAGES_ALLOC 231
PAGES_FREE 232
PAUSE 233
PIPE 233
PREEMPT_OFF 234
PREEMPT_ON 235
PUTC 235
PUTHEX 236
PUTS 236
READ 237
REBOOT 238
REMOVE 239
ROMREAD 240
RTC 240
SEEK 241
SEGINFO 242

SEG_ATTACH 243
SEG_CREATE 243
SEG_CREATE_IN 244
SEG_DETACH 245
SEG_MAP 245
SEM_ACQUIRE 246
SEM_FREE 247
SEM_NEW 247
SEM_RELEASE 248
SEM_TRY 249
SLEEP 249
SLEEP_UNTIL 250
SPAWN 251
SRV_NAP 252
SRV_REGISTER 253
SRV_REPLY 253
SRV_TAKE 254
STAT 255
SYSINFO 255
TASKINFO 256
TASKMEM 257
TASKREAD 258
TASKSTEP 259
TASKSTOP 260
TICKS 261
TIME 261
TIME_SET 262
UNMOUNT 262
WAIT 263
WAKE 264
WRITE 265
WSTAT 265
XCALL 266
YIELD 267
Constants 267
Error codes 276

4 HyForth Words 279

% 280
', ['] 281
(282
*/, */mod 282
+, -, * 283
+! 284
,, c, 284
., u. 285
." 285
.(286
/, mod, /mod 286
: 287

; 288
<#, #, #s, hold, holds, sign, #> 289
<>, u>, 0<>, 0> 290
=, <, >, u< 290
? 291
@, ! 291
[,] 292
\ 292
{: 293
0=, 0< 293
1+, 1-, 2*, 2/ 294
2>r, 2r>, 2r@ 295
2@, 2! 295
2constant, 2variable, 2value, 2literal 295
2dup, 2drop, 2swap, 2over 296
accept 296
again 297
ahead, cs-pick, cs-roll 297
align, aligned 297
allocate 298
allot 298
and, or, xor 299
argc, arg 300
at-xy 300
bank!, bank@ 301
bank-window 302
base 302
beep 303
begin, until 303
begin-structure, end-structure 303
bin 304
bind, mount, unmount, newns 305
bitmap, bitmap-off 305
bl 306
black, red, green, yellow, blue, magenta, cyan, white 306
blank 307
blk, scr 307
block, buffer 308
>body 308
bright 309
buffer: 309
bye 310
c" 310
c@, c! 311
case, of, endof, endcase 311
catch 312
cd 313
cells, cell+, chars, char+ 313
char, [char] 314

circle, disc 315
clear 315
clear-line, clear-below 315
close-file 316
cmove, cmove> 316
code-banks 317
color, bgcolor 317
compare 318
compile, 318
[compile] 319
complex 319
count 320
cr, space, spaces 320
create 321
cs, home, fd, bk, rt, lt, pu, pd, heading, seth 321
ctl 322
cursor-save, cursor-restore, cursor-off, cursor-on 323
cursor-up, cursor-down, cursor-right, cursor-left 323
d+, d-, m+, dnegate, dabs 323
d., d.r 324
d0=, d0<, d=, d<, du<, dmax, dmin 324
d2*, d2/, d>s, m/*, 2rot 325
decimal 325
defer, is, defer!, defer@, action-of 325
[defined], [undefined] 326
definitions 327
delete-file, rename-file 327
depth 328
digits 328
disasm 329
disk-ctl, disk-cmd, disk-start, disk-stop, cards 329
?do 330
do, loop, +loop 331
does> 332
drop 332
dump 333
?dup 333
dup 334
ekey, ekey? 334
ekey>char, ekey>fkey 335
emit 335
emit? 336
environment? 336
erase 336
evaluate 337
execute 337
exit 338
exits 339
+field, field:, cfield: 339

file-position, reposition-file 340
file-size, resize-file 340
file-status 341
fill 341
find 342
flush-file 343
form 343
forth-wordlist 343
free 344
get-current, set-current 344
get-dir, set-dir 344
get-order, set-order 345
getenv, setenv, unsetenv 346
gpio, gpio! 347
gpio-ca1!, gpio-ca2!, gpio-wait 347
gpio-in, gpio-out, gpio-ddr! 348
gpio-port, gpio-port! 348
gpio-state 349
here 349
hex 350
i, j 350
i2c-read, i2c-write, i2c-devices, i2c?, i2c-speed, i2c-reg-size 351
if, else, then 352
[if], [else], [then] 353
immediate 353
include-file 354
included, include 354
int?, fixed?, rational?, complex? 355
invert 356
ior>text 356
k-resize, k-focus 357
k-shift-mask, k-ctrl-mask, k-alt-mask 357
k-up, k-down, k-left, k-right, k-home, k-end, k-prior, k-next, k-insert, k-delete, k-f1, k-f12 358
key 358
key? 359
leave, unloop 359
-lib 360
lib 361
library, end-library 361
libs 362
line, box, bar 363
list 363
literal 363
load, thru 364
lshift, rshift 364
marker 365
min, max 365
=mkdir 366
mouse, mouse-wait 366

move 367
ms 367
>n 368
n+, n-, n*, n/ 368
n., n.base 369
n.s 370
n/mod 370
n=, n<, n>, n0=, n0< 371
n>r, nr> 372
n>s, n>d 372
n>str 373
name>string, name>interpret, name>compile 373
nand, nor, nxor, ninvert 374
nbit? 374
nbytes, nfrom-bytes 375
ncompare 375
ndepth 376
ndrop, ndup, nover, nswap, nrot 377
negate, abs 377
nfib 378
nformat 378
nip, tuck 379
nliteral, nconstant, nvariable, nvalue, n@, n! 379
nlshift, nrshift 380
nnegate, nabs, ngcd, npow 380
:noname 381
note, note-group 382
note-of 382
nrandom, nseed 383
nsqrt, nexp, nlog, nsin, ncos, ntan, natan, npi 384
>number 385
on-note 385
only, also, previous, forth, order 386
open-blocks 387
open-dir, read-dir, close-dir 387
open-file, create-file 387
output-of 388
over 389
pad 389
page 390
palette!, sprite!, sprite-at, sprite-off, vsync 390
parse, parse-name 390
pause 391
pc? 392
pen, border 392
pick 392
piped 393
plain, bold, dim, underline, blink, reverse 393
plot 394

postpone 394
prompt, prompt2 395
quit, abort, abort" 395
.r, u.r 396
>r, r>, r@ 397
r/o, w/o, r/w 397
rand, rand32 398
random 398
read-file 399
read-line 399
recurse 400
refill, source-id, save-input, restore-input 400
replaces, substitute, unescape 401
required, require 401
resize 402
roll 402
rot 403
rseed 403
run 404
.s 404
s" 405
s>d 405
s>n, d>n 406
s" 406
sbit, cbit 407
search 408
search-wordlist 408
see 409
seg-bank! 410
send 410
set-base, get-base 411
set-date, rtc 411
sgr 412
sh 412
sh-out 413
sliteral 413
snd-bend, snd-glide 414
snd-claim, snd-release, snd-claim-psg, snd-release-psg 414
snd-drum 415
snd-freq 415
snd-level, snd-volume, snd-pan 415
snd-mml, snd-chord, snd-play 416
snd-note, snd-off 416
snd-patch 417
snd-regs 417
snd-reset 418
snd-sens, snd-lfo, snd-noise, snd-reg 418
snd-wave 419
spawn 419

spi, spi-read, spi-mode 419
state 420
status 421
/string 421
swap 422
synonym 422
sys 423
sys- 423
task-args, task-cwd, task-env, task-ns, task-regs, task-mem, task-ram 424
tbit 425
text 425
throw 426
time&date 428
to-fixed 428
to-rational, rational.n, rational.d 429
-trailing 429
traverse-wordlist 430
true, false 430
truncate, nfloor, nround 431
tune 431
type 432
um, *m* 432
um/mod, fm/mod, sm/rem 433
unused 434
update, save-buffers, flush, empty-buffers 434
value, to 435
variable, constant 436
vpeek, vpoke, vram@, vram! 436
wait 437
while, repeat 437
window, windows, new-window, new-group, show-window, window-label, window-status, window-ctl 438
within 439
word, source, >in 439
wordlist 440
words 441
write-file, write-line 441
>z, zcount 442

5 hylang Forms 443

+, -, *, / 444
<, >, <=, >=, cmp 445
<=> 446
==, eq, neq 447
?? 448
=0, =1, >0, <0, >=0, <=0, !=0, !=1 449
1+, 1-, abs 450
alpha?, digit?, space?, upper?, lower? 451
and, or 452
any?, all? 452
banks, bank-alloc, bank-free, bank-read, bank-write 453

base 455
between?, ><?, not-between? 456
bind, mount, unmount, ns 457
bit-and, bit-or, bit-xor, bit-not 458
buffer 459
buffer-cmp 461
buffer-fill, buffer-copy 462
buffer-get, buffer-put 463
buffer? 464
bytes, from-bytes 465
cd, cwd 466
char-code, code-char 467
clock library 468
close 468
complex 469
cond, case 470
cons 471
cons library 472
def 473
defined?, expr? 474
dev library 474
dice library 476
digits 477
disk library 478
div, mod, divmod, fdiv 479
do 480
dotimes 480
each 481
elem?, in? 481
env, setenv, unsetenv 482
error 483
error-message, error-code 484
eval 485
exists?, dir?, file? 486
exit 486
fexpr 487
fib 487
filter 488
find, count 489
first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tenth 490
flip, comp, unpack, apply, curry, pack, uncurry 491
floor, ceil, round, trunc 492
fn 493
foldl, foldr 494
format 495
from 496
fst, snd, thd, last, nth 497
fun 498
gensym 499

gpio library 499
hash-add-tag, hash-tag? 501
hash-call 502
hash-clone 503
hash-create, to 504
hash-get 506
hash-key?, hash-keys, hash-values 507
hash-lock, hash-make-const, hash-make-private, hash-make-not-nil 507
hash-locked?, hash-private?, hash-const? 509
hash-put 510
hash-remove 511
head, tail 512
hex, bin 513
hold 514
i2c library 514
if 516
index-of, last-index-of 516
init, end 517
item-at 518
join 519
key, key? 519
len 520
let 521
list 522
lo, hi, word 523
load 524
ls, dir, stat, glob 525
map 526
math.pi, math.e, math.phi, math.gamma 527
min, max 528
minimum, maximum, sort-desc, best-of, worst-of 529
mkdir, remove, rename, copy-file 530
neg?, pos?, zero?, one?, not 531
newns 532
note, note-group, on-note 533
num?, int?, fixed?, rational?, complex? 534
number-bytes, bytes-number 535
odd?, even?, positive?, negative? 536
open 537
output-of 539
pc library 539
peek, poke, peek-word, poke-word 540
pid 541
platform, hydra? 541
print 542
print-to, write-to 543
proc library 543
qexpr?, sexpr?, function?, error? 545
random 546

range 547
rational.n, rational.d 548
read 548
read-buffer 549
read-bytes, write-bytes 550
read-file, read-lines 552
read-line, read-byte, read-all 553
remove-fst, remove-all, remove-min, remove-max 554
repeat, times 555
reverse 556
run, spawn, wait, kill 556
save 558
screen library 559
seek, tell 560
seg-create, seg-attach, seg-detach, seg-read, seg-write 561
set 563
set! 563
sh, sh-out 564
shl, shr, bit? 565
sleep, sleep-until, yield 566
snd library 567
sort 569
spi library 570
split, zip, flatten, distinct, avg 571
sqrt, exp, log, sin, cos, tan, atan, pi, pow 573
square, cube, recip, inverse 575
starts-with?, ends-with?, contains? 575
str-pad-left, str-pad-right 576
str-replace, str-repeat 577
str-split, str-join, str-chars 578
str-upper, str-lower, str-trim 580
string?, char?, atom?, symbol? 581
subset 582
substring, char-at 582
sum, product 583
sys 584
sysinfo, mods, ps, task-info, errstr, free 586
t?, nil? 587
take, drop 588
ticks, tick-rate, clock 589
time, date, date-parts, seconds-of 590
to-fixed, to-rational 591
to-str, repr 592
to-sym, to-atom 593
truncate 594
try 595
type-of 595
use 597
val 598

video library 599
while 601
write 602
write-file, append-file 602
xor, <>, != 603

CHAPTER 1

HyBASIC Keywords

HyBASIC compiles a program whole before it runs it: every line is read, every name resolved and every block matched, and only then does the first statement execute. An error the compiler finds (a `FOR` with no `NEXT`, a label that isn't there) is therefore reported before anything has run, with the line it was found on. A line typed at the prompt without a number is compiled and run the same way, as a program of one line.

An error is reported as its message, in small letters. `ERR` then holds its number, `ERR$` the message and `ERL` the line; the numbers given in each entry are `ERR`'s. A failure reported by the system (a disk that is full, a name that isn't there) has HyBASIC's own number where there is one that fits (53, file not found), and otherwise 256 plus the system's code (Appendix G).

A number is an integer of any length, a rational, a decimal or a complex number, and arithmetic on them is exact (Chapter 18). A string is a sequence of bytes, each 0 to 255, up to 64K of them. A string written in a program is enclosed in double quotes and can hold any character but the double quote itself: there is no doubled form for one inside a string. `CHR$(34)` is the way to make one.

ABS

A numeric function: the absolute value of a real number.

SYNTAX

```
ABS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an integer, a rational or a decimal.

RESULT

x if *x* is zero or more, and *–x* if it is less. The result is the same kind of number as *x*, and exact: an integer gives an integer, of whatever length; a rational gives a rational in lowest terms; a decimal gives a decimal with the same digits. There is no largest value and no overflow.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex. A complex number has no sign, and `ABS` does not give its modulus
13	`type mismatch`	*x* is a string
2	`syntax error`	No argument, or more than one

NOTES

For the modulus of a complex number *z*, use `SQR(REAL(z) ^ 2 + IMAG(z) ^ 2)`.

SEE ALSO

`SGN`, `INT`, `FIX`, `REAL`, `IMAG`.

EXAMPLE

```
> PRINT ABS(-7); ABS(7); ABS(-7/2); ABS(-2.5)
 7  7  7/2  2.5
> PRINT ABS(-2 ^ 100)
 1267650600228229401496703205376
> PRINT ABS(3+4i)
illegal function call
> PRINT ERR; SQR(REAL(3+4i) ^ 2 + IMAG(3+4i) ^ 2)
 5  5
```

ABSOLUTE

Part of `CALL ABSOLUTE`, which calls machine code at an address. See `CALL`.

AND

An operator: the bitwise AND of two integers, which is also the logical AND of two truth values.

SYNTAX

```
a AND b
```

ARGUMENTS

a*, *b

Numeric expressions whose values are real.

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in both *a* and *b*, and 0 elsewhere.

DESCRIPTION

Both operands are evaluated, always: there is no short cut when the first is 0. Each is then rounded to the nearest integer, a half going to the even one (1.5 and 2.5 both become 2), and the two integers are combined bit by bit.

The integers can be of any length. A negative one is taken in two's complement with as many sign bits as are needed, so –1 is all ones, and `-1 AND x` is *x* for every integer *x*.

A comparison gives –1 for true and 0 for false. Since –1 is all ones and 0 is none, `AND` of two comparisons is their logical AND, and `IF a > 0 AND b > 0 THEN` means what it says. `AND` of a truth value and some other number is a bit operation still: `(1 = 1) AND 2` is 2, which counts as true, but `1 AND 2` is 0, which is false, though both of its operands count as true.

`AND` binds more tighter than `OR`, `XOR`, `EQV` and `IMP`, and less tightly than `NOT` and the comparisons: `NOT a AND b = c OR d` is `((NOT a) AND (b = c)) OR d`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An operand is a string
5	`illegal function call`	An operand is complex

SEE ALSO

`OR`, `XOR`, `NOT`, `EQV`, `IMP`, `BIT`, `SHL`.

EXAMPLE

```
> PRINT 12 AND 10; -1 AND 255; (1 < 2) AND (3 < 2); 1 AND 2
 8  255  0  0
> PRINT 2.5 AND 3; 1.5 AND 3; 2 ^ 40 + 5 AND 2 ^ 40 + 3
 2  2  1099511627777
> PRINT 0 AND (1 / 0)
division by zero
```

APPEND

A mode of `OPEN`: `OPEN path$ FOR APPEND AS #f` opens a file for writing at its end. See `OPEN`.

ARG$

A string function: one of the arguments the program was started with.

SYNTAX

```
ARG$(n)
```

ARGUMENTS

n

A numeric expression: which argument. 0 is the program's own name, 1 the first argument after it.

RESULT

The argument, as a string. The arguments are those of the command that started HyBASIC with a program to run: after `basic prog.bas one two`, `ARG$(0)` is `prog.bas`, `ARG$(1)` is `one` and `ARG$(2)` is `two`. An argument is passed whole, as the shell made it: one that was quoted there can hold spaces.

The empty string if there is no argument *n*: past the last one, for a negative *n*, and for every *n* when HyBASIC was started without a program (at its prompt).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*n* is a string

NOTES

There is no count of the arguments. Read them until one is empty, or split `COMMAND$`.

SEE ALSO

`COMMAND$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; ARG$(0); "]["; ARG$(1); "]["; ARG$(2); "]["; ARG$(3); "]"' >args.bas
% basic args.bas one 'two words'
[args.bas][one][two words][]
```

AS

Part of four statements, where it introduces a name, a number or a type: `DIM v AS type`, `OPEN path$ FOR mode AS #f`, `NAME old$ AS new$`, and a field's line in `TYPE`. See those.

ASC

A numeric function: the code of a string's first byte.

SYNTAX

```
ASC(s$)
```

ARGUMENTS

s$

A string expression of one byte or more.

RESULT

An integer from 0 to 255: the value of the first byte of *s$*. The rest of the string is not looked at. A string holds bytes, not characters of any one set, so the result is the byte itself; which character the console draws for it depends on the character set in use (Appendix E). The codes 32 to 126 are ASCII's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is the empty string
13	`type mismatch`	The argument is a number

NOTES

`ASC` is the inverse of `CHR$` for one byte: `ASC(CHR$(n))` is *n* for every *n* from 0 to 255, including 0 (a zero byte doesn't end a string). To test a string that may be empty, test `LEN(s$)` first.

SEE ALSO

`CHR$`, `LEN`, `MID$`, `VAL`.

EXAMPLE

```
> PRINT ASC("A"); ASC("Abc"); ASC(" "); ASC(CHR$(0) + "x"); ASC(CHR$(255))
 65  65  32  0  255
> PRINT ASC("")
illegal function call
```

ASM ... END ASM

A block: 65C02 assembly language inside a program, assembled when the program is compiled.

SYNTAX

```
ASM
    assembly language lines
END ASM

CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

label

A label defined in one of the program's `ASM` blocks, spelled as it is there. The assembler's names keep their case: `Add1` and `add1` are two labels.

a, x, y

Numeric expressions: the values the A, X and Y registers hold as the routine is entered, a byte each. Each is 0 if it is left out.

DESCRIPTION

`ASM` alone on a line starts a block and `END ASM` alone on a line ends it. The lines between are not HyBASIC: they are in the language of the `as` assembler (Chapter 27), with its labels, directives and expressions. A program can have several blocks; they are assembled together, as one source, in the order they appear, so a label in one can be used in another.

The blocks are assembled when the program is compiled, before it runs. Their code goes into a RAM bank of its own, 8K long, which is at `$8000` to `$9FFF` while the code runs. Statements don't run "through" a block: when control reaches `ASM`, it goes on after `END ASM`.

`CALL ASM label` calls the code at *label* as a subroutine (`jsr`), which returns with `rts`. `RREG` then gives the registers it returned with.

The assembler is given some of the program's names as symbols. A global numeric variable's name is the address of its value: five bytes, the first its kind (0 for an integer) and the next four an integer's value, low byte first. An integer `CONST`'s name is its value. Each is defined in capitals and in small letters. The names `A`, `X` and `Y` are left out, since they are the registers'.

ERRORS

ERR	MESSAGE	CAUSE
90	`ASM:` and the assembler's message	An error in a block, reported at its line when the program is compiled
90	`ASM:` *label*`: no such label in an ASM block`	`CALL ASM` names a label that no block defines

NOTES

The code's bank is at `$8000` only while the code runs, so the code can't use `BANK`'s bank at the same time, and a `PEEK` of `$8000` from HyBASIC doesn't see the code.

A program can name at most 64 different labels in its `CALL ASM` statements.

The code runs as part of HyBASIC's task, with the whole machine open to it. A routine that doesn't return, or that writes where it shouldn't, takes HyBASIC down with it.

SEE ALSO

`CALL`, `SYS`, `RREG`, `PEEK`, `POKE`, `BANK`.

EXAMPLE

```
> 10 CALL ASM seven : RREG A : PRINT A
> 20 CALL ASM add1, 41 : RREG A : PRINT A
> 30 END
> 40 ASM
> 50 seven: lda #7
> 60        rts
> 70 add1:  inc a
> 80        rts
> 90 END ASM
> RUN
 7
 42
```

ATN

A numeric function: the arctangent of a real number.

SYNTAX

```
ATN(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The angle, in radians, whose tangent is *x*: more than $-\pi/2$ and less than $\pi/2$. `ATN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed), whatever kind of number *x* is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

`4 * ATN(1)` is the usual way to get π in a BASIC, but here it multiplies the rounding of `ATN(1)` by four. `PI` gives π to the digits set, directly.

SEE ALSO

`TAN`, `SIN`, `COS`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT ATN(1); ATN(0); ATN(-1); ATN(1/2)
 0.785398163397  0 -0.785398163397  0.463647609001
> DIGITS 30 : PRINT ATN(1) : DIGITS 12
 0.785398163397448309615660845820
```

BANK

A statement and a function: chooses, or reports, the RAM bank that `PEEK` and `POKE` see at `$8000`.

SYNTAX

```
BANK n
BANK()
```

ARGUMENTS

n

A numeric expression from 0 to 255: a RAM bank's number.

RESULT

`BANK()` is the number last set by `BANK`, or 0 if none has been.

DESCRIPTION

The addresses `$8000` to `$9FFF` are a window, 8K wide, onto one RAM bank at a time (Appendix F). `BANK n` sets the bank that a `PEEK` or `POKE` of an address in that range will use. It changes nothing else: the bank is put in the window for the moment of each `PEEK` and `POKE` and then taken out again, because HyBASIC keeps its own data in banks and uses the window itself.

HyBASIC takes RAM banks for its own use as it needs them, for the program's text, its strings and its arrays, and `BANK` doesn't check for them: a `POKE` into a bank HyBASIC is using damages it. A program that wants a bank to itself should take one high in the task's range, which HyBASIC reaches last if at all.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

n is not checked against the banks the task has. A task with three RAM modules has banks 0 to 47.

`BANK` has no effect on the bank an `ASM` block's code runs in.

SEE ALSO

`PEEK`, `POKE`, `SYS`, `ASM`.

EXAMPLE

```
> PRINT BANK() : BANK 40 : POKE &H8000, 65 : PRINT PEEK(&H8000); BANK()
 0
 65  40
> BANK 41 : POKE &H8000, 66 : BANK 40 : PRINT PEEK(&H8000)
 65
```

BASE

A statement: sets the base that numbers are written and read in.

SYNTAX

```
BASE b$
```

ARGUMENTS

b$

A string expression: a base string (sections 18.4 to 18.6). `"d"` is decimal, `"x"` hexadecimal, `"b"` binary, `"o"` octal; `"7r"` is base 7.

DESCRIPTION

From the next statement on, every number that HyBASIC converts to text is written in the base, and every number it converts from text is read in it. That covers `PRINT`, `WRITE`, `STR$` and `PRINT #`; `INPUT`, `READ` and `VAL`; and the numbers written in the program's own lines that are compiled after the statement has run, which at the prompt means the lines typed after it.

A number with a prefix says its own base whatever the setting: `&HFF` and `&B1010` are 255 and 10 in every base.

The setting stays until another `BASE`, and is `"d"` when HyBASIC starts.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*b$* isn't a base string
13	`type mismatch`	*b$* is a number

NOTES

`OPTION BASE`, which sets the lowest index of arrays, is a different statement: see `OPTION`.

To write one number in another base without changing the setting, use `HEX$`, `OCT$`, or `STR$` with a base string.

SEE ALSO

`BASE$`, `DIGITS`, `HEX$`, `OCT$`, `STR$`, `VAL`.

EXAMPLE

On the second line the 10 and the 6 are read in hex, as sixteen and six, and their sum, twenty-two, is written in hex:

```
> X = 255 : BASE "x" : PRINT X
 FF
> PRINT 10 + 6; BASE$
 16 x
> BASE "d" : PRINT X
 255
```

BASE$

A string function: the base string that `BASE` last set.

SYNTAX

```
BASE$
```

RESULT

The string given to the last `BASE` statement, as it was given; `d` if there has been none.

SEE ALSO

`BASE`.

EXAMPLE

```
> PRINT BASE$ : BASE "16r" : PRINT BASE$ : BASE "d"
d
16r
```

BEEP

A statement: sounds the terminal's bell.

SYNTAX

```
BEEP
```

DESCRIPTION

Writes the byte 7 (BEL) to standard output. A terminal sounds its bell, or flashes its window, as it is set to. If output has been sent to a file, the byte goes into the file.

SEE ALSO

`SOUND`, `PLAY`.

BINARY

A mode of `OPEN`: `OPEN path$ FOR BINARY AS #f` opens a file for reading and writing bytes at any position. See `OPEN`, `GET` and `PUT`.

BIT

A numeric function: one bit of an integer.

SYNTAX

```
BIT(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real. It is rounded to the nearest integer, a half going to the even one.

n

A numeric expression: which bit. Bit 0 is the lowest, worth 1; bit *n* is worth 2 to the power *n*.

RESULT

1 if bit *n* of *x* is set, and 0 if it is clear.

A negative *x* is taken in two's complement with endless sign bits, so every bit of −1 is 1, however high. 0 if *n* is negative.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

SEE ALSO

`AND`, `OR`, `XOR`, `SHL`, `SHR`.

EXAMPLE

```
> PRINT BIT(5, 0); BIT(5, 1); BIT(5, 2); BIT(-1, 100); BIT(2 ^ 70, 70)
 1  0  1  1  1
```

BYE

A statement: leaves HyBASIC. `BYE [n]` is another spelling of `SYSTEM [n]`. See `SYSTEM`.

CALL

A statement: calls a `SUB`, machine code at an address, or a label in an `ASM` block.

SYNTAX

```
[CALL] name [(] [argument [, argument] ...] [)]
CALL ABSOLUTE (address)
CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

name

The name of a `SUB` defined anywhere in the program.

argument

An expression, one for each of the `SUB`'s parameters, of the parameter's type.

address

A numeric expression, 0 to 65,535: where the machine code is.

DESCRIPTION

A `SUB`. The word `CALL` is optional, and so are the parentheses round the arguments: `Show "x", 2`, `CALL Show("x", 2)` and `CALL Show "x", 2` are the same call. The `SUB` runs, and the program goes on with the statement after the call.

An argument that is a variable, or an array's element, is passed **by reference**: the parameter is another name for it, and an assignment to the parameter changes it. Any other expression is passed by value. To pass a variable by value, put it in parentheses of its own, which makes it an expression: `Inc (X)`.

`CALL ABSOLUTE` calls the machine code at *address* as a subroutine, with the A, X and Y registers 0. It is `SYS address` by QuickBASIC's name. `RREG` gives the registers afterwards. See `SYS`.

`CALL ASM` calls a label of an `ASM` block. See `ASM`.

ERRORS

ERR	MESSAGE	CAUSE
35	`subprogram not defined`	No `SUB` has that name. Found when the program is compiled
37	`argument-count mismatch`	More arguments, or fewer, than the `SUB` has parameters
13	`type mismatch`	A string where the parameter is a number, or a number where it is a string

NOTES

A `SUB` can be called before the line that defines it: the whole program is compiled before any of it runs.

A `FUNCTION` is not called with `CALL`. It is used in an expression.

SEE ALSO

`SUB`, `FUNCTION`, `DECLARE`, `SYS`, `ASM`, `RREG`.

EXAMPLE

```
> 10 SUB Inc (N) : N = N + 1 : END SUB
> 20 X = 1 : Inc X : PRINT X
> 30 CALL Inc((X)) : PRINT X
> 40 DIM A(3) : A(1) = 5 : CALL Inc(A(1)) : PRINT A(1)
> RUN
 2
 2
 6
```

CASE

Part of `SELECT CASE`: `CASE` starts each of its choices. See `SELECT`.

CDBL, CSNG

Numeric functions: their argument, unchanged.

SYNTAX

```
CDBL(x)
CSNG(x)
```

ARGUMENTS

x

A numeric expression.

RESULT

x. In QuickBASIC these convert to double and to single precision. HyBASIC has one kind of numeric variable, which holds any number exactly, so there is nothing to convert: a rational stays a rational, and no digits are lost or gained. They are here so that programs written for QuickBASIC run.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`CINT`, `FIXED`, `RATIONAL`, `DIGITS`.

EXAMPLE

```
> PRINT CDBL(1/3); CSNG(2.5); CDBL(7)
 1/3  2.5  7
```

CHDIR

A statement: changes the current directory.

SYNTAX

```
CHDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name. One that doesn't start with `/` is taken from the current directory.

DESCRIPTION

Sets the current directory of HyBASIC's task, which is where a file name that doesn't start with `/` is looked for by `OPEN`, `LOAD`, `SAVE`, `FILES`, `KILL` and the rest. The change lasts until the next `CHDIR` or until HyBASIC ends. It doesn't change the directory of the shell that started HyBASIC.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name
76	`path not found`	The name is a file, not a directory
13	`type mismatch`	*path$* is a number

SEE ALSO

`DIR$`, `FILES`, `MKDIR`, `RMDIR`, `SHELL`.

EXAMPLE

```
> CHDIR "/ram" : OPEN "note.txt" FOR OUTPUT AS #1 : CLOSE : FILES
bin/            lib/            note.txt
> CHDIR "/nothere"
file not found
> CHDIR "/rom/bin/ls"
path not found
```

CHR$

A string function: a string of one byte.

SYNTAX

```
CHR$(n)
```

ARGUMENTS

n

A numeric expression from 0 to 255. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

A string one byte long, whose byte is *n*. Any value can be made, 0 included, and a string can hold any of them anywhere: `LEN(CHR$(0))` is 1.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

What the console shows for a byte depends on the byte and on the character set (Appendix E). The codes below 32 are controls to a terminal: `CHR$(7)` rings its bell, `CHR$(10)` starts a new line, and `CHR$(27)` begins an escape sequence (section 13.7).

SEE ALSO

`ASC`, `STRING$`, `SPACE$`.

EXAMPLE

```
> PRINT CHR$(72) + CHR$(105); LEN(CHR$(0)); ASC(CHR$(65.5)); ASC(CHR$(66.5))
Hi 1  66  66
> PRINT CHR$(256)
illegal function call
```

CINT, CLNG

Numeric functions: a real number rounded to the nearest integer.

SYNTAX

```
CINT(x)
CLNG(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer nearest to *x*. A value exactly half way between two integers goes to the even one: 2.5 gives 2, 3.5 gives 4, and −2.5 gives −2. This is the rounding QuickBASIC's `CINT` does.

The two functions are the same. An integer here can be of any length, so there is no range to pass, where QuickBASIC's `CINT` overflows beyond 32,767.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT` (rounds down), `FIX` (rounds toward zero), `FIXED`.

EXAMPLE

```
> PRINT CINT(2.5); CINT(3.5); CINT(-2.5); CINT(7/2); CINT(2.4); CINT(-2.6)
 2  4 -2  4  2 -3
> PRINT CLNG(2 ^ 40 + 1/2)
 1099511627776
```

CIRCLE

A statement: draws a circle, or a filled disc, on the bitmap.

SYNTAX

```
CIRCLE [STEP] (x, y), r [, c [, start [, end [, aspect [, F]]]]]
```

ARGUMENTS

x, y
: Numeric expressions: the centre (see `SCREEN` for coordinates). With `STEP`, they are an offset from the last point referenced.

r
: A numeric expression, 0 or more: the radius.

c
: A numeric expression, 0 to 255: the colour. A mode with fewer colours uses the number's low bits. If it is left out, the pen's colour.

start, end, aspect
: Accepted, evaluated and ignored. QuickBASIC draws arcs and ellipses with them; HyBASIC always draws a whole circle.

F
: The letter F: fill the circle, making a disc.

DESCRIPTION

Needs a graphics mode. The radius is in pixels, rounded to the nearest; with a `WINDOW` in force it is in the window's x units. The centre becomes the last point referenced.

A radius of 0 draws the centre's one dot.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
258	`invalid argument`	The driver refused it: a negative radius, or a centre far off the bitmap
13	`type mismatch`	An argument is a string

NOTES

In HydraOS 1.0, a circle drawn while a `WINDOW` is in force is filled whether or not `F` is given. This is a fault, not a feature.

SEE ALSO

`SCREEN`, `COLOR`, `PSET`, `LINE`, `PAINT`, `WINDOW`, `POINT`.

EXAMPLE

With a Vera X card. (`SCREEN` clears the console, so the graphics examples are shown as they were typed, not as replayed transcripts.)

```
> SCREEN 13 : CIRCLE (100, 100), 20, 200
> PRINT POINT(120, 100); POINT(100, 80); POINT(100, 100)
 200  200  0
> CIRCLE (100, 100), 10, 3, , , , F : PRINT POINT(100, 100); POINT(105, 100)
 3  3
> SCREEN 0
```

CLEAR

A command of the prompt: forgets every variable.

SYNTAX

```
CLEAR
```

DESCRIPTION

Every variable and array of the program is dropped, so that each is 0, empty or not yet made when it is next used. The program's lines stay. A program that was stopped can no longer be continued with `CONT`.

`CLEAR` is one of the commands that work only when typed at the prompt, alone on its line. `RUN` clears the variables itself as it starts.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`CLEAR` in a program's line, or after other statements on a line typed at the prompt

SEE ALSO

`NEW`, `RUN`, `ERASE`, `CONT`.

CLOSE

A statement: closes files.

SYNTAX

```
CLOSE [[#]f [, [#]f] ...]
```

ARGUMENTS

f

A numeric expression: the number of a file that `OPEN` opened.

DESCRIPTION

Each file named is closed, and its number is free to be used by another `OPEN`. With no numbers, every open file is closed.

Closing a number that is not open is not an error: nothing happens. That holds for any number, 0 and numbers above 255 included.

What a program has written with `PRINT #`, `WRITE #` or `PUT` is in the file as each statement finishes, not held back until `CLOSE`; closing only gives up the file.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*f* is a string

NOTES

`RUN` closes every file as it starts, and files are closed when HyBASIC ends.

SEE ALSO

`OPEN`, `FREEFILE`.

CLS

A statement: clears the screen.

SYNTAX

```
CLS
```

DESCRIPTION

Clears the console's text and puts the cursor at the top left. In a graphics mode (`SCREEN` 1 or more) the bitmap is cleared to colour 0 as well, and the last point referenced becomes its middle.

If output is not a console, the bytes that would clear one (an escape sequence, section 13.7) go where output goes.

SEE ALSO

`LOCATE`, `SCREEN`, `COLOR`.

COLOR

A statement: sets the colours of text, or in a graphics mode the colour drawn with.

SYNTAX

```
COLOR [fg] [, bg]
```

ARGUMENTS

fg

A numeric expression: the foreground colour.

bg

A numeric expression: the background colour.

DESCRIPTION

With no graphics mode (`SCREEN 0`), `COLOR` sets the colours of the text written from then on. Each is 0 to 15: 0 to 7 are the terminal's eight colours (black, red, green, yellow, blue, magenta, cyan, white), and 8 to 15 are their bright forms. One that is left out is not changed. The colours are set by writing an escape sequence (section 13.7), so what they look like is the terminal's choice.

In a graphics mode (`SCREEN` 1 or more), *fg* sets the pen: the colour used by the drawing statements when they are given none. *bg* is accepted and ignored.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	A text colour that isn't 0 to 15
13	`type mismatch`	An argument is a string

SEE ALSO

`SCREEN`, `PALETTE`, `CLS`, `PSET`, `LINE`, `CIRCLE`.

COMMAND$

A string function: the arguments the program was started with, as one string.

SYNTAX

```
COMMAND$
```

RESULT

The arguments after the program's name, joined with one space between each, 255 bytes at most. The empty string if there were none, or if HyBASIC was started without a program.

An argument that held spaces of its own can't be told from two arguments here. `ARG$` gives each one whole.

SEE ALSO

`ARG$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; COMMAND$; "]"' >cmd.bas
% basic cmd.bas one 'two words'
[one two words]
% basic cmd.bas
[]
```

COMPLEX

A numeric function: a complex number made from its two parts.

SYNTAX

```
COMPLEX(re, im)
```

ARGUMENTS

re, im

Numeric expressions whose values are real: the real part and the imaginary part.

RESULT

The number *re* + *im* i. The parts keep their kinds: a rational real part with a decimal imaginary part is a complex number of exactly those. If *im* is 0 the result is the real number *re*, not a complex number with nothing in its imaginary part.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*re* or *im* is itself complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

A complex number can also be written directly, as `1+2i` or `0.5i`: digits with an `i` after them are an imaginary number. The `i` must follow a number. `1+i` is 1 plus the variable `I`.

SEE ALSO

REAL, IMAG, SQR.

EXAMPLE

```
> PRINT COMPLEX(1, 2); COMPLEX(1/2, 0.5); COMPLEX(3, 0); COMPLEX(0, 1) ^ 2
 1+2i  1/2+0.5i  3 -1
> PRINT COMPLEX(1+2i, 3)
illegal function call
```

CONST

A statement: gives names to constant values.

SYNTAX

```
CONST name = value [, name = value] ...
```

ARGUMENTS

name

A name that has not been used: a string's if it ends in `$`, otherwise a number's.

value

A constant expression of the name's type: numbers and strings written out, other constants, and operators and built-in functions applied to them.

DESCRIPTION

The value is worked out when the program is compiled, and the name stands for it from then on. A constant can be used wherever a value can, and it is known in every `SUB` and `FUNCTION` without being `SHARED`.

A constant can't be assigned to, and its name can't be given to anything else.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	*name* is already a constant or a variable; or an assignment to a constant
87	`not a constant`	*value* uses a variable, an array or a `FUNCTION`
13	`type mismatch`	A string for a numeric name, or a number for a string name

NOTES

An integer constant's name is also a symbol in the program's `ASM` blocks, where it stands for its value.

SEE ALSO

DIM, LET, ASM.

EXAMPLE

```
> CONST LIMIT = 5, UNIT$ = "cm", TWICE = LIMIT * 2 : PRINT LIMIT; UNIT$; TWICE
 5 cm 10
> LIMIT = 6
duplicate definition
> V = 3 : CONST LATER = V
not a constant
```

CONT

A command of the prompt: goes on with a program that was stopped.

SYNTAX

```
CONT
```

DESCRIPTION

A program stops, with its state kept, at a STOP statement or when CTRL-C is pressed. CONT carries on from the statement after the one it stopped at, with every variable as it is now: variables can be looked at, and changed, at the prompt in between.

A program can no longer be continued once it has ended, once one of its lines has been changed, added or removed, or after NEW, LOAD or CLEAR.

ERRORS

ERR	MESSAGE	CAUSE
17	cannot continue	No program is stopped
89	only at the prompt	CONT in a program's line

SEE ALSO

STOP, END, RUN.

EXAMPLE

```
> 10 PRINT "one" : STOP : PRINT "two"
> 20 PRINT "three"
> RUN
one
Break in line 10
> CONT
two
three
> CONT
cannot continue
```

COS

A numeric function: the cosine of a real number.

SYNTAX

```
COS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The cosine of *x*, from −1 to 1. `COS(0)` is the integer 1. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

An angle in degrees is turned into radians by multiplying by `PI / 180`.

`PI` is itself rounded to the digits set, so `COS(PI / 2)` is not exactly 0. It is a very small number: the cosine of the angle that `PI / 2` really is.

SEE ALSO

`SIN`, `TAN`, `ATN`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT COS(0); COS(1); COS(PI); COS(60 * PI / 180)
 1  0.540302305868 -1  0.5
```

CSRLIN

A numeric function: the row the cursor is on.

SYNTAX

```
CSRLIN
```

RESULT

The cursor's row, counted from 1 at the top of the screen. 0 if HyBASIC has no console (its input or output is a file or a pipe).

DESCRIPTION

HyBASIC does not keep track of the row. It asks the terminal each time, by writing a cursor position request (section 13.7) and reading the terminal's reply, so the answer is right whatever has been written to the screen, by this program or another.

NOTES

The column is `POS(0)`.

SEE ALSO

`POS`, `LOCATE`.

CVN

A numeric function: the number that a string of bytes holds, in the stored format.

SYNTAX

```
CVN(s$)
```

ARGUMENTS

s$

A string expression: a number's bytes, as `MKN$` makes them and as `PUT` writes a numeric variable to a file.

RESULT

The number. It is exactly the number that was stored, of the same kind: `CVN(MKN$(x))` is *x* for every number *x*.

DESCRIPTION

A number's stored form is as long as the number needs: a small integer takes one byte, a rational or a long integer more.

It replaces QuickBASIC's `CVI`, `CVL`, `CVS` and `CVD`, as `MKN$` replaces `MKI$` and the rest: there is one kind of number, so one pair of functions.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is not a number's stored form
13	`type mismatch`	*s$* is a number

NOTES

A string that was never a number may still be some number's stored form, and then there is no error: `CVN("x")` is –8.

SEE ALSO

MKN$, GET, PUT.

EXAMPLE

```
> PRINT CVN(MKN$(1/3)); CVN(MKN$(-7)); CVN(MKN$(2+3i)); LEN(MKN$(7)); LEN(MKN$(1/3))
 1/3 -7  2+3i  1  3
```

DATA

A statement: holds values in the program for READ to take.

SYNTAX

```
DATA item [, item] ...
```

ARGUMENTS

item

A value, written out. It is not an expression: nothing in it is evaluated.

DESCRIPTION

DATA does nothing when control passes over it. Its items are collected when the program is compiled: all the DATA statements of the program, in the order of its lines, make one list, and READ takes the next item from that list each time it fills a variable. RESTORE moves the place back.

An item is the text between two commas, with the spaces before and after it dropped. Text in double quotes is taken as it stands, commas and spaces included, without the quotes. An empty item, two commas with nothing between them, is allowed.

What an item means is decided by the variable that reads it. Read into a string variable, it is its text. Read into a numeric variable, it is read as a number in any form a program's numbers may take (12, -7/2, 2.5, &HFF); text that isn't a number, and an empty item, give 0.

ERRORS

ERR	MESSAGE	CAUSE
4	out of DATA	A READ when every item has been taken. The error is READ's, at its line

NOTES

Because the items aren't expressions, DATA 2 + 2 holds the five characters 2 + 2: nothing adds them up.

A DATA statement can be anywhere in the program, before or after the READ that uses it.

SEE ALSO

READ, RESTORE.

EXAMPLE

```
> 10 DATA 12, hello there , "a, b", 1/2, , &HFF
> 20 READ A, B$, C$, D, E$, F
> 30 PRINT A; "["; B$; "]["; C$; "]"; D; "["; E$; "]"; F
> 40 READ G
> RUN
 12 [hello there][a, b] 1/2 [] 255
line 40: out of DATA
```

DATE$

A string function: today's date.

SYNTAX

```
DATE$
```

RESULT

A string of ten characters, `yyyy-mm-dd`: the year, the month and the day of the system's clock. On a Hydra with no clock chip, the clock starts at 2000-01-01 when the system starts.

ERRORS

ERR	MESSAGE	CAUSE
73	`not done yet`	An assignment to `DATE$`. QuickBASIC sets the date that way; HyBASIC doesn't set the clock

NOTES

The form is not QuickBASIC's `mm-dd-yyyy`. Strings in this form sort in date order.

SEE ALSO

`TIME$`, `TIMER`.

DECLARE

A statement: accepted, and ignored.

SYNTAX

```
DECLARE SUB name [(parameters)]
DECLARE FUNCTION name [(parameters)]
```

DESCRIPTION

QuickBASIC needs a `DECLARE` for a procedure that is used before it is defined. HyBASIC compiles the whole program before it runs any of it, and finds every `SUB` and `FUNCTION` first, so a declaration tells it nothing. The statement is read past without being checked: its parameter list need not match the procedure's.

It is here so that programs written for QuickBASIC load unchanged.

SEE ALSO

`SUB`, `FUNCTION`.

DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR

Statements: set the type of names by their first letter.

SYNTAX

```
DEFSTR letters [, letters] ...
DEFINT letters [, letters] ...
```

ARGUMENTS

letters

One letter, or a range of letters with a hyphen between: `S`, `A-C`.

DESCRIPTION

A name normally belongs to a string if it ends in `$` and to a number if it doesn't. After `DEFSTR`, a name with no `$` whose first letter is one of those given is a string's too.

`DEFINT`, `DEFLNG`, `DEFSNG` and `DEFDBL` all do the opposite, and are the same statement: they make names that start with those letters numbers' again. (In QuickBASIC they choose between four kinds of number. HyBASIC has one.)

The statement takes effect where it stands in the program's text, for the lines after it, not when control reaches it.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	A number assigned to a name that `DEFSTR` has made a string's, or the reverse

SEE ALSO

`DIM`, `OPTION`.

EXAMPLE

```
> 10 DEFSTR S-T : S = "str" : T1 = "t" : PRINT S; T1
> 20 DEFINT S : S2 = 5 : PRINT S2
> RUN
strt
 5
```

DELETE

A command of the prompt: removes lines from the program.

SYNTAX

```
DELETE from[-to]
```

ARGUMENTS

from, to
: Line numbers.

DESCRIPTION

The lines numbered *from* to *to*, both included, are removed. With *from* alone, that one line is. Numbers in the range that no line has are passed over, so a range can name lines that aren't there, and a `DELETE` that finds none is not an error.

A program that was stopped can't be continued after a `DELETE`.

Typing a line number with nothing after it removes that one line, which is usually quicker.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	No line number; or a range with no *from*
89	`only at the prompt`	`DELETE` in a program's line

SEE ALSO

`LIST`, `NEW`, `EDIT`.

EXAMPLE

```
> 10 PRINT 1
> 20 PRINT 2
> 30 PRINT 3
> 40 PRINT 4
> DELETE 20-30
> LIST
10 PRINT 1
40 PRINT 4
```

DENOMINATOR

A numeric function: the denominator of a real number written as a fraction in lowest terms.

SYNTAX

```
DENOMINATOR(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

A positive integer. A rational gives its denominator, with any common factor already taken out: 6/8 is kept as 3/4, so it gives 4. An integer gives 1. A decimal is turned into the fraction it exactly equals first: 0.75 is 3/4, and gives 4.

`NUMERATOR(x) / DENOMINATOR(x)` is *x* for every real *x*; the sign is the numerator's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`NUMERATOR`, `RATIONAL`, `FIXED`, `GCD`.

EXAMPLE

```
> PRINT NUMERATOR(-6/8); DENOMINATOR(-6/8); DENOMINATOR(7); NUMERATOR(0.75);
DENOMINATOR(0.75)
-3  4  1  3  4
```

DIGITS

A statement and a function: sets, or reports, how many significant digits the mathematical functions work to.

SYNTAX

```
DIGITS n
DIGITS
```

ARGUMENTS

n

A numeric expression from 1 to 100, rounded to the nearest integer.

RESULT

`DIGITS`, used as a function, is the current setting: 12 when HyBASIC starts.

DESCRIPTION

Addition, subtraction, multiplication and division of integers, rationals and decimals are exact, and `DIGITS` has nothing to do with them. It governs the functions whose results generally can't be written exactly: `SQR`, `EXP`, `LOG`, `SIN`, `COS`, `TAN`, `ATN`, `PI` and a power with a fractional exponent. Each of those

returns a decimal rounded to *n* significant digits. `RND` returns a decimal of *n* digits too.

A result that happens to be exact comes back exact whatever the setting: `SQR(16)` is the integer 4.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 100
13	`type mismatch`	*n* is a string

NOTES

A value that was worked out at one setting keeps the digits it has when the setting changes. To cut a number to some number of decimal places, use `FIXED`.

SEE ALSO

`BASE`, `FIXED`, `PI`, `SQR`, `RND`.

EXAMPLE

```
> PRINT DIGITS; SQR(2); 1/3
 12  1.41421356237  1/3
> DIGITS 5 : PRINT SQR(2); PI; 1/3 : DIGITS 12
 1.4142  3.1416  1/3
> DIGITS 0
illegal function call
```

DIM

A statement: makes arrays, and declares variables with a type.

SYNTAX

```
DIM [SHARED] name[(bounds [, bounds] ...)] [AS type] [, name ...] ...
```

where *bounds* is

```
upper
lower TO upper
```

ARGUMENTS

name

The array's or the variable's name.

lower, upper

Numeric expressions, each rounded to the nearest integer: the lowest and the highest index in that dimension. If *lower* is left out it is 0, or 1 after `OPTION BASE 1`. An array can have from 1 to 8 dimensions.

type

`INTEGER`, `LONG`, `SINGLE`, `DOUBLE` or `NUMBER`, which all mean a number; `STRING`; or the name of a `TYPE`, for a record. Without `AS`, the type is the name's own: a string if it ends in `$`.

DESCRIPTION

An array is made when the statement runs, so its bounds can be worked out by the program. Every element starts as 0, or as the empty string. `DIM A(3)` has the four elements `A(0)` to `A(3)`.

An array that is used without having been made is made on the spot, with indexes 0 to 10 in each dimension it is used with (from 1 after `OPTION BASE 1`). `DIM` is needed only for other sizes.

A variable with no bounds is declared with its type: `DIM Count AS INTEGER`, `DIM P AS Point`. A numeric type is a statement of intent only. Every numeric variable holds any number, so a variable declared `INTEGER` can be given 1/3 and keeps it.

`SHARED` makes the names known inside every `SUB` and `FUNCTION` as well as in the main program: see `SHARED`.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	The array has been made already. `ERASE` it first, or use `REDIM`
9	`subscript out of range`	A negative *upper*; *lower* above *upper*; more than 8 dimensions. Also: an index outside an array's bounds, wherever the array is used
7	`out of memory`	No room for the array
13	`type mismatch`	A bound that is a string

NOTES

A name followed by parentheses is taken to be an array's unless it is a `FUNCTION`'s. A misspelled function name is therefore not an error: it is a new array, whose elements are all 0.

SEE ALSO

`REDIM`, `ERASE`, `OPTION`, `LBOUND`, `UBOUND`, `TYPE`, `SHARED`, `STATIC`.

EXAMPLE

```
> DIM A(3) : A(3) = 7 : PRINT A(0); A(3); LBOUND(A); UBOUND(A)
 0  7  0  3
> A(4) = 1
subscript out of range
> DIM A(5)
duplicate definition
> DIM C(1 TO 3, -2 TO 2) : C(3, -2) = 9 : PRINT C(3, -2); LBOUND(C, 2); UBOUND(C, 2)
 9 -2  2
> B(10) = 1 : PRINT UBOUND(B)
 10
> DIM Q AS INTEGER : Q = 1/3 : PRINT Q
 1/3
```

DIR$

A string function: the names in a directory, one at a time.

SYNTAX

```
DIR$(path$)
DIR$
```

ARGUMENTS

path$

A string expression: a directory's name.

RESULT

`DIR$(path$)` is the first name in the directory, and each `DIR$` after it, with no argument, is the next. A name that is itself a directory has a `/` after it. The empty string when there are no more, and for every call after that. The names come in the directory's own order, which is not alphabetical.

The empty string, too, if *path$* is a file and not a directory.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name as *path$*
13	`type mismatch`	*path$* is a number

NOTES

path$ names a directory. It is not a pattern: QuickBASIC's `DIR$("*.BAS")` has no meaning here. To pick out some of the names, test each one as it comes.

SEE ALSO

`FILES`, `CHDIR`, `OPEN`.

EXAMPLE

```
> CHDIR "/ram" : MKDIR "sub" : OPEN "a.txt" FOR OUTPUT AS #1 : CLOSE
> N$ = DIR$("/ram") : DO WHILE N$ <> "" : PRINT N$; " "; : N$ = DIR$ : LOOP : PRINT
bin/ lib/ sub/ a.txt
```

DO ... LOOP

A statement block: a loop that repeats while, or until, a condition holds.

SYNTAX

```
DO [{WHILE | UNTIL} condition]
    [statements]
LOOP

DO
    [statements]
LOOP [{WHILE | UNTIL} condition]
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0.

DESCRIPTION

With the condition after `DO`, it is tested before each pass, the first included, so the statements may not run at all. With the condition after `LOOP`, it is tested after each pass, so the statements run at least once.

`WHILE` goes on for as long as the condition holds. `UNTIL` goes on for as long as it does not.

With no condition in either place the loop repeats for ever, until `EXIT DO` leaves it, an error stops it, or CTRL-C is pressed. `EXIT DO` leaves the innermost `DO` loop at once, whatever its condition, and goes on after its `LOOP`.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	A `DO` with no `LOOP`, or a `LOOP` with no `DO`; a condition in both places; the loop and another block overlapping. Found when the program is compiled
13	`type mismatch`	The condition is a string

NOTES

At the prompt a loop must be whole on its line, since each line typed is compiled alone.

`WHILE` ... `WEND` is the same as `DO WHILE` ... `LOOP`.

SEE ALSO

`EXIT`, `WHILE`, `FOR`.

EXAMPLE

```
> I = 0 : DO : I = I + 1 : LOOP UNTIL I = 3 : PRINT I
 3
> I = 5 : DO WHILE I < 3 : I = I + 1 : LOOP : PRINT I
 5
> I = 5 : DO : I = I + 1 : LOOP WHILE I < 3 : PRINT I
 6
```

DRAW

A statement: draws with a string of pen movements, QuickBASIC's turtle language.

SYNTAX

```
DRAW commands$
```

ARGUMENTS

commands$

A string expression: movement commands, one after another. Spaces between them are optional, and capitals and small letters are the same.

DESCRIPTION

Needs a graphics mode. Drawing starts at the last point referenced, and each move draws a line from where the pen is to where it goes, in the pen's colour, and leaves the pen there.

COMMAND	MOVES THE PEN
`U` *n*, `D` *n*, `L` *n*, `R` *n*	Up, down, left, right by *n* units
`E` *n*, `F` *n*, `G` *n*, `H` *n*	Diagonally: up and right, down and right, down and left, up and left, *n* units each way
`M` *x*, *y*	To the point (*x*, *y*)
`M` +*x*, *y* or `M` –*x*, *y*	By that much from where it is: a sign on the first number makes both numbers offsets

n is 1 if it is left out. A unit is one pixel to begin with.

BEFORE A MOVE	EFFECT
`B`	Move without drawing
`N`	Draw, then put the pen back where it started

SETTING	EFFECT
`C` *n*	The colour to draw in. It becomes the pen's colour, and stays so after the `DRAW`
`S` *n*	The scale: a unit is *n* / 4 pixels. 4 to begin with
`A` *n*	Turn every direction by *n* × 90 degrees, anticlockwise: *n* is 0 to 3
`TA` *n*	Turn every direction by *n* degrees, anticlockwise

The scale and the turn stay in force for the `DRAW` statements that follow, until they are set again. A line that runs off the bitmap is drawn up to its edge.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or a letter in the string that is not a command
13	`type mismatch`	The argument is a number

NOTES

QuickBASIC's `X`, which runs another string, and `P`, which paints, are not here. Join strings with `+`, and use `PAINT`.

SEE ALSO

`LINE`, `PSET`, `COLOR`, `PAINT`, `SCREEN`.

EXAMPLE

A square of side 10, and a line at 45 degrees:

```
> SCREEN 13 : DRAW "BM150,100 C5 R10 D10 L10 U10"
> PRINT POINT(155, 100); POINT(160, 105); POINT(150, 105); POINT(155, 105)
 5  5  5  0
> DRAW "BM150,150 C7 TA45 R10" : PRINT POINT(157, 143)
 7
> SCREEN 0
```

EDIT

A command of the prompt: opens the program in the screen editor.

SYNTAX

```
EDIT [line]
```

ARGUMENTS

line

A line number or a label: where the editor's cursor starts.

DESCRIPTION

The program's text is written to a file in `/ram`, and the screen editor, `edit` (Chapter 11), is started on it with the cursor at *line*. With no *line*, the cursor starts at the line of the last error, if there has been one.

When the editor is left, the file is read back as the program and compiled, and the first error found, if there is one, is reported with its line. `EDIT` again goes to it.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`EDIT` in a program's line

SEE ALSO

LIST, LOAD, SAVE, DELETE.

ELSE, ELSEIF

Parts of IF: ELSE begins the statements for a condition that didn't hold, and ELSEIF tests another. CASE ELSE is the last choice of a SELECT CASE. See IF and SELECT.

END

A statement: ends the program. Also the first word of the lines that close a block.

SYNTAX

```
END
END {IF | SELECT | SUB | FUNCTION | TYPE | ASM}
```

DESCRIPTION

END alone stops the program and returns to the prompt, or, when HyBASIC was started with a program to run, ends HyBASIC. Nothing is printed. A program also ends when control runs off its last line, so END is needed only to stop sooner, typically before the lines of a subroutine that GOSUB reaches.

A program that has ended can't be continued with CONT. To stop in a way that can be, use STOP.

The two-word forms each close the block of that name: see IF, SELECT, SUB, FUNCTION, TYPE and ASM.

SEE ALSO

STOP, SYSTEM, CONT.

EXAMPLE

```
> 10 PRINT "a" : END : PRINT "b"
> RUN
a
> CONT
cannot continue
```

ENV$, ENVIRON$

String functions: the value of a variable of the environment.

SYNTAX

```
ENVIRON$(name$)
ENV$(name$)
```

ARGUMENTS

name$

A string expression: the variable's name, in the case it has. The shell's own variables are in small letters.

RESULT

The variable's value. A value is a list of words (section 8.4); they are returned joined with one space between each. The empty string if there is no such variable.

The two names are the same function.

DESCRIPTION

The environment is the set of named values a task starts with, copied from the task that started it: for HyBASIC, usually a shell, whose variables they are: `path`, `prompt` and the rest. Each is also a file in `/env` (section 17.7).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*name$* is a number

SEE ALSO

`ENVIRON`, `ARG$`, `COMMAND$`, `SHELL`.

EXAMPLE

```
> PRINT ENVIRON$("path"); "|"; ENV$("nosuch"); "|"
. /bin||
```

ENVIRON

A statement: sets, or removes, a variable of the environment.

SYNTAX

```
ENVIRON "name=value"
```

ARGUMENTS

A string expression: the variable's name, an equals sign, and its value.

DESCRIPTION

The variable is set in HyBASIC's own environment. The value is one word, whatever spaces are in it. Programs that HyBASIC starts afterwards with `SHELL` get a copy of the environment and see the variable; the shell that started HyBASIC does not, since a task can't change its parent's environment.

With nothing after the equals sign, the variable is removed.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No equals sign in the string
13	`type mismatch`	The argument is a number

SEE ALSO

`ENVIRON$`, `SHELL`.

EXAMPLE

```
> ENVIRON "colour=red" : PRINT ENVIRON$("colour") : SHELL "cat /env/colour; echo"
red
red
> ENVIRON "colour=" : PRINT "["; ENVIRON$("colour"); "]"
[]
```

EOF

A numeric function: whether a file has been read to its end.

SYNTAX

```
EOF(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file.

RESULT

−1 (true) if there is nothing more to read from file *f*, and 0 (false) if there is. For a file opened `FOR OUTPUT` or `FOR APPEND`, always −1.

DESCRIPTION

The usual loop reads while there is something to read: `DO UNTIL EOF(1) : LINE INPUT #1, L$ : ... : LOOP`. Testing first matters, because reading at the end is an error (62, `input past end of file`), and an empty file is at its end from the start.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
13	`type mismatch`	*f* is a string

SEE ALSO

`OPEN`, `INPUT`, `LINE INPUT`, `LOF`, `LOC`.

EXAMPLE

```
> OPEN "/ram/two.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT #1, "two" : CLOSE
> OPEN "/ram/two.txt" FOR INPUT AS #1
> DO UNTIL EOF(1) : LINE INPUT #1, L$ : PRINT L$; EOF(1) : LOOP : CLOSE
one 0
two-1
```

EQV

An operator: bitwise equivalence.

SYNTAX

```
a EQV b
```

RESULT

An integer. Each bit of it is 1 where the same bit of *a* and of *b* are alike (both 0 or both 1), and 0 where they differ. It is `NOT (a XOR b)`.

For two truth values, –1 and 0, the result is true when they are the same.

The operands are evaluated, rounded and treated as `AND`'s are: see `AND`. `EQV` binds less tightly than `AND`, `OR` and `XOR`, and more tightly than `IMP`.

EXAMPLE

```
> PRINT 5 EQV 3; (1 = 1) EQV (2 = 3); 0 EQV 0
-7  0 -1
```

ERASE

A statement: drops arrays.

SYNTAX

```
ERASE array [, array] ...
```

ARGUMENTS

array

An array's name, with no parentheses.

DESCRIPTION

Each array named is dropped, and the memory it took is free again. The name is then as if it had never been used: a `DIM` can make it again with other bounds, and using it without one makes it afresh with indexes 0 to 10, every element 0 or empty.

A name that isn't an array's is passed over without an error.

SEE ALSO

DIM, REDIM, CLEAR.

EXAMPLE

```
> DIM A(3) : A(3) = 7 : ERASE A : DIM A(50) : PRINT A(3); UBOUND(A)
 0  50
```

ERL, ERR, ERR$

Functions: the line, the number and the message of the last error.

SYNTAX

```
ERR
ERR$
ERL
```

RESULT

`ERR` is the number of the most recent error, and 0 if there has been none. The numbers are QuickBASIC's where it has an error of the same meaning (5 is `illegal function call`, 53 is `file not found`); the Hydra's own start at 80. An error reported by the system that has no number of HyBASIC's is 256 plus the system's code (Appendix G).

`ERR$` is that error's message, as it is printed: in small letters, with no line number. `error 0` if there has been none.

`ERL` is the number of the line the error was on. If that line has no number, it is the number of the nearest numbered line before it; in a program with no line numbers at all, it is the line's position in its file, counted from 1. 0 if the error was in a line typed at the prompt, or if there has been none.

DESCRIPTION

The three keep their values until the next error: a statement that succeeds doesn't clear them. They are meant to be read in an error handler (`ON ERROR GOTO`), where they tell it what happened and where.

SEE ALSO

ON, RESUME, ERROR.

EXAMPLE

```
> PRINT ERR; ERL; ERR$
 0  0 error 0
> 10 X = 1 / 0
> RUN
line 10: division by zero
> PRINT ERR; ERL; ERR$
 11  10 division by zero
```

ERROR

A statement: raises an error.

SYNTAX

```
ERROR n
```

ARGUMENTS

n

A numeric expression from 1 to 255: the error's number.

DESCRIPTION

The program behaves exactly as if error *n* had happened at this statement. If an `ON ERROR GOTO` handler is in force, control goes to it, with `ERR` equal to *n* and `ERL` this line; otherwise the program stops and the error's message is printed.

If *n* is one of HyBASIC's error numbers, the message is that error's. Any other number from 1 to 255 can be used for an error of the program's own; its message is `error` and the number.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 255

SEE ALSO

`ON`, `RESUME`, `ERR`.

EXAMPLE

```
> 10 ON ERROR GOTO 50
> 20 ERROR 200
> 30 PRINT "after"
> 40 END
> 50 PRINT "caught"; ERR; ERL; ERR$ : RESUME NEXT
> RUN
caught 200  20 error 200
after
```

EXIT

A statement: leaves a loop or a procedure early.

SYNTAX

```
EXIT {FOR | DO | SUB | FUNCTION}
```

DESCRIPTION

`EXIT FOR` and `EXIT DO` leave the innermost loop of that kind that the statement is inside, and the program goes on with the statement after the loop's `NEXT` or `LOOP`. A `FOR` loop's variable keeps the value it has. An `EXIT DO` inside a `FOR` loop that is itself inside a `DO` loop leaves both.

`EXIT SUB` and `EXIT FUNCTION` return from the procedure at once. A function returns the value last assigned to its name, or 0 or the empty string if none has been.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	`EXIT FOR` or `EXIT DO` that is not inside such a loop
85	`not in a procedure`	`EXIT SUB` or `EXIT FUNCTION` outside a `SUB` or a `FUNCTION`

NOTES

There is no `EXIT WHILE`, and `EXIT DO` doesn't leave a `WHILE` ... `WEND` loop. A loop that needs an early way out should be written with `DO`.

SEE ALSO

`FOR`, `DO`, `SUB`, `FUNCTION`.

EXAMPLE

```
> 10 FOR I = 1 TO 100 : IF I * I > 50 THEN EXIT FOR
> 20 NEXT : PRINT I
> RUN
 8
```

EXP

A numeric function: e raised to a power.

SYNTAX

```
EXP(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

e to the power *x*, where e is 2.71828... `EXP(0)` is the integer 1. Any other result is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

There is no overflow. A large result is a decimal of that many significant digits followed by zeros, and is printed in full.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`LOG`, `SQR`, `DIGITS`.

EXAMPLE

```
> PRINT EXP(0); EXP(1); EXP(-1); EXP(1/2)
 1  2.71828182846  0.367879441171  1.6487212707
> PRINT EXP(100)
 26881171418200000000000000000000000000000000
```

FIB

A numeric function: a Fibonacci number.

SYNTAX

```
FIB(n)
```

ARGUMENTS

n

A numeric expression whose value is an integer, 0 or more.

RESULT

The *n*th Fibonacci number, an exact integer of whatever length: `FIB(0)` is 0, `FIB(1)` is 1, and each one after is the sum of the two before it.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative, or is not an integer
13	`type mismatch`	*n* is a string

SEE ALSO

`GCD`.

EXAMPLE

```
> PRINT FIB(0); FIB(1); FIB(2); FIB(30); FIB(100)
 0  1  1  832040  354224848179261915075
```

FILES

A statement: lists the names in a directory.

SYNTAX

```
FILES [path$]
```

ARGUMENTS

path$

A string expression: a directory's name. The current directory if it is left out.

DESCRIPTION

The names are printed across the screen, each in a column 16 characters wide, five to a line, in the directory's own order. A name that is a directory has a `/` after it. Nothing else is shown: for sizes and dates, use `SHELL "ls -l"`.

If *path$* is a file and not a directory, nothing is listed.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name as *path$*
13	`type mismatch`	*path$* is a number

NOTES

path$ is a directory, not a pattern: `FILES "*.BAS"` looks for a directory of that name.

SEE ALSO

`DIR$`, `CHDIR`, `KILL`, `NAME`, `SHELL`.

EXAMPLE

```
> FILES "/rom/lib"
as/             basic/          edit/           font/           forth/
hylang/         namespace       profile         shell           windows
```

FIX

A numeric function: a real number with its fraction removed.

SYNTAX

```
FIX(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer part of *x*: *x* rounded toward zero. For a positive number this is `INT(x)`. For a negative number that isn't whole it is one more: `FIX(-2.5)` is −2, where `INT(-2.5)` is −3. The result is an exact integer of whatever length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT`, `CINT`, `FIXED`.

EXAMPLE

```
> PRINT FIX(2.5); FIX(-2.5); FIX(7/2); FIX(-7/2); INT(-2.5)
 2 -2  3 -3 -3
```

FIXED

A numeric function: a real number as a decimal with a given number of places.

SYNTAX

```
FIXED(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real.

n

A numeric expression, 0 or more: the number of decimal places.

RESULT

A decimal: *x* with the digits after its *n*th decimal place cut off. The digits are cut, not rounded, so the result is never further from zero than *x*: `FIXED(2/3, 2)` is 0.66, and `FIXED(-2/3, 2)` is −0.66. Zeros at the end are not kept, so `FIXED(5, 2)` is 5.

DESCRIPTION

This is the way to turn a rational into a decimal. `1/3` is kept and printed as `1/3`; `FIXED(1/3, 6)` is 0.333333. *n* is not limited by `DIGITS`, which is about the mathematical functions.

To round instead of cutting, add half of the last place first: `FIXED(x + 0.005, 2)` rounds a positive *x* to two places.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	An argument is a string

SEE ALSO

`RATIONAL`, `FIX`, `CINT`, `DIGITS`, `PRINT` (`USING`).

EXAMPLE

```
> PRINT FIXED(22/7, 4); FIXED(2/3, 2); FIXED(-2/3, 2); FIXED(5, 2); FIXED(1.239, 2)
 3.1428  0.66 -0.66  5  1.23
> PRINT FIXED(1/3, 30)
 0.333333333333333333333333333333
```

FOR ... NEXT

A statement block: a loop that counts a variable from one value to another.

SYNTAX

```
FOR v = start TO limit [STEP step]
    [statements]
NEXT [v [, v] ...]
```

ARGUMENTS

v

The loop's counter: a simple numeric variable. An array's element can't be one.

start, limit, step

Numeric expressions. *step* is 1 if `STEP` is left out.

DESCRIPTION

1. *start* is assigned to *v*, and *limit* and *step* are evaluated and kept. They are evaluated this once only: a variable they were worked out from can change inside the loop without changing them.
2. The test: if *step* is zero or more and *v* is greater than *limit*, or *step* is negative and *v* is less than *limit*, the loop ends, and the program goes on after its `NEXT`. The test is made before the first pass too, so the statements may not run at all; *v* then keeps the value *start*.
3. Otherwise the statements run.
4. At `NEXT`, *step* is added to *v*, and the loop goes back to the test at step 2.

When the loop ends at the test, *v* holds the first value that failed it: after `FOR I = 1 TO 3`, `I` is 4.

The addition is exact. A loop with a fractional step, `STEP 0.1` or `STEP 1/3`, reaches its limit on the last pass, where a binary floating-point BASIC may stop one short.

The statements may assign to *v*; `NEXT` adds *step* to whatever it then holds.

`EXIT FOR` leaves the innermost loop at once, with *v* as it is.

`NEXT` with no variable closes the innermost open loop. `NEXT v` does the same, and the compiler checks that *v* is that loop's counter. `NEXT j, i` is `NEXT j : NEXT i`.

ERRORS

ERR	MESSAGE	CAUSE
26	`FOR without NEXT`	A `FOR` whose `NEXT` isn't there. Found when the program is compiled: nothing has run
1	`NEXT without FOR`	A `NEXT` with no loop open, or naming a variable that isn't the innermost loop's counter
80	`block without its start or end`	The loop and another block overlap: a `NEXT` inside a one-line `IF`, say
13	`type mismatch`	*v* is a string variable, or *start*, *limit* or *step* a string
2	`syntax error`	*v* is not a simple variable

NOTES

With a *step* of 0 the counter never changes and the loop doesn't end by itself: only `EXIT FOR`, an error or CTRL-C ends it.

At the prompt, a line is compiled alone, so a loop typed there must be whole on its line: `FOR I = 1 TO 3` by itself is `FOR without NEXT`.

SEE ALSO

`EXIT`, `DO...LOOP`, `WHILE...WEND`.

EXAMPLE

```
> FOR I = 1 TO 3 : PRINT I; : NEXT : PRINT "then"; I
 1  2  3 then 4
> FOR I = 5 TO 1 : PRINT I; : NEXT : PRINT "then"; I
then 5
> FOR X = 0 TO 1 STEP 1/3 : PRINT X; : NEXT : PRINT
 0  1/3  2/3  1
> N = 3 : FOR I = 1 TO N : N = 10 : PRINT I; : NEXT : PRINT
 1  2  3
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : PRINT I; J; : NEXT J, I : PRINT
 1  1  1  2  2  1  2  2
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : NEXT I
NEXT without FOR
```

FRE

A numeric function: how much memory is free.

SYNTAX

```
FRE()
FRE(x)
```

ARGUMENTS

x

Any expression. It is evaluated and ignored; QuickBASIC's forms `FRE(0)` and `FRE("")` both work.

RESULT

The number of bytes left for the program's data: its strings, its arrays and its numbers too long for a variable's own five bytes. The figure is the room in the RAM banks HyBASIC has not yet used, so it falls as arrays are made and strings grow, and rises when they are dropped.

NOTES

With three RAM modules a new HyBASIC has about 300K free.

SEE ALSO

`DIM`, `ERASE`, `CLEAR`.

EXAMPLE

The figure itself depends on the machine. An array of 5,001 numbers takes five bytes for each, and twelve for the array:

```
> A = FRE() : DIM BIG(5000) : PRINT A - FRE()
 25017
```

FREEFILE

A numeric function: a file number that is not in use.

SYNTAX

```
FREEFILE
```

RESULT

The lowest number, from 1 up, that no open file has. It is the number to give the next `OPEN` in a routine that can't know which numbers the rest of the program is using.

NOTES

`FREEFILE` finds a number. It doesn't reserve it: call it again only after the `OPEN`, or it returns the same one.

Since 8 files can be open at once and numbers go to 255, there is always a free number. When 8 are open the `OPEN` fails instead, with `too many files`.

SEE ALSO

`OPEN`, `CLOSE`.

EXAMPLE

```
> PRINT FREEFILE : OPEN "/ram/x" FOR OUTPUT AS #1 : OPEN "/ram/y" FOR OUTPUT AS #3
 1
> PRINT FREEFILE : CLOSE #1 : PRINT FREEFILE : CLOSE
 2
 1
```

FUNCTION ... END FUNCTION

A block: defines a function, which returns a value and is used in expressions.

SYNTAX

```
FUNCTION name [(parameter [, parameter] ...)] [STATIC]
    [statements]
    name = value
END FUNCTION
```

ARGUMENTS

name

The function's name. It returns a string if the name ends in `$`, and a number otherwise.

parameter

A variable's name, optionally followed by `AS type`; or an array's, written with empty parentheses.

STATIC

The function's variables keep their values from one call to the next. See `STATIC`.

DESCRIPTION

The function is called by using its name in an expression, with an argument for each parameter: `PRINT Area(3, 4)`. A function with no parameters is used as a bare name.

The value returned is whatever was last assigned to the function's name when the function ends, at `END FUNCTION` or at an `EXIT FUNCTION`. If nothing was assigned, it is 0, or the empty string.

Arguments are passed as a `SUB`'s are (see `CALL`): a variable or an array's element by reference, so that assigning to the parameter changes it; any other expression by value. To protect a variable, pass it in parentheses of its own.

The variables used inside the function are its own. A name there is a new variable, 0 or empty at each call, even if the main program has a variable of the same name; the main program's is not touched. Names declared `SHARED`, and constants, are the exceptions.

A function may call itself. Each call has its own parameters and variables.

Functions are found when the program is compiled, wherever they are in its text, so one can be used on a line before its definition. When control reaches a `FUNCTION` line in the ordinary run of the program, it skips to after `END FUNCTION`.

ERRORS

ERR	MESSAGE	CAUSE
37	`argument-count mismatch`	A call with more arguments, or fewer, than the function has parameters
10	`duplicate definition`	Two procedures of one name
13	`type mismatch`	An argument of the wrong type; a string assigned to a numeric function's name
80	`block without its start or end`	`FUNCTION` with no `END FUNCTION`
2	`syntax error`	`FUNCTION` typed at the prompt: a function can be defined only in a program

NOTES

A name followed by parentheses that is not a function's is an array's. Calling a function that doesn't exist, or misspelling one, is therefore not an error: it makes an array of that name and returns its element, 0.

SEE ALSO

`SUB`, `CALL`, `EXIT`, `SHARED`, `STATIC`, `DECLARE`.

EXAMPLE

```
> 10 FUNCTION Fact (N)
> 20 IF N <= 1 THEN Fact = 1 ELSE Fact = N * Fact(N - 1)
> 30 END FUNCTION
> 40 FUNCTION Bump (V) : V = V + 1 : X = 99 : Bump = V : END FUNCTION
> 50 PRINT Fact(5); Fact(25)
> 60 X = 1 : Z = 1 : PRINT Bump(Z); Z; Bump((Z)); Z; X
> 70 PRINT Fcat(5)
> RUN
 120  15511210043330985984000000
 2  2  3  2  1
 0
```

GCD

A numeric function: the greatest common divisor of two integers.

SYNTAX

```
GCD(a, b)
```

ARGUMENTS

a, b

Numeric expressions whose values are real. Each is rounded to the nearest integer, a half going to the even one.

RESULT

The largest integer that divides both *a* and *b* exactly. It is never negative: the signs of *a* and *b* make no difference. `GCD(a, 0)` is the size of *a*, and `GCD(0, 0)` is 0. The integers can be of any length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	An argument is complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

The least common multiple of two positive integers is `a * b / GCD(a, b)`.

Rationals are always kept in lowest terms, so there is no need for `GCD` to reduce a fraction.

SEE ALSO

`MOD`, `NUMERATOR`, `DENOMINATOR`, `FIB`.

EXAMPLE

```
> PRINT GCD(12, 18); GCD(-12, 18); GCD(0, 5); GCD(17, 5); GCD(2 ^ 64, 2 ^ 40)
 6  6  5  1  1099511627776
```

GET

A statement: reads a value from a `BINARY` file, at a given position.

SYNTAX

```
GET [#]f, [position], v
```

ARGUMENTS

f

A numeric expression: the number of a file opened `FOR BINARY`.

position

A numeric expression: where to read, counted in bytes from 1 for the file's first byte. If it is left out (the two commas are still written) or is less than 1, the read starts where the last `GET` or `PUT` on the file ended.

v

A variable: where the value goes. Its type and, for a string, its present length decide how many bytes are read.

DESCRIPTION

A string variable is filled with as many bytes from the file as it holds now. To read ten bytes, make the variable ten bytes long first, with `SPACE$(10)` say. If the file ends first, the bytes that weren't there are left as zero bytes; the variable keeps its length.

A numeric variable reads one number in the stored format, the form `PUT` writes and `MKN$` makes. A number takes as many bytes as it needs, which the number's own first bytes say, so numbers written one after another are read back one after another without their lengths being known.

After the read, the file's position is just past what was read.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* was not opened `FOR BINARY`
62	`input past end of file`	A number was to be read, and the file had ended

NOTES

There are no records and no `FIELD`: QuickBASIC's random files are not here. `GET` and `PUT` always work in bytes, and a program that wants records of a fixed size works out each one's position itself.

SEE ALSO

`PUT`, `OPEN`, `SEEK`, `LOC`, `LOF`, `CVN`, `MKN$`, `INPUT$`.

EXAMPLE

```
> OPEN "/ram/b.dat" FOR BINARY AS #1 : A$ = "HELLO" : PUT #1, 1, A$ : N = 1/3 : PUT #1,
, N
> PRINT LOF(1) : B$ = SPACE$(3) : GET #1, 2, B$ : PRINT B$
 8
ELL
> GET #1, 6, M : PRINT M; EOF(1) : CLOSE
 1/3 -1
```

GOSUB

A statement: calls a subroutine, which is a part of the program that ends with `RETURN`.

SYNTAX

```
GOSUB {line | label}
```

ARGUMENTS

line

A line number.

label

A label: a name with a colon after it at the start of a line that has no number.

DESCRIPTION

Control goes to the line named, and the place to come back to is remembered. The next `RETURN` that runs comes back to the statement after the `GOSUB`.

Subroutines can call others, and themselves; each `RETURN` goes back to the most recent `GOSUB` that hasn't been returned from.

A subroutine is not a block: it is only lines of the program, and nothing stops control from running into them from above. The usual arrangement is to put the subroutines at the end of the program, with an `END` before them.

A subroutine shares every variable with the code that called it. A `SUB` or a `FUNCTION`, with parameters and variables of its own, is nearly always the better tool.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or that label. Found when the program is compiled
3	`RETURN without GOSUB`	A `RETURN` ran with no `GOSUB` to go back to
81	`out of stack space`	Too many `GOSUB`s without a `RETURN`

NOTES

A `GOSUB` inside a `SUB` or `FUNCTION` can only reach lines of that same procedure.

SEE ALSO

`RETURN`, `GOTO`, `ON`, `SUB`, `FUNCTION`.

EXAMPLE

```
> 10 GOSUB 100 : PRINT "back" : END
> 100 PRINT "in the subroutine" : RETURN
> RUN
in the subroutine
back
```

GOTO

A statement: goes to another line.

SYNTAX

```
GOTO {line | label}
```

DESCRIPTION

Control goes to the first statement of the line with that number or that label, and carries on from there.

The same jump is written inside an `IF` as `IF c THEN 100` or `IF c GOTO 100`.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or that label. Found when the program is compiled

NOTES

A label marks a line that has no number. In a program typed with line numbers, the targets are numbers; in a program written in a file without them, they are labels, and a line may still be given a number so that an old listing's `GOTO 100` works.

`GOTO` typed at the prompt can't jump into the program: a line typed there is compiled alone, and the program's lines are not known to it (`label not defined`).

SEE ALSO

`GOSUB`, `ON`, `IF`, `DO`, `EXIT`.

GPRINT

A statement: writes text on the bitmap.

SYNTAX

```
GPRINT [STEP] (x, y), text$
```

ARGUMENTS

x, y
: Numeric expressions: where the top left corner of the first character goes. With `STEP`, an offset from the last point referenced.

text$
: A string expression. Its first 40 characters are drawn.

DESCRIPTION

Needs a graphics mode. The characters are drawn as dots, in the console's font and the pen's colour.

This is text that is part of the picture. It is not the console's text, which `PRINT` writes and which lies over the bitmap in its own layer; `CLS` clears both.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set
258	`invalid argument`	The point is far off the bitmap
13	`type mismatch`	*text$* is a number

SEE ALSO

`PRINT`, `LOCATE`, `COLOR`, `SCREEN`.

HEX$

A string function: a number written in hexadecimal.

SYNTAX

```
HEX$(x)
```

ARGUMENTS

x

A numeric expression of any kind.

RESULT

The text of *x* in base 16, as `STR$(x, "x")` would give it, with no space in front. The digits above 9 are the capitals A to F.

This is the number itself written in another base, not the contents of a machine word. A negative number has a minus sign, so `HEX$(-1)` is `-1`, where QuickBASIC gives `FFFF`. A number that isn't an integer is written as what it is: 2.5 is `2.8` (two and eight sixteenths), and 1/3 is `1/3`. An integer can be of any length.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

NOTES

For the two's complement form of a negative number in some number of bits, mask it first: `HEX$(-1 AND &HFFFF)` is `FFFF`.

To read hexadecimal text back, put `&H` before it and use `VAL`.

SEE ALSO

`OCT$`, `STR$`, `BASE`, `VAL`.

EXAMPLE

```
> PRINT HEX$(255); " "; HEX$(-1); " "; HEX$(2 ^ 40); " "; HEX$(2.5); " "; HEX$(-1 AND
&HFFFF)
FF -1 10000000000 2.8 FFFF
```

IF

A statement: runs statements only if a condition holds. It has a one-line form and a block form.

SYNTAX

```
IF condition THEN statements [ELSE statements]
IF condition THEN line
IF condition GOTO line

IF condition THEN
    [statements]
[ELSEIF condition THEN
    [statements]] ...
[ELSE
    [statements]]
END IF
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0. A comparison gives –1 or 0, but any number will do: 1/2 holds.

line

A line number or a label, which is jumped to.

DESCRIPTION

The one-line form is all on one line. If the condition holds, the statements after `THEN` run, up to the `ELSE` or the end of the line; if it doesn't, the statements after `ELSE` run, if there is an `ELSE`. Either part can be several statements with colons between them, and they all belong to the `IF`: nothing on the line after `THEN` runs when the condition fails, except what follows `ELSE`.

A one-line `IF` can hold another. An `ELSE` belongs to the nearest `IF` before it that has none.

The block form has nothing after `THEN` on the first line. The conditions are tested in order, the `IF`'s and then each `ELSEIF`'s, and the statements under the first one that holds are run; the rest are skipped, their conditions not evaluated. If none holds, the statements under `ELSE` run. Control then goes on after `END IF`.

Block `IF`s nest to any depth, and can hold loops and other blocks, each closed inside the part it began in.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	The condition is a string. (Two strings can be compared: the comparison is a number)
80	`block without its start or end`	A block `IF` with no `END IF`; `ELSE`, `ELSEIF` or `END IF` with no block `IF`
8	`label not defined`	`THEN` or `GOTO` with a line that isn't there

NOTES

At the prompt only the one-line form can be used, since a line typed there is compiled alone.

Both sides of an `AND` or an `OR` in a condition are always evaluated. To test one thing only if another holds, use two `IF`s.

SEE ALSO

`SELECT`, `AND`, `OR`, `NOT`, `GOTO`.

EXAMPLE

```
> 10 IF 1 THEN PRINT "a"; : PRINT "b" ELSE PRINT "c"; : PRINT "d"
> 20 IF 0 THEN PRINT "a"; : PRINT "b" ELSE PRINT "c"; : PRINT "d"
> 30 IF 5 THEN IF 0 THEN PRINT "x" ELSE PRINT "inner else"
> 40 X = 3
> 50 IF X = 1 THEN
> 60   PRINT "one"
> 70 ELSEIF X = 3 THEN
> 80   PRINT "three"
> 90 ELSE
> 100  PRINT "other"
> 110 END IF
> RUN
ab
cd
inner else
three
```

IMAG

A numeric function: the imaginary part of a number.

SYNTAX

```
IMAG(z)
```

ARGUMENTS

z

A numeric expression of any kind.

RESULT

The imaginary part of *z*, a real number of the kind it has in *z*. 0 if *z* is real.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*z* is a string

SEE ALSO

`REAL`, `COMPLEX`.

EXAMPLE

```
> PRINT IMAG(1+2i); REAL(1+2i); IMAG(5); IMAG(1/2+0.5i)
 2  1  0  0.5
```

IMP

An operator: bitwise implication.

SYNTAX

```
a IMP b
```

RESULT

An integer. Each bit of it is 0 where that bit is 1 in *a* and 0 in *b*, and 1 everywhere else. It is `(NOT a) OR b`.

For two truth values, −1 and 0, the result is false only when *a* is true and *b* is false: "if *a*, then *b*".

The operands are evaluated, rounded and treated as `AND`'s are: see `AND`. `IMP` binds less tightly than every other operator.

EXAMPLE

```
> PRINT -1 IMP 0; 0 IMP 0; 0 IMP -1; -1 IMP -1; 5 IMP 3
 0 -1 -1 -1 -5
```

INCLUDE

A statement: reads another file's lines into the program at this point.

SYNTAX

```
INCLUDE "file"
'$INCLUDE: 'file'
REM $INCLUDE: 'file'
```

ARGUMENTS

file

The file's name, written out: a literal, not an expression.

DESCRIPTION

The lines of *file* are compiled as if they were written where the `INCLUDE` line is. The statement must be on a line of its own. The second and third forms are QuickBASIC's, in which the statement is disguised as a comment; they do the same.

A name that starts with `/` is used as it is. Any other name is looked for in the directory the program itself came from, and then in `/lib/basic`.

An included file can include others, four deep. A program can include eight files in all.

An error in an included file is reported with that file's name and its own line number there.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	The file is in neither place
12	`illegal in direct mode`	`INCLUDE` typed at the prompt
2	`syntax error`	`INCLUDE` with other statements on its line

NOTES

Including is done when the program is compiled, not when control reaches the line, so a file can't be chosen by the running program. To run another program from a program, use `SHELL "basic file"`.

SEE ALSO

`LOAD`, `SHELL`, `DECLARE`.

EXAMPLE

```
% echo 'PRINT "included"' >inc.bas
% echo 'PRINT "main"' >main.bas
% echo 'INCLUDE "inc.bas"' >>main.bas
% echo 'PRINT "after"' >>main.bas
% basic main.bas
main
included
after
```

INKEY$

A string function: a key that has been pressed, without waiting for one.

SYNTAX

```
INKEY$
```

RESULT

The empty string if no key is waiting. Otherwise one key, which is taken out of the keyboard's queue:

- An ordinary key is a string of one byte, the key's character.
- A cursor key or a function key is a string of two bytes: `CHR$(0)`, then a code for the key. The codes are QuickBASIC's: 72 is up, 80 is down, 59 is F1, and so on.

DESCRIPTION

`INKEY$` never waits. The usual use is in a loop that does something else as well, or one that waits for any key: `DO : K$ = INKEY$ : LOOP UNTIL K$ <> ""`.

NOTES

A loop that only polls `INKEY$` takes all the processor time it can get. Put a `SLEEP` of a short time in it if other tasks are running.

To wait for a set number of keys, use `INPUT$`.

SEE ALSO

`INPUT$`, `INPUT`, `LINE INPUT`, `SLEEP`.

INPUT

A statement: reads a line and assigns its fields to variables. The line comes from the console, or from a file.

SYNTAX

```
INPUT [;] ["prompt" {; | ,}] v [, v] ...
INPUT #f, v [, v] ...
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. It must be a literal: a string variable there is a syntax error. After it, `;` adds a question mark and a space, and `,` adds nothing. With no prompt at all, `?` is written.

v

A numeric or string variable, or an array's element. Each takes one field of the line.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

From the console. The prompt is written and one line is read, with the console's line editing (section 13.2). Only its first 255 bytes are used. The line is split into fields at its commas, and each variable takes a field, in order:

- A numeric variable's field is read as a number, in any form a program's own numbers may take: `12`, `-7/2`, `2.5`, `1e3`, `&HFF`. An empty field is 0.
- A string variable's field is the text up to the next comma, with the spaces before and after it dropped. A field that starts with a double quote runs to the closing quote instead, and can hold commas and spaces.

If the line has fewer fields than there are variables, or more, or a numeric field that isn't a number, HyBASIC writes `?Redo from start` and the statement starts again from its prompt. It is never an error.

From a file. One line of file *f* is read, and split and assigned in the same way, with three differences. There is no prompt and nothing is asked again: a numeric field that isn't a number is 0. If the line runs out of fields before the variables do, the next line is read and its fields carry on. Fields left on the line after the last variable is filled are dropped: the next `INPUT #` starts at the next line.

The semicolon after `INPUT` matters only when input doesn't come from the console (`basic prog <answers`). The console echoes the Return that ends a line; a file doesn't, so HyBASIC writes a new line itself after reading one. `INPUT ;` leaves it out, and what is printed next follows the prompt on the same line.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`
2	`syntax error`	The prompt isn't a string literal

CTRL-C at the prompt stops the program, as it does anywhere else (`Break`).

NOTES

To read a whole line, commas and all, into one string, use `LINE INPUT`. To read keys without waiting for Return, use `INKEY$` or `INPUT$`.

SEE ALSO

`LINE INPUT`, `INPUT$`, `INKEY$`, `OPEN`, `EOF`, `READ`.

EXAMPLE

```
> 10 INPUT "Two numbers"; A, B
> 20 PRINT A + B
> 30 INPUT "Name, age: ", N$, G
> 40 PRINT "["; N$; "]"; G
> 50 INPUT X, Y$
> 60 PRINT X; "["; Y$; "]"
> RUN
Two numbers? 3
?Redo from start
Two numbers? 1/2, 2.5
 3
Name, age: Sam Smith , 12
[Sam Smith] 12
? , "x, y"
 0 [x, y]
```

Fields that carry on from one line of a file to the next:

```
> 10 OPEN "/ram/n.txt" FOR OUTPUT AS #1
> 20 PRINT #1, "1" : PRINT #1, "2, 3" : PRINT #1, "4"
> 30 CLOSE #1 : OPEN "/ram/n.txt" FOR INPUT AS #1
> 40 INPUT #1, A, B : PRINT A; B
> 50 INPUT #1, C : PRINT C
> 60 INPUT #1, D
> RUN
 1  2
 4
line 60: input past end of file
```

INPUT$

A string function: a set number of keys from the keyboard, or of bytes from a file.

SYNTAX

```
INPUT$(n)
INPUT$(n, [#]f)
```

ARGUMENTS

n

A numeric expression: how many keys or bytes.

f

A numeric expression: the number of a file opened `FOR INPUT` or `FOR BINARY`.

RESULT

From the keyboard, `INPUT$(n)` waits until *n* keys have been typed and returns them as a string of *n* bytes. Nothing is echoed and Return is a key like any other, so this is the way to read a key press, a password, or a menu choice without the user pressing Return. When standard input is not a console, it

reads *n* bytes from it instead.

From a file, `INPUT$(n, #f)` returns the next *n* bytes of file *f*, whatever they are, line ends included, and moves the file's position past them.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	The file, or standard input, ends before *n* bytes
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`

NOTES

To read a whole file into a string, ask for its length: `INPUT$(LOF(1), #1)`.

SEE ALSO

`INKEY$`, `INPUT`, `LINE INPUT`, `GET`, `LOF`.

EXAMPLE

```
> OPEN "/ram/t.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : CLOSE
> OPEN "/ram/t.txt" FOR INPUT AS #1 : PRINT INPUT$(3, #1); "|"; INPUT$(2, 1); "|";
LOC(1)
abc|de| 5
> CLOSE
```

INSTR

A numeric function: where one string is found inside another.

SYNTAX

```
INSTR([start,] s$, t$)
```

ARGUMENTS

start

A numeric expression, an integer from 1 up: the position in *s$* to start looking from. 1 if it is left out.

s$

A string expression: the string to look in.

t$

A string expression: the string to look for.

RESULT

The position in *s$*, counting its first byte as 1, of the first place at or after *start* where *t$* appears whole. 0 if it appears nowhere from there on, which includes the cases where *start* is beyond the end of *s$* and where *t$* is longer than what is left.

If *t$* is empty, the result is *start*, as long as *start* is within *s$*.

The comparison is of bytes: capitals and small letters are different.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*start* is less than 1, or is not an integer
13	`type mismatch`	*s$* or *t$* is a number

NOTES

To find something without regard to case, compare capitals: `INSTR(UCASE$(s$), UCASE$(t$))`.

To find every place, call it again with *start* one past the last result.

SEE ALSO

`MID$`, `LEFT$`, `RIGHT$`, `LEN`.

EXAMPLE

```
> PRINT INSTR("hello", "l"); INSTR(4, "hello", "l"); INSTR("hello", "z"); INSTR("Hello",
"h")
 3  4  0  0
> PRINT INSTR("hello", ""); INSTR(9, "hello", "l")
 1  0
```

INT

A numeric function: a real number rounded down.

SYNTAX

```
INT(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The largest integer that is not more than *x*. For a negative number that isn't whole this is the next integer *below* it: `INT(-2.5)` is –3. The result is an exact integer of whatever length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

The three ways to make an integer differ only for numbers that aren't whole: `INT` rounds down, `FIX` rounds toward zero, and `CINT` rounds to the nearest.

SEE ALSO

`FIX`, `CINT`, `MOD`, `FIXED`.

EXAMPLE

```
> PRINT INT(2.5); INT(-2.5); INT(7/2); INT(-7/2); INT(9)
 2 -3  3 -4  9
```

IS

Part of `SELECT CASE`: `CASE IS < 10` tests the selected value with a comparison. See `SELECT`.

KILL

A statement: removes a file.

SYNTAX

```
KILL path$
```

ARGUMENTS

path$

A string expression: the file's name. It is one name, not a pattern.

DESCRIPTION

The file is removed from its directory, and the space it took is free. There is no way to get it back.

A directory can be removed with `KILL` too, if it is empty, though `RMDIR` says more clearly what is meant.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such file
263	`busy`	The file is open, in this program or any other
70	`permission denied`	The file is read-only, or its disk is
13	`type mismatch`	*path$* is a number

SEE ALSO

`NAME`, `RMDIR`, `FILES`, `OPEN`.

EXAMPLE

```
> OPEN "/ram/u.txt" FOR OUTPUT AS #1 : KILL "/ram/u.txt"
busy
> CLOSE : KILL "/ram/u.txt" : KILL "/ram/u.txt"
file not found
```

LBOUND, UBOUND

Numeric functions: the lowest and the highest index of an array.

SYNTAX

```
LBOUND(array [, dimension])
UBOUND(array [, dimension])
```

ARGUMENTS

array

An array's name, with no parentheses.

dimension

A numeric expression: which dimension, counting the first as 1. 1 if it is left out.

RESULT

The lowest, or the highest, index the array allows in that dimension, as `DIM` set them.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*dimension* is less than 1 or more than the array has

NOTES

If the name has not been used as an array, asking for its bounds makes it one, with the usual indexes 0 to 10: `UBOUND` of a misspelled name is 10, not an error.

A loop over a whole array is best written `FOR I = LBOUND(A) TO UBOUND(A)`, which goes on being right if the `DIM` is changed.

SEE ALSO

`DIM`, `REDIM`, `OPTION`.

EXAMPLE

```
> DIM A(2 TO 5, 7) : PRINT LBOUND(A); UBOUND(A); LBOUND(A, 2); UBOUND(A, 2)
 2  5  0  7
```

LCASE$, UCASE$

String functions: a string with its letters made small, or made capital.

SYNTAX

```
LCASE$(s$)
UCASE$(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

A string of the same length. `LCASE$` turns each of the capitals A to Z into its small letter; `UCASE$` turns each of a to z into its capital. Every other byte is copied as it is, including the accented letters above 127 in the console's character sets.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

SEE ALSO

`INSTR`, `LTRIM$`.

EXAMPLE

```
> PRINT LCASE$("Hello World 123"); " "; UCASE$("Hello World 123")
hello world 123 HELLO WORLD 123
```

LEFT$, RIGHT$

String functions: the first, or the last, bytes of a string.

SYNTAX

```
LEFT$(s$, n)
RIGHT$(s$, n)
```

ARGUMENTS

s$

A string expression.

n

A numeric expression, 0 or more: how many bytes. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

`LEFT$` is the first *n* bytes of *s$*, and `RIGHT$` the last *n*. If *s$* has fewer than *n* bytes, the result is all of it. If *n* is 0, the result is the empty string.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative
13	`type mismatch`	*s$* is a number, or *n* a string

SEE ALSO

`MID$`, `LEN`, `INSTR`.

EXAMPLE

```
> PRINT "["; LEFT$("hello", 2); "]["; RIGHT$("hello", 2); "]["; LEFT$("hello", 9); "][";
RIGHT$("hi", 0); "]"
[he][lo][hello][]
```

LEN

A numeric function: the length of a string.

SYNTAX

```
LEN(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

The number of bytes in *s$*: 0 for the empty string. Every byte counts, zero bytes and line ends included.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number. `LEN` does not give the size of a numeric variable

NOTES

`LEN = n` at the end of an `OPEN` statement is a different thing: see `OPEN`.

SEE ALSO

`LEFT$`, `MID$`, `INSTR`, `STR$`.

EXAMPLE

```
> PRINT LEN("hello"); LEN(""); LEN(CHR$(0) + CHR$(0)); LEN(STRING$(1000, "x"))
 5  0  2  1000
```

LET

A statement: gives a variable a value.

SYNTAX

```
[LET] variable = expression
```

ARGUMENTS

variable

A variable, an array's element, a record or one of its fields.

expression

An expression of the variable's type.

DESCRIPTION

The expression is evaluated, and its value becomes the variable's. The word `LET` is optional and changes nothing.

A numeric variable takes any number as it is: an integer of any length, a rational, a decimal or a complex number. Nothing is rounded or converted on the way in. A string variable takes a string of any length up to 64K.

Assigning to a function's own name, inside the function, sets the value it will return.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	A string for a numeric variable, or a number for a string variable
10	`duplicate definition`	The name is a constant's
9	`subscript out of range`	An array's element that isn't there

NOTES

= is also the comparison "is equal to". Which one is meant depends on where it stands: the first = of an assignment assigns, and any after it compare, so `A = B = C` gives `A` the truth of `B = C`.

A line typed at the prompt that starts with a number is taken as a line of the program, whatever follows the number.

SEE ALSO

`CONST`, `DIM`, `SWAP`, `READ`.

EXAMPLE

```
> B = 2 : C = 2 : A = B = C : PRINT A : A = B = 3 : PRINT A
-1
 0
> LET N = 2 ^ 100 / 3 : PRINT N
 12676506002282294014967032053 76/3
```

LINE

A statement: draws a line, a box, or a filled box. (`LINE INPUT`, which reads a line of text, is the next entry.)

SYNTAX

```
LINE [[STEP] (x1, y1)] - [STEP] (x2, y2) [, [c] [, {B | BF}]]
```

ARGUMENTS

x1, y1
: Numeric expressions: where the line starts. If the first point is left out, the line starts at the last point referenced: `LINE -(x, y)` carries on from where the last statement stopped.

x2, y2
: Numeric expressions: where it ends.

STEP
: Before either point: its coordinates are offsets from the last point referenced. For the second point, that is the first point.

c
: A numeric expression, 0 to 255: the colour. The pen's colour if it is left out.

B, BF
: `B` draws the box that has the two points as opposite corners, in outline. `BF` draws it filled.

DESCRIPTION

Needs a graphics mode. Both end points are drawn. The second point becomes the last point referenced, so a chain of `LINE -(x, y)` statements draws a connected path.

A line that goes off the bitmap is drawn as far as the edge.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`PSET`, `CIRCLE`, `DRAW`, `PAINT`, `COLOR`, `SCREEN`.

EXAMPLE

```
> SCREEN 13 : LINE (0, 0)-(10, 0), 3 : LINE -(10, 10), 4 : PRINT POINT(5, 0); POINT(10,
5)
 3  4
> LINE (30, 30)-(40, 40), 6, B : PRINT POINT(30, 35); POINT(35, 35)
 6  0
> LINE (50, 50)-(60, 60), 7, BF : PRINT POINT(55, 55)
 7
> SCREEN 0
```

LINE INPUT

A statement: reads a whole line into a string variable.

SYNTAX

```
LINE INPUT [;] ["prompt";] v$
LINE INPUT #f, v$
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. Nothing is added to it: no question mark.

v$

A string variable, or an array's element.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

One line is read and assigned to *v$* as it stands: commas, quotes and spaces at either end are all kept, where `INPUT` would split the line at its commas and trim it. The line's end is not part of the string. A line longer than 255 bytes is cut to 255.

From the console the line is typed with the console's line editing (section 13.2). From a file, the next line of the file is read, and the file's position moves past its line end.

The semicolon after `INPUT` has the meaning it has for `INPUT`: it matters only when input is not a console.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
13	`type mismatch`	The variable is numeric
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`

NOTES

`LINE` with an opening parenthesis after it is the graphics statement that draws a line.

SEE ALSO

`INPUT`, `INPUT$`, `EOF`, `OPEN`.

EXAMPLE

```
> Q$ = CHR$(34) : OPEN "/ram/t.txt" FOR OUTPUT AS #1 : PRINT #1, "  two, "; Q$; "words";
Q$; "  " : CLOSE
> OPEN "/ram/t.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT "["; L$; "]"; EOF(1)
[  two, "words"  ]-1
> LINE INPUT #1, L$
input past end of file
```

LIST

A command of the prompt: shows the program's lines.

SYNTAX

```
LIST [from][-[to]]
```

ARGUMENTS

from, to

Line numbers or labels.

DESCRIPTION

TYPED	LISTS
`LIST`	The whole program
`LIST 100`	Line 100 alone
`LIST 100-`	From line 100 to the end
`LIST -100`	From the start to line 100
`LIST 100-200`	Lines 100 to 200

A number need not be a line that exists: the lines in the range are listed, and if there are none nothing is. The lines are shown as they were typed or loaded, with their spacing and their case.

CTRL-C stops a long listing.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`LIST` in a program's line

SEE ALSO

`EDIT`, `DELETE`, `SAVE`, `LOAD`.

EXAMPLE

```
> 10 PRINT "one"
> 20 PRINT "two"
> 30 PRINT "three"
> LIST 20-
20 PRINT "two"
30 PRINT "three"
> LIST 15-25
20 PRINT "two"
```

LOAD

A command of the prompt: reads a program from a file, in place of the one in memory.

SYNTAX

```
LOAD "file"
```

ARGUMENTS

file

The file's name, in quotes. Nothing is added to it: a program saved as `game.bas` is loaded as `"game.bas"`.

DESCRIPTION

The program in memory is dropped first, with its variables and any files it had open, as `NEW` does; then the file's lines become the program. The file is plain text, a line of the program to each line, with or without line numbers. A carriage return before a line feed is dropped, so a file written on a PC loads as it is.

The program is not run. `RUN` runs it; `basic file`, typed at the shell, loads and runs a file in one step.

ERRORS

ERR	MESSAGE	CAUSE
288	`not found`	There is no such file. The message is preceded by the file's name
89	`only at the prompt`	`LOAD` in a program's line

NOTES

The old program is dropped before the file is opened. If the `LOAD` then fails, there is no program in memory at all: `SAVE` first.

SEE ALSO

`SAVE`, `NEW`, `RUN`, `INCLUDE`, `EDIT`.

EXAMPLE

```
> 10 PRINT "saved"
> SAVE "/ram/s.bas"
> NEW
> LOAD "/ram/s.bas"
> LIST
10 PRINT "saved"
> LOAD "/ram/nothere.bas"
/ram/nothere.bas: not found
> LIST
```

LOC, LOF

Numeric functions: an open file's position, and its length.

SYNTAX

```
LOC(f)
LOF(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file, in any mode.

RESULT

`LOC(f)` is the number of bytes before the file's position: how many have been read or written so far, in a file used from start to end. It is 0 when a file has just been opened, except `FOR APPEND`, where it is the file's length.

`LOF(f)` is the file's length in bytes, as it is now, including what this program has written to it.

`SEEK(f)` is `LOC(f) + 1`: the position counted from 1, which is the form the `SEEK` and `GET` statements take.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open

SEE ALSO

`SEEK`, `EOF`, `OPEN`, `GET`, `PUT`.

EXAMPLE

```
> OPEN "/ram/l.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : PRINT LOC(1); LOF(1);
SEEK(1) : CLOSE
 7  7  8
```

LOCATE

A statement: moves the cursor, and shows or hides it.

SYNTAX

```
LOCATE [row] [, [column] [, cursor]]
```

ARGUMENTS

row, column

Numeric expressions, from 1: the top line of the screen is row 1 and its left edge column 1. Each is rounded to the nearest integer. One that is left out keeps its present value.

cursor

A numeric expression: 0 hides the cursor and 1 shows it.

DESCRIPTION

The cursor is moved by writing an escape sequence to standard output (section 13.7), and the next thing printed starts there. `POS(0)` follows the move.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*row* or *column* is less than 1, or more than 255
13	`type mismatch`	An argument is a string

NOTES

The screen's real size isn't checked. A position beyond the edge of the terminal is the terminal's to deal with; most stop at the edge.

SEE ALSO

`CSRLIN`, `POS`, `CLS`, `PRINT`, `TAB`.

EXAMPLE

The transcript shows what was printed, not where on the screen it went. `POS(0)` is read after `CSRLIN` has been printed, so it is 17, not 14:

```
> LOCATE 5, 10 : PRINT "here"; : PRINT CSRLIN; POS(0)
here 5  17
```

LOG

A numeric function: the natural logarithm.

SYNTAX

```
LOG(x)
```

ARGUMENTS

x

A numeric expression whose value is real and not 0.

RESULT

The logarithm of *x* to the base e. `LOG(1)` is the integer 0. Any other result is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

For a negative *x* the result is complex: the logarithm of the size of *x*, plus π i. `LOG(-1)` is π i.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is 0, or is complex
13	`type mismatch`	*x* is a string

NOTES

A logarithm to another base is a quotient: `LOG(x) / LOG(10)` is the logarithm to base 10.

SEE ALSO

`EXP`, `SQR`, `DIGITS`.

EXAMPLE

```
> PRINT LOG(1); LOG(10); LOG(1/2); LOG(2 ^ 100); LOG(-1)
 0  2.30258509299 -0.69314718056  69.314718056  3.14159265359i
> PRINT LOG(0)
illegal function call
```

LOOP

The line that ends a `DO` loop, with or without a condition. See `DO`.

LTRIM$, RTRIM$

String functions: a string without the spaces at its start, or at its end.

SYNTAX

```
LTRIM$(s$)
RTRIM$(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

`LTRIM$` is *s$* from its first byte that is not a space; `RTRIM$` is *s$* up to its last byte that is not a space. Only the space, byte 32, is removed: tabs and line ends are kept. A string that is all spaces gives the empty string.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

NOTES

`STR$` puts a space in front of a number that isn't negative. `LTRIM$(STR$(n))` is the number's digits alone.

SEE ALSO

`STR$`, `LEFT$`, `RIGHT$`, `SPACE$`.

EXAMPLE

```
> PRINT "["; LTRIM$("  a b  "); "]["; RTRIM$("  a b  "); "]["; LTRIM$("   "); "]";
LTRIM$(STR$(42)); "|"
[a b  ][  a b][]42|
```

MID$

A string function, and a statement: a part of a string, taken out or replaced.

SYNTAX

```
MID$(s$, start [, length])
MID$(v$, start [, length]) = new$
```

ARGUMENTS

s$

A string expression.

v$

A string variable, or an array's element.

start

A numeric expression, 1 or more: the position of the first byte, counting the string's first byte as 1.

length

A numeric expression, 0 or more: how many bytes. If it is left out, all the rest of the string.

new$

A string expression.

Numbers that aren't integers are rounded to the nearest, a half going to the even one.

RESULT

The function is the *length* bytes of *s$* that start at *start*, or as many as there are if the string ends sooner. It is the empty string if *start* is past the end, or *length* is 0.

DESCRIPTION

The statement overwrites part of the variable's string with the bytes of *new$*, starting at *start*. It never changes the string's length. The number of bytes replaced is the smallest of three: *length* (if it is given), the length of *new$*, and what is left of *v$* from *start* to its end. Bytes of *new$* that don't fit are not used.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*start* is less than 1, or *length* is negative; in the statement, *start* is past the end of *v$*
13	`type mismatch`	A number where a string is wanted, or a string where a number is

SEE ALSO

`LEFT$`, `RIGHT$`, `INSTR`, `LEN`.

EXAMPLE

```
> PRINT "["; MID$("hello", 2, 3); "]["; MID$("hello", 4); "]["; MID$("hello", 4, 9); "]
["; MID$("hello", 9); "]"
[ell][lo][lo][]
> A$ = "hello" : MID$(A$, 2, 2) = "XYZ" : PRINT A$ : MID$(A$, 4) = "12345" : PRINT A$;
LEN(A$)
hXYlo
hXY12 5
```

MKDIR, RMDIR

Statements: make a directory, and remove one.

SYNTAX

```
MKDIR path$
RMDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name.

DESCRIPTION

`MKDIR` makes a new, empty directory. The directory it is to go in must exist already: one level is made at a time.

`RMDIR` removes a directory, which must be empty.

ERRORS

ERR	MESSAGE	CAUSE
58	`file already exists`	`MKDIR`: there is a file or a directory of that name
53	`file not found`	`MKDIR`: the directory it would go in isn't there. `RMDIR`: there is no such directory
292	`directory not empty`	`RMDIR`: the directory still has something in it
70	`permission denied`	The disk can't be written

SEE ALSO

`CHDIR`, `FILES`, `KILL`, `NAME`.

EXAMPLE

```
> MKDIR "/ram/d1" : MKDIR "/ram/d1/sub" : RMDIR "/ram/d1"
directory not empty
> RMDIR "/ram/d1/sub" : RMDIR "/ram/d1" : RMDIR "/ram/d1"
file not found
```

MKN$

A string function: a number's bytes in the stored format.

SYNTAX

```
MKN$(x)
```

ARGUMENTS

x

A numeric expression of any kind.

RESULT

A string holding *x* in the form numbers are stored in: the form `PUT` writes for a numeric variable, and `CVN` reads back. Its length depends on the number. A small integer takes one byte; larger integers, rationals, decimals and complex numbers take as many as they need.

DESCRIPTION

`MKN$` and `CVN` are the pair that stands in place of QuickBASIC's `MKI$`, `MKL$`, `MKS$`, `MKD$` and their `CV` functions. `CVN(MKN$(x))` is exactly *x*, for every number.

Because the length varies, a number can't be given a fixed place in a record by `MKN$` alone. Pad the string to the width wanted, or keep the number as text.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

CVN, PUT, GET, STR$.

EXAMPLE

```
> PRINT LEN(MKN$(7)); LEN(MKN$(1/3)); LEN(MKN$(2 ^ 100)); CVN(MKN$(2 ^ 100))
 1  3  14  126765060022822940149670320 5376
```

MOD

An operator: the remainder of an integer division. The entry covers `\`, integer division, too.

SYNTAX

```
a MOD b
a \ b
```

ARGUMENTS

a, b

Numeric expressions whose values are real.

RESULT

Both operands are first rounded to the nearest integer, a half going to the even one. Then `a \ b` is the quotient with its fraction dropped, which rounds toward zero, and `a MOD b` is what is left over: `a - (a \ b)

- b`. The remainder has the sign of *a*, or is 0.

The integers can be of any length.

DESCRIPTION

`\` binds more tightly than `MOD`, and both less tightly than `*` and `/` and more tightly than `+` and –. So `A + B MOD C \ D` is `A + (B MOD (C \ D))`.

ERRORS

ERR	MESSAGE	CAUSE
11	`division by zero`	*b* is 0, or rounds to 0
13	`type mismatch`	An operand is a string

NOTES

Because the operands are rounded first, `7.5 MOD 2` is 0: 7.5 rounds to 8. For the remainder of a division of numbers that aren't whole, work it out: `x - INT(x / y) * y`.

`/` is exact division, and gives a rational when the integers don't divide: `7 / 2` is 7/2.

SEE ALSO

INT, FIX, GCD.

EXAMPLE

```
> PRINT 7 MOD 3; -7 MOD 3; 7 MOD -3; 7 \ 2; -7 \ 2; 7.5 MOD 2; 7 / 2
 1 -1  1  3 -3  0  7/2
> PRINT 2 ^ 70 MOD 1000; 2 + 3 * 4 ^ 2 MOD 5 \ 2
 424  2
```

NAME

A statement: renames a file.

SYNTAX

```
NAME old$ AS new$
```

ARGUMENTS

old$

A string expression: the file's present name.

new$

A string expression: its new name.

DESCRIPTION

The file, or directory, *old$* is given the name *new$* in the directory it is already in. Only the last part of *new$* is used, the name after its last `/`: a rename can't move a file to another directory, and a directory written in *new$* is ignored without an error. To move a file, copy it and remove the original.

Renaming a file to the name it has does nothing.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no *old$*
58	`file already exists`	There is a *new$* in that directory already
70	`permission denied`	The disk can't be written

SEE ALSO

`KILL`, `FILES`, `MKDIR`, `SHELL`.

EXAMPLE

```
> OPEN "/ram/n1" FOR OUTPUT AS #1 : CLOSE : MKDIR "/ram/dd"
> NAME "/ram/n1" AS "/ram/dd/moved" : FILES "/ram" : FILES "/ram/dd"
bin/            lib/            moved           dd/
```

NEW

A command of the prompt: drops the program.

SYNTAX

```
NEW
```

DESCRIPTION

The program's lines are removed, with everything that belonged to it: its variables and arrays, its procedures and labels, and any files it had open, which are closed. A stopped program is gone, so there is nothing to `CONT`. The settings that are HyBASIC's and not the program's stay as they are: `BASE`, `DIGITS`, the current directory.

There is no question first, and no way back. `SAVE` what is wanted.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`NEW` in a program's line

SEE ALSO

`LOAD`, `SAVE`, `CLEAR`, `DELETE`.

NEXT

The line that ends a `FOR` loop. See `FOR`.

NOT

An operator: the bitwise complement of an integer, which is also the logical NOT of a truth value.

SYNTAX

```
NOT a
```

RESULT

An integer: a, rounded to the nearest integer if it isn't one, with every bit turned over. In two's complement that is $-a-1$: `NOT 0` is −1, `NOT -1` is 0, and `NOT 5` is −6.

For the truth values −1 and 0, `NOT` gives the other one. For any other number it does not: 5 counts as true in an `IF`, and so does `NOT 5`.

DESCRIPTION

`NOT` binds less tightly than the comparisons and more tightly than `AND`: `NOT A = B` is `NOT (A = B)`, and `NOT A AND B` is `(NOT A) AND B`.

The operand is rounded and treated as `AND`'s are: see `AND`.

EXAMPLE

```
> PRINT NOT 0; NOT -1; NOT 5; NOT 1 = 2
-1  0 -6 -1
```

NUMERATOR

A numeric function: the numerator of a real number written as a fraction in lowest terms.

SYNTAX

```
NUMERATOR(x)
```

RESULT

An integer with the sign of *x*. A rational gives its numerator, an integer gives itself, and a decimal is turned into the fraction it exactly equals first: −0.75 is −3/4, and gives −3. See `DENOMINATOR`, which has the errors and an example.

OCT$

A string function: a number written in octal.

SYNTAX

```
OCT$(x)
```

RESULT

The text of *x* in base 8, with no space in front, as `STR$(x, "o")` would give it. As with `HEX$`, it is the number itself in another base: a negative number has a minus sign, and a number that isn't an integer is written as what it is. See `HEX$`.

EXAMPLE

```
> PRINT OCT$(8); " "; OCT$(255); " "; OCT$(-8); " "; OCT$(0.5)
10 377 -10 0.4
```

ON

A statement: jumps to one of several lines by a number; or sets where errors go.

SYNTAX

```
ON n GOTO line [, line] ...
ON n GOSUB line [, line] ...
ON ERROR GOTO line
ON ERROR GOTO 0
```

ARGUMENTS

n

A numeric expression, rounded to the nearest integer.

line

A line number or a label.

DESCRIPTION

ON n GOTO goes to the first line in the list if *n* is 1, to the second if it is 2, and so on. If *n* is less than 1 or more than the number of lines listed, nothing happens: the program goes on with the next statement. **ON n GOSUB** is the same, but calls the line as a subroutine, and its RETURN comes back to the statement after the ON.

ON ERROR GOTO line names an error handler. From then on, an error that would have stopped the program makes it go to *line* instead, with ERR, ERL and ERR$ set. The handler ends with a RESUME, which says where to carry on: at the statement that failed, at the one after it, or at another line.

While the handler is running, a second error is not sent to it again. It stops the program in the ordinary way.

ON ERROR GOTO 0 turns the handler off. Used inside a handler, it gives up: the error being handled stops the program, with its message, as if there had been no handler.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	A line in the list, or the handler's, isn't in the program
13	`type mismatch`	*n* is a string

NOTES

CTRL-C is not an error, and no handler sees it.

SEE ALSO

RESUME, ERR, ERROR, GOTO, GOSUB, SELECT.

EXAMPLE

```
> 10 ON ERROR GOTO 100
> 20 X = 1 / 0 : PRINT "after the error"
> 30 OPEN "/ram/nothere" FOR INPUT AS #1
> 40 PRINT "line 40" : ON ERROR GOTO 0 : ERROR 5
> 50 PRINT "not reached"
> 100 PRINT "handler:"; ERR; ERL
> 110 IF ERR = 11 THEN RESUME NEXT
> 120 RESUME 40
> RUN
handler: 11  20
after the error
handler: 53  30
line 40
line 40: illegal function call
```

OPEN

A statement: opens a file, and gives it a number for the statements that read and write it.

SYNTAX

```
OPEN path$ FOR {INPUT | OUTPUT | APPEND | BINARY} AS [#]f [LEN = n]
```

ARGUMENTS

path$

A string expression: the file's name. A name that doesn't start with `/` is taken from the current directory (`CHDIR`). It can be any name in the program's namespace: a file on a disk, or a device such as `/dev/cons`.

f

A numeric expression from 1 to 255: the number the file is known by until it is closed. The `#` is optional.

n

Accepted, evaluated and ignored. QuickBASIC's record length has no meaning here: `GET` and `PUT` take a position in bytes.

DESCRIPTION

The mode decides how the file is opened, and which statements may then use it:

MODE	A FILE THAT EXISTS	A FILE THAT DOESN'T	STARTS AT	MAY THEN BE USED BY
`INPUT`	Opened for reading	Error 53	Its first byte	`INPUT #`, `LINE INPUT #`, `INPUT$`, `EOF`, `LOF`, `LOC`, `SEEK`
`OUTPUT`	Emptied, and opened for writing	Made	Its first byte	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`APPEND`	Opened for writing, its contents kept	Made	Its end	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`BINARY`	Opened for reading and writing	Made	Its first byte	All of the above, and `GET` and `PUT`

Up to 8 files can be open at once. A file stays open until `CLOSE` names it, `CLOSE` with no number closes them all, or a `RUN` starts (which closes every file first). Leaving HyBASIC closes them too.

`EOF(f)` is always true (–1) for a file opened `FOR OUTPUT` or `FOR APPEND`.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	*f* is less than 1 or more than 255
55	`file already open`	Number *f* is in use
67	`too many files`	8 files are open already
53	`file not found`	`FOR INPUT`: there's no such file. Any mode: a directory in the path isn't there
70	`permission denied`	The file, or its disk (`/rom`), can't be written
13	`type mismatch`	*path$* is a number, or *f* a string

The system's other refusals come through as its own errors (a full disk is 61, `disk full`).

NOTES

The limit of 8 is HyBASIC's own. Each open file also takes one of the task's 16 file descriptors, three of which are the console's.

One file can be open under two numbers at once, each with its own position: opening it `FOR INPUT` as `#2` while it is open `FOR OUTPUT` as `#1` is allowed, and reads what has been written so far.

A file written with `PRINT #` holds exactly the bytes `PRINT` would have sent to the screen, each line ended by a line feed (byte 10).

SEE ALSO

`CLOSE`, `PRINT`, `WRITE`, `INPUT`, `LINE INPUT`, `GET`, `PUT`, `SEEK`, `EOF`, `LOF`, `LOC`, `KILL`, `NAME`.

EXAMPLE

```
> OPEN "/ram/log.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT LOF(1); EOF(1) : CLOSE
#1
 4 -1
> OPEN "/ram/log.txt" FOR APPEND AS #1 : PRINT #1, "two" : PRINT LOF(1) : CLOSE #1
 8
> OPEN "/ram/log.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$; EOF(1)
one 0
> PRINT #1, "three"
bad file mode
> OPEN "/ram/other.txt" FOR OUTPUT AS #1
file already open
> CLOSE : OPEN "/ram/nothere.txt" FOR INPUT AS #1
file not found
> PRINT ERR
 53
```

OPTION

A statement: sets the lowest index of arrays.

SYNTAX

```
OPTION BASE {0 | 1}
```

DESCRIPTION

An array made with only its highest index given, as in `DIM A(10)`, has indexes from 0. After `OPTION BASE 1` they are from 1, so that `DIM A(10)` has ten elements and no `A(0)`. `OPTION BASE 0` puts it back.

The setting applies to arrays made after the statement, whether by `DIM`, by `REDIM`, or by being used without either (whose indexes then run from 1 to 10). Arrays that exist already keep the bounds they were made with. A `DIM` that states its own lowest index, `DIM A(5 TO 9)`, is not affected.

The digit must be written out as 0 or 1: it is part of the statement, not an expression.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	Anything but 0 or 1 after `BASE`

NOTES

This `BASE` has nothing to do with the `BASE` statement, which sets the base numbers are written in.

SEE ALSO

`DIM`, `REDIM`, `LBOUND`, `UBOUND`.

EXAMPLE

```
> OPTION BASE 1 : DIM A(3) : B(1) = 1 : PRINT LBOUND(A); UBOUND(A); LBOUND(B); UBOUND(B)
 1  3  1  10
> OPTION BASE 0 : DIM C(3) : PRINT LBOUND(C); LBOUND(A)
 0  1
```

OR

An operator: the bitwise OR of two integers, which is also the logical OR of two truth values.

SYNTAX

```
a OR b
```

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in *a*, in *b*, or in both.

For two truth values, −1 and 0, the result is true if either is.

Both operands are always evaluated, and they are rounded and treated as `AND`'s are: see `AND`. `OR` binds less tightly than `AND` and more tightly than `XOR`.

EXAMPLE

```
> PRINT 5 OR 3; 0 OR 0; (1 = 2) OR (2 = 2); 1.5 OR 0
 7  0 -1  2
```

OUTPUT

A mode of `OPEN`: `OPEN path$ FOR OUTPUT AS #f` makes a file, or empties one, and opens it for writing. See `OPEN`.

PAINT

A statement: fills an area of the bitmap with a colour.

SYNTAX

```
PAINT [STEP] (x, y) [, c [, border]]
```

ARGUMENTS

x*, *y
Numeric expressions: a point inside the area.

c
A numeric expression, 0 to 255: the colour to fill with. The pen's colour if it is left out.

border

A numeric expression, 0 to 255: the colour that bounds the area. If it is left out, it is *c*.

DESCRIPTION

Needs a graphics mode. Starting from the point, the fill spreads to every pixel it can reach by moving up, down, left and right without crossing a pixel of the *border* colour, or one that is already colour *c*. It stops at the edges of the bitmap. If the starting pixel is itself the border's colour or *c*, nothing is filled.

The outline must be closed. A gap of a single pixel lets the colour out, and it fills everything outside too.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or a colour is not 0 to 255
13	`type mismatch`	An argument is a string

NOTES

The fill is done by HyBASIC, a row at a time, reading the bitmap back from the screen's memory. A large or intricate area takes a noticeable time. A box is filled far faster by `LINE` with `BF`, and a disc by `CIRCLE` with `F`.

A starting point off the bitmap fills nothing, without an error.

SEE ALSO

`LINE`, `CIRCLE`, `DRAW`, `COLOR`, `SCREEN`.

EXAMPLE

A box outlined in colour 6, filled with colour 2 up to that border:

```
> SCREEN 13 : LINE (30, 30)-(40, 40), 6, B : PAINT (35, 35), 2, 6
> PRINT POINT(35, 35); POINT(31, 31); POINT(30, 30); POINT(45, 45)
 2  2  6  0
> SCREEN 0
```

PALETTE

A statement: sets what colour one of the colour numbers shows.

SYNTAX

```
PALETTE n, rgb
```

ARGUMENTS

n

A numeric expression, 0 to 255: a colour number.

rgb

A numeric expression, 0 to 4095: the colour, as three hex digits, one each for red, green and blue. `&HF00` is the brightest red, `&HFFF` white and `&H000` black.

DESCRIPTION

A pixel holds a colour number, and the palette says what colour each number is shown as. `PALETTE` changes one entry. Everything on the screen that has that number changes colour at once, without being drawn again: the text as well as the bitmap, since they share the one palette.

It does not need a graphics mode, only a Vera X card.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is not 0 to 255, or *rgb* is not 0 to 4095
13	`type mismatch`	An argument is a string

NOTES

The palette is the screen's, not HyBASIC's: a change stays after the program ends, and after HyBASIC does.

SEE ALSO

`COLOR`, `SCREEN`.

PEEK, POKE

A function and a statement: read, and write, one byte of memory.

SYNTAX

```
PEEK(address)
POKE address, value
```

ARGUMENTS

address

A numeric expression: an address in HyBASIC's task, 0 to 65,535. A negative number from −32,768 to −1 is accepted too, and means the address 65,536 more: −1 is `$FFFF`.

value

A numeric expression from 0 to 255, rounded to the nearest integer.

RESULT

`PEEK` is the byte at the address, 0 to 255.

DESCRIPTION

The addresses are those of HyBASIC's own task: its zero page and stack, the RAM that holds its state, the bank window at `$8000` to `$9FFF`, the paged ROM at `$A000` to `$DFFF` and the BIOS above (Appendix F). Another task's memory can't be reached.

For an address in the bank window, the bank is the one `BANK` has set. HyBASIC puts it there for the read or the write and takes it away again, so the window is of no use to a program between the two: see `BANK`.

`POKE` does no checking beyond the two ranges. Writing into HyBASIC's own memory, which is most of what is below `$8000`, will usually stop it working; and a write to an address that is ROM does nothing. The hardware's registers are reached through the drivers' files (Chapter 12), not through `POKE`.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	The address is outside −32,768 to 65,535; or the value is outside 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`BANK`, `SYS`, `CALL`, `ASM`.

EXAMPLE

```
> BANK 40 : POKE &H8000, 65 : POKE &H8001, 66.5 : PRINT PEEK(&H8000); PEEK(&H8001);
PEEK(-32768)
 65  66  65
> POKE &H8000, 256
illegal function call
```

PI

A numeric function: π.

SYNTAX

```
PI
```

RESULT

The ratio of a circle's circumference to its diameter, as a decimal rounded to the number of significant digits that `DIGITS` has set: 3.14159265359 at the usual 12.

It takes no argument and no parentheses.

SEE ALSO

`DIGITS`, `SIN`, `COS`, `TAN`, `ATN`.

EXAMPLE

```
> PRINT PI : DIGITS 30 : PRINT PI : DIGITS 12
 3.14159265359
 3.14159265358979323846264338328
```

PLAY

A statement: plays music, and waits until it has finished.

SYNTAX

```
PLAY [channel,] score$
PLAY file$
```

ARGUMENTS

score$

A string expression: a line of the score language, MML (section 15.6).

channel

A numeric expression: the sound channel to play it on (section 15.1). Channel 0 if it is left out.

file$

A string expression ending in `.zsm` or `.mml`: the name of a song file, or of a file of score lines.

DESCRIPTION

HyBASIC does not make the sound itself. It runs the `play` program (section 15.7) with the string, and waits for it to end: `play -m channel score` for a line of score, and `play file` for a file. Whatever `play` can play, `PLAY` can. The program stops while the music lasts.

A string is taken as a file's name if it ends in `.zsm` or `.mml`, and as a score otherwise.

ERRORS

ERR	MESSAGE	CAUSE
511	`play`'s own message	`play` could not play it: a mistake in the score, a file that isn't there. `play` prints its message itself first, with its name before it
5	`illegal function call`	A channel number that is not one
13	`type mismatch`	The score is a number

NOTES

This is not QuickBASIC's `PLAY` language, though simple tunes look alike in both: the letters `c` to `b` are notes, `o` sets the octave and `l` the length. Section 15.6 has the whole language.

For a single tone, `SOUND` is simpler.

SEE ALSO

SOUND, BEEP, SHELL.

EXAMPLE

```
> PLAY "t240 l16 c e g" : PRINT "played"
played
> PLAY "zzz!"
play: zzz!: channel 0: what is z
channel 0: what is z
> PRINT ERR
 511
```

POINT

A numeric function: the colour of a pixel.

SYNTAX

```
POINT(x, y)
```

ARGUMENTS

x, y

Numeric expressions: the pixel, in the coordinates the drawing statements use, with the WINDOW and VIEW that are in force.

RESULT

The pixel's colour number. −1 if the point is off the bitmap, or if no graphics mode is set.

ERRORS

ERR	MESSAGE	CAUSE
13	type mismatch	An argument is a string

NOTES

The result is what is in the bitmap. Sprites and the console's text, which are shown over it, are not seen.

SEE ALSO

PSET, SCREEN, WINDOW, VIEW.

POS

A numeric function: the column the next character will be printed in.

SYNTAX

```
POS(x)
POS
```

ARGUMENTS

x

Any numeric expression. It is ignored; `POS(0)` is the usual form, from QuickBASIC, and `POS` alone means the same.

RESULT

The column, counted from 1 at the left edge. HyBASIC keeps count of what it has written since the last line end, so this is the column of its own output: after `PRINT "ab";` it is 3.

NOTES

The count is HyBASIC's, not the terminal's. It follows `LOCATE`, but it knows nothing of what other programs have written, or of escape sequences a program prints for itself. The row is `CSRLIN`, which does ask the terminal.

`TAB` and the print zones of a comma work from the same count.

SEE ALSO

`CSRLIN`, `LOCATE`, `TAB`, `SPC`, `WIDTH`.

EXAMPLE

```
> PRINT POS(0); : PRINT "ab"; : PRINT POS(0)
 1 ab 6
```

PRESERVE

Part of `REDIM`: `REDIM PRESERVE` changes an array's size and keeps what is in it. See `REDIM`.

PRESET, PSET

Statements: set one pixel.

SYNTAX

```
PSET [STEP] (x, y) [, c]
PRESET [STEP] (x, y) [, c]
```

ARGUMENTS

x, y

Numeric expressions: the pixel. With `STEP`, an offset from the last point referenced.

c

A numeric expression, 0 to 255: the colour. A mode with fewer colours uses the number's low bits.

DESCRIPTION

Needs a graphics mode. The two differ only when no colour is given: `PSET` then uses the pen's colour, and `PRESET` uses colour 0, the background, which rubs the pixel out.

The point becomes the last point referenced. A point off the bitmap sets nothing, and is not an error.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`POINT`, `LINE`, `COLOR`, `SCREEN`.

EXAMPLE

```
> SCREEN 13 : PSET (10, 10), 5 : PSET (20, 20) : PRINT POINT(10, 10); POINT(20, 20);
POINT(11, 10)
 5  15  0
> PRESET (20, 20) : PSET STEP (5, 0), 9 : PRINT POINT(20, 20); POINT(25, 20)
 0  9
> SCREEN 0
```

PRINT

A statement: writes values as text, to the screen or to a file.

SYNTAX

```
PRINT [#f,] [item [{; | ,} item] ...] [; | ,]
PRINT [#f,] USING format$; item [{; | ,} item] ... [; | ,]
? ...
```

ARGUMENTS

item

An expression of any type, or `TAB(n)` or `SPC(n)`.

f

The number of a file opened `FOR OUTPUT`, `FOR APPEND` or `FOR BINARY`. The output goes to the file in place of the screen, byte for byte the same.

format$

A string expression: a picture of how the items are to be written (below).

`?` is another spelling of `PRINT`.

DESCRIPTION

Values. A string is written as its bytes. A number is written as its text in the base that `BASE` has set, with a space before it if it isn't negative (a negative number has its minus sign there instead) and a space after it. The text is the number's exact value: an integer has all its digits however many there are, a rational is written with a `/`, and a decimal with its point and every digit it holds. There is no exponent form: a very large or very small number is written out in full.

Between items. A semicolon writes nothing: the next item follows at once. A comma moves to the next print zone: the zones are 14 columns wide, starting at columns 1, 15, 29 and so on, and if the next zone would start past the width of the line the item starts a new line instead. Two commas skip a zone.

At the end. `PRINT` ends the line, by writing a line feed, unless the statement ends with a semicolon or a comma: then the next `PRINT` carries on from there. `PRINT` with no items writes an empty line.

`TAB(n)` moves to column *n*, counting from 1, by writing spaces. If the output is already past column *n*, it starts a new line and moves to column *n* there. **`SPC(n)`** writes *n* spaces. Both are used only inside `PRINT`.

The line's width is 80 unless `WIDTH` has set another. It decides where the zones run out. Text longer than the width is not broken by HyBASIC; the terminal wraps it.

PRINT USING

The format is text with **fields** in it. Each item is written by the next field; the text between fields is written as it stands. If there are more items than fields, the format is used again from its start.

A number's field is a picture of the number:

IN THE FIELD	MEANS
`#`	A digit's place. The number is put at the right of its places, with spaces before it
`.`	The decimal point. The `#`s after it are decimal places, always filled, with zeros if need be
`,`	Before the point: put commas between each three digits
`+`	At the start: write the sign, `+` or `-`, always
`$$`	At the start: write a `$` just before the number
`**`	At the start: fill the spaces before the number with `*`

The number is **rounded** to the field's decimal places, a half going away from zero: 2.5 in `###` is 3, and −3.145 in `###.##` is −3.15. (This is not `CINT`'s rule, which sends a half to the even integer.) A minus sign takes a place. If the number needs more places than the field has, it is written in full with a `%` before it to show that the column is broken.

A string's field is one of three:

IN THE FIELD	MEANS
`!`	The first character of the string
`&`	The whole string
`\`, spaces, `\`	As many characters as the field is wide, counting both backslashes; a shorter string is padded with spaces

Two more things can be in a format:

IN THE FORMAT	MEANS
`{}`	The item as `PRINT` would write it, a number or a string, with no spaces added
`{`*base*`}`	A number in that base: `{x}` in hex, `{b}` in binary (a base string, section 18.4)
`_`	The next character is text, not part of a field: `_#` writes a `#`

A semicolon or a comma between the items of `PRINT USING` makes no difference: the format decides the spacing. At the end of the statement they hold the line open, as in `PRINT`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	`USING`: a string for a number's field, or a number for a string's
5	`illegal function call`	`USING`: the format has no field in it
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR INPUT`

NOTES

`PRINT #` writes to a file exactly what `PRINT` would have put on the screen, zones and spaces included. For data that a program will read back with `INPUT #`, `WRITE #` is the better choice: it puts commas between the items and quotes round the strings.

To print a number without its spaces, use `PRINT USING "{}"`, or `LTRIM$(STR$(n))`.

SEE ALSO

`WRITE`, `WIDTH`, `LOCATE`, `POS`, `STR$`, `BASE`, `FIXED`, `OPEN`.

EXAMPLE

```
> PRINT 1; -2; 3 : PRINT "a"; "b"; 5; "c" : PRINT 1, 2, "x", -4
 1 -2  3
ab 5 c
 1             2             x             -4
> PRINT 1/3; 2.50; 2 ^ 64; 1+2i : PRINT "one"; : PRINT " line"
 1/3  2.5  18446744073709551616  1+2i
one line
> PRINT "A"; TAB(10); "B"; TAB(5); "C"; SPC(3); "D"
A         B
    C   D
> PRINT USING "###.##"; 3.14159 : PRINT USING "###.##|"; -3.145; 1234.5
  3.14
 -3.15|%1234.50|
> PRINT USING "#,###.# +##.# $$###.## **###.##"; 1234.56; 5; 12.5; 12.5
1,234.6  +5.0   $12.50 ***12.50
> PRINT USING "[!][&][\  \] {} {x} _#"; "hello"; "world"; "abcdefg"; 255; 255
[h][world][abcd] 255 FF #
```

PUT

A statement: writes a value to a `BINARY` file, at a given position.

SYNTAX

```
PUT [#]f, [position], value
```

ARGUMENTS

f

A numeric expression: the number of a file opened `FOR BINARY`.

position

A numeric expression: where to write, counted in bytes from 1 for the file's first byte. If it is left out (the two commas are still written) or is less than 1, the write starts where the last `GET` or `PUT` on the file ended.

value

An expression of either type. A variable is the usual thing, but any expression will do.

DESCRIPTION

A string is written as its bytes, as many as it has, with nothing before or after them. A number is written in the stored format, the form `MKN$` makes, which is as long as the number needs.

The bytes replace whatever was at that position. Writing past the end makes the file longer; writing at a position beyond the end leaves the gap between filled with zero bytes.

After the write, the file's position is just past what was written.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* was not opened `FOR BINARY`
61	`disk full`	No room left on the disk

NOTES

To write text to a file opened `FOR BINARY`, `PRINT #` works too, and adds its line end.

Since a number's stored form varies in length, a file of numbers written with `PUT` is read back in the same order with `GET`; a number can't be changed in place for a longer one without overwriting what follows it.

SEE ALSO

`GET`, `OPEN`, `SEEK`, `MKN$`, `CVN`, `PRINT`.

EXAMPLE

```
> OPEN "/ram/p.dat" FOR BINARY AS #1 : PUT #1, 1, "HELLO" : PUT #1, 3, "xy" : PUT #1, 9,
"!" : PRINT LOF(1)
 9
> A$ = SPACE$(9) : GET #1, 1, A$ : PRINT A$; ASC(MID$(A$, 6)) : CLOSE
HExyO! 0
```

RANDOMIZE

A statement: seeds the random number generator.

SYNTAX

```
RANDOMIZE [n]
```

ARGUMENTS

n

A numeric expression: the seed. Only its low 16 bits are used, so there are 65,536 different sequences.

DESCRIPTION

`RND` produces a fixed sequence of numbers from wherever the generator stands. `RANDOMIZE n` puts the generator at the start of sequence *n*: the same *n* always gives the same numbers afterwards, which is what a program being tested wants.

`RANDOMIZE` with no number, or with 0, takes the seed from the system's clock, so that each run is different. That is what a game wants, once, at its start.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*n* is a string

NOTES

QuickBASIC asks `Random-number seed?` when `RANDOMIZE` has no number. HyBASIC asks nothing.

SEE ALSO

`RND`, `TIMER`.

EXAMPLE

```
> RANDOMIZE 1 : A = RND : B = RND : RANDOMIZE 1 : PRINT A = RND; B = RND
-1 -1
```

RATIONAL

A numeric function: a real number as an exact fraction.

SYNTAX

```
RATIONAL(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The rational number equal to *x*, in lowest terms. A decimal becomes the fraction it exactly is: 0.75 becomes 3/4, and 0.1 becomes 1/10, with no error at all, since a decimal here is held in decimal. An integer or a rational is returned unchanged.

DESCRIPTION

Adding, subtracting and multiplying decimals gives a decimal. Division is always exact, and gives a fraction when the quotient isn't whole, a decimal's too: `0.75 / 3` is 1/4. `RATIONAL` turns a decimal into a fraction without dividing it by anything; `FIXED` goes the other way.

The result is exact for the decimal it is given. If that decimal was itself rounded, as the result of `SQR` or `SIN` is, the fraction is the rounded value's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

FIXED, NUMERATOR, DENOMINATOR.

EXAMPLE

```
> PRINT RATIONAL(0.75); RATIONAL(2.5); RATIONAL(0.1); RATIONAL(7); RATIONAL(SQR(2))
 3/4  5/2  1/10  7  141421356237/100000000000
```

READ

A statement: takes the next items of the program's DATA into variables.

SYNTAX

```
READ variable [, variable] ...
```

ARGUMENTS

variable

A numeric or string variable, an array's element, or a record's field.

DESCRIPTION

Each variable is given the next unread item of the program's DATA, in the order the DATA statements stand in the program. The place carries on from one READ to the next, until RESTORE moves it.

An item read into a string variable is its text. An item read into a numeric variable is read as a number, in any form a program's numbers may take; an item that isn't a number, and an empty item, give 0 with no error. DATA describes the items.

ERRORS

ERR	MESSAGE	CAUSE
4	`out of DATA`	No item is left

SEE ALSO

DATA, RESTORE, INPUT.

EXAMPLE

```
> 10 DATA 1, 2
> 20 DATA 3, text
> 30 READ A, B, C, D$ : PRINT A; B; C; D$
> 40 RESTORE 20 : READ E : PRINT E
> RUN
 1  2  3 text
 3
```

REAL

A numeric function: the real part of a number.

SYNTAX

```
REAL(z)
```

RESULT

The real part of *z*, of the kind it has in *z*. A real number is returned unchanged. See `IMAG`, which has the errors and an example.

REDIM

A statement: makes an array again, with new bounds.

SYNTAX

```
REDIM [PRESERVE] name(bounds [, bounds] ...) [AS type] [, name ...] ...
```

ARGUMENTS

As `DIM`'s.

DESCRIPTION

Where `DIM` refuses to make an array that exists, `REDIM` replaces it. An array that doesn't exist yet is simply made, so `REDIM` can be used for the first time as well as the next.

Without `PRESERVE`, the new array is empty: every element 0 or the empty string.

With `PRESERVE`, the old array's elements are copied into the new one **in order of position**, not by index: the old array's first element goes to the new array's first element, the second to the second, and so on, for as many as both arrays have. If the two have the same lowest index, as they usually do, an element keeps its index; if the lowest index changes, every element moves. The number of dimensions may change too, the elements still being taken in their stored order.

ERRORS

ERR	MESSAGE	CAUSE
9	`subscript out of range`	Bounds that `DIM` would refuse
7	`out of memory`	No room for the new array

SEE ALSO

`DIM`, `ERASE`, `LBOUND`, `UBOUND`.

EXAMPLE

In the second line the lowest index moves from 0 to 1, so the old `A(0)` and `A(1)` become the new `A(1)` and `A(2)`:

```
> DIM A(3) : A(0) = 5 : A(1) = 6 : REDIM PRESERVE A(10) : PRINT A(0); A(1); UBOUND(A)
 5  6  10
> REDIM PRESERVE A(1 TO 2) : PRINT A(1); A(2) : REDIM A(4) : PRINT A(1)
 5  6
 0
```

REM

A statement: a comment.

SYNTAX

```
REM text
' text
```

DESCRIPTION

Everything from `REM` to the end of the line is ignored, colons included: a statement can't follow a comment on the same line. An apostrophe does the same, and can come after a statement without a colon before it.

NOTES

One comment is not ignored: `REM $INCLUDE: 'file'`, and the same with an apostrophe, is QuickBASIC's way of writing `INCLUDE`. See `INCLUDE`.

EXAMPLE

```
> REM this is ignored : PRINT "not printed"
> PRINT "printed" ' and this is ignored too : PRINT "nor this"
printed
```

RESTORE

A statement: moves the place `READ` takes `DATA` from.

SYNTAX

```
RESTORE [line | label]
```

DESCRIPTION

With nothing after it, the next `READ` starts again at the program's first `DATA` item. With a line number or a label, it starts at the first item of the first `DATA` statement at or after that line.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or label

SEE ALSO

READ, DATA.

RESUME

A statement: ends an error handler, and says where the program carries on.

SYNTAX

```
RESUME [0]
RESUME NEXT
RESUME {line | label}
```

DESCRIPTION

FORM	CARRIES ON
`RESUME`, `RESUME 0`	At the statement that failed, which is run again
`RESUME NEXT`	At the statement after the one that failed
`RESUME line`	At that line

The statement that failed is the one in which the error happened, not the line: with several statements on a line, `RESUME NEXT` goes on with the next statement of that line.

`RESUME` is what marks the handler's end. After it, a new error goes to the handler again; until it, a second error stops the program.

`RESUME` alone suits an error that the handler has put right, a missing file it has now made. If the cause is still there the statement fails again, and the program goes round for ever.

ERRORS

ERR	MESSAGE	CAUSE
20	`RESUME without error`	A `RESUME` that is not in a handler that an error started
8	`label not defined`	No line has that number or label

SEE ALSO

ON, ERR, ERROR.

EXAMPLE

See `ON`.

RETURN

A statement: comes back from a subroutine that `GOSUB` called.

SYNTAX

```
RETURN
```

DESCRIPTION

Control goes back to the statement after the most recent `GOSUB` that hasn't been returned from.

ERRORS

ERR	MESSAGE	CAUSE
3	`RETURN without GOSUB`	There is no `GOSUB` to go back to: usually, control has run into a subroutine's lines from above

NOTES

To leave a `SUB` or a `FUNCTION`, use `EXIT SUB` or `EXIT FUNCTION`, not `RETURN`.

SEE ALSO

`GOSUB`, `ON`, `EXIT`.

RND

A numeric function: a random number.

SYNTAX

```
RND
RND(x)
```

ARGUMENTS

x

A numeric expression. Its sign chooses what happens:

X	RESULT
Left out, or more than 0	The next number of the sequence
0	The last number again
Less than 0	The generator is seeded with *x* first, then the next number. The same *x* gives the same number every time

RESULT

A decimal from 0 up to, but never reaching, 1, with as many digits as `DIGITS` has set (12 unless it has been changed).

DESCRIPTION

The numbers are a fixed sequence worked out from a seed, and they look random. HyBASIC seeds the generator afresh each time it starts, so two sessions give different numbers; within a session the sequence simply carries on, from one `RUN` to the next. `RANDOMIZE n` gives a sequence that can be repeated.

A whole number from 1 to *n* is `INT(RND * n) + 1`.

NOTES

The generator has 16 bits of seed. It is good enough for games and simulations and not for anything that has to be unguessable.

SEE ALSO

`RANDOMIZE`, `DIGITS`, `INT`.

EXAMPLE

```
> RANDOMIZE 1 : A = RND : PRINT A; RND(0) = A; A >= 0 AND A < 1
 0.778176127105 -1 -1
> X = RND(-5) : Y = RND(-5) : PRINT X = Y : DIGITS 5 : PRINT RND : DIGITS 12
-1
 0.25799
```

RREG

A statement: copies the processor's registers, as machine code left them, into variables.

SYNTAX

```
RREG a [, x [, y [, p]]]
```

ARGUMENTS

a, x, y, p

Numeric variables. Each is given one register's value, 0 to 255: A, X, Y, and the status flags, in that order. The list can stop after any of them.

DESCRIPTION

`SYS`, `CALL ABSOLUTE` and `CALL ASM` each run machine code and then save the registers it returned with. `RREG` reads them: the values are those of the most recent such call, and stay until the next.

In the flags, bit 0 is the carry (C), bit 1 zero (Z), bit 6 overflow (V) and bit 7 negative (N). A system call reports failure by returning with the carry set and an error code in A, so `BIT(p, 0)` tests whether a call worked.

ERRORS

ERR	MESSAGE	CAUSE
40	`variable required`	An argument that is not a variable

SEE ALSO

SYS, CALL, ASM, BIT.

EXAMPLE

GETPID returns the task's number in A. Here HyBASIC is task 2:

```
> SYS "GETPID" : RREG A, X, Y, P : PRINT A; BIT(P, 0)
 2  0
```

RUN

A command of the prompt: runs the program.

SYNTAX

```
RUN ["file"]
```

ARGUMENTS

file

A file's name, in quotes: a program to load first, as LOAD would, and then run.

DESCRIPTION

RUN first clears away what an earlier run left: every variable and array is dropped, every file is closed, DATA is back at its first item and any stopped program is forgotten. Then the whole program is compiled, and if that finds an error it is reported with its line and nothing runs. Otherwise the program runs from its first line.

The program's numbers are always read in decimal as it is compiled, whatever BASE was left at.

RUN must be alone on its line.

ERRORS

ERR	MESSAGE	CAUSE
89	only at the prompt	RUN in a program's line, or after other statements on a line typed at the prompt
288	not found	There is no such *file*

Any error the compiler finds in the program is reported by RUN, with its line.

NOTES

A line number after RUN is accepted and ignored: RUN 100 runs the program from its first line. To start part-way, put a GOTO at the top.

SEE ALSO

CONT, LOAD, NEW, END, STOP.

EXAMPLE

```
> 10 PRINT "running"; X : X = X + 1
> X = 5
> RUN
running 0
> RUN
running 0
```

SAVE

A command of the prompt: writes the program to a file.

SYNTAX

```
SAVE ["file"]
```

ARGUMENTS

file

The file's name, in quotes. Nothing is added to it.

DESCRIPTION

The program's lines are written to the file as plain text, a line to a line, as `LIST` shows them. A file of that name is replaced.

With no name, the program is saved to the file it was last loaded from or saved to.

ERRORS

ERR	MESSAGE	CAUSE
70	`permission denied`	The file, or its disk, can't be written
53	`file not found`	A directory in the name isn't there
61	`disk full`	No room left
89	`only at the prompt`	`SAVE` in a program's line

NOTES

`/ram` is emptied when the Hydra is switched off or reset. Save what matters to an SD card (`/sd/0`) or to the PC (`/pc`).

SEE ALSO

`LOAD`, `LIST`, `NEW`.

SCREEN

A statement: chooses a graphics mode, or returns to text alone.

SYNTAX

```
SCREEN n
```

ARGUMENTS

n

A numeric expression: the mode.

N	BITMAP	COLOURS	THE PEN STARTS AS
0	None: text alone		
1	320 × 240	4	3
2	640 × 480	2	1
7	320 × 240	16	15
12	640 × 480	4	3
13	320 × 240	256	15

DESCRIPTION

Graphics need a Vera X card (Chapter 16). In a graphics mode the screen has a **bitmap**, a grid of pixels each holding a colour number, with the console's text shown over it. The drawing statements change the bitmap, and `PRINT` goes on writing text as before.

`SCREEN n` sets the mode, clears the bitmap to colour 0, sets the **pen** (the colour used when a drawing statement is given none) to the mode's brightest, up to 15, and puts the **last point referenced** at the middle of the bitmap. Any `WINDOW` and `VIEW` are cancelled. `SCREEN 0` takes the bitmap away.

Coordinates. A point is written `(x, y)`. Unless `WINDOW` says otherwise, *x* counts pixels from the left edge, starting at 0, and *y* counts them down from the top: in a 320 × 240 mode the corners are (0, 0) and (319, 239). A coordinate that isn't an integer is rounded to the nearest, a half going to the even one.

The last point referenced is where the most recent drawing statement finished. `STEP (dx, dy)` means the point that far from it, and `LINE -(x, y)` starts from it.

Colours. A colour is a number from 0 to 255. A mode with fewer colours uses as many low bits as it has, so in a 4-colour mode 5 is colour 1. What colour a number looks like is the palette's business: see `PALETTE`.

The mode numbers are QuickBASIC's for the nearest thing each mode is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is not one of the six modes
53	`file not found`	There is no Vera X card: the screen driver's files are not there

NOTES

The drawing is done by the screen's driver, to which HyBASIC writes commands (`/dev/vid/draw`, section 16.4). A program in another language, or the shell, can draw on the same bitmap by the same route.

SEE ALSO

`PSET`, `LINE`, `CIRCLE`, `PAINT`, `DRAW`, `GPRINT`, `POINT`, `COLOR`, `PALETTE`, `WINDOW`, `VIEW`, `SPRITE`, `CLS`.

SEEK

A statement and a function: sets, or reports, an open file's position.

SYNTAX

```
SEEK [#]f, position
SEEK(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file, in any mode.

position

A numeric expression, 1 or more: the byte the next read or write will start at, counting the file's first byte as 1.

RESULT

`SEEK(f)` is the file's present position, counted from 1: one more than `LOC(f)`.

DESCRIPTION

The statement moves the position. What is read or written next starts there, whatever statement does it.

A position past the end of the file is allowed. A read there finds the end of the file; a write there makes the file longer, the gap filled with zero bytes.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*position* is less than 1
52	`bad file number`	File *f* isn't open

SEE ALSO

`LOC`, `LOF`, `GET`, `PUT`, `OPEN`.

EXAMPLE

```
> OPEN "/ram/s.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : SEEK #1, 3 : PRINT #1, "X";
: PRINT SEEK(1) : CLOSE
 4
> OPEN "/ram/s.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$ : SEEK #1, 100 :
PRINT EOF(1) : CLOSE
abXdef
-1
```

SELECT CASE

A statement block: runs one of several groups of statements, chosen by a value.

SYNTAX

```
SELECT CASE value
CASE test [, test] ...
    [statements]
[CASE ...] ...
[CASE ELSE
    [statements]]
END SELECT
```

where *test* is one of

```
expression
low TO high
IS operator expression
```

ARGUMENTS

value

An expression, numeric or string. It is evaluated once.

test

An expression of the same type as *value*. *operator* is =, <>, <, >, <= or >=.

DESCRIPTION

The `CASE` lines are taken in order from the top, and the tests of each from left to right. A plain expression matches if *value* equals it; `low TO high` if *value* is from *low* to *high*, both included; and `IS` if the comparison of *value* with the expression holds. The statements under the first `CASE` that has a matching test are run, and then control goes on after `END SELECT`: at most one group runs, even if a later `CASE` would match as well.

`CASE ELSE` matches anything, so it is put last. If nothing matches and there is no `CASE ELSE`, no group runs, and that is not an error.

Strings are compared byte by byte, as < and > compare them, so ranges of strings work: `CASE "a" TO "m"`.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	`SELECT CASE` with no `END SELECT`, or `CASE` or `END SELECT` with none open
13	`type mismatch`	A test of the other type from *value*

NOTES

Statements written between `SELECT CASE` and the first `CASE` are run every time, before the choice is made. QuickBASIC doesn't allow them.

The whole block can be written on one line with colons, which is the only form the prompt can take.

SEE ALSO

`IF`, `ON`.

EXAMPLE

```
> 10 X = 2
> 20 SELECT CASE X
> 30 CASE 1 : PRINT "one"
> 40 CASE 2, 3 : PRINT "two or three"
> 50 CASE IS > 1 : PRINT "more than one: not run, an earlier CASE matched"
> 60 CASE ELSE : PRINT "other"
> 70 END SELECT
> 80 SELECT CASE "pear" : CASE "apple" TO "fig" : PRINT "a to f" : CASE IS > "m" : PRINT
"after m" : END SELECT
> RUN
two or three
after m
```

SGN

A numeric function: the sign of a number.

SYNTAX

```
SGN(x)
```

ARGUMENTS

x

A numeric expression.

RESULT

−1 if *x* is negative, 0 if it is zero, and 1 if it is positive. For a complex number, the sign of its real part.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`ABS`.

EXAMPLE

```
> PRINT SGN(-5); SGN(0); SGN(7/2); SGN(-0.1); SGN(-1+2i)
-1  0  1 -1 -1
```

SHARED

A statement, in a procedure: makes variables of the main program visible there.

SYNTAX

```
SHARED name[()] [AS type] [, name ...] ...
DIM SHARED ...
```

ARGUMENTS

name

A variable of the main program. An array is written with empty parentheses.

DESCRIPTION

A `SUB` or a `FUNCTION` has variables of its own: a name used inside it is a new variable, which starts at 0 or empty on each call, whatever the main program calls its own variables. There are two ways to reach the main program's variables instead.

`SHARED`, inside a procedure, names the main program's variables that this procedure uses. From that line on, within the procedure, those names mean the main program's variables.

`DIM SHARED`, in the main program, declares variables and arrays that every procedure can use without saying so. It is `DIM` with one more word: see `DIM`.

Constants need neither. A `CONST` is known everywhere.

ERRORS

ERR	MESSAGE	CAUSE
85	`not in a procedure`	`SHARED` on its own in the main program. (`DIM SHARED` is the form for there)

SEE ALSO

`DIM`, `STATIC`, `SUB`, `FUNCTION`, `CONST`.

EXAMPLE

```
> 10 DIM SHARED G : G = 1 : L = 2
> 20 SUB Show : PRINT "G ="; G; "L ="; L : END SUB
> 30 SUB Uses : SHARED L : L = L + 10 : END SUB
> 40 Show : Uses : PRINT L
> RUN
G = 1 L = 0
 12
```

SHELL

A statement: runs a command line in the shell, and waits for it.

SYNTAX

```
SHELL [line$]
```

ARGUMENTS

line$

A string expression: a line as it would be typed at rc's prompt (Chapter 8). It can be anything rc accepts: a program and its arguments, a pipeline, several commands.

DESCRIPTION

HyBASIC starts rc with the line (`rc -c line`), with the program's standard input and output, its current directory and a copy of its environment, and waits until rc has finished. What the commands print appears on the screen in the ordinary way. `STATUS` then gives how the last command ended.

With no line, rc is started by itself: an interactive shell, which lasts until `exit` or CTRL-D, when the program carries on.

Nothing the shell does to its own variables or its own current directory comes back to HyBASIC: it is another task. Files it makes and removes are, of course, made and removed.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*line$* is a number

A command that fails is not an error in the program. Its message is printed by rc or by the command, and `STATUS` is not 0.

SEE ALSO

`SHELL$`, `STATUS`, `ENVIRON`, `SYSTEM`.

EXAMPLE

```
> SHELL "echo from the shell" : PRINT STATUS
from the shell
 0
> SHELL "ls /nothere" : PRINT STATUS
ls: /nothere: not found
 1
```

SHELL$

A string function: what a command line prints.

SYNTAX

```
SHELL$(line$)
```

ARGUMENTS

line$

A string expression: a line for rc, as `SHELL`'s.

RESULT

Everything the line's commands wrote to standard output, as one string, 8K at most. The line end after the last line is not included; the ones between lines are, as `CHR$(10)`.

What the commands write to standard error is not captured. It goes to the screen, as their messages should.

`STATUS` is set as `SHELL` sets it.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*line$* is a number

SEE ALSO

`SHELL`, `STATUS`.

EXAMPLE

```
> A$ = SHELL$("echo one; echo two") : PRINT LEN(A$); INSTR(A$, CHR$(10)); "["; LEFT$(A$,
3); "]"
 7  4 [one]
> A$ = SHELL$("ls /nothere") : PRINT "["; A$; "]"; STATUS
ls: /nothere: not found
[] 1
```

SHL, SHR

Numeric functions: an integer shifted left, or right, by a number of bits.

SYNTAX

```
SHL(x, n)
SHR(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real, rounded to the nearest integer: the number to shift.

n

A numeric expression: how many bits.

RESULT

`SHL(x, n)` is *x* multiplied by 2 to the power *n*. There is no word size, so nothing is ever shifted out at the top: `SHL(1, 100)` is 2 to the power 100.

`SHR(x, n)` is *x* divided by 2 to the power *n* and rounded down. For a negative *x* this is the arithmetic shift, which keeps the sign: `SHR(-8, 1)` is −4.

A negative *n* shifts the other way: `SHR(1, -2)` is `SHL(1, 2)`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

SEE ALSO

AND, OR, XOR, BIT.

EXAMPLE

```
> PRINT SHL(1, 4); SHR(256, 4); SHL(1, 100); SHR(-8, 1); SHR(1, 5); SHR(1, -2)
 16  16  1267650600228229401496703205376 -4  0  4
```

SIN

A numeric function: the sine of a real number.

SYNTAX

```
SIN(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The sine of *x*, from −1 to 1. `SIN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

`PI` is itself rounded to the digits set, so `SIN(PI)` is not exactly 0: it is a very small number, the sine of the angle that `PI` really is.

SEE ALSO

`COS`, `TAN`, `ATN`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT SIN(0); SIN(1); SIN(PI / 2); SIN(-1); SIN(PI)
 0  0.841470984808  1 -0.841470984808 -0.00000000000002067615373 57
```

SLEEP

A statement: waits for a time, or for a key.

SYNTAX

```
SLEEP [seconds]
```

ARGUMENTS

seconds

A numeric expression: how long to wait. It need not be whole: `SLEEP 0.5` waits half a second.

DESCRIPTION

The program stops for that long and then goes on. While it waits it uses no processor time, so other tasks run freely; this is the right way to slow a loop down. The time is counted in the system's ticks, 200 to the second, so the shortest wait is five thousandths of a second, and a wait can be a tick longer than asked.

With no argument, or with a negative one, `SLEEP` waits until a key is pressed. The key is used up by the wait: the program doesn't see it.

`SLEEP 0` returns at once.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*seconds* is a string

SEE ALSO

`TIMER`, `INKEY$`, `INPUT$`.

EXAMPLE

```
> T = TIMER : SLEEP 0.5 : PRINT TIMER - T >= 0.5; TIMER - T < 0.6
-1 -1
```

SOUND

A statement: starts or stops a note; or sends the sound driver a command.

SYNTAX

```
SOUND channel [, note [, patch [, volume]]]
SOUND command$
```

ARGUMENTS

channel

A numeric expression: one of the sound driver's channels (section 15.1).

note

A numeric expression: the note's number. 60 is middle C and each step is a semitone, so 69 is the A above it (section 15.2). If it is left out, the channel's note is turned off.

patch

A numeric expression: the instrument to give the channel first (section 15.4).

volume

A numeric expression: the channel's volume, set before the note starts. It can be given only after a patch.

command$

A string expression: a line for the sound driver, as it would be written to `/dev/sndctl` (section 15.3): `"volume 150"`, `"pan 0 left"`.

DESCRIPTION

`SOUND channel, note` starts a note and returns at once. The note goes on sounding until `SOUND channel` stops it, or another note is started on the same channel. Timing is the program's business: `SOUND 0, 60 : SLEEP 0.5 : SOUND 0` plays middle C for half a second.

The patch and the volume, when they are given, are set before the note starts, and all of it is sent to the driver in one write, so that another program's commands can't come between.

The string form passes its text to the driver unchanged, and whatever the driver's commands can do, it does.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	A channel, note or patch number the driver doesn't have
258	`invalid argument`	The driver did not understand *command$*
2	`syntax error`	No arguments; or an argument skipped

NOTES

This is not QuickBASIC's `SOUND frequency, duration`. `SOUND 440, 18` is an error here: 440 is not a channel.

For a tune, `PLAY` is far less work.

SEE ALSO

`PLAY`, `BEEP`, `SLEEP`.

EXAMPLE

```
> SOUND 0, 60 : SLEEP 0.1 : SOUND 0 : PRINT "one note"
one note
> SOUND "volume 150" : SOUND "nonsense here"
invalid argument
```

SPACE$

A string function: a string of spaces.

SYNTAX

```
SPACE$(n)
```

ARGUMENTS

n

A numeric expression, 0 or more, rounded to the nearest integer.

RESULT

A string of *n* spaces; the empty string if *n* is 0.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative
13	`type mismatch`	*n* is a string

NOTES

`SPACE$` is the usual way to give a string variable a length before `GET` fills it.

SEE ALSO

`STRING$`, `SPC`, `GET`.

EXAMPLE

```
> PRINT "["; SPACE$(3); "]"; LEN(SPACE$(0)); LEN(SPACE$(1000))
[   ] 0  1000
```

SPC

Used in `PRINT`: `SPC(n)` writes *n* spaces. See `PRINT`.

SPRITE

A statement: shows, moves or hides a sprite.

SYNTAX

```
SPRITE n, x, y [, image$]
SPRITE n OFF
```

ARGUMENTS

n

A numeric expression, 1 to 127: which sprite.

x, y

Numeric expressions: where on the screen the sprite's top left corner goes, in pixels.

image$

A string expression of exactly 64 or 256 bytes: the sprite's picture, one byte to a pixel, each byte a colour number, row by row from the top left. 64 bytes make a sprite 8 pixels square; 256 make one 16 square. A pixel of colour 0 is transparent.

DESCRIPTION

A sprite is a small picture that the screen shows over everything else, wherever it is told to, without anything being drawn into the bitmap. Moving one is instant and leaves the bitmap as it was underneath.

`SPRITE n, x, y, image$` gives sprite *n* its picture and shows it at the point. `SPRITE n, x, y,` without a picture, moves it there with the picture it has. `SPRITE n OFF` hides it; its picture is kept.

Sprites need a Vera X card but no graphics mode: they show over text too.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is not 1 to 127; or *image$* is not 64 or 256 bytes long
13	`type mismatch`	A number where the picture should be, or a string where a number should be

NOTES

A picture is most easily built from `CHR$` and `STRING$`, or read from `DATA`. `STRING$(64, CHR$(7))` is a solid square of colour 7.

Sprites are the screen's, not HyBASIC's. One left showing stays on the screen after the program ends: hide it first.

SEE ALSO

`SCREEN`, `PALETTE`, `CHR$`, `STRING$`.

SQR

A numeric function: the square root.

SYNTAX

```
SQR(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The square root of *x*.

- If the root is exact, it is returned exactly, as an integer, a rational or a decimal: `SQR(16)` is 4, `SQR(1/4)` is 1/2, and `SQR(2 ^ 100)` is 2 to the power 50.
- Otherwise it is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).
- If *x* is negative, the result is imaginary: the root of its size, times i. `SQR(-16)` is `4i`. It is not an error.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

A program brought from another BASIC that relies on `SQR` of a negative number being an error will go on with an imaginary number instead, and fail later, at the first function that won't take one. Test the sign first.

SEE ALSO

`EXP`, `LOG`, `DIGITS`, `COMPLEX`.

EXAMPLE

```
> PRINT SQR(16); SQR(2); SQR(1/4); SQR(0.25); SQR(-16); SQR(-2); SQR(2 ^ 100)
 4  1.41421356237  1/2  0.5  4i  1.41421356237i  1125899906842624
```

STATIC

A statement, and a word on a procedure's first line: makes a procedure's variables keep their values between calls.

SYNTAX

```
STATIC name[()] [AS type] [, name ...] ...
SUB name [(parameters)] STATIC
FUNCTION name [(parameters)] STATIC
```

DESCRIPTION

A procedure's own variables are normally made afresh, 0 or empty, each time it is called. A variable named in a `STATIC` statement inside the procedure is made once and keeps its value from one call to the next. It is still the procedure's own: no other part of the program can see it.

`STATIC` at the end of a `SUB` or `FUNCTION` line does the same for every variable of the procedure.

ERRORS

ERR	MESSAGE	CAUSE
85	`not in a procedure`	A `STATIC` statement in the main program

SEE ALSO

`SHARED`, `SUB`, `FUNCTION`, `DIM`.

EXAMPLE

`K` is static and counts the calls; `M` is not, and starts at 0 each time:

```
> 10 SUB Keeps : STATIC K : K = K + 1 : M = M + 1 : PRINT K; M : END SUB
> 20 Keeps : Keeps : Keeps
> RUN
 1  1
 2  1
 3  1
```

STATUS

A numeric function: how the last `SHELL` command ended.

SYNTAX

```
STATUS
```

RESULT

The exit code of the command line that the most recent `SHELL` or `SHELL$` ran: 0 if it succeeded, and not 0 if it failed. For a line of several commands it is the last one's.

SEE ALSO

`SHELL`, `SHELL$`, `SYSTEM`.

STEP

Part of `FOR`, where it gives the amount the counter changes by each time. In the graphics statements, `STEP` before a point means that its coordinates are an offset from the last point. See `FOR` and `PSET`.

STOP

A statement: stops the program in a way that can be continued.

SYNTAX

```
STOP
```

DESCRIPTION

The program stops, `Break in line` and the line's number are printed, and the prompt returns. Everything is as it was at that moment: the variables can be looked at and changed, and `CONT` carries on with the statement after the `STOP`. CTRL-C stops a running program in the same way, wherever it is.

SEE ALSO

`CONT`, `END`.

EXAMPLE

See `CONT`.

STR$

A string function: a number's text.

SYNTAX

```
STR$(x [, base$])
```

ARGUMENTS

x

A numeric expression of any kind.

base$

A string expression: a base string (section 18.4), `"x"` for hex, `"b"` for binary.

RESULT

With one argument, the text `PRINT` would write for *x* in the base that `BASE` has set, without the space after it: a number that isn't negative has a space in front, where a negative one has its minus sign.

With a base, the text of *x* in that base, with no space in front.

Either way the text is the number's exact value: all of an integer's digits, a rational with its `/`, a decimal with its point.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*base$* isn't a base string
13	`type mismatch`	*x* is a string, or *base$* a number

NOTES

To drop the leading space, `LTRIM$(STR$(x))`; or give the base, `STR$(x, "d")`.

SEE ALSO

`VAL`, `HEX$`, `OCT$`, `BASE`, `PRINT`.

EXAMPLE

```
> PRINT "["; STR$(5); "]["; STR$(-5); "]["; STR$(1/3); "]["; STR$(255, "x"); "][";
STR$(5, "b"); "]"
[ 5][-5][ 1/3][FF][101]
```

STRING$

A string function: one character, repeated. `STRING` is also the name of the string type, in `AS STRING`.

SYNTAX

```
STRING$(n, s$)
STRING$(n, code)
```

ARGUMENTS

n

A numeric expression, 0 or more, rounded to the nearest integer: how many.

s$

A string expression of one byte or more. Its first byte is the one repeated; the rest are not used.

code

A numeric expression from 0 to 255: the byte to repeat, given as its code.

RESULT

A string of *n* bytes, all the same.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative; *s$* is empty; *code* is not 0 to 255

SEE ALSO

`SPACE$`, `CHR$`.

EXAMPLE

```
> PRINT STRING$(3, "ab"); "|"; STRING$(3, 65); "|"; STRING$(0, "x"); "|"
aaa|AAA||
```

SUB ... END SUB

A block: defines a subprogram, a named piece of the program with parameters and variables of its own.

SYNTAX

```
SUB name [(parameter [, parameter] ...)] [STATIC]
    [statements]
END SUB
```

ARGUMENTS

name

The subprogram's name.

parameter

A variable's name, optionally followed by `AS type`; or an array's, written with empty parentheses.

STATIC

The subprogram's variables keep their values from one call to the next. See `STATIC`.

DESCRIPTION

A `SUB` is run by calling it: its name, then an argument for each parameter, with or without the word `CALL` and with or without parentheses. See `CALL`, which describes how arguments are passed: a variable by reference, anything else by value.

Inside the SUB, the names of variables are its own. Each is made anew at every call, 0 or empty, and is gone when the SUB returns, and it has nothing to do with a variable of the same name anywhere else. The exceptions are the parameters, the names declared SHARED or STATIC, and constants.

The SUB returns when control reaches END SUB or an EXIT SUB. A SUB may call itself.

Subprograms are found when the program is compiled, wherever they are in its text, so one can be called from a line before it. When control reaches a SUB line in the ordinary run of the program, it skips to after END SUB.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	Two procedures of one name
80	`block without its start or end`	SUB with no END SUB
2	`syntax error`	SUB typed at the prompt: a subprogram can be defined only in a program

The errors of calling one are under CALL.

SEE ALSO

CALL, FUNCTION, EXIT, SHARED, STATIC, GOSUB.

EXAMPLE

```
> 10 SUB Greet (N$, Times)
> 20   FOR I = 1 TO Times : PRINT "Hello, "; N$ : NEXT
> 30 END SUB
> 40 I = 99 : Greet "Sam", 2 : PRINT I
> RUN
Hello, Sam
Hello, Sam
 99
```

SWAP

A statement: exchanges the values of two variables.

SYNTAX

```
SWAP a, b
```

ARGUMENTS

a, b

Two variables, array elements or record fields, of the same type.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	One is a number's and the other a string's
40	`variable required`	An argument that is not a variable

EXAMPLE

```
> A = 1 : B = 2 : SWAP A, B : PRINT A; B : DIM V(1) : V(0) = 7 : V(1) = 8 : SWAP V(0),
V(1) : PRINT V(0); V(1)
 2  1
 8  7
```

SYS

A statement: calls machine code at an address, or a system call by its name.

SYNTAX

```
SYS address [, a [, x [, y]]]
SYS "NAME" [, a [, x [, y]]]
```

ARGUMENTS

address

A numeric expression, 0 to 65,535: the address of a machine code routine.

NAME

A string expression: the name of a system call (Appendix K), in capitals or small letters.

a, x, y

Numeric expressions: the values of the A, X and Y registers as the code is entered. Each is 0 if it is left out.

DESCRIPTION

The code is called as a subroutine (`jsr`) and must return with `rts`. When it does, its A, X and Y and the flags are kept for `RREG` to read, and the program goes on.

Anything the program has printed is written out before the call, so output appears in order even if the code prints too.

A system call that takes its arguments in the call registers r0 to r15 needs them put there first. They are two bytes each in the zero page from address 2 (r0 is at 2 and 3, r1 at 4 and 5), and `POKE` sets them; HyBASIC leaves r0 and r1 as the program's `POKE`s made them until the call.

ERRORS

ERR	MESSAGE	CAUSE
259	`no such call`	*NAME* is not a system call's name
13	`type mismatch`	A register's value is a string

A system call that fails is not an error in the program. It returns with the carry flag set and its error code in A: read them with `RREG`.

NOTES

The code runs inside HyBASIC's task with nothing to protect HyBASIC from it. Code that doesn't return, or writes over HyBASIC's memory, ends the session.

For code of the program's own, an `ASM` block is easier than `POKE`s: it is assembled with the program, and `CALL ASM` finds it by name.

SEE ALSO

`RREG`, `CALL`, `ASM`, `PEEK`, `POKE`.

EXAMPLE

```
> SYS "getpid" : RREG A : PRINT A
 2
> SYS "NOSUCH"
no such call
```

SYSTEM

A statement: leaves HyBASIC.

SYNTAX

```
SYSTEM [n]
BYE [n]
```

ARGUMENTS

n

A numeric expression from 0 to 255, rounded to the nearest integer: the exit code. 0, for success, if it is left out.

DESCRIPTION

HyBASIC ends, from a program or from the prompt, and control returns to whatever started it, usually a shell. Open files are closed. The program in memory is lost unless it has been saved; there is no question first.

The exit code is what the shell sees as the command's status (section 8.9): 0 is success, and anything else a failure of the program's own choosing.

`BYE` is the same statement.

SEE ALSO

`END`, `STOP`, `SHELL`.

EXAMPLE

```
% echo 'PRINT "leaving" : SYSTEM 3 : PRINT "not printed"' >bye.bas
% basic bye.bas; echo status $status
leaving
status 3
```

TAB

Used in `PRINT`: `TAB(n)` moves to column *n*. See `PRINT`.

TAN

A numeric function: the tangent of a real number.

SYNTAX

```
TAN(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The tangent of *x*. `TAN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

The tangent has no value at π/2, but no number a program can write is exactly π/2. `TAN(PI / 2)` is therefore not an error: it is a very large number, the tangent of the angle that `PI / 2` really is.

SEE ALSO

`SIN`, `COS`, `ATN`, `PI`.

EXAMPLE

```
> PRINT TAN(0); TAN(1); TAN(PI / 4)
 0  1.55740772465  1
```

THEN

Part of `IF`. See `IF`.

TIME$, TIMER

Functions: the time of day, as text and as a number of seconds.

SYNTAX

```
TIME$
TIMER
```

RESULT

`TIME$` is a string of eight characters, `hh:mm:ss`, on the 24-hour clock.

`TIMER` is the number of seconds since midnight, as a decimal with a fraction: the system's clock ticks 200 times a second, so it moves in steps of 0.005.

Both come from the system's clock. On a Hydra with no clock chip that clock starts at midnight when the system starts, so both then tell the time since it was switched on.

NOTES

To time something, take `TIMER` before and after and subtract. The answer is wrong, by a day's worth of seconds, if midnight comes between.

SEE ALSO

`DATE$`, `SLEEP`, `RANDOMIZE`.

EXAMPLE

```
> PRINT LEN(TIME$); MID$(TIME$, 3, 1) : T = TIMER : SLEEP 1 : PRINT TIMER - T >= 1
 8 :
-1
```

TO

Part of `FOR` (`FOR i = a TO b`), of `DIM` (`DIM a(1 TO 10)`) and of `CASE` (`CASE 1 TO 5`). See those.

TYPE ... END TYPE

A block: defines a record type, a group of named fields that travel together.

SYNTAX

```
TYPE name
    field AS type
    [field AS type] ...
END TYPE
```

ARGUMENTS

name

The type's name.

field

A field's name.

type

`INTEGER`, `LONG`, `SINGLE`, `DOUBLE` or `NUMBER`, for a number; `STRING`, for a string of any length.

DESCRIPTION

`TYPE` makes no variable. It describes a kind of variable, which `DIM` then makes: `DIM P AS Point`, or an array of them, `DIM Pts(10) AS Point`. A field is written with a full stop: `P.X`, `Pts(3).Label`. It is used like any variable of its type.

Assigning one record to another of the same type copies every field. A record can be written to and read from a `BINARY` file with `PUT` and `GET`.

The type is known in the whole program, procedures included, from where it is defined.

ERRORS

ERR	MESSAGE	CAUSE
88	`no such field`	A field name that the record's type doesn't have. Found when the program is compiled
80	`block without its start or end`	`TYPE` with no `END TYPE`

NOTES

QuickBASIC's fixed-length strings, `STRING * 10`, are not here. A `STRING` field holds a string of whatever length it is given.

A full stop is also a legal character in an ordinary name. `P.X` is the field X of P only if `P` has been declared as a record; otherwise it is a variable called `P.X`.

SEE ALSO

`DIM`, `PUT`, `GET`, `SUB`.

EXAMPLE

```
> 10 TYPE Point
> 20   X AS INTEGER
> 30   Y AS INTEGER
> 40   Label AS STRING
> 50 END TYPE
> 60 DIM P AS Point, Q AS Point : P.X = 3 : P.Y = 4 : P.Label = "here" : Q = P : Q.X = 9
> 70 PRINT P.X; P.Y; P.Label; Q.X; Q.Label
> 80 DIM Pts(2) AS Point : Pts(1).X = 5 : PRINT Pts(1).X; Pts(0).X
> RUN
 3  4 here 9 here
 5  0
```

UNTIL

Part of `DO` loops: `DO UNTIL condition` and `LOOP UNTIL condition`. See `DO`.

USING

Part of `PRINT USING`, which writes values in a format. See `PRINT`.

VAL

A numeric function: the number that a string's text says.

SYNTAX

```
VAL(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

The number written at the start of *s$*, after any spaces. It can be in any form a program's own numbers take: a whole number, a decimal, a rational (`1/3`), a complex number (`1+2i`), one with an exponent (`-3.5e2`), or one with a prefix that says its base (`&HFF`). Without a prefix it is read in the base that `BASE` has set.

Reading stops at the first character that can't be part of the number, and what was read up to there is the result: `VAL("12abc")` is 12, and `VAL("1,000")` is 1. If there is no number at the start, the result is 0. It is never an error for the text to be wrong.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

NOTES

`VAL` reads a number. It doesn't work anything out: `VAL("2^3")` is 2.

Since a wrong text gives 0, `VAL` can't tell "0" from nonsense. A program that must know checks the text itself.

SEE ALSO

`STR$`, `INPUT`, `BASE`, `CVN`.

EXAMPLE

```
> PRINT VAL("12"); VAL(" -3.5e2"); VAL("1/3"); VAL("&HFF"); VAL("12abc"); VAL("abc");
VAL("")
 12 -350  1/3  255  12  0  0
```

VIEW

A statement: moves the origin of the drawing coordinates to a rectangle of the bitmap.

SYNTAX

```
VIEW [[SCREEN] (x1, y1)-(x2, y2) [, fill [, border]]]
```

ARGUMENTS

x1, y1, x2, y2

Numeric expressions: two opposite corners of a rectangle, the **viewport**, in pixels of the bitmap.

fill, border

Accepted, evaluated and ignored.

DESCRIPTION

After `VIEW (x1, y1)-(x2, y2)`, the drawing statements' coordinates are measured from the viewport's top left corner: the point (0, 0) is the pixel (x1, y1). A drawing can be moved about the screen by changing one statement.

With `SCREEN`, the coordinates stay the bitmap's own, and the viewport matters only to a `WINDOW`.

A `WINDOW` set afterwards spreads its coordinates over the viewport instead of over the whole bitmap, which is the usual reason for a viewport: a graph in one corner of the screen.

`VIEW` alone makes the viewport the whole bitmap again.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set
13	`type mismatch`	An argument is a string

NOTES

Unlike QuickBASIC's, the viewport does not clip. A point outside it is still drawn, as long as it is on the bitmap, and the viewport is neither cleared nor outlined.

SEE ALSO

`WINDOW`, `SCREEN`, `POINT`.

EXAMPLE

```
> SCREEN 13 : VIEW (100, 100)-(200, 200) : PSET (0, 0), 13 : PRINT POINT(0, 0)
 13
> VIEW : PRINT POINT(100, 100); POINT(0, 0)
 13  0
> SCREEN 0
```

WHILE ... WEND

A statement block: a loop that repeats while a condition holds.

SYNTAX

```
WHILE condition
    [statements]
WEND
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0.

DESCRIPTION

The condition is tested before each pass, the first included. While it holds, the statements run and control goes back to the test; when it doesn't, the program goes on after `WEND`.

It is the same loop as `DO WHILE condition` ... `LOOP`, kept for older programs, with one difference: there is no `EXIT` for it. `EXIT DO` inside a `WHILE` loop is an error unless a `DO` loop encloses it, and then it leaves that one.

ERRORS

ERR	MESSAGE	CAUSE
29	`WHILE without WEND`	A `WHILE` whose `WEND` isn't there. Found when the program is compiled
30	`WEND without WHILE`	A `WEND` with no loop open
13	`type mismatch`	The condition is a string

SEE ALSO

`DO`, `FOR`, `EXIT`.

EXAMPLE

```
> I = 0 : WHILE I < 3 : I = I + 1 : WEND : PRINT I : WHILE 0 : PRINT "never" : WEND
 3
```

WIDTH

A statement: sets the width of the line that `PRINT` works to.

SYNTAX

```
WIDTH n
```

ARGUMENTS

n

A numeric expression from 1 to 255: the number of columns. 80 when HyBASIC starts.

DESCRIPTION

The width decides how many print zones fit on a line: `PRINT a, b, c` moves to a new line when the next zone would start beyond it. It is HyBASIC's idea of the line, used for its own layout.

It does not change the terminal or the screen, and it does not break long text: a string longer than the width is written whole, and the terminal wraps it wherever its own edge is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 255

SEE ALSO

`PRINT`, `POS`, `TAB`.

EXAMPLE

```
> WIDTH 40 : PRINT 1, 2, 3, 4 : WIDTH 80
 1             2             3
 4
```

WINDOW

A statement: gives the drawing statements coordinates of the program's own choosing.

SYNTAX

```
WINDOW [[SCREEN] (x1, y1)-(x2, y2)]
```

ARGUMENTS

x1*, *y1*, *x2*, *y2

Numeric expressions: the coordinates that the left edge, one horizontal edge, the right edge and the other horizontal edge of the drawing area are to have.

DESCRIPTION

After `WINDOW (x1, y1)-(x2, y2)`, a point's *x* runs from *x1* at the left edge of the bitmap to *x2* at the right, and its *y* from *y1* at the **bottom** to *y2* at the top, as a graph is drawn on paper. Each point a statement is given is worked out exactly in those coordinates and then rounded to a pixel. `WINDOW (-1, -1)-(1, 1)` puts (0, 0) at the middle of the screen.

With `SCREEN`, *y* runs the screen's way instead: *y1* is at the top and *y2* at the bottom.

The window covers the viewport, if `VIEW` has set one, and the whole bitmap otherwise. Every statement that takes a point follows it, `POINT` included; a `CIRCLE`'s radius is in the window's x units.

`WINDOW` alone returns to pixels.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set
13	`type mismatch`	An argument is a string

SEE ALSO

`VIEW`, `SCREEN`, `POINT`, `CIRCLE`.

EXAMPLE

The point (50, 50) of a window 100 units each way is the pixel in the middle of the bitmap:

```
> SCREEN 13 : WINDOW (0, 0)-(100, 100) : PSET (50, 50), 12 : PRINT POINT(50, 50)
 12
> WINDOW : PRINT POINT(160, 120)
 12
> SCREEN 0
```

WRITE

A statement: writes values in a form that `INPUT` can read back.

SYNTAX

```
WRITE [#f,] [item [, item] ...]
```

ARGUMENTS

item

An expression of either type.

f

The number of a file opened `FOR OUTPUT`, `FOR APPEND` or `FOR BINARY`.

DESCRIPTION

The items are written on one line with a comma between each and nothing else: no spaces, no zones. A string is put in double quotes. A number is written as its text with no space before or after it. The line is then ended. `WRITE` with no items writes an empty line.

This is exactly the form `INPUT` and `INPUT #` take apart: the commas separate the fields, and the quotes keep a string's own commas and spaces inside one field. A line written by `WRITE #` and read by `INPUT #` with the same list of variables gives back the values that were written.

The items are separated by commas only. A semicolon is a syntax error here.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	A semicolon between items
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR INPUT`

NOTES

A string that itself contains a double quote can't be read back whole: the quote ends the field. There is no way to write one that `INPUT` will accept.

SEE ALSO

`PRINT`, `INPUT`, `OPEN`.

EXAMPLE

```
> WRITE 1, "two", 1/3, -4.5 : WRITE "a,b", 5
1,"two",1/3,-4.5
"a,b",5
> OPEN "/ram/w.txt" FOR OUTPUT AS #1 : WRITE #1, "Sam", 12, "x, y" : CLOSE
> OPEN "/ram/w.txt" FOR INPUT AS #1 : INPUT #1, N$, G, T$ : PRINT N$; G; "["; T$; "]" :
CLOSE
Sam 12 [x, y]
```

XOR

An operator: the bitwise exclusive OR of two integers.

SYNTAX

```
a XOR b
```

RESULT

An integer. Each bit of it is 1 where the same bit of *a* and of *b* differ, and 0 where they are alike.

For two truth values, –1 and 0, the result is true if exactly one of them is.

Both operands are always evaluated, and they are rounded and treated as `AND`'s are: see `AND`. `XOR` binds less tightly than `OR` and more tightly than `EQV`.

EXAMPLE

```
> PRINT 5 XOR 3; -1 XOR -1; (1 = 1) XOR (1 = 2); -1 XOR 1
 6  0 -1 -2
```

CHAPTER 2

Commands

A command is a program in `/bin`, or one of the shell's own, which are at the end of the chapter. This chapter describes each one's command line: what it takes, what it does with it, what it prints when it can't, and the status it ends with. The languages and the larger tools have chapters of their own in the manual, to which their entries here point.

The exit status. Every program ends with a status, which rc keeps in `$status` (section 8.9). It is empty for success. For failure it is a short text: most often `1`, and `usage` when the arguments made no sense, in which case the program has printed its usage line. A few programs have statuses of their own, and their entries say so.

Messages. What a program has to say about a failure goes to standard error, not into a pipe's data, as `name: what: why`. The *why* is usually the system's own text for an error (Appendix G): `not found`, `read-only`, `is a directory`.

Options are single letters after a `-`, and letters can be joined: `-ld` is `-l -d`. They come before the other arguments. `-` alone is not an option, and unless an entry says otherwise it is not standard input either: it is a file called `-`.

Paths. A path in an argument can be at most 63 characters long once it has been made absolute.

Standard input. The programs that read files read standard input when they are given none, which is how they are used in a pipe.

as

A program: the 65C02 assembler.

SYNTAX

```
as [-bl] {file.s | -} [out | -]
```

ARGUMENTS

file.s

The source. `-` reads the source from standard input.

out
: The file to write. If it is left out, the source's name without its `.s`; for a source read from standard input, standard output. `-` writes to standard output.

`-l`
: Write a labels file too, *out*`.lbl`: every label and its address, which `db` and `dis -l` read. It needs a named output.

`-b`
: Write the bytes alone, with no program header, assembled from the address `.org` gives, or `$0800` if there is none.

DESCRIPTION

The source is in ca65's assembly language (Chapter 27). Without `-b` the output is a program in the form the system loads and runs: `as hi.s` makes `hi`, which `./hi` runs. The assembler is the same code that assembles the `ASM` blocks of a HyBASIC program.

EXIT STATUS

Empty on success. `1` if the source can't be read, or has a mistake, which is reported as `as: file:line: message`. `usage` for bad arguments.

SEE ALSO

`dis`, `db`; Chapter 27.

EXAMPLE

```
% echo ' lda #' >bad.s
% as bad.s; echo status $status
as: bad.s:1: a bad expression
status 1
```

basic

A program: HyBASIC.

SYNTAX

```
basic [-l] [file [argument ...]]
```

ARGUMENTS

file
: A program to load and run. When it ends, HyBASIC ends. The arguments after it are the program's own, which it reads with `ARG$` and `COMMAND$`.

`-l`
: Start as a login shell would: make the default namespace and run the profile first (section 8.16).

DESCRIPTION

With no file, HyBASIC starts at its prompt, where lines are run as they are typed and numbered lines build a program. `SYSTEM` or `BYE` leaves it.

EXIT STATUS

At the prompt: the status given to `SYSTEM` or `BYE`, empty if none. Running a file: empty if the program ran to its end; the status given to `SYSTEM`; `1` if the file can't be read or the program stopped with an error, which is printed as `file:line: message`.

SEE ALSO

Chapter 1 of this book; Chapters 23 and 24.

calc

A program: works out an expression in the Hydra's numbers.

SYNTAX

```
calc [-b base] [-d digits] [expression ...]
```

ARGUMENTS

expression
: The arguments, joined with spaces, are one expression. With none, `calc` reads standard input and works out each line, until the end of the input.

-b *base*
: Print answers in this base: a base string (section 18.4).

-d *digits*
: The number of significant digits for results that can't be exact, 1 to 100. 12 if it is not given.

DESCRIPTION

The numbers are those of Chapter 18: integers of any length, exact fractions, decimals and complex numbers. The operators are `+`, `-`, `*`, `/`, `%` (the remainder of dividing integers) and `^` (a power), with parentheses. The functions, each applied to what follows it, are `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `atan`, `abs`, `floor`, `round`, `truncate`, `fib`, `numerator`, `denominator`, `re`, `im`, `rational` and `random`; and with two arguments in parentheses, `gcd`, `pow`, `complex` and `fixed`. The constants are `pi`, `e` and `i`.

Each answer is printed on a line of its own. A mistake in one expression is reported and, when lines are being read, the next line is gone on with.

The shell reads the command line first. `^`, `(`, `)`, `#`, `*` and `'` all mean something to rc, so an expression that uses them goes in quotes: `calc '2^10'`.

EXIT STATUS

Empty on success. The message itself for an expression that failed: `division by zero`, `not a base`. `usage` for a bad option.

SEE ALSO

Chapter 18.

EXAMPLE

```
/> calc 1/3 + 1/6
1/2
/> calc -b x 255
FF
/> calc -d 5 sqrt 2
1.4142
/> calc 1 / 0
calc: division by zero
```

cat

A program: copies files to standard output, one after another.

SYNTAX

```
cat [file ...]
```

DESCRIPTION

Each file is read and written to standard output, 512 bytes at a time, in the order given. Nothing is added between them and nothing is changed: `cat` copies bytes, and is as good for a program or a picture as for text. With no files it copies standard input.

A file that can't be read is reported, and `cat` goes on with the next.

Anything that can be read can be given: a device (`cat /dev/time`), or a directory, which reads as its files' 64-byte records.

EXIT STATUS

Empty if every file was copied. `1` if any could not be read. `write error` if standard output could not be written: a full disk, or a pipe whose reader has gone.

NOTES

`-` is not standard input here: `cat -` looks for a file named `-`.

SEE ALSO

`more`, `head`, `tail`, `xd`, `cp`, `tee`.

EXAMPLE

```
% echo one >a; echo two >b
% cat a nothere b; echo status $status
one
cat: nothere: not found
two
status 1
```

cmp

A program: compares two files, byte by byte.

SYNTAX

```
cmp file1 file2
```

DESCRIPTION

The two files are read together. If every byte is the same and they end together, `cmp` prints nothing. At the first byte that differs, it prints the files' names and the byte's position, counted from 1, and stops. If one file ends while they still agree, it says which.

EXIT STATUS

Empty if the files are the same. `1` if they differ, or if either can't be read. `usage` unless there are exactly two names.

SEE ALSO

`cp`, `xd`.

EXAMPLE

```
% echo one >a; echo two >b; echo onetwo >ab
% cmp a a; echo status $status
status
% cmp a b; echo status $status
a b differ: byte 1
status 1
% cmp a ab
a ab differ: byte 4
```

cp

A program: copies files, and with `-r` directories.

SYNTAX

```
cp [-r] from to
cp [-r] from ... dir
```

ARGUMENTS

-r

Copy directories too: each one's whole tree.

from

A file to copy, or with `-r` a directory. There can be several if the last name is a directory.

to

The copy's name. If it is an existing directory, the copy goes into it under *from*'s own name.

dir

An existing directory. Each *from* is copied into it under its own name.

DESCRIPTION

Each *from* is handled in turn, and one that fails doesn't stop the rest.

A file is copied by opening it for reading, creating the destination, and reading and writing 512 bytes at a time until a read returns nothing. Creating the destination empties a file that is already there. The copy is a new file: it has its own date, the time of copying, and is not read-only even if the original is.

Before copying, `cp` compares the two files' device, disk and file identifier. If they are the same file, by whatever two names, it refuses to copy it onto itself, which would otherwise empty it.

Anything that can be opened and read can be a *from*, a device included: `cp /dev/time now` copies what one read of the clock gives.

Without `-r`, a *from* that is a directory is reported and skipped. With `-r`, the directory is made at the destination (one already there is used as it is), and then everything in it is copied in the same way, down to 16 directories deep.

EXIT STATUS

Empty if every file was copied. `1` if any was not. `usage`, after a line saying how `cp` is used, if there are fewer than two names or an option other than `-r`.

ERRORS

Each is printed as `cp: name: reason`, where *name* is the file that could not be read or written.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and there's no `-r`
`not a directory`	Several *from*s, and the last name isn't a directory
`the same file`	*from* and *to* are one file
`read-only`	The destination's disk can't be written (`/rom`)
`name too long`	The destination's path would pass 63 characters

Any other reason is the system's own error message for what failed (Appendix G): a full disk, for one.

NOTES

If a read or a write fails part-way, the destination is left holding what was written up to then.

`cp` doesn't ask before it overwrites.

SEE ALSO

`mv`, `rm`, `mkdir`, `cat`, `cmp`.

EXAMPLE

```
% echo hello >a.txt
% cp a.txt b.txt; cat b.txt
hello
% mkdir d; cp a.txt b.txt d; ls d
a.txt
b.txt
% cp d e; echo $status
cp: d: is a directory
1
% cp -r d e; ls e
a.txt
b.txt
% cp a.txt a.txt
cp: a.txt: the same file
% cp a.txt /rom/x
cp: /rom/x: read-only
% cp a.txt; echo $status
usage: cp [-r] from to, or cp [-r] from ... dir
usage
```

date

A program: prints the date and the time.

SYNTAX

```
date [-n]
```

ARGUMENTS

-n

Print the time as one number: the seconds since the start of 1 January 2000.

DESCRIPTION

Without `-n`, the date and time are printed as `yyyy-mm-dd hh:mm:ss`, which is what `/dev/time` reads as. The clock is the system's; on a Hydra with no clock chip it starts at 2000-01-01 00:00:00 when the system does.

`date` doesn't set the clock. Write the time to it: `echo 2026-10-03 15:04:05 >/dev/time`.

EXIT STATUS

Empty on success. `usage` for any argument but `-n`.

SEE ALSO

`sleep`; section 17.5.

db

A program: the debugger.

SYNTAX

```
db program [argument ...]
db -p task
```

ARGUMENTS

program
: A program to start under the debugger, stopped before its first instruction, with its arguments.

`-p` *`task`*
: Attach to a task that is already running, by its number.

DESCRIPTION

`db` shows a program's registers and memory, runs it an instruction or a call at a time, and stops it at breakpoints. Its own commands are typed at its `db>` prompt (Chapter 29).

EXIT STATUS

`usage` with no arguments.

SEE ALSO

`as`, `dis`, `ps`; Chapter 29.

df

A program: lists the disks, with their sizes and the space free.

SYNTAX

```
df
```

DESCRIPTION

One line is printed for each disk the storage driver has started, under a heading: its name (the name it has under `/sd`, or the letter of a built-in disk), its kind, the size of its file system, how much of that is free, and its label. A disk that has no file system is listed with its own size.

Arguments are ignored.

EXIT STATUS

Empty.

SEE ALSO

`du`, `fsck`, `label`, `mkfs`; Chapter 14.

EXAMPLE

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      13 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

dis

A program: the disassembler.

SYNTAX

```
dis [-cnw] [-l labels] [-o addr] file
```

ARGUMENTS

file
: A program, as `as` makes one, or with `-o` a file of raw bytes.

-l *labels*
: Use the names in a labels file (`as -l`'s) for the addresses they belong to.

-n
: Don't name the system calls and the call registers: show their addresses.

-w
: Wide: set the instructions out in the columns the system's own sources use.

-c
: Treat every byte as code. Without it, `dis` follows the program from its entry and takes what is never reached as data.

-o *addr*
: The file has no program header: it is bytes to be loaded at *addr*, given in hex.

DESCRIPTION

The program is written to standard output as assembly language that `as` accepts: `dis prog | as - copy` makes a copy that is byte for byte the same (Chapter 28).

EXIT STATUS

Empty on success. `1` if the file can't be read, and for bad arguments, after the usage line.

SEE ALSO

`as`, `db`, `xd`; Chapter 28.

du

A program: reports how much space files take.

SYNTAX

```
du [-a] [name ...]
```

ARGUMENTS

name

A file or a directory. The current directory if none is given.

-a

Print a line for every file as well as for every directory.

DESCRIPTION

For each name, `du` adds up the lengths of everything in its tree, each file's length rounded up to a whole kilobyte, and prints a line for each directory with the total of what is in and under it: the number of kilobytes, a tab, and the path. The deepest directories come first and the name itself last. A name that is a file gets its own line.

The figure is the files' lengths, not the space the file system gives them, which is in whole clusters and is a little more.

EXIT STATUS

Empty on success. `1` if a name isn't there. `usage` for an unknown option.

SEE ALSO

`df`, `ls`.

EXAMPLE

```
% echo 12345 >s1; mkdir t; echo x >t/s2
% du -a s1 t
1       s1
1       t/s2
1       t
```

echo

A program: prints its arguments.

SYNTAX

```
echo [-n] [word ...]
```

ARGUMENTS

-n

Don't end the line.

DESCRIPTION

The words are written to standard output with one space between each, and then a line end. The spaces are `echo`'s: however many there were between the words on the command line, the shell has already made them separate arguments. To keep spaces, quote them.

`-n` is the only option, and only as the first argument. Anything else that starts with – is a word and is printed.

The whole line is written in one piece (up to 256 bytes at a time), so two programs writing lines to the same place don't mix their lines' halves.

EXIT STATUS

Empty on success. `write error` if the output can't be written.

SEE ALSO

`cat`; section 8.3.

EXAMPLE

```
/> echo a  b   c; echo -n x; echo y; echo -x z
a b c
xy
-x z
```

ed

A program: the line editor.

SYNTAX

```
ed [file]
```

ARGUMENTS

file

The file to edit. One that doesn't exist is not an error: `ed` says `new file` and starts with nothing, and the file is made when the text is first written with `w`.

DESCRIPTION

`ed` holds the file's lines in memory and changes them by commands typed at its `*` prompt, each naming the lines it is about by their numbers. It needs nothing of the terminal but lines in and lines out, so it works over any connection, in a script, and with its input from a pipe. Its commands are in Chapter 11.

Nothing is written to the file until a `w` command.

SEE ALSO

`edit`; Chapter 11, under `ed`.

edit

A program: the screen editor.

SYNTAX

```
edit [+n] [file ...]
```

ARGUMENTS

file
: A file to edit. Several can be given, each in a buffer of its own; one that doesn't exist starts empty. With none, `edit` starts with an empty, unnamed buffer.

+n
: Start with the cursor on line *n*.

DESCRIPTION

`edit` shows the text on the whole screen and the cursor moves through it; what is typed goes in where the cursor is. Its commands are control keys, listed along the bottom of its screen, in the manner of `nano` (Chapter 11). It needs a terminal that understands the cursor's escape sequences (section 13.7), which HydraOS's own screen and any terminal emulator do.

The text is kept in the task's RAM banks, so a file's size is limited by those, not by the 32K of ordinary memory.

HyBASIC's `EDIT` command runs this program on the program in memory.

SEE ALSO

`ed`, `more`; Chapter 11, under `edit`.

forth

A program: HyForth.

SYNTAX

```
forth [-l] [file [argument ...]]
```

ARGUMENTS

file
: A file of Forth to run. When it has been read to its end, HyForth ends. The arguments after it are the script's own (`argc` and `arg`, section 20.9).

-l
: Start as a login shell: build the namespace and run the profile, `/lib/forth/profile.fs`, first (section 8.16).

DESCRIPTION

With no file, HyForth starts at its prompt, which shows the current directory. It is the shell a Hydra starts with: besides Forth's words it runs programs by name (section 8.15). `bye` leaves it.

EXIT STATUS

`1` if the file can't be read.

SEE ALSO

Chapter 4 of this book; Chapters 19 and 20.

free

A program: reports how much memory there is.

SYNTAX

```
free
```

DESCRIPTION

Two lines are printed. The first is the RAM every task has for itself in banks: 128K for each RAM module fitted. The second is the shared RAM: its size, how much of it is in shared segments and how many of those there are, and how much is free.

The figures are for the machine, not for one program. To see what a task is using, look in `/proc` (section 17.8).

Arguments are ignored.

EXIT STATUS

Empty.

SEE ALSO

`ps`, `df`; Appendix F.

EXAMPLE

```
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

fsck

A program: checks a disk's file system, and can repair it.

SYNTAX

```
fsck [-f] disk
```

ARGUMENTS

disk

The disk's name: the name it has under `/sd`, or the letter of a built-in disk (`r` is the RAM disk).

`-f`

Fix what is found wrong.

DESCRIPTION

The check is done by the storage driver, which `fsck` asks through the disk's control file. It reads every directory and every file's list of clusters, and compares what the files use with the free map. It then prints the disk's label, its free space, and what it found:

- **lost**: clusters marked as used that no file has. They are wasted space.
- **unmarked**: clusters a file has that are marked free. Something else could be given them.
- **twice**: clusters that two files both claim.

All three are 0 on a sound disk. `-f` puts the free map right, which cures the first two.

EXIT STATUS

`1` if there is no such disk. `usage` with no disk.

SEE ALSO

`df`, `mkfs`, `label`; section 14.6.

EXAMPLE

```
/> fsck r
hydrafs label=RAM
free 253 KB of 255 KB
check: lost 0, unmarked 0, twice 0
banks $00-$1f
```

grep

A program: prints the lines that match a pattern.

SYNTAX

```
grep [-chilnsv] [-e] pattern [file ...]
```

ARGUMENTS

pattern

A regular expression.

file

The files to search. Standard input if there are none.

OPTION	EFFECT
`-c`	Print a count of the matching lines, not the lines
`-h`	Don't put the file's name before each line
`-i`	Treat capitals and small letters as the same
`-l`	Print only the names of the files that have a matching line
`-n`	Put each line's number before it
`-s`	Say nothing about files that can't be read
`-v`	Print the lines that don't match
`-e`	The next argument is the pattern, even if it starts with -

DESCRIPTION

Each line of each file is tested, and printed if some part of it matches the pattern. When more than one file is named, each line printed has its file's name and a colon before it.

The pattern's language:

PATTERN	MATCHES
c	The character *c*, if it is not one of those below
`\`*c*	The character *c*, whatever it is
`.`	Any one character
`[abc]`, `[a-z]`	One character of those listed, or in the range
`[^abc]`	One character not listed
`^`, `$`	The start of the line, and its end
e`*`	*e* any number of times, none included
e`+`	*e* once or more
e`?`	*e* once or not at all
e1`|`*e2*	Either
`(`*e*`)`	*e*, as a group

A pattern matches anywhere in a line unless `^` and `$` tie it down. The shell reads the command line first, so a pattern with any of rc's own characters in it goes in quotes.

EXIT STATUS

Empty if a line matched. `no matches` if none did. `1` if a file could not be read. `bad expression`, after a message saying what is wrong, for a pattern that isn't one. `usage` for bad arguments.

A script can therefore test for a match directly: `if(grep -s pattern file >/dev/null) ...`.

SEE ALSO

`sort`, `uniq`, `wc`.

EXAMPLE

```
% echo one >a; echo two >b
% grep o a b; grep -c o a; grep -l t a b
a:one
b:two
1
b
% grep z a; echo status $status
status no matches
% grep '[' a
grep: bad expression: no ]
```

head

A program: prints the first lines of files.

SYNTAX

```
head [-N] [file ...]
```

ARGUMENTS

-N

How many lines: a number. 10 if it is not given.

DESCRIPTION

The first *N* lines of each file are written to standard output, or all of a file that has fewer. With no files, standard input's. Several files are printed one after another with nothing between them.

EXIT STATUS

Empty on success. `1` if a file can't be read. `usage` for an option that isn't a number.

SEE ALSO

`tail`, `more`, `cat`.

hello

A program: a test program, which greets its arguments.

SYNTAX

```
hello [word ...]
```

DESCRIPTION

`hello` is the program `init` runs first when the system starts, to show that programs load and run. It prints `hello,` and its first argument, and ends on purpose with the exit code 7 and the status `bye`.

EXIT STATUS

`bye`, always.

hwtest

A program: restarts the Hydra into its hardware test.

SYNTAX

```
hwtest
```

DESCRIPTION

The machine is reset and the hardware test, which is in the ROM, runs in place of HydraOS (section 5.7). Everything that was running stops, as at any reset.

SEE ALSO

Section 5.7.

hylang

A program: hylang.

SYNTAX

```
hylang [-l] [file]
```

ARGUMENTS

file
: A file of hylang to run. When it has been read to its end, hylang ends.

-l
: Start as a login shell: build the namespace and run the profile, `/lib/hylang/profile.hl`, first (section 8.16).

DESCRIPTION

With no file, hylang starts at its `hylang>` prompt, where each expression typed is evaluated and its value printed after `=>`. `exit` leaves it.

EXIT STATUS

`1` if the file can't be read.

SEE ALSO

Chapter 5 of this book; Chapters 21 and 22.

kill

A program: ends tasks, by their numbers.

SYNTAX

```
kill [-i] task ...
```

ARGUMENTS

task
: A task's number, as `ps` shows it.

`-i`
: Interrupt the task instead of ending it.

DESCRIPTION

Each task is sent a note. Without `-i` it is the kill note, which no program can catch or ignore: the task ends at once, its files are closed and its memory is freed. With `-i` it is the interrupt note, the one CTRL-C sends: a program that handles it can tidy up, or carry on.

EXIT STATUS

Empty if every task was sent its note. `1` if any is not a task: `no such task`, or `invalid argument` for something that is not a number. `usage` with no task.

SEE ALSO

`slay`, `ps`.

EXAMPLE

```
% sleep 30 &
% kill $apid; echo status $status
status
% kill 99
kill: 99: no such task
```

label

A program: shows or sets a disk's label.

SYNTAX

```
label disk [word ...]
```

ARGUMENTS

disk
: The disk's name: the name it has under `/sd`, or the letter of a built-in disk.

word
: The new label: the words, joined with one space between each.

DESCRIPTION

With no words, the disk's label is printed. With words, it is set. The label is a name kept in the file system itself, which `df` shows; it travels with a card.

EXIT STATUS

Empty on success. `1` if the disk isn't there or can't be written. `usage` with no disk.

SEE ALSO

`df`, `mkfs`.

ls

A program: lists files.

SYNTAX

```
ls [-ld] [name ...]
```

ARGUMENTS

name
: A file or a directory. The current directory if none is given.

`-l`
: The long form: a line of details for each.

`-d`
: List a directory itself, not what is in it.

DESCRIPTION

For a directory, each entry in it is listed, one to a line, in the directory's own order, which is the order the entries were made in and not alphabetical. A directory's name has a `/` after it. For a file, its own name is listed.

The long form's line has five parts: the mode, the device, the length in bytes, the date and time the file was last changed, and the name.

```
d-rw-rw-rw- fr        0 2000-01-01 00:00 lib
```

The mode's first character is `d` for a directory and `-` otherwise; the second is `a` for an append-only file. Then come three groups of `r`, `w` and `x`, for reading, writing and running, as Plan 9 lists them; HydraOS has no users, so the three groups are always alike. A file that can't be written, such as anything on the ROM disk, has no `w`. The device is the letter of the driver the file belongs to, and the letter or digit of its disk if it has one: `fr` is the file system on the RAM disk, `fx` on the ROM disk, and `c-` the console.

EXIT STATUS

Empty on success. `1` if a name isn't there; the others are still listed. `usage` for an unknown option.

SEE ALSO

`du`, `df`, `pwd`.

EXAMPLE

```
% echo one >a; mkdir d
% ls
bin/
lib/
a
d/
% ls -ld d nothere
d-rw-rw-rw- fr        0 2000-01-01 00:00 d
ls: nothere: not found
```

mkdir

A program: makes directories.

SYNTAX

```
mkdir [-p] dir ...
```

ARGUMENTS

`-p`

Make the directories on the way too, as needed, and don't complain about one that exists.

DESCRIPTION

Each directory is made, empty. Without `-p`, the directory it goes in must exist, and the directory itself must not.

EXIT STATUS

Empty on success. `1` if any could not be made: `already exists`, `not found` (the directory it would go in), `read-only`.

SEE ALSO

`rmdir`, `rm`, `ls`.

EXAMPLE

```
% mkdir d; mkdir d; echo status $status
mkdir: d: already exists
status 1
% mkdir p/q
mkdir: p/q: not found
% mkdir -p d p/q; ls p
q/
```

mkfs

A program: makes a new, empty file system on a disk.

SYNTAX

```
mkfs [-fp] disk [word ...]
```

ARGUMENTS

disk
: The disk's name: the name it has under `/sd`, or the letter of a built-in disk.

word
: The new file system's label.

`-f`
: A full format: write the whole of the free map. Without it only what is needed is written, which is much quicker on a large card.

`-p`
: Make the file system in a new partition of its own, after the partitions a card already has, leaving them alone.

DESCRIPTION

Everything on the disk is lost. `mkfs` does not ask first. The disk gets a HydraFS file system with nothing in it, which the storage driver makes.

EXIT STATUS

Empty on success. `1` if there is no such disk, or it can't be done. `usage` with no disk.

SEE ALSO

`fsck`, `label`, `df`; Chapter 14.

mods

A program: lists the modules in the paged ROM.

SYNTAX

```
mods
```

DESCRIPTION

One line is printed for each module, under a heading: the ROM bank or banks it is in, its type, and its name. A module that shares a bank with others is shown as the bank, a `+`, and the page in the bank where it starts.

The types are `program` (run by name), `driver`, `library` (code that programs call), and `boot`: a driver that the kernel starts itself as the system comes up.

Programs that are not modules, the ones in `/rom/bin`, are files on the ROM disk and are not listed here: `ls /rom/bin` shows those.

EXIT STATUS

Empty.

SEE ALSO

`ps`; section 17.10.

more

A program: shows text a screen at a time.

SYNTAX

```
more [file ...]
```

DESCRIPTION

Each file, or standard input if there are none, is shown 22 lines at a time. After each 22 lines `more` prints `--more--` and waits for a line from the console: Return alone shows the next 22, and `q` then Return stops.

The answer is read from the console itself, not from standard input, so `more` works at the end of a pipe: `ls /rom/bin | more`.

Text that ends before 22 lines is printed and `more` ends without asking.

EXIT STATUS

Empty on success. `1` if a file can't be read.

SEE ALSO

`head`, `tail`, `cat`, `edit`.

mv

A program: moves or renames files.

SYNTAX

```
mv from to
mv from ... dir
```

ARGUMENTS

from

A file or directory to move. There can be several if the last name is a directory.

to

The new name. If it is an existing directory, *from* goes into it under its own name.

DESCRIPTION

Within one directory, a move is a rename: the entry's name is changed, and nothing is copied. A file already at the new name is removed first. Files and directories can both be renamed.

To another directory, a file is copied there and the original is then removed. A directory can't be moved to another directory: `mv` refuses, as Plan 9's does. Copy it with `cp -r` and remove the original with `rm -r`.

Moving a file to itself does nothing.

EXIT STATUS

Empty on success. `1` if anything could not be moved. `usage` with fewer than two names.

ERRORS

Each is printed as `mv: name: reason`.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and *to* is in another directory
`read-only`	A disk that can't be written

SEE ALSO

`cp`, `rm`.

EXAMPLE

```
% echo one >a; echo two >b; mkdir d e
% mv a c; mv c b; cat b
one
% mv b d; ls d
b
% mv d e; echo status $status
mv: d: is a directory
status 1
```

new-window

A program: opens a window and runs a command in it.

SYNTAX

```
new-window [-g] [command [argument ...]]
```

ARGUMENTS

command

What to run in the window. With none, a shell: the one the system starts in each window.

-g

Put the new window in a group of its own. Without it, the window joins the group of the window the command was typed in (Chapter 10).

DESCRIPTION

A new window is made and shown, and the command is started in it with the window as its standard input, output and error. The command is run by a login shell (`rc -l -c`), so it has the default namespace and the profile's settings, and `$window` in its environment is the new window's number.

`new-window` does not wait: it ends as soon as the command has been started, and the prompt returns in the old window. The new window closes by itself when the command ends.

The command has a note group of its own, so CTRL-C typed in one window doesn't reach the programs of another.

SEE ALSO

Chapter 10; section 13.5.

ns

A program: shows a task's namespace.

SYNTAX

```
ns [task]
```

ARGUMENTS

task

A task's number. This shell's own task if it is left out.

DESCRIPTION

The namespace is printed as the `bind` and `mount` commands that would build it, in the order they were made (section 9.8). It is read from the task's file in `/proc`.

EXIT STATUS

Empty on success. `usage` for more than one argument, or a number that is no task's.

SEE ALSO

`bind`, `mount`, `newns`; Chapter 9.

EXAMPLE

```
/> ns | head -3
bind '#/' /
bind '#/dev' /dev
bind -a '#n' /dev
```

play

A program: plays a song, or a line of music.

SYNTAX

```
play [-l] song [n]
play -o score.mml song.zsm
play [-lx] -m channel mml ...
play [-lx] -c channel note ...
```

ARGUMENTS

song
A ZSM file, a score in MML (a name ending `.mml`), or on a Vera X a WAV file.

n
Play the song's repeating part this many more times.

-l
Play again and again, until CTRL-C.

-o
Don't play: turn the MML score into a ZSM file.

-m *channel*
Play the words that follow as one line of MML on that channel.

-c *channel*
Play the notes that follow as a chord: one on each channel from *channel* on.

-x
Read the MML in the Commander X16's dialect.

DESCRIPTION

`play` reads the music and sends the sound driver its notes at the times they fall due, and ends when the song does. It is the program HyBASIC's `PLAY` statement runs. Chapter 15 describes the sound channels and MML.

EXIT STATUS

Empty when the song has been played. `1` if the file can't be read or the music has a mistake in it, which is reported. `usage` for bad arguments.

SEE ALSO

Chapter 15.

ps

A program: lists the tasks.

SYNTAX

```
ps [-a]
```

ARGUMENTS

`-a`

Show each program's arguments after its name.

DESCRIPTION

One line is printed for each task in use, under a heading:

COLUMN	MEANING
`task`	The task's number, 0 to 15
`state`	`ready` to run; `wait`ing for a child to end; `sleep`ing; waiting for an `event`; or `idle`, a driver with nothing asked of it
`parent`	The task that started it; – for those the kernel started
`cpu`	The processor time it has had, in seconds to a tenth
`group`	Its note group: the tasks a CTRL-C reaches together
`name`	The program it is running

Task 0 is the kernel. The drivers are the tasks at the top of the range, and the programs a user starts are given the lowest free numbers.

Arguments other than `-a` are ignored.

EXIT STATUS

Empty.

SEE ALSO

`top`, `kill`, `slay`, `ns`; section 17.8.

pwd

A program: prints the current directory.

SYNTAX

```
pwd
```

DESCRIPTION

The current directory's full path is written to standard output, with a line end. Arguments are ignored.

A program's current directory is its own, copied from the program that started it; `pwd` shows the one it was started with, which is the shell's.

EXIT STATUS

Empty.

SEE ALSO

`cd`, `ls`.

---

rc

A program: the shell.

SYNTAX

```
rc [-l] [-i] [file [argument ...]]
rc [-l] -c commands
```

ARGUMENTS

file

A script: a file of commands to run. The arguments after it are the script's, `$1`, `$2` and so on, with the file's name as `$0`.

-c *commands*

Run the commands in this one argument, then end.

-l

A login shell: build the default namespace and run the profile, `/rom/lib/profile`, before anything else.

-i

Interactive: print the prompt and carry on after errors, even when the commands come from a file.

DESCRIPTION

With no file and no `-c`, rc reads commands from standard input, printing its prompt first if that is a console. It ends at the end of its input (CTRL-D at the keyboard) or at `exit`.

rc's language is described in Chapter 8, and its own commands at the end of this chapter.

EXIT STATUS

The status of the last command it ran, or the status given to `exit`. `1` if the file can't be read.

SEE ALSO

`forth`, `hylang`, `basic`; Chapter 8.

EXAMPLE

```
/> rc -c 'echo one; exit 3'
one
```

rm

A program: removes files and directories.

SYNTAX

```
rm [-rf] name ...
```

ARGUMENTS

-r

Remove a directory and everything in it, to any depth.

-f

Say nothing about what can't be removed, and don't count it a failure.

DESCRIPTION

Each name is removed. A directory is removed only if it is empty, unless `-r` is given, in which case what is in it is removed first, from the bottom up.

There is no question first and nothing can be brought back.

A file that is open, in any task, is not removed: the system refuses.

EXIT STATUS

Empty on success. `1` if anything could not be removed and `-f` was not given. `usage` for an unknown option.

ERRORS

Each is printed as `rm: name: reason`.

REASON	CAUSE
`not found`	There is no such name
`directory not empty`	A directory with something in it, and no `-r`
`read-only`	The file's disk can't be written
`busy`	The file is open

SEE ALSO

`rmdir`, `mv`, `cp`.

EXAMPLE

```
% mkdir d; echo x >d/f
% rm d; echo status $status
rm: d: directory not empty
status 1
% rm -r d; rm nothere; rm -f nothere; echo status $status
rm: nothere: not found
status
```

rmdir

A program: removes empty directories.

SYNTAX

```
rmdir dir ...
```

DESCRIPTION

Each directory is removed, if it has nothing in it. Unlike `rm`, it will not remove a file, which makes it the safe way to say "this should be an empty directory".

EXIT STATUS

Empty on success. `1` if any could not be removed: `directory not empty`, `not a directory`, `not found`.

SEE ALSO

`rm`, `mkdir`.

slay

A program: ends tasks, by the name of the program they are running.

SYNTAX

```
slay [-i] name ...
```

ARGUMENTS

name

A program's name, as `ps` shows it.

`-i`

Interrupt the tasks instead of ending them.

DESCRIPTION

Every task running a program of that name is sent the kill note, or with `-i` the interrupt note, as `kill` sends them. `slay` never ends itself.

EXIT STATUS

Empty if each name matched a task. `1` if a name matched none: `no such task`. `usage` with no name.

SEE ALSO

`kill`, `ps`.

EXAMPLE

```
% sleep 30 &
% slay sleep; echo status $status
status
% slay sleep
slay: sleep: no such task
```

sleep

A program: waits.

SYNTAX

```
sleep seconds
```

ARGUMENTS

seconds

A whole number of seconds.

DESCRIPTION

`sleep` does nothing for that long, and ends. It uses no processor time while it waits. CTRL-C ends it sooner.

EXIT STATUS

Empty when the time is up. `usage` if the argument is missing or is not a whole number: `sleep 0.5` is not accepted.

SEE ALSO

`date`.

sort

A program: sorts lines.

SYNTAX

```
sort [-bfnru] [file ...]
```

ARGUMENTS

OPTION	EFFECT
`-b`	Ignore spaces and tabs at the start of each line when comparing
`-f`	Fold: treat capitals and small letters as the same
`-n`	Compare the numbers that the lines start with, not their text
`-r`	Reverse the order
`-u`	Unique: print one of each set of lines that compare equal

DESCRIPTION

All the lines of all the files (or of standard input) are read into memory, sorted, and written to standard output. The order is that of the bytes' values, so every capital comes before every small letter, and digits before both, unless `-f` is given.

With `-n`, a line's number may have a sign and a decimal point; lines are in the order of their numbers' values, so 9 comes before 10, where plain sorting puts `10` first.

EXIT STATUS

Empty on success. `1` if a file can't be read; the others are still sorted. `usage` for an unknown option.

NOTES

The text must fit in memory: `sort` is for files of a size a person would edit, not for megabytes.

SEE ALSO

`uniq`, `grep`.

EXAMPLE

```
% echo 10 >n; echo 9 >>n; echo -5 >>n; echo 2.5 >>n
% sort n | tail -2; sort -n n | tail -2
2.5
9
9
10
```

tail

A program: prints the last lines of a file.

SYNTAX

```
tail [-N] [file]
```

ARGUMENTS

-N

How many lines: a number. 10 if it is not given.

file

One file. Standard input if it is left out.

DESCRIPTION

The file's last *N* lines are written to standard output, or all of it if it has fewer.

The file is read through to its end, with the most recent 16K of it kept in memory, and the lines are found there. What `tail` prints can therefore never be more than the last 16K of the file, however many lines are asked for.

EXIT STATUS

Empty on success. `1` if the file can't be read. `usage` for more than one file, or an option that isn't a number.

SEE ALSO

`head`, `more`.

tee

A program: copies its input to files as well as to its output.

SYNTAX

```
tee [-a] [file ...]
```

ARGUMENTS

-a

Add to the end of each file. Without it, each file is made, or emptied if it exists.

DESCRIPTION

Standard input is copied to standard output and, at the same time, to each file. It is for the middle of a pipe, to keep a copy of what is passing: `ls -l | tee listing | wc -l`.

A file that can't be opened or written is reported, and `tee` goes on with the others and with its output.

EXIT STATUS

Empty on success. `1` if any file could not be written.

SEE ALSO

`cat`.

EXAMPLE

```
% echo x | tee t1 /rom/no; echo status $status; cat t1
tee: /rom/no: read-only
x
status 1
x
```

top

A program: shows which tasks are using the processor.

SYNTAX

```
top
```

DESCRIPTION

Once a second, `top` clears the screen and lists the tasks, each with its number, its state, its name, and the share of the processor it took in that second, as a percentage. It goes on until CTRL-C.

The kernel's share, task 0's, includes the time when nothing needed the processor.

SEE ALSO

`ps`.

touch

A program: makes empty files, or marks files as just changed.

SYNTAX

```
touch name ...
```

DESCRIPTION

A name that doesn't exist is made, as an empty file. A name that does exist has its time of last change set to now, and nothing else about it altered.

EXIT STATUS

Empty on success. `1` if any could not be made or changed.

SEE ALSO

`ls`, `date`.

uniq

A program: removes repeated lines that are next to each other.

SYNTAX

```
uniq [-c] [file]
```

ARGUMENTS

`-c`
: Put before each line the number of times it came in a row.

DESCRIPTION

The file, or standard input, is copied to standard output with each run of identical lines shown once. Only lines that follow one another are compared: the same line appearing again later, after a different one, is printed again. To remove every repeat, sort first: `sort file | uniq`.

A line is compared as its first 255 bytes; a longer line is treated as more than one.

EXIT STATUS

Empty on success. `1` if the file can't be read. `usage` for more than one file or an unknown option.

SEE ALSO

`sort`, `wc`.

EXAMPLE

```
% echo a >u; echo a >>u; echo b >>u; echo a >>u
% uniq -c u
      2 a
      1 b
      1 a
```

WC

A program: counts lines, words and bytes.

SYNTAX

```
wc [-lwc] [file ...]
```

ARGUMENTS

`-l, -w, -c`
: Print only the lines, the words, or the bytes. They can be combined, and the counts always come in that order, whatever order the letters are in.

DESCRIPTION

For each file, or for standard input, `wc` prints its number of lines, of words and of bytes, each in a column seven characters wide, and then the file's name. With more than one file, a line of totals follows.

A line is counted at each line end, so a last line with no line end is not counted. A word is a run of anything but spaces, tabs and line ends.

EXIT STATUS

Empty on success. `1` if a file can't be read. `usage` for an unknown option.

SEE ALSO

`grep`, `sort`, `uniq`.

EXAMPLE

```
% echo one two three >a; echo -n 'no newline' >c
% wc a c
      1       3      14 a
      0       2      10 c
      1       5      24 total
% wc -l <a
      1
```

xd

A program: shows a file's bytes in hexadecimal.

SYNTAX

```
xd [file ...]
```

DESCRIPTION

Each file, or standard input, is printed 16 bytes to a line: the offset of the line's first byte, in hex; the bytes, each as two hex digits; and the same bytes as text, with a `.` for any that isn't a printable character. Each file's offsets start again at 0.

EXIT STATUS

Empty on success. `1` if a file can't be read.

SEE ALSO

`cat`, `cmp`, `dis`.

EXAMPLE

```
/> echo hello | xd
0000000  68 65 6c 6c 6f 0a                                 hello.
```

xmodem

A program: sends or receives a file over the serial port, by the XMODEM protocol.

SYNTAX

```
xmodem -r file
xmodem -s [-k] file
```

ARGUMENTS

-r file

Receive: the terminal program at the other end sends, and what arrives is written to *file*.

`-s` *`file`*
Send *file* to the terminal program.

`-k`
Send in blocks of 1K, with a CRC. Without it, in blocks of 128 bytes.

DESCRIPTION

XMODEM is the simplest of the file transfer protocols that terminal programs offer, and any of them can be the other end. Start `xmodem` on the Hydra, then start the transfer in the terminal program within a minute.

While the transfer runs, the serial line carries nothing else: the console holds back the screen's output until it is over, and keys are not acted on.

Receiving, `xmodem` asks first for a CRC and, if the sender doesn't answer to that, for a plain checksum; it takes blocks of 128 bytes or of 1K as they are sent. Each block is acknowledged if it arrived whole and asked for again if it didn't. XMODEM pads the last block; the padding bytes are not written to the file.

When it has finished it prints the file's name and its length.

EXIT STATUS

Empty when the file has been transferred. `1` otherwise, after a message: `too many errors` (a block failed ten times), `cancelled` (the other end, or CTRL-C before the first block), `no answer` (nothing for a minute). `usage` for bad arguments.

NOTES

`/pc` (section 9.9) is a far easier way to move files to and from a PC that runs the Hydra's own PC tool. `xmodem` is for when there is only an ordinary terminal program.

SEE ALSO

Sections 9.9 and 14.7.

The system's own programs

One program is started by the system itself, and is not meant to be typed.

`init`
The first program, and the first program of every window. The kernel starts it as the system comes up; again for each window the user opens (CTRL-] C), with the window's number as its argument; and again whenever the first window's shell ends. It puts its standard input, output and error on its window's console, builds its namespace, and then becomes the shell: it runs the shell in its own task, in its own place, as `exec` does, so that no task is left behind only to wait for the shell. The shell is the one `/lib/shell` names (`/bin/forth -l` in the ROM's), or `rc -l` with no such file. The first time, as the system starts, it also starts the RAM disks, sets the windows up from `/lib/windows`, lists the tasks and runs `hello` (section 7.4).

Because no task waits for a shell, a task whose parent has ended is simply let go by the kernel when it ends itself: there is nothing for it to be handed to.

The keyboard and mouse of a Vera X card, the pins and the I²C bus are all served by `kdev`, the kernel's own device driver, and are not programs or tasks of their own.

The shell's own commands

These are built into rc. Each changes something about the shell itself, its directory, its variables or its namespace, which a separate program could not do: a program is another task, and changes only its own.

A built-in command's failure sets `$status` to the reason, and prints it as `rc: command: reason`.

. (dot)

```
. file [argument ...]
```

Reads *file* and runs its commands in this shell, as if they had been typed. Unlike running the file as a script, which starts another rc, anything the commands set stays set here: variables, functions, the current directory. Inside the file, `$1`, `$2` and `$*` are the arguments given.

Status: that of the file's last command; `not found` if there is no such file.

bind

```
bind [-a | -b] [-c] new old
```

Makes the directory or file *new* appear at the name *old* in this shell's namespace (Chapter 9). With no option *new* replaces what was at *old*; `-a` adds it after what is there, making a union directory, and `-b` before. `-c` lets files be created in it through the union.

Status: `not found` if either name doesn't exist.

builtin

```
builtin command [argument ...]
```

Runs *command* as rc's built-in of that name, or as a program, passing over any function of the same name. It is how a function that wraps a command calls the real one: `fn cd { builtin cd $*; echo now in` {pwd} }`.

cd

```
cd [dir]
```

Makes *dir* the shell's current directory. With no *dir*, it goes to the directory named by `$home`, or to `/` if `$home` isn't set. Every program the shell starts afterwards begins there.

Status: `not found`; `not a directory`.

eval

```
eval [word ...]
```

Joins its words with spaces and runs the result as a command line. The line is read again from scratch, so variables and quoting in the words are interpreted a second time: `eval echo '$'$v` prints the value of the variable whose name is in `v`. With no words it does nothing.

exec

```
exec command [argument ...]
```

Runs *command* in place of the shell. The shell does not start another task and wait: its own task becomes the program, with the same task number, files and environment, and there is no shell left to return to. Whatever started the shell sees the program's status as the shell's.

exit

```
exit [status ...]
```

Ends the shell. The words given are its exit status; with none, the status is that of the last command.

mount

```
mount [-a | -b] [-c] '#x' old [spec]
```

Attaches a driver's files, named by its device letter, at *old* in this shell's namespace. *spec* chooses among what the driver has: which disk, for the file system's driver. The options are `bind`'s. The device name is quoted because `#` starts a comment in rc.

Status: `no such device` if there is no such driver; `not found` if *old* doesn't exist.

newns

```
newns
```

Throws away this shell's namespace and builds the default one again, from `/lib/namespace`, as a login shell does when it starts. It undoes every `bind`, `mount` and `unmount` the shell has made.

The default namespace gives a task its own area of the RAM disk as `/ram`, named for the task's number, and building it empties that area. So in a shell that the system started, the first shell of a window, `newns` removes every file in its `/ram`, `/ram/bin` and `/ram/lib` included. In a shell started from another shell, which until then shared its parent's `/ram`, the parent's files are untouched: the shell simply has a new, empty `/ram` of its own from then on.

shift

```
shift [n]
```

Drops the first *n* words of `$*`, the arguments of the script or function it is used in, so that `$2` becomes `$1`. *n* is 1 if it is left out. Shifting by more than there are leaves none.

unmount

```
unmount [new] old
```

Takes away what was bound or mounted at *old*. With *new*, only that one member of a union directory is taken out, and the others stay.

Status: `not found` if nothing is bound there.

wait

```
wait [task]
```

Waits for a background command to end. With a task's number, such as `$apid`, it waits for that one; with none, for all of them. A number that is not one of the shell's own children is not an error: there is nothing to wait for.

whatis

```
whatis name ...
```

Says what each name means to the shell, in a form that could be typed back in: a variable as its assignment, a function as its definition, a built-in as `builtin name`, and a program as the full path it would be run from. A name that means nothing prints nothing.

CHAPTER 3

System Calls

A system call is a `jsr` to a fixed address in the jump table at `$F800`. Its arguments go in A, X, Y and the call registers r0 to r15 (two bytes each, in the zero page from `$02`); its results come back in the same places, a 16-bit result in A (low byte) and X (high byte). On return, C = 0 means success. C = 1 means failure, and A holds an error code (Appendix G). A call may change A, X, Y, the flags and r0 to r15, including registers it doesn't use for a result; it never changes the program's own zero page, `$22` to `$7F`.

A call that *can wait* may suspend the task until something happens elsewhere: a key, a disk, another task. A note delivered to the task ends such a wait, and the call returns `E_INTR`.

Each call's name is defined in `hydra.inc` (`/lib/as`), with the constants and error codes used here. The constants and the error codes are listed at the end of the chapter.

How the entries are made. The kernel is built from a specification that gives, for every call, its registers in and out, the errors it can return and why, whether it can wait, and what it does. The entries of this chapter are made from that same specification, so they describe the calls the kernel has, in the kernel's own terms. Names in `THIS FORM` are constants or other calls. `OPEN` and `READ`, which most programs begin with, are written out at more length.

From the languages. A call that a program can use, as opposed to one meant for drivers or the debugger, can be made from each language:

- **hylang** has a function for it, `sys-` and the call's name in small letters. Each entry gives its arguments. A call that fails is an error carrying the system's message.
- **HyForth**, after `lib hydra`, has a word of the same name. It takes the call's inputs from the stack in the order the entry lists them, the first deepest, and leaves the results, with an *ior* on top if the call can fail. A path is passed as the address of a zero-terminated string, which `>z` makes (section 20.9).
- **HyBASIC** has `SYS "NAME" [, a [, x [, y]]]`, which sets A, X and Y. Arguments that go in the call registers must be put there with `POKE` first (r0 is at addresses 2 and 3, r1 at 4 and 5, and so on), and the results are read with `RREG` and `PEEK`.

The calls by group

GROUP	FROM	CALLS
The system	`$F800`	SYSINFO ERRSTR KMESG REBOOT XCALL
Tasks	`$F830`	EXITS WAIT SPAWN GETPID YIELD SLEEP SLEEP_UNTIL TICKS PREEMPT_OFF PREEMPT_ON TASKINFO NOTIFY NOTE MODINFO ENV_GET ENV_PUT ENV_DEL ENV_NAME TASKREAD GETPPID SEM_NEW SEM_ACQUIRE SEM_TRY SEM_RELEASE SEM_FREE EXEC
Memory	`$F890`	BREAK PAGES_ALLOC PAGES_FREE BANKS BANKS_ALLOC BANKS_FREE SEG_CREATE SEG_ATTACH SEG_DETACH SEG_MAP SEGINFO BANKS_ALLOC_IN SEG_CREATE_IN
Files	`$F8C0`	OPEN CREATE CLOSE READ WRITE SEEK STAT FSTAT WSTAT FWSTAT REMOVE DUP DUP2 CHDIR GETCWD PIPE FD2PATH
Namespaces	`$F920`	BIND MOUNT UNMOUNT NSINFO
The console	`$F950`	PUTC PUTS GETC PUTHEX
The clock	`$F980`	TIME TIME_SET RTC
Drivers	`$F9B0`	IRQ_OWN IRQ_RELEASE WAKE PAUSE NOTE_POST SRV_REGISTER SRV_TAKE SRV_REPLY CLIENT_READ CLIENT_WRITE NOTE_QUEUE ROMREAD TASKMEM TASKSTOP TASKSTEP SRV_NAP INIT_RUN
Debugging	`$FA10`	DBG_SCALL DBG_KCOPY DBG_PS

BANKS

System call `$F899`, one of those for memory. It never waits.

ARGUMENTS

None.

RESULT

A

The banks the task has (16 for each good RAM module)

r0

The good modules (bit = module)

DESCRIPTION

The task's own RAM banks: module m's are $m0-$mF, selected by writing `$00` (the X16's way). Modules POST found bad aren't counted.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-banks)`. It returns a list: A, r0.

HyForth: `sys-banks`. HyBASIC: `SYS "BANKS"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

BANKS_ALLOC

System call `$F89C`, one of those for memory. It never waits.

ARGUMENTS

A

Banks.

RESULT

A

The first (for `$00`)

DESCRIPTION

A run of the task's banks, from the lowest. Only bookkeeping, so that the libraries in one program don't use the same banks: a program may use its banks without it.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	0
`$06`	`E_NOMEM`	out of memory	

FROM THE LANGUAGES

hylang: `(sys-banks-alloc A)`, where *A* is a number. It returns A.

HyForth: `sys-banks-alloc`. HyBASIC: `SYS "BANKS_ALLOC"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

BANKS_ALLOC_IN

System call `$F8B1`, one of those for memory. It never waits.

ARGUMENTS

A

How many.

X

The lowest bank it may take.

Y

The highest (module m's are $m0-$mF)

RESULT

A

The first.

DESCRIPTION

`BANKS_ALLOC`, from a range of the task's banks (the RAM disk's start FROM-TO: the modules its banks are on).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	0
`$06`	`E_NOMEM`	out of memory	No run that long there

FROM THE LANGUAGES

hylang: `(sys-banks-alloc-in A X Y)`, where *A* is a number; *X* is a number; *Y* is a number. It returns A.

HyForth: `sys-banks-alloc-in`. HyBASIC: `SYS "BANKS_ALLOC_IN"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `SEG_CREATE_IN`.

BANKS_FREE

System call `$F89F`, one of those for memory. It never waits.

ARGUMENTS

A

The first bank.

X

How many.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Give banks back.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not banks `BANKS_ALLOC` gave this task

FROM THE LANGUAGES

hylang: `(sys-banks-free A X)`, where *A* is a number; *X* is a number. It returns `NIL`.

HyForth: `sys-banks-free`. HyBASIC: `SYS "BANKS_FREE"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

BIND

System call `$F920`, one of those for namespaces. It can wait.

ARGUMENTS

r0
: New (a path)

r1
: Old (a path: absolute)

A
: `MREPL`, `MBEFORE` or `MAFTER`, and `MCREATE`.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Bind new at old: in old's place (`MREPL`), or a union with what's there, new first (`MBEFORE`) or last (`MAFTER`); with `MCREATE`, a `CREATE` in the union comes to new. New is what it is now: a mount point itself is all of its union's members, anything else the first of its candidates that's there. A union's first bind at a directory that's there (and isn't a mount point itself) keeps it as a member.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$02`	`E_INVAL`	invalid argument	
`$50`	`E_NSFULL`	namespace full	
`$2B`	`E_NODEV`	no such device	
`$0A`	`E_NAMETOOLONG`	name too long	

FROM THE LANGUAGES

hylang: `(sys-bind r0 r1 A)`, where *r0* is a string; *r1* is a string; *A* is a number, or left out. It returns `NIL`.

HyForth: `sys-bind`. HyBASIC: `SYS "BIND"`.

SEE ALSO

The other calls for namespaces: `MOUNT`, `UNMOUNT`, `NSINFO`.

BREAK

System call `$F890`, one of those for memory. It never waits.

ARGUMENTS

r0

The new end of the program's data (the first byte past it), or 0 to ask.

RESULT

r0

The end.

DESCRIPTION

Move the end of the program's data area: Unix's brk (BRK is the 65C02's instruction, so the call is `BREAK`). It starts at the top of the module's data and BSS. The top of a task's RAM is `$7FFF` (`$7EFF` in task F).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Below the program's own data
`$06`	`E_NOMEM`	out of memory	Over pages `PAGES_ALLOC` gave, or past the top of RAM

FROM THE LANGUAGES

hylang: `(sys-break r0)`, where *r0* is a number, or left out. It returns r0.

HyForth: `sys-break`. HyBASIC: `SYS "BREAK"`.

SEE ALSO

The other calls for memory: `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

CHDIR

System call `$F8E7`, one of those for files. It can wait.

ARGUMENTS

r0

A directory's path.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Change the current directory: the name a relative path starts from. It's kept whole and clean (`GETCWD`).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$22`	`E_NOTDIR`	not a directory	
`$20`	`E_NOENT`	not found	
`$2B`	`E_NODEV`	no such device	
`$0A`	`E_NAMETOOLONG`	name too long	

FROM THE LANGUAGES

hylang: `(sys-chdir r0)`, where *r0* is a string. It returns `NIL`.

HyForth: `sys-chdir`. HyBASIC: `SYS "CHDIR"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `GETCWD`, `PIPE`, `FD2PATH`.

CLIENT_READ

System call `$F9C8`, one of those for drivers. It never waits.

ARGUMENTS

r0

Here.

r1

There, in the client's view.

r2

A count.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Bytes from the client of the request being served (kcopy).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

CLIENT_WRITE

System call `$F9CB`, one of those for drivers. It never waits.

ARGUMENTS

r0
: Here (the data)

r1
: There, in the client's view.

r2
: A count.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Bytes to the client of the request being served (kcopy).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

CLOSE

System call `$F8C6`, one of those for files. It can wait.

ARGUMENTS

A

An fd.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Close an fd. The last fd naming a channel (`DUP`, `SPAWN` share them) tells its server.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	Not open

FROM THE LANGUAGES

hylang: `(sys-close A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-close`. HyBASIC: `SYS "CLOSE"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

CREATE

System call `$F8C3`, one of those for files. It can wait.

ARGUMENTS

r0

A path.

A

The mode.

X

The new file's mode, high byte (`DM_DIR`: a directory)

RESULT

A

An fd.

DESCRIPTION

Make a file or a directory, and open it.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$2B`	`E_NODEV`	no such device	
`$0A`	`E_NAMETOOLONG`	name too long	
`$26`	`E_MFILE`	no free file descriptor	
`$27`	`E_NFILE`	too many open files	
`$21`	`E_EXIST`	already exists	

FROM THE LANGUAGES

hylang: `(sys-create r0 A X)`, where *r0* is a string; *A* is a number, or left out; *X* is a number, or left out. It returns A.

HyForth: `sys-create`. HyBASIC: `SYS "CREATE"`.

SEE ALSO

The other calls for files: `OPEN`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

DBG_KCOPY

System call `$FA13`, one of those for debugging. It never waits.

ARGUMENTS

A
: Another task.

r0
: An address in this task.

r1
: An address in that task.

r2
: A byte count.

C
: 0 to copy from this task to that one, C = 1 the other way.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Copy bytes between this task's memory and another's (the kernel's kcopy).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$41`	`E_SRCH`	no such task	Not a task

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for debugging: `DBG_SCALL`, `DBG_PS`.

DBG_PS

System call `$FA16`, one of those for debugging. It can wait.

ARGUMENTS

None.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Print a line for each task in use: its number, state, flags, parent, CPU time and name.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for debugging: `DBG_SCALL`, `DBG_KCOPY`.

DBG_SCALL

System call `$FA10`, one of those for debugging. It can wait.

ARGUMENTS

`Y`
: A driver task.

`A, X`
: What its serve entry gets.

RESULT

A, X and C from its serve entry.

DESCRIPTION

Call a driver's serve entry directly, in the driver's task (the file layer will make these calls in phase 2). A driver serves one call at a time: a second caller waits.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$41`	`E_SRCH`	no such task	Not a task
`$2B`	`E_NODEV`	no such device	It's free or ended, or has no serve entry: a program

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for debugging: `DBG_KCOPY`, `DBG_PS`.

DUP

System call `$F8E1`, one of those for files. It never waits.

ARGUMENTS

A

An fd.

RESULT

A

A new fd (the lowest free), on the same channel (the same offset)

DESCRIPTION

Another fd for an open file.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	
`$26`	`E_MFILE`	no free file descriptor	

FROM THE LANGUAGES

hylang: `(sys-dup A)`, where *A* is a number. It returns A.

HyForth: `sys-dup`. HyBASIC: `SYS "DUP"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

DUP2

System call `$F8E4`, one of those for files. It can wait.

ARGUMENTS

A

An fd.

X

Another (0-15)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Make fd X name the same channel as fd A, closing what it named first.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	

FROM THE LANGUAGES

hylang: `(sys-dup2 A X)`, where *A* is a number; *X* is a number. It returns `NIL`.

HyForth: `sys-dup2`. HyBASIC: `SYS "DUP2"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

ENV_DEL

System call `$F860`, one of those for tasks. It never waits.

ARGUMENTS

A

A task (`$FF`: this one)

r0

A variable's name.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Remove a variable.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$41`	`E_SRCH`	no such task	
`$02`	`E_INVAL`	invalid argument	
`$0A`	`E_NAMETOOLONG`	name too long	

FROM THE LANGUAGES

hylang: `(sys-env-del A r0)`, where *A* is a number; *r0* is a string. It returns `NIL`.

HyForth: `sys-env-del`. HyBASIC: `SYS "ENV_DEL"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

ENV_GET

System call `$F85A`, one of those for tasks. It never waits.

ARGUMENTS

A
: A task (`$FF`: this one)

r0
: A variable's name.

r1
: A buffer.

r2
: Its size.

r3
: An offset in the value.

RESULT

A/X
: The value's length from the offset (0 at its end or past it)

As much of it as fits, in the buffer.

DESCRIPTION

A variable of a task's environment (each task has its own: `ENV_MAX` bytes; `SPAWN` copies its parent's). A value is any bytes (rc's lists: each word and a 0). `#e` serves the caller's as files, `/env/NAME`..

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	No such variable
`$41`	`E_SRCH`	no such task	No such task
`$02`	`E_INVAL`	invalid argument	An empty name, or one with a /
`$0A`	`E_NAMETOOLONG`	name too long	

FROM THE LANGUAGES

hylang: `(sys-env-get A r0 size r3)`, where *A* is a number; *r0* is a string; *size* is the most to take: r2; *r3* is a number, or left out. It returns a list: A/X, the first A/X bytes of the buffer, as a string. (Here r1 is a buffer the function makes itself.)

HyForth: `sys-env-get`. HyBASIC: `SYS "ENV_GET"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

ENV_NAME

System call `$F863`, one of those for tasks. It never waits.

ARGUMENTS

A
: A task (`$FF`: this one)

X
: Which variable (0 on)

r0
: A buffer (`ENV_NAME_MAX` + 1 bytes)

RESULT

A/X
: Its value's length.

Its name in the buffer, zero-terminated.

DESCRIPTION

The variables of an environment, one at a time, in the order they were made.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	Past the last
`$41`	`E_SRCH`	no such task	

FROM THE LANGUAGES

hylang: `(sys-env-name A X)`, where *A* is a number; *X* is a number. It returns a list: the buffer's text, A/X. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-env-name`. HyBASIC: `SYS "ENV_NAME"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

ENV_PUT

System call `$F85D`, one of those for tasks. It never waits.

ARGUMENTS

A
: A task (`$FF`: this one)

r0
: A variable's name.

r1
: Bytes.

r2
: Their count.

r3
: Where in its value they go (the value cut there first: 0 for a new value; its length at most)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Set a variable, or make it (offset 0). An offset is for writing a long value in parts.

ERRORS

CODE	NAME	MESSAGE	CAUSE
$06	E_NOMEM	out of memory	No room in the environment: ENV_MAX bytes; or no RAM module for one
$02	E_INVAL	invalid argument	Past the value's end; a bad name
$41	E_SRCH	no such task	
$20	E_NOENT	not found	An offset given, and no such variable
$0A	E_NAMETOOLONG	name too long	

FROM THE LANGUAGES

hylang: `(sys-env-put A r0 r1 r3)`, where *A* is a number; *r0* is a string; *r1* is a string or a list of bytes (its length goes in r2); *r3* is a number, or left out. It returns `NIL`.

HyForth: `sys-env-put`. HyBASIC: `SYS "ENV_PUT"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

ERRSTR

System call `$F803`, one of those for the system. It never waits.

ARGUMENTS

A
: An error code.

r0
: A buffer (32 bytes)

RESULT

The error's text in the buffer, zero-terminated ("error $xx" for an unknown code)

DESCRIPTION

An error code as text, for messages.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-errstr A)`, where *A* is a number. It returns the buffer's text. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-errstr`. HyBASIC: `SYS "ERRSTR"`.

SEE ALSO

The other calls for the system: `SYSINFO`, `KMESG`, `REBOOT`, `XCALL`.

EXEC

System call `$F87B`, one of those for tasks. It can wait.

ARGUMENTS

r0

The program's path (as `SPAWN`'s)

r1

Its arguments (as `SPAWN`'s: 176 bytes at most), or 0.

RESULT

(doesn't return, once the program is found: it runs in this task's place)

DESCRIPTION

Run a program in this task, in place of the one that calls it (Plan 9's exec): the program is found as `SPAWN` finds it (a module of the paged ROM runs in place; any other is read into this task's RAM at its load address), and starts at its entry point with r0 pointing to its arguments, its name the header's. The task keeps what isn't the old program's: its number, its parent (whose `WAIT` gets the new program's exit status), its note group, every fd (a RAM program's file takes fd 15 as it loads), its current directory, its namespace and its environment, and its CPU time. What was the old program's goes as at its end: its IRQ lines, shared segments, semaphores (and the mutexes it holds), and the device letters it served; its note handler; its pages and banks. A note that comes meanwhile waits for the new program. If the program can't be found or isn't one, `EXEC` returns the error and the caller goes on; if it's found but can't be read (a RAM program's file cut short), the task ends with the error as its code. rc runs the last command of rc -c this way, and its exec built-in.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	No such file
`$2D`	`E_NOEXEC`	not a program	Not a program this kernel runs
`$0B`	`E_TOOBIG`	too big	The arguments
`$01`	`E_PERM`	not allowed	A driver
`$05`	`E_INTR`	interrupted	

Also what else `OPEN` and `READ` give.

FROM THE LANGUAGES

hylang: `(sys-exec r0 r1)`, where *r0* is a string; *r1* is a list of strings, or `NIL`. It returns `NIL`.

HyForth: `sys-exec`. HyBASIC: `SYS "EXEC"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`.

EXITS

System call `$F830`, one of those for tasks. It never waits.

ARGUMENTS

A

The exit code (0: success)

r0

A message (zero-terminated, 31 characters at most), or 0 for none.

RESULT

(doesn't return)

DESCRIPTION

End the calling task, with an exit status (Plan 9's exits). Returning from a program's entry point is `EXITS` with code 0 and no message. Everything the task had is released; its parent can `WAIT` for the status.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-exits A r0)`, where *A* is a number, or left out; *r0* is a string, or `NIL`. It returns `NIL`.

HyForth: `sys-exits`. HyBASIC: `SYS "EXITS"`.

SEE ALSO

The other calls for tasks: `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

FD2PATH

System call `$F8F0`, one of those for files. It never waits.

ARGUMENTS

A

An fd.

r0

A buffer (`PATH_MAX` + 1 bytes)

RESULT

The name in the buffer, zero-terminated.

DESCRIPTION

The name an fd's file was opened by (Plan 9's fd2path), whole and clean, as `OPEN` or `CREATE` made it (an fd `SPAWN` or `DUP` gave shares its channel, and its name): the namespace's, or a `#` name; a pipe's `#|/pipe`.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	Not an open fd

FROM THE LANGUAGES

hylang: `(sys-fd2path A)`, where *A* is a number. It returns the buffer's text. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-fd2path`. HyBASIC: `SYS "FD2PATH"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`.

FSTAT

System call `$F8D5`, one of those for files. It can wait.

ARGUMENTS

A

An fd.

r0

A buffer (`SR_SIZE` bytes)

RESULT

The stat record in the buffer.

DESCRIPTION

An open file's stat record.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	

FROM THE LANGUAGES

hylang: `(sys-fstat A)`, where *A* is a number. It returns the stat record, as a hash. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-fstat`. HyBASIC: `SYS "FSTAT"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

FWSTAT

System call `$F8DB`, one of those for files. It can wait.

ARGUMENTS

A

An fd.

r0

A stat record.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Change an open file from a stat record.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	
`$03`	`E_NOSYS`	no such call	

FROM THE LANGUAGES

hylang: `(sys-fwstat A r0)`, where *A* is a number; *r0* is a hash, as `stat` gives. It returns `NIL`.

HyForth: `sys-fwstat`. HyBASIC: `SYS "FWSTAT"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

GETC

System call `$F956`, one of those for the console. It can wait.

ARGUMENTS

None.

RESULT

A

A byte.

DESCRIPTION

Read a byte from stdin, waiting for one (the X16's CHRIN, unechoed).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$2E`	`E_EOF`	end of file	The end of stdin
`$05`	`E_INTR`	interrupted	A note came: taken on the way out

FROM THE LANGUAGES

hylang: `(sys-getc)`. It returns A.

HyForth: `sys-getc`. HyBASIC: `SYS "GETC"`.

SEE ALSO

The other calls for the console: `PUTC`, `PUTS`, `PUTHEX`.

GETCWD

System call `$F8EA`, one of those for files. It never waits.

ARGUMENTS

r0

A buffer (`PATH_MAX` + 1 bytes)

RESULT

A

The current directory's length.

The directory in the buffer, zero-terminated.

DESCRIPTION

The current directory.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-getcwd)`. It returns the buffer's text. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-getcwd`. HyBASIC: `SYS "GETCWD"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `PIPE`, `FD2PATH`.

GETPID

System call `$F839`, one of those for tasks. It never waits.

ARGUMENTS

None.

RESULT

A

This task's number.

DESCRIPTION

The calling task's number (0-15).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-getpid)`. It returns A.

HyForth: `sys-getpid`. HyBASIC: `SYS "GETPID"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

GETPPID

System call `$F869`, one of those for tasks. It never waits.

ARGUMENTS

None.

RESULT

A

This task's parent (`$FF`: none)

DESCRIPTION

The task that started this one (Plan 9's getppid): init once that one has ended; none for init, the drivers and the kernel task.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-getppid)`. It returns A.

HyForth: `sys-getppid`. HyBASIC: `SYS "GETPPID"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

INIT_RUN

System call `$F9E0`, one of those for drivers. It never waits.

ARGUMENTS

A

A number, 0-15.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Start init (the module the directory names) again, in a task of its own, with the number in decimal as its argument: as the boot starts it (no parent, no fds, an empty namespace and environment, a note group of its own), but the boot's has no argument. The kernel task starts it as it next runs; with no task free it isn't started (but 0's, which waits for one). The kernel itself asks for 0's when the task init was last started in for 0 (or by the boot) has EXECed and then ends: so an init that EXECs the login shell is started again when that shell ends (an init that ends itself is left ended). HydraOS's console asks for n's when the user makes window n. From a task or an irq entry.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: IRQ_OWN, IRQ_RELEASE, WAKE, PAUSE, NOTE_POST, SRV_REGISTER, SRV_TAKE, SRV_REPLY, CLIENT_READ, CLIENT_WRITE, NOTE_QUEUE, ROMREAD, TASKMEM, TASKSTOP, TASKSTEP, SRV_NAP.

IRQ_OWN

System call $F9B0, one of those for drivers. It never waits.

ARGUMENTS

A

An IRQ line (0-14), or LINE_VIA_T2 (VIA timer 2), or LINE_VIA_CA1 (VIA CA1)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Own an IRQ line: its interrupts run the calling task's irq entry, in the calling task. The entry gets the line in A, and returns A = 0, or IRQ_RESCHED to ask for a task switch (it woke a task). It runs with interrupts off and must not make calls that wait, and with the dispatch's own cycles it must stay under the 200-cycle budget: about 70 cycles of its own (TASK_EVENT, not WAKE, to wake clients; NOTE_QUEUE for a group). A task's lines are released when it ends. LINE_VIA_T2 is VIA timer 2: owning it sets the timer one-shot with its interrupt on; the owner starts it (T2CL, then T2CH), and its entry clears the interrupt (reading T2CL, or starting it again). LINE_VIA_CA1 is CA1: owning it turns CA1's interrupt on (off as it's given back), and the entry clears CA1's flag (IFR).

ERRORS

CODE	NAME	MESSAGE	CAUSE
$07	E_BUSY	busy	Another task owns it
$08	E_RANGE	out of range	Not a line
$02	E_INVAL	invalid argument	The task has no IRQ entry

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: IRQ_RELEASE, WAKE, PAUSE, NOTE_POST, SRV_REGISTER, SRV_TAKE, SRV_REPLY, CLIENT_READ, CLIENT_WRITE, NOTE_QUEUE, ROMREAD, TASKMEM, TASKSTOP, TASKSTEP, SRV_NAP, INIT_RUN.

IRQ_RELEASE

System call `$F9B3`, one of those for drivers. It never waits.

ARGUMENTS

A

An IRQ line.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Give an IRQ line back.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$01`	`E_PERM`	not allowed	Not the caller's
`$08`	`E_RANGE`	out of range	

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

KMESG

System call `$F806`, one of those for the system. It never waits.

ARGUMENTS

r0

A buffer.

r1

Its size, the most it copies.

r2

An offset in the messages held, 0 the oldest.

RESULT

A/X

The bytes held, `KMESG_SIZE` at most.

The buffer has those from r2, r1 at most.

DESCRIPTION

The kernel's messages: what it printed on the bring-up console (the banner, POST, the boot's tasks, a driver's own lines), its last `KMESG_SIZE` bytes. `/dev/kmesg` (kdev's `#n/kmesg`) is them as a file.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-kmesg count r2)`, where *count* is the most to take: r1; *r2* is a number, or left out. It returns a list: the buffer, as a string, A/X. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-kmesg`. HyBASIC: `SYS "KMESG"`.

SEE ALSO

The other calls for the system: `SYSINFO`, `ERRSTR`, `REBOOT`, `XCALL`.

MODINFO

System call `$F857`, one of those for tasks. It never waits.

ARGUMENTS

A
: An entry of the module directory (0 on)

r0
: A buffer (`ME_SIZE` bytes)

RESULT

The buffer: `ME_BANK`, `ME_BANKS`, `ME_TYPE` (`HT_*`), `ME_FLAGS` (`HF_*`), `ME_NAME` (11, zero-padded), `ME_PAGE`.

DESCRIPTION

A module in the paged ROM (`SPAWN`'s `#m/NAME`), as the module directory has it. A module of a bank may share it with others: its header is at `$A000` + 256 * `ME_PAGE` there (0 for a module of banks).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	Past the last

FROM THE LANGUAGES

hylang: `(sys-modinfo A)`, where *A* is a number. It returns the buffer, as a string. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-modinfo`. HyBASIC: `SYS "MODINFO"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

MOUNT

System call `$F923`, one of those for namespaces. It can wait.

ARGUMENTS

X
: A device letter.

r0
: Its spec (a string, 8 characters at most), or 0.

r1
: Old.

A
: `BIND`'s flags.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Mount a device's server, with a spec (which of its file trees: '#f' with x is the ROM disk), at old, as `BIND` does.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$2B`	`E_NODEV`	no such device	
`$20`	`E_NOENT`	not found	
`$02`	`E_INVAL`	invalid argument	
`$50`	`E_NSFULL`	namespace full	
`$0A`	`E_NAMETOOLONG`	name too long	

FROM THE LANGUAGES

hylang: `(sys-mount X r0 r1 A)`, where *X* is a number; *r0* is a string, or `NIL`; *r1* is a string; *A* is a number, or left out. It returns `NIL`.

HyForth: `sys-mount`. HyBASIC: `SYS "MOUNT"`.

SEE ALSO

The other calls for namespaces: `BIND`, `UNMOUNT`, `NSINFO`.

NOTE

System call `$F854`, one of those for tasks. It never waits.

ARGUMENTS

A

A task, or `NOTE_GROUP` | a note group.

X

The note (1-31: `NOTE_*`)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Send a note (Plan 9's postnote). It wakes the task if it waits: `WAIT`, `PAUSE`, `SLEEP` and `GETC` end with `E_INTR`. With no handler, and for `NOTE_KILL` always, the task ends with 128 + the note's Unix number (130 for `NOTE_INTERRUPT`, 137 for `NOTE_KILL`) and its name as the message.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	No such note
`$41`	`E_SRCH`	no such task	No task, or nobody in the group
`$01`	`E_PERM`	not allowed	The kernel task, a driver

FROM THE LANGUAGES

hylang: `(sys-note A X)`, where *A* is a number; *X* is a number. It returns `NIL`.

HyForth: `sys-note`. HyBASIC: `SYS "NOTE"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

NOTE_POST

System call `$F9BC`, one of those for drivers. It never waits.

ARGUMENTS

A

A task, or `NOTE_GROUP` | a note group.

X

The note.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

`NOTE` without taking the caller's own notes: the note posted, the tasks woken. From a task, or an irq entry for a task (a group is too many tasks to look at with interrupts off: `NOTE_QUEUE`).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	
`$41`	`E_SRCH`	no such task	
`$01`	`E_PERM`	not allowed	

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

NOTE_QUEUE

System call `$F9CE`, one of those for drivers. It never waits.

ARGUMENTS

X

A note group (0-15)

A

The note's bit: 1 << (note - 1), for a note 1-8.

IRQs off.

RESULT

X kept.

DESCRIPTION

A note to a note group, from an irq entry (a console's Ctrl-C): queued, and posted by the kernel task as soon as it runs. Short enough for an irq entry: nothing is checked.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

NOTIFY

System call `$F851`, one of those for tasks. It never waits.

ARGUMENTS

r0

The note handler, or 0 for none.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Set the task's note handler (Plan 9's notify). A note runs it in the task, in its own code (never in the middle of a system call), with A = the note: it returns C = 0 to go on where the task was, or C = 1 for the default; it may call `EXITS`. A note that comes meanwhile waits for it to return.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

NSINFO

System call `$F929`, one of those for namespaces. It never waits.

ARGUMENTS

A

A task (`$FF`: this one)

X

Which entry of its namespace (0 on)

r0

A buffer (`NI_SIZE` bytes)

RESULT

The entry in the buffer (`NI_*`): its mount point, its device letter and spec, the path in that device, its flags (`MCREATE`) and its place among the union's members.

DESCRIPTION

A namespace's mount entries, one at a time, in the order of the kernel's table: each a member of the union at its mount point, the members in `NI_SEQ`'s order (lowest first). `/proc`'s ns, and ns.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$08`	`E_RANGE`	out of range	Past the last
`$41`	`E_SRCH`	no such task	

FROM THE LANGUAGES

hylang: `(sys-nsinfo A X)`, where *A* is a number; *X* is a number. It returns the buffer, as a string. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-nsinfo`. HyBASIC: `SYS "NSINFO"`.

SEE ALSO

The other calls for namespaces: `BIND`, `MOUNT`, `UNMOUNT`.

OPEN

System call `$F8C0`: opens a file that exists, and returns a file descriptor for it. It can wait.

SYNTAX

```
            LDR r0, path        ; r0 = the address of the path
            lda #mode
            jsr OPEN
            bcs failed          ; A = the error code
            sta fd              ; A = the file descriptor
```

ARGUMENTS

r0

The address of the path: its bytes, then a zero byte.

A

The mode: one of `O_READ` (0), `O_WRITE` (1) or `O_RDWR` (2), to which can be added `O_TRUNC` (`$10`), to empty the file as it is opened, and `O_NONBLOCK` (`$40`), to make reads and writes of this descriptor return `E_AGAIN` where they would otherwise wait.

RESULT

C = 0, and A = the file descriptor: a number from 0 to 15, the lowest one the task has free.

DESCRIPTION

1. The path is made absolute and clean. One that doesn't start with `/` (or with `#`, a device's own name) has the task's current directory put before it; then `.` elements and doubled slashes are removed, and each `..` removes the element before it. The result may be at most 63 bytes (`PATH_MAX`).
2. The path is looked up in the task's namespace (section 9.5). Where a directory is a union of several (section 9.7), each member is tried in turn, and the first that has the name is the one opened. A member whose device has gone is passed over as if it didn't have the name.
3. The device's driver is asked to open the file in the mode given. If it refuses, its error is the call's.
4. The descriptor is set to the file, at offset 0.

A descriptor has an offset of its own, which `READ`, `WRITE` and `SEEK` move. Descriptors made from it by `DUP`, and those a child task is given by `SPAWN`, share that offset; a second `OPEN` of the same file does not.

Opening a directory with `O_READ` succeeds: it reads as a sequence of 64-byte stat records, one for each file in it. When the directory is a union, the records of all its members follow one another.

`OPEN` never makes a file. `CREATE` does, and opens it.

ERRORS

CODE	NAME	CAUSE
`$20`	`E_NOENT`	No such file, in any member of a union
`$2B`	`E_NODEV`	The path names a device that isn't there
`$0A`	`E_NAMETOOLONG`	The path, made absolute, is longer than 63 bytes
`$26`	`E_MFILE`	The task has all 16 of its descriptors in use
`$27`	`E_NFILE`	All 48 of the system's channels are in use (every open file in every task takes one; `DUP`s share)
`$05`	`E_INTR`	A note arrived while the call was waiting for the driver

A driver adds its own: `E_ROFS` (`$29`) for a write mode on a read-only disk, and `E_ISDIR` (`$23`) for a write mode on a directory.

NOTES

Descriptors 0, 1 and 2 are standard input, output and error, already open when a program starts. A task that closes one of them will get that number from its next `OPEN`.

A descriptor that isn't closed is closed when the task ends.

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-open` (*z-addr mode -- fd ior*), the path zero-terminated (`>z` makes one)
hylang	`(sys-open path [mode])`: the descriptor. A failure is an error, its text the path and the system's message
HyBASIC	`SYS "OPEN"`; or, for most purposes, the `OPEN` statement (Chapter 1)

SEE ALSO

`CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `DUP`, `FSTAT`.

EXAMPLE

From hylang. Descriptors 0 to 2 are in use, so the first file opened is 3:

```
hylang> (sys-open "/dev/time" 0)
=> 3
hylang> (sys-open "/dev/nothing" 0)
=> Error: /dev/nothing: not found
hylang> (sys-open "/rom/bin/ls" 1)
=> Error: /rom/bin/ls: read-only
hylang> (sys-open "/ram" 1)
=> Error: /ram: is a directory
hylang> (sys-open "#q/x" 0)
=> Error: #q/x: no such device
```

PAGES_ALLOC

System call `$F893`, one of those for memory. It never waits.

ARGUMENTS

A

Pages (256 bytes each)

RESULT

r0

The first one's address.

A

Its page.

DESCRIPTION

A run of pages of the task's RAM, above the break, taken from the top down: for a language runtime's heap, or a buffer.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	0
`$06`	`E_NOMEM`	out of memory	

FROM THE LANGUAGES

hylang: `(sys-pages-alloc A)`, where *A* is a number. It returns a list: r0, A.

HyForth: `sys-pages-alloc`. HyBASIC: `SYS "PAGES_ALLOC"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

PAGES_FREE

System call `$F896`, one of those for memory. It never waits.

ARGUMENTS

A
: Pages.

r0
: The first one's address.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Give pages back.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not pages `PAGES_ALLOC` gave this task

FROM THE LANGUAGES

hylang: `(sys-pages-free A r0)`, where *A* is a number; *r0* is a number. It returns `NIL`.

HyForth: `sys-pages-free`. HyBASIC: `SYS "PAGES_FREE"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

PAUSE

System call `$F9B9`, one of those for drivers. It can wait.

ARGUMENTS

None.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Wait until another task or an irq entry WAKEs this one, or a note comes (taken on the way out). (Wake-ups can come early: look again after.)

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

PIPE

System call `$F8ED`, one of those for files. It can wait.

ARGUMENTS

None.

RESULT

A
: An fd to read from.

X
: An fd to write to.

DESCRIPTION

A pipe (`#|`): what's written to one end is read from the other, 512 bytes held between them. A read waits for something to read, and is the end (0) once no writer is left; a write waits for room, and is `E_PIPE` once no reader is left.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$2B`	`E_NODEV`	no such device	No pipe server
`$26`	`E_MFILE`	no free file descriptor	
`$27`	`E_NFILE`	too many open files	
`$06`	`E_NOMEM`	out of memory	No pipe free

FROM THE LANGUAGES

hylang: `(sys-pipe)`. It returns a list: A, X.

HyForth: `sys-pipe`. HyBASIC: `SYS "PIPE"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `FD2PATH`.

PREEMPT_OFF

System call `$F848`, one of those for tasks. It never waits.

ARGUMENTS

None.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Hold the CPU: no task switch until the matching `PREEMPT_ON` (they nest). Interrupts keep running.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-preempt-off)`. It returns `NIL`.

HyForth: `sys-preempt-off`. HyBASIC: `SYS "PREEMPT_OFF"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

PREEMPT_ON

System call `$F84B`, one of those for tasks. It never waits.

ARGUMENTS

None.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Undo `PREEMPT_OFF`; a task switch that came due meanwhile happens now.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-preempt-on)`. It returns `NIL`.

HyForth: `sys-preempt-on`. HyBASIC: `SYS "PREEMPT_ON"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

PUTC

System call `$F950`, one of those for the console. It can wait.

ARGUMENTS

A

A byte.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Write a byte to stdout (the X16's CHROUT).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-putc A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-putc`. HyBASIC: `SYS "PUTC"`.

SEE ALSO

The other calls for the console: `PUTS`, `GETC`, `PUTHEX`.

PUTHEX

System call `$F959`, one of those for the console. It can wait.

ARGUMENTS

`A`

A byte.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Write a byte to stdout as two hex digits (for debugging).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-puthex A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-puthex`. HyBASIC: `SYS "PUTHEX"`.

SEE ALSO

The other calls for the console: `PUTC`, `PUTS`, `GETC`.

PUTS

System call `$F953`, one of those for the console. It can wait.

ARGUMENTS

`r0`

A string, zero-terminated.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Write a string to stdout.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$05`	`E_INTR`	interrupted	A note came while it waited: taken on the way out

FROM THE LANGUAGES

hylang: `(sys-puts r0)`, where *r0* is a string. It returns `NIL`.

HyForth: `sys-puts`. HyBASIC: `SYS "PUTS"`.

SEE ALSO

The other calls for the console: `PUTC`, `GETC`, `PUTHEX`.

READ

System call `$F8C9`: reads bytes from an open file into memory. It can wait.

SYNTAX

```
            LDR r0, buf         ; r0 = where the bytes go
            LDR r1, count       ; r1 = how many, at most
            lda fd
            jsr READ
            bcs failed          ; A = the error code
                                ; A/X = how many were read; 0 is the end of the file
```

ARGUMENTS

A

A file descriptor, open with `O_READ` or `O_RDWR`.

r0

The address of a buffer in the task's memory.

r1

The most bytes to read: 0 to 65,535. The buffer must have room for them.

RESULT

C = 0, and A/X = the number of bytes read, which were put in the buffer from its start. The descriptor's offset has moved on by that many.

DESCRIPTION

The kernel asks the file's driver for the bytes at the descriptor's offset, at most 512 at a time (`IO_UNIT`), and repeats until one of three things happens:

- **The count is reached.** The result equals r1.
- **The file ends.** The result is what there was before the end; a read that starts at the end returns 0. A result of 0 is how the end of a file is recognised, and it is not an error.
- **The driver returns fewer bytes than it was asked for.** The kernel stops there and returns them. This is how a device says "that is all for now": the console returns one line, however large the buffer.

A result smaller than r1 therefore does not by itself mean the end of the file. A program that needs exactly *n* bytes must call `READ` again for the remainder until it has them or gets 0.

When the driver has nothing yet (no key typed, an empty pipe), the task waits. With `O_NONBLOCK` set on the descriptor, the call returns `E_AGAIN` instead.

A directory reads as 64-byte stat records (`SR_SIZE`), one for each file in it: ask for a multiple of 64.

ERRORS

CODE	NAME	CAUSE
`$25`	`E_BADF`	A isn't an open descriptor, or it was opened with `O_WRITE`
`$04`	`E_AGAIN`	Nothing to read yet, and the descriptor is non-blocking
`$05`	`E_INTR`	A note arrived while the call was waiting

A driver adds its own: a disk that fails answers `E_IO` (`$2A`).

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-read` (*fd c-addr u -- u2 ior*): *u2* bytes were read
hylang	`(sys-read fd count)`: the bytes read, as a string
HyBASIC	`SYS "READ"`; or `INPUT #`, `LINE INPUT #`, `INPUT$` and `GET`

SEE ALSO

`OPEN`, `WRITE`, `SEEK`, `CLOSE`, `PIPE`.

EXAMPLE

The offset moving on, a count cut short by the end of the file, and 0 at the end:

```
hylang> (set {fd} (sys-open "/ram/t" 0))
=> NIL
hylang> (sys-read fd 5)
=> "hello"
hylang> (sys-read fd 100)
=> " world\n"
hylang> (sys-read fd 100)
=> ""
hylang> (sys-close fd)
=> NIL
```

REBOOT

System call `$F809`, one of those for the system. It never waits.

ARGUMENTS

A

0, or `REBOOT_HWTEST`.

RESULT

(doesn't return)

DESCRIPTION

Start the system again, as the reset button does (the VIA's and ACIA's interrupts quieted first: only the button resets the chips); with `REBOOT_HWTEST`, into the hardware test, as a T typed during POST does. Every task ends where it is: files written are on their disks already (the disks write through).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-reboot A)`, where *A* is a number, or left out. It returns `NIL`.

HyForth: `sys-reboot`. HyBASIC: `SYS "REBOOT"`.

SEE ALSO

The other calls for the system: `SYSINFO`, `ERRSTR`, `KMESG`, `XCALL`.

REMOVE

System call `$F8DE`, one of those for files. It can wait.

ARGUMENTS

`r0`
: A path.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Remove a file, or an empty directory.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$2B`	`E_NODEV`	no such device	
`$0A`	`E_NAMETOOLONG`	name too long	
`$24`	`E_NOTEMPTY`	directory not empty	
`$03`	`E_NOSYS`	no such call	

FROM THE LANGUAGES

hylang: `(sys-remove r0)`, where *r0* is a string. It returns `NIL`.

HyForth: `sys-remove`. HyBASIC: `SYS "REMOVE"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

ROMREAD

System call `$F9D1`, one of those for drivers. It never waits.

ARGUMENTS

A
: A paged ROM bank.

r0
: Where in it (`$A000-$DFFF`)

r1
: A buffer (not at `$A000-$DFFF`)

r2
: A count.

RESULT

The bytes in the buffer.

DESCRIPTION

Copy from any bank of the paged ROM, for a module run in place (the storage driver's ROM disk): the bank is the task's own meanwhile, and its module's is put back.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$01`	`E_PERM`	not allowed	The task owns an IRQ line: its irq entry would find the wrong bank at `$A000`

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

RTC

System call `$F986`, one of those for the clock. It never waits.

ARGUMENTS

A

0, read, or 1, write.

r0

8 bytes: the DS1747's registers (RTC_REGS, hw.inc), BCD.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

The DS1747 in U7, its registers a byte at a time (task F's `$7FF8-$7FFF`): read with its updates halted (R) for the copy; written with W set, the control (its century) last with W clear, which starts it from them. No chip: the RAM there, as it is. A read must check the fields hold a date and time.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	A neither

FROM THE LANGUAGES

hylang: `(sys-rtc A r0)`, where *A* is a number; *r0* is a string of 8 bytes, or `NIL` for zeros. It returns the buffer, as a string.

HyForth: `sys-rtc`. HyBASIC: `SYS "RTC"`.

SEE ALSO

The other calls for the clock: `TIME`, `TIME_SET`.

SEEK

System call `$F8CF`, one of those for files. It can wait.

ARGUMENTS

A

An fd.

r0, r1

An offset (low word, high word: signed)

X

From: 0 the start, 1 the offset, 2 the end.

RESULT

r0, r1

The new offset.

DESCRIPTION

Move an fd's offset (shared by the fds that `DUP` or `SPAWN` made of it).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	
`$02`	`E_INVAL`	invalid argument	Before the start, or no such from

FROM THE LANGUAGES

hylang: `(sys-seek A r0/r1 X)`, where *A* is a number; *r0/r1* is a number of 32 bits; *X* is a number, or left out. It returns r0/r1.

HyForth: `sys-seek`. HyBASIC: `SYS "SEEK"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

SEGINFO

System call `$F8AE`, one of those for memory. It never waits.

ARGUMENTS

None.

RESULT

A
: The shared banks segments can have (8K each: the good chips')

X
: Those segments have now.

r0
: The segments in use (bit = segment)

DESCRIPTION

The shared RAM, as free shows it.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-seginfo)`. It returns a list: A, X, r0.

HyForth: `sys-seginfo`. HyBASIC: `SYS "SEGINFO"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

SEG_ATTACH

System call `$F8A5`, one of those for memory. It never waits.

ARGUMENTS

A

A segment.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Attach the calling task to a segment (attaching twice is once).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	No such segment

FROM THE LANGUAGES

hylang: `(sys-seg-attach A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-seg-attach`. HyBASIC: `SYS "SEG_ATTACH"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

SEG_CREATE

System call `$F8A2`, one of those for memory. It never waits.

ARGUMENTS

A

Banks (1-128, 8K each)

RESULT

A

The segment (0-15)

DESCRIPTION

A shared segment: shared RAM banks any task can attach to, with the calling task attached. It stays while a task is attached (`SEG_DETACH`, or the task's end).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	0
`$06`	`E_NOMEM`	out of memory	

FROM THE LANGUAGES

hylang: `(sys-seg-create A)`, where *A* is a number. It returns A.

HyForth: `sys-seg-create`. HyBASIC: `SYS "SEG_CREATE"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

SEG_CREATE_IN

System call `$F8B4`, one of those for memory. It never waits.

ARGUMENTS

A

How many banks (1-128)

X

The lowest shared bank ID it may take.

Y

The highest (U << 4 | the bank's low nibble; segments have `$80`-`$FF`)

RESULT

A

The segment.

DESCRIPTION

`SEG_CREATE`, from a range of the shared RAM (the shared RAM disk's start FROM-TO).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	0
`$06`	`E_NOMEM`	out of memory	No run that long there

FROM THE LANGUAGES

hylang: `(sys-seg-create-in A X Y)`, where *A* is a number; *X* is a number; *Y* is a number. It returns A.

HyForth: `sys-seg-create-in`. HyBASIC: `SYS "SEG_CREATE_IN"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`.

SEG_DETACH

System call `$F8A8`, one of those for memory. It never waits.

ARGUMENTS

A

A segment.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Detach the calling task from a segment; the last task to go frees it.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not attached

FROM THE LANGUAGES

hylang: `(sys-seg-detach A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-seg-detach`. HyBASIC: `SYS "SEG_DETACH"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_MAP`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

SEG_MAP

System call `$F8AB`, one of those for memory. It never waits.

ARGUMENTS

A

A segment.

X

One of its banks (0 on)

RESULT

A

The value for U.

X

The value for `$00`.

DESCRIPTION

How to reach a bank of a segment: write U, then `$00`, and it's at `$8000`-`$9FFF`.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not attached
`$08`	`E_RANGE`	out of range	No such bank in it

FROM THE LANGUAGES

hylang: `(sys-seg-map A X)`, where *A* is a number; *X* is a number. It returns a list: A, X.

HyForth: `sys-seg-map`. HyBASIC: `SYS "SEG_MAP"`.

SEE ALSO

The other calls for memory: `BREAK`, `PAGES_ALLOC`, `PAGES_FREE`, `BANKS`, `BANKS_ALLOC`, `BANKS_FREE`, `SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEGINFO`, `BANKS_ALLOC_IN`, `SEG_CREATE_IN`.

SEM_ACQUIRE

System call `$F86F`, one of those for tasks. It can wait.

ARGUMENTS

A

A semaphore.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Take one of a semaphore's count; if there's none, wait (using no CPU) till `SEM_RELEASE` gives one back. A mutex's holder is then this task.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not a semaphore, or freed while this task waited
`$07`	`E_BUSY`	busy	A mutex this task holds already
`$05`	`E_INTR`	interrupted	A note came: taken on the way out

FROM THE LANGUAGES

hylang: `(sys-sem-acquire A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-sem-acquire`. HyBASIC: `SYS "SEM_ACQUIRE"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

SEM_FREE

System call `$F878`, one of those for tasks. It never waits.

ARGUMENTS

A

A semaphore.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Free a semaphore (any task may): its number can be made again, and the tasks waiting for it wake with `E_INVAL`. The end of the task that made it frees it too, and a task's end gives back the mutexes it holds.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not a semaphore

FROM THE LANGUAGES

hylang: `(sys-sem-free A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-sem-free`. HyBASIC: `SYS "SEM_FREE"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `EXEC`.

SEM_NEW

System call `$F86C`, one of those for tasks. It never waits.

ARGUMENTS

A

Its count: how many can take it before a taker waits (0-255)

X

0, or `SEM_MUTEX`: a mutex (its count 1, and it has a holder: only the task that took it gives it back)

RESULT

A

The semaphore (0 to `SEM_MAX` - 1)

DESCRIPTION

Make a semaphore, for tasks that share something (a shared segment, a device) or wait for each other. Every task sees the same ones, by number. It lasts till `SEM_FREE`, or the end of the task that made it.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$06`	`E_NOMEM`	out of memory	None free: `SEM_MAX` in all

FROM THE LANGUAGES

hylang: `(sys-sem-new A X)`, where *A* is a number; *X* is a number, or left out. It returns A.

HyForth: `sys-sem-new`. HyBASIC: `SYS "SEM_NEW"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

SEM_RELEASE

System call `$F875`, one of those for tasks. It never waits.

ARGUMENTS

A

A semaphore.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Give one back to a semaphore's count, and wake the tasks waiting for it: the first to run takes it.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not a semaphore
`$01`	`E_PERM`	not allowed	A mutex this task doesn't hold
`$08`	`E_RANGE`	out of range	Its count is 255

FROM THE LANGUAGES

hylang: `(sys-sem-release A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-sem-release`. HyBASIC: `SYS "SEM_RELEASE"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_FREE`, `EXEC`.

SEM_TRY

System call `$F872`, one of those for tasks. It never waits.

ARGUMENTS

A

A semaphore.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

`SEM_ACQUIRE` without the wait.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not a semaphore
`$07`	`E_BUSY`	busy	A mutex this task holds already
`$04`	`E_AGAIN`	not yet: try again	None to take

FROM THE LANGUAGES

hylang: `(sys-sem-try A)`, where *A* is a number. It returns `NIL`.

HyForth: `sys-sem-try`. HyBASIC: `SYS "SEM_TRY"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

SLEEP

System call `$F83F`, one of those for tasks. It can wait.

ARGUMENTS

A/X

Ticks (`TICK_HZ` a second; 0-32767)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Sleep for a number of ticks. The other tasks run meanwhile, or the system idles.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$05`	`E_INTR`	interrupted	A note woke it early: taken on the way out

FROM THE LANGUAGES

hylang: `(sys-sleep A/X)`, where *A/X* is a number. It returns `NIL`.

HyForth: `sys-sleep`. HyBASIC: `SYS "SLEEP"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

SLEEP_UNTIL

System call `$F842`, one of those for tasks. It can wait.

ARGUMENTS

A/X

A tick count (at most 32767 ticks ahead)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Sleep until the tick count reaches a value: for steady timing (a time that's passed returns at once).

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$05`	`E_INTR`	interrupted	A note woke it early: taken on the way out

FROM THE LANGUAGES

hylang: `(sys-sleep-until A/X)`, where *A/X* is a number. It returns `NIL`.

HyForth: `sys-sleep-until`. HyBASIC: `SYS "SLEEP_UNTIL"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

SPAWN

System call `$F836`, one of those for tasks. It can wait.

ARGUMENTS

r0

The program's path (through the namespace: `/bin/NAME`; or `#m/NAME`, a module in the paged ROM)

r1

Its arguments (zero-terminated strings, an empty one after the last: 176 bytes at most), or 0.

A

Flags (`SPAWN_NEWGROUP`, `SPAWN_NEWNS`, `SPAWN_FDMAP`, `SPAWN_NOENV`, `SPAWN_STOPPED`)

r2

An fd map (with `SPAWN_FDMAP`)

RESULT

A

The new task.

DESCRIPTION

Start a program in a task of its own. `SPAWN` opens the file and reads its HYX2 header (`HX_*`): a module of the paged ROM (`HF_INPLACE`: `#m`'s files) runs in place, in its bank (the module directory's of its name); any other program is read into the new task's RAM at its load address (`$0800`), by the task itself as it starts, and its BSS cleared (one that can't be read ends at once, with the error as its exit code). It starts at its entry point with r0 pointing to its arguments (at `TASK_ARGS`: the list), its name the header's. Its fds: the caller's 0, 1 and 2; or with `SPAWN_FDMAP`, the map's: a count (`SPAWN_FDS` at most), then for each of the child's fds from 0 the caller's fd that becomes it (`$FF`, or a closed one: closed). Each shares the caller's channel and offset; the rest are closed. Its parent is the caller, which can `WAIT` for it. It's in the caller's note group unless `SPAWN_NEWGROUP` starts one (its number: the child's), and it shares the caller's namespace unless `SPAWN_NEWNS` gives it an empty one. Its environment is a copy of the caller's (`SPAWN_NOENV`: empty). `SPAWN_STOPPED`: it's stopped at its entry point, for the debugger (`TASKSTEP`; `/proc/N/ctl`'s step, or start).

ERRORS

CODE	NAME	MESSAGE	CAUSE
$20	E_NOENT	not found	No such file
$2D	E_NOEXEC	not a program	Not a program this kernel runs
$40	E_NOTASK	no free task	No free task
$0B	E_TOOBIG	too big	The arguments
$02	E_INVAL	invalid argument	The map
$26	E_MFILE	no free file descriptor	
$05	E_INTR	interrupted	

Also what else OPEN and READ give.

FROM THE LANGUAGES

hylang: `(sys-spawn r0 r1 A r2)`, where *r0* is a string; *r1* is a list of strings, or NIL; *A* is a number, or left out; *r2* is a list of bytes, or NIL. It returns A.

HyForth: `sys-spawn`. HyBASIC: `SYS "SPAWN"`.

SEE ALSO

The other calls for tasks: EXITS, WAIT, GETPID, YIELD, SLEEP, SLEEP_UNTIL, TICKS, PREEMPT_OFF, PREEMPT_ON, TASKINFO, NOTIFY, NOTE, MODINFO, ENV_GET, ENV_PUT, ENV_DEL, ENV_NAME, TASKREAD, GETPPID, SEM_NEW, SEM_ACQUIRE, SEM_TRY, SEM_RELEASE, SEM_FREE, EXEC.

SRV_NAP

System call $F9DD, one of those for drivers. It can wait.

ARGUMENTS

A/X

Ticks (1-32767)

Or 0, and Y = the driver's event count (TASK_EVENT) as it last saw it.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

For a driver with work of its own between its calls (a device to look at now and then): its init doesn't return, but waits here between its turns. While it waits it serves its calls, as a driver that has started does (the boot goes on); when the time has come it runs again, busy (a call to it waits) till its next SRV_NAP, or till its init returns (it's an ordinary driver from then). With 0 ticks it waits till its own event count isn't Y any more (its irq entry adds to it, or one of its calls): for work that follows what happens, not the clock. ps shows it asleep, or waiting for an event.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `INIT_RUN`.

SRV_REGISTER

System call `$F9BF`, one of those for drivers. It never waits.

ARGUMENTS

A

A device letter (printable; not #)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Serve a device: its requests ("#x...") come to the calling task's serve entry. A task may serve several letters (the request block names it: `RQ_DEV`). Freed when the task ends.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$02`	`E_INVAL`	invalid argument	Not a letter
`$21`	`E_EXIST`	already exists	Another task's
`$06`	`E_NOMEM`	out of memory	No room

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

SRV_REPLY

System call `$F9C5`, one of those for drivers. It never waits.

ARGUMENTS

`TASK_INBOX`'s `RQ_FID`, `RQ_DONE`, `RQ_PERM`, `RQ_EVENT`: what the request returns. A, C: the answer.

RESULT

A and C kept.

DESCRIPTION

At the end of the serve entry: what the request returns, back to the client (srvlib does it).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

SRV_TAKE

System call `$F9C2`, one of those for drivers. It never waits.

ARGUMENTS

None.

RESULT

A
: The request (`R_*`)

Y
: The client.

The request block in `TASK_INBOX`, and (`R_OPEN`, `R_CREATE`, `R_REMOVE`) the name in `TASK_PATH`.

DESCRIPTION

In the serve entry: take the request being served from the client (srvlib does it). It notes the server's event count (`TASK_EVENT`) in `RQ_EVENT`: a client answered `E_AGAIN` waits for it to change.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

STAT

System call `$F8D2`, one of those for files. It can wait.

ARGUMENTS

r0

A path.

r1

A buffer (`SR_SIZE` bytes)

RESULT

The file's stat record in the buffer (`SR_*`)

DESCRIPTION

A file's stat record: its name, qid, mode, length, time, device letter.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$2B`	`E_NODEV`	no such device	
`$0A`	`E_NAMETOOLONG`	name too long	
`$26`	`E_MFILE`	no free file descriptor	
`$27`	`E_NFILE`	too many open files	

FROM THE LANGUAGES

hylang: `(sys-stat r0)`, where *r0* is a string. It returns the stat record, as a hash. (Here r1 is a buffer the function makes itself.)

HyForth: `sys-stat`. HyBASIC: `SYS "STAT"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

SYSINFO

System call `$F800`, one of those for the system. It never waits.

ARGUMENTS

None.

RESULT

A
: The ABI version (1)

X
: How many RAM modules are installed.

r0
: The free tasks (bit = task)

DESCRIPTION

What the system has.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-sysinfo)`. It returns a list: A, X, r0.

HyForth: `sys-sysinfo`. HyBASIC: `SYS "SYSINFO"`.

SEE ALSO

The other calls for the system: `ERRSTR`, `KMESG`, `REBOOT`, `XCALL`.

TASKINFO

System call `$F84E`, one of those for tasks. It never waits.

ARGUMENTS

A
: A task (0-15)

r0
: A buffer (`TI_SIZE` bytes)

RESULT

The buffer: `TI_STATE`, `TI_FLAGS`, `TI_PARENT`, `TI_CPU` (3 bytes: ticks), `TI_BANK`, `TI_TYPE`, `TI_NAME` (16)

DESCRIPTION

What the kernel knows of a task, for ps (and `/proc`'s source). A task's CPU time is the ticks that interrupted it. `TI_STATE` is 0 for a free task.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$41`	`E_SRCH`	no such task	Not a task

FROM THE LANGUAGES

hylang: `(sys-taskinfo A)`, where *A* is a number. It returns the buffer, as a string. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-taskinfo`. HyBASIC: `SYS "TASKINFO"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

TASKMEM

System call `$F9D4`, one of those for drivers. It never waits.

ARGUMENTS

A
: A task.

X
: `TM_READ` or `TM_WRITE`, | `TM_BANK`.

r0
: A buffer.

r1
: An address in the task's view, `$0000-$DFFF` (with `TM_BANK`: `$8000-$9FFF`, in its RAM bank r3)

r2
: A count (the range ends at `$E000`, or with `TM_BANK` at `$A000`, at most)

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Bytes from a task's memory into the buffer, or from the buffer into it: `/proc`'s mem and ram (kdev, which decides who may: a program reaches another task's memory only through those files). Its task RAM, and its RAM bank and paged ROM bank as it has them selected; or with `TM_BANK` any of its RAM banks (its bank register points at it meanwhile: not for a driver, whose irq entry might run). A byte at a time, IRQs off for each.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$01`	`E_PERM`	not allowed	The caller isn't a driver
`$41`	`E_SRCH`	no such task	Not a task
`$02`	`E_INVAL`	invalid argument	Past the range

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

TASKREAD

System call `$F866`, one of those for tasks. It never waits.

ARGUMENTS

A

A task (`$FF`: this one)

X

What: `TR_ARGS` (its arguments: `ARGS_MAX` bytes), `TR_CWD` (its current directory: `PATH_MAX` + 1 bytes), `TR_ENV` (its environment's first variables, whole, and a 0: `ENV_SIZE` bytes at most), `TR_FRAME` (its state and the frame it left as it was switched out: `TF_SIZE` bytes, `TF_*`) or `TR_ENVAT` (part of its environment), or `TR_FD` (its fd r2: `FI_SIZE` bytes, `FI_*`)

r0

A buffer that big.

r1

`TR_ENVAT`'s count (the most it copies)

r2

`TR_ENVAT`'s offset, or `TR_FD`'s fd.

RESULT

A/X

`TR_ENVAT`'s: the environment's bytes in use (its 0 too: it copied as many as there are from r2, r1 at most)

The bytes in the buffer.

DESCRIPTION

Another task's arguments, current directory, environment, registers or open files, as they are now: `/proc`'s args, cwd, env, regs and fd, ps. The kernel task copies them a byte at a time, with IRQs off for each. An environment may be `ENV_MAX` bytes: `TR_ENVAT` reads one whole, a part at a time.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$41`	`E_SRCH`	no such task	
`$02`	`E_INVAL`	invalid argument	Another what
`$25`	`E_BADF`	bad file descriptor	`TR_FD`: not an open fd

FROM THE LANGUAGES

hylang: `(sys-taskread A X count r2)`, where *A* is a number; *X* is a number; *count* is the most to take: r1; *r2* is a number, or left out. It returns a list: the buffer, as a string, A/X. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-taskread`. HyBASIC: `SYS "TASKREAD"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

TASKSTEP

System call `$F9DA`, one of those for drivers. It never waits.

ARGUMENTS

A

A task.

X

`TS_STEP`, `TS_NEXT`, `TS_BREAKS` or `TS_NOBREAKS`.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

The debugger's step (`/proc`'s ctl: kdev). `TS_STEP`: a stopped task runs its next instruction and stops again (`TI_FLAGS`: `TF_STOPPED` once it has). JMP, JSR, RTS and RTI are done at once, on its frame; others run out of line, in its zero page, a BRK after them. `TS_NEXT`: the same, but a JSR runs its subroutine whole, as a JSR to the kernel's jump table always does (a subroutine that reads bytes after its JSR can't be stepped over). `TS_BREAKS`: from now any BRK stops the task, its PC back on the BRK (the debugger's breakpoints, BRKs it writes into the program); `TS_NOBREAKS`: a BRK is the program's own again (`NOTE_BRK`). `SPAWN_STOPPED` starts a program stopped at its entry point.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$01`	`E_PERM`	not allowed	The caller isn't a driver; the kernel task, a driver
`$41`	`E_SRCH`	no such task	Not a task, or not started
`$02`	`E_INVAL`	invalid argument	Another X; to step: a BRK or STP
`$07`	`E_BUSY`	busy	To step: not stopped, or a step still out

Or in the kernel: in a call, its PC at `$E000` or above, or W not 0.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `SRV_NAP`, `INIT_RUN`.

TASKSTOP

System call `$F9D7`, one of those for drivers. It never waits.

ARGUMENTS

A

A task.

X

1 to stop it, 0 to start it again.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Stop a task where it is (the scheduler passes it by: its waits go on, and end, but it doesn't run), or start it again: `/proc`'s ctl (kdev, which decides who may), for the debugger. `TASKINFO`'s `TI_FLAGS` has `TF_STOPPED` while it's stopped. A kill note starts it, to end it.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$01`	`E_PERM`	not allowed	The caller isn't a driver; the kernel task, a driver
`$41`	`E_SRCH`	no such task	Not a task, or not started

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

TICKS

System call `$F845`, one of those for tasks. It never waits.

ARGUMENTS

None.

RESULT

A/X

The tick count (`TICK_HZ` a second; it wraps after about 5.5 minutes)

DESCRIPTION

The scheduler's tick count.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-ticks)`. It returns A/X.

HyForth: `sys-ticks`. HyBASIC: `SYS "TICKS"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

TIME

System call `$F980`, one of those for the clock. It never waits.

ARGUMENTS

None.

RESULT

r0/r1

The clock: seconds since 2000-01-01 00:00:00.

DESCRIPTION

The clock: the boot's time and the ticks since. It starts at 0 (2000-01-01) unless kdev finds a DS1747 with a date and time as it starts; a write of `/dev/time` sets it (and the chip).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-time)`. It returns r0/r1.

HyForth: `sys-time`. HyBASIC: `SYS "TIME"`.

SEE ALSO

The other calls for the clock: `TIME_SET`, `RTC`.

TIME_SET

System call `$F983`, one of those for the clock. It never waits.

ARGUMENTS

r0/r1
: Seconds since 2000-01-01 00:00:00.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Set the clock (from now: the boot's time becomes the clock less the ticks since). The DS1747 is the caller's to set (`RTC`).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-time-set r0/r1)`, where *r0/r1* is a number. It returns `NIL`.

HyForth: `sys-time-set`. HyBASIC: `SYS "TIME_SET"`.

SEE ALSO

The other calls for the clock: `TIME`, `RTC`.

UNMOUNT

System call `$F926`, one of those for namespaces. It can wait.

ARGUMENTS

r0
: New, or 0.

r1
: Old, or 0.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Undo a bind or a mount at old: all of what's there (r0 = 0), or new's member (as `BIND` finds new). With no old either (r1 = 0), everything: the caller has an empty namespace of its own (the tasks it shared one with keep theirs): what Plan 9's newns starts from, and nslib's ns_default does first.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$02`	`E_INVAL`	invalid argument	
`$50`	`E_NSFULL`	namespace full	
`$0A`	`E_NAMETOOLONG`	name too long	

FROM THE LANGUAGES

hylang: `(sys-unmount r0 r1)`, where *r0* is a string, or `NIL`; *r1* is a string, or `NIL`. It returns `NIL`.

HyForth: `sys-unmount`. HyBASIC: `SYS "UNMOUNT"`.

SEE ALSO

The other calls for namespaces: `BIND`, `MOUNT`, `NSINFO`.

WAIT

System call `$F833`, one of those for tasks. It can wait.

ARGUMENTS

A

A task this one started, or `$FF` for any.

r0

A buffer for the message (32 bytes), or 0.

RESULT

A

The task that ended.

X

Its exit code.

Its message in r0's buffer ("" for none)

DESCRIPTION

Wait for a child to end, and take its exit status. A child that has ended already isn't waited for.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$42`	`E_CHILD`	no such child	No such child, or none at all
`$05`	`E_INTR`	interrupted	A note came: taken on the way out

FROM THE LANGUAGES

hylang: `(sys-wait A)`, where *A* is a number. It returns a list: A, X, the buffer's text. (Here r0 is a buffer the function makes itself.)

HyForth: `sys-wait`. HyBASIC: `SYS "WAIT"`.

SEE ALSO

The other calls for tasks: `EXITS`, `SPAWN`, `GETPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

WAKE

System call `$F9B6`, one of those for drivers. It never waits.

ARGUMENTS

A

A task.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Make a waiting task (`PAUSE`) runnable. From a task or an irq entry. Waking a task that isn't waiting does nothing.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for drivers: `IRQ_OWN`, `IRQ_RELEASE`, `PAUSE`, `NOTE_POST`, `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP`, `INIT_RUN`.

WRITE

System call `$F8CC`, one of those for files. It can wait.

ARGUMENTS

A
: An fd.

r0
: The data.

r1
: Its count.

RESULT

A/X
: The count written.

DESCRIPTION

Write at the fd's offset, and move it on.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$25`	`E_BADF`	bad file descriptor	Not open for writing
`$04`	`E_AGAIN`	not yet: try again	
`$05`	`E_INTR`	interrupted	

FROM THE LANGUAGES

hylang: `(sys-write A r0)`, where *A* is a number; *r0* is a string or a list of bytes (its length goes in r1). It returns A/X.

HyForth: `sys-write`. HyBASIC: `SYS "WRITE"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

WSTAT

System call `$F8D8`, one of those for files. It can wait.

ARGUMENTS

r0
: A path.

r1
: A stat record.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Change a file from a stat record: its name, its mode, its length (what its server allows: HydraFS a file's length, longer with zeros or shorter). What's all `$FF` in the record (an empty name) is left as it is.

ERRORS

CODE	NAME	MESSAGE	CAUSE
`$20`	`E_NOENT`	not found	
`$2B`	`E_NODEV`	no such device	
`$0A`	`E_NAMETOOLONG`	name too long	
`$26`	`E_MFILE`	no free file descriptor	
`$27`	`E_NFILE`	too many open files	
`$03`	`E_NOSYS`	no such call	Its server changes nothing

FROM THE LANGUAGES

hylang: `(sys-wstat r0 r1)`, where *r0* is a string; *r1* is a hash, as `stat` gives. It returns `NIL`.

HyForth: `sys-wstat`. HyBASIC: `SYS "WSTAT"`.

SEE ALSO

The other calls for files: `OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`.

XCALL

System call `$F80C`, one of those for the system. It never waits.

ARGUMENTS

r15

A routine's address (`$A000-$DFFF`)

r14 (its low byte)

The paged ROM bank it's in.

A, X, Y, the flags and r0-r13: the routine's.

RESULT

A, X, Y, the flags and r0-r15 as the routine leaves them.

DESCRIPTION

Call a routine in another bank of the paged ROM (a library module's: `HT_LIBRARY`, its bank from `MODINFO`), as the X16's jsrfar: the caller's bank register (`$01`) is set to the routine's for the call and back after it, and everything else passes through both ways. The routine runs in the caller's task, with its RAM and zero page. A note may come while it runs: a program that makes far calls keeps its note handler in RAM, as a module of several banks does.

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

Not in the languages: it is a call for drivers, or for the debugger.

SEE ALSO

The other calls for the system: `SYSINFO`, `ERRSTR`, `KMESG`, `REBOOT`.

YIELD

System call `$F83C`, one of those for tasks. It never waits.

ARGUMENTS

None.

RESULT

Nothing, but the carry: C = 0 for success.

DESCRIPTION

Let the other tasks run; come back when it's this one's turn again (at once, if nothing else can run).

ERRORS

None: the call always succeeds.

FROM THE LANGUAGES

hylang: `(sys-yield)`. It returns `NIL`.

HyForth: `sys-yield`. HyBASIC: `SYS "YIELD"`.

SEE ALSO

The other calls for tasks: `EXITS`, `WAIT`, `SPAWN`, `GETPID`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF`, `PREEMPT_ON`, `TASKINFO`, `NOTIFY`, `NOTE`, `MODINFO`, `ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`, `TASKREAD`, `GETPPID`, `SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `EXEC`.

Constants

Every constant that `hydra.inc` defines for programs, by family. A value with a `$` is hexadecimal.

NAME	VALUE	MEANING
ABI_VERSION	1	The ABI version this include describes
TICK_HZ	200	Ticks a second
INIT_TASK	1	init's task (and its note group: the console's foreground at first)
PROG_ZP	$22	A program's own zero page: $22-$7F
PROG_ZP_END	$80	
TASK_ARGS	$0350	A task's arguments: zero-terminated strings, an empty one after the last (176 bytes at most)
TASK_NAME	$0230	A task's name, zero-terminated
TASK_EVENT	$BD	A task's event count: a server adds 1 (inc TASK_EVENT) when what its clients wait for has happened
TASK_INBOX	$0200	A server's copy of the request it's serving (RQ_*)
TASK_PATH	$02C0	A server's copy of the name a request names (64 bytes)
ARGS_MAX	176	SPAWN's arguments: their bytes at most (each string's 0, and the empty one that ends them)
IRQ_RESCHED	$80	An irq entry's answer: a task switch, please
LINE_VIA_T2	16	IRQ_OWN's line for VIA timer 2 (the hardware's lines: hw.inc)
LINE_VIA_CA1	17	IRQ_OWN's line for VIA CA1 (J27's pin 11): owning it turns CA1's interrupt on; the owner clears its flag
HT_PROGRAM	1	HYX2 module types
HT_DRIVER	2	
HT_LIBRARY	3	
HF_INPLACE	$01	HYX2 flags: runs in place in the paged ROM
HF_BOOT	$02	HYX2 flags: a driver the kernel starts at boot
HX_MAGIC	0	The HYX2 header (HX_SIZE bytes, a program's first): HYX2
HX_HSIZE	4	HYX2: the header's size (48)
HX_TYPE	5	HYX2: the type (HT_*)
HX_FLAGS	6	HYX2: the flags (HF_*)
HX_ABI	7	HYX2: the ABI version it was built for
HX_LOAD	8	HYX2: its load address (2: $A000 in place, $0800 a RAM program)
HX_LENGTH	10	HYX2: its image's length (2: in its last bank, in place)
HX_DATA_LOAD	12	HYX2: its data, where it is in the image (2) ...

NAME	VALUE	MEANING
HX_DATA_RUN	14	HYX2: ... where it runs (2) ...
HX_DATA_LEN	16	HYX2: ... and its length (2)
HX_BSS	18	HYX2: its BSS (2) ...
HX_BSS_LEN	20	HYX2: ... and its length (2: cleared as it starts)
HX_TOP	22	HYX2: the top of the RAM it uses (2: its break starts there)
HX_MAIN	24	HYX2: entry: main (a program), init (a driver)
HX_SERVE	26	HYX2: entry: serve (a driver)
HX_IRQ	28	HYX2: entry: irq (a driver)
HX_STOP	30	HYX2: entry: stop (a driver)
HX_DEVICE	32	HYX2: its device letter (a driver)
HX_BANKS	33	HYX2: the banks it spans (in place)
HX_VERSION	34	HYX2: its version (2)
HX_NAME	36	HYX2: its name (12: zero-terminated)
HX_SIZE	48	
ME_BANK	0	MODINFO's answer (ME_SIZE bytes): the module's first paged ROM bank
ME_BANKS	1	MODINFO: the banks it spans
ME_TYPE	2	MODINFO: its type (HT_*)
ME_FLAGS	3	MODINFO: its flags (HF_*)
ME_NAME	4	MODINFO: its name (11 bytes, zero-padded: 10 characters at most)
ME_PAGE	15	MODINFO: its header's page in its first bank, 0-63 ($A000 + 256 * it: a module of a bank, packed with others)
ME_SIZE	16	
TI_STATE	0	TASKINFO's answer: the task's state (0: free, 1 ready, 2 wait, 3 call, 4 idle, 5 new, 6 sleep, 7 blocked, 8 event)
TI_FLAGS	1	TASKINFO: its flags (TF_DRIVER, TF_STOPPED)
TI_PARENT	2	TASKINFO: its parent ($FF: none)
TI_CPU	3	TASKINFO: its CPU time, in ticks (3 bytes)
TI_BANK	6	TASKINFO: its module's paged ROM bank ($FF: none)
TI_TYPE	7	TASKINFO: its module's type (HT_*; 0: none)
TI_NAME	8	TASKINFO: its name, zero-terminated (16 bytes)
TI_GROUP	24	TASKINFO: its note group

NAME	VALUE	MEANING
TI_SIZE	25	TASKINFO: the answer's size
TF_DRIVER	$01	TI_FLAGS: a driver
TF_STOPPED	$80	TI_FLAGS: stopped (/proc's ctl: TASKSTOP)
TF_STATE	0	TR_FRAME's: the task's state (0 free, 1 ready, 2 waiting, 3 calling, 4 idle, 5 new, 6 sleeping, 7 blocked, 8 waiting for an event)
TF_S	1	TR_FRAME's: its stack pointer, as it was before the frame
TF_U	2	TR_FRAME's: the frame: U
TF_Y	3	
TF_W	4	
TF_X	5	
TF_A	6	
TF_P	7	
TF_PC	8	TR_FRAME's: PC (2)
TF_RAM	10	TR_FRAME's: its RAM bank register ($00)
TF_ROM	11	TR_FRAME's: its paged ROM bank register ($01)
TF_SIZE	12	
NOTE_INTERRUPT	1	Notes: Ctrl-C (default: the end, 130)
NOTE_KILL	2	Notes: kill, never caught (137)
NOTE_HANGUP	3	Notes: hangup (129)
NOTE_ALARM	4	Notes: alarm (142)
NOTE_BRK	5	Notes: sys: brk, a BRK instruction (133)
NOTE_USER	16	Notes: 16-31 are the programs' own (128 + the note)
NOTE_GROUP	$80	NOTE's A: a note group, not a task
SEM_MUTEX	$01	SEM_NEW's X: a mutex (its count 1, given back only by its holder)
SEM_MAX	16	Semaphores: as many as there can be at once
REBOOT_HWTEST	$01	REBOOT's A: into the hardware test
SPAWN_NEWGROUP	$01	SPAWN's flags: the child starts a note group of its own
SPAWN_NEWNS	$02	SPAWN's flags: the child starts with an empty namespace of its own (else it shares its parent's)
SPAWN_FDMAP	$04	SPAWN's flags: the child's fds are r2's map (else the caller's 0, 1 and 2)

NAME	VALUE	MEANING
SPAWN_NOENV	$08	SPAWN's flags: the child starts with an empty environment (else a copy of its parent's)
SPAWN_STOPPED	$10	SPAWN's flags: the child stops at its entry point, for the debugger (/proc/N/ctl's step)
SPAWN_FDS	15	SPAWN's fd map: the child's fds it can give, at most (its fd 15 is its loader's as it starts)
ENV_SIZE	1024	TASKREAD's TR_ENV: its buffer, an environment's first variables, whole, and a 0
ENV_MAX	8192	An environment's bytes at most: each variable its name's length, its name, its value's length (2), its value; a 0
ENV_NAME_MAX	31	A variable's name's length at most
MREPL	0	BIND's and MOUNT's flags: in place of what's at old
MBEFORE	1	BIND's and MOUNT's flags: a union, new first
MAFTER	2	BIND's and MOUNT's flags: a union, new last
MCREATE	4	BIND's and MOUNT's flags: a CREATE in the union comes to new
EXIT_MSG_MAX	31	An exit message's length at most (WAIT's buffer: 32 bytes)
KMESG_SIZE	4096	KMESG: the kernel's messages held at most (then the oldest go)
TR_ARGS	0	TASKREAD: a task's arguments (ARGS_MAX bytes)
TR_CWD	1	TASKREAD: a task's current directory (PATH_MAX + 1 bytes)
TR_ENV	2	TASKREAD: a task's environment's first variables, whole, and a 0 (ENV_SIZE bytes at most)
TR_FRAME	3	TASKREAD: a task's state and frame (TF_SIZE bytes)
TR_ENVAT	4	TASKREAD: part of a task's environment (r1 bytes at most, from offset r2; A/X its bytes in use)
TR_FD	5	TASKREAD: a task's fd r2 (FI_SIZE bytes)
FI_NAME	0	TR_FD's: the name its file was opened by (PATH_MAX + 1 bytes, zero-terminated) ...
FI_MODE	64	TR_FD's: its open mode (O_*) ...
FI_DEV	65	TR_FD's: its server's device letter ...
FI_QTYPE	66	TR_FD's: its qid type (QT_DIR ...) ...
FI_OFFSET	67	TR_FD's: and its offset (4 bytes, low first)
FI_SIZE	71	
TM_READ	0	TASKMEM: from the task's memory
TM_WRITE	$01	TASKMEM: to the task's memory

NAME	VALUE	MEANING
TM_BANK	$80	TASKMEM's flag: its RAM bank r3, at $8000-$9FFF
TS_STEP	0	TASKSTEP: one instruction
TS_NEXT	1	TASKSTEP: one instruction, a JSR's subroutine whole
TS_BREAKS	2	TASKSTEP: from now, the task's BRKs stop it (the debugger's breakpoints)
TS_NOBREAKS	3	TASKSTEP: its BRKs are its own again (NOTE_BRK)
NI_FROM	0	NSINFO's answer (NI_SIZE bytes): the mount point (PATH_MAX + 1 bytes, zero-terminated)
NI_PATH	64	NSINFO: the path in the device (zero-terminated, no leading /)
NI_DEV	128	NSINFO: the device letter
NI_SPEC	129	NSINFO: the spec (8 bytes, zero-padded, and a 0)
NI_FLAGS	138	NSINFO: the flags (MCREATE)
NI_SEQ	139	NSINFO: its place among the union's members (lowest first)
NI_SIZE	140	
KEY_UP	$80	Keys: a raw console read gives the terminal's cursor and function keys as one code each
KEY_DOWN	$81	
KEY_RIGHT	$82	
KEY_LEFT	$83	
KEY_HOME	$84	
KEY_END	$85	
KEY_INS	$86	
KEY_DEL	$87	
KEY_PGUP	$88	
KEY_PGDN	$89	
KEY_F1	$8A	Keys: F1-F12 are $8A-$95
KEY_F2	$8B	
KEY_F3	$8C	
KEY_F4	$8D	
KEY_F5	$8E	
KEY_F6	$8F	
KEY_F7	$90	

NAME	VALUE	MEANING
KEY_F8	$91	
KEY_F9	$92	
KEY_F10	$93	
KEY_F11	$94	
KEY_F12	$95	
KEY_FOCUS	$97	Keys: the focus moved to another window of this one's group; its number is the next key (a raw read's, keys hydra's)
KEY_RESIZE	$96	Keys: the window's size changed (a raw read's, keys hydra's: consctl reads with the new one)
KEY_MOD	$98	Keys: a key the terminal sent modified (keys mods's): the modifiers next (1 Shift, 2 Alt, 4 Ctrl), then the key
O_READ	0	Open modes
O_WRITE	1	
O_RDWR	2	
O_RW_MASK	3	
O_TRUNC	$10	Open modes: empty the file first
O_NONBLOCK	$40	Open modes: E_AGAIN rather than wait
FD_MAX	16	Fds a task has (0 stdin, 1 stdout, 2 stderr)
PATH_MAX	63	A path's length at most
IO_UNIT	512	The most a READ or WRITE asks of a server in one request
SR_NAME	0	The stat record (SR_SIZE bytes): its name (32, zero-terminated)
SR_QTYPE	32	Stat: the qid's type (QT_*)
SR_QVERS	33	Stat: the qid's version
SR_QPATH	34	Stat: the qid's path (4: unique in its server)
SR_MODE	38	Stat: the mode (2: permissions; high byte DM_DIR)
SR_LENGTH	40	Stat: the length (4)
SR_MTIME	44	Stat: the time it changed (4: seconds since 2000)
SR_DEV	48	Stat: the device letter
SR_INST	49	Stat: the device's instance (its spec's first character)
SR_SIZE	64	
QT_DIR	$80	Qid types: a directory

NAME	VALUE	MEANING
QT_FILE	$00	
DM_DIR	$80	A mode's high byte: a directory
DM_APPEND	$40	A mode's high byte: append-only (writes go to the end)
RQ_TYPE	0	The request block (RQ_SIZE bytes): the request (R_*)
RQ_FID	1	Request: the fid (out: the new fid, R_OPEN and R_CREATE)
RQ_MODE	2	Request: the open mode
RQ_CLIENT	3	Request: the client task
RQ_U	4	Request: the client's U
RQ_FLAGS	5	Request: RF_*
RQ_OFFSET	6	Request: the offset (4)
RQ_COUNT	10	Request: the count asked for (2)
RQ_DONE	12	Request: the count done (2, out)
RQ_BUF	14	Request: the buffer, in the client's view (2)
RQ_PERM	16	Request: R_CREATE's new mode (high byte); out (R_OPEN, R_CREATE): the qid type
RQ_NAMELEN	17	Request: the name's length
RQ_GROUP	18	Request: the client's note group
RQ_DEV	19	Request: the device letter
RQ_SPEC	20	Request: the mount's spec (8, zero-padded)
RQ_EVENT	28	Request: the server's event count as it took it (after E_AGAIN, the client waits for a change)
RQ_SIZE	32	
RF_NONBLOCK	$01	Request flags: answer E_AGAIN rather than wait
R_OPEN	1	Requests
R_CREATE	2	
R_READ	3	
R_WRITE	4	
R_CLUNK	5	
R_STAT	6	
R_WSTAT	7	
R_REMOVE	8	

NAME	VALUE	MEANING
R_FLUSH	9	
R_DUP	10	
SND_CHANNELS	24	Sound (#a): its channels: 0-7 the YM2151's, 8-23 the Vera X's PSG (its voices 0-15)
SND_PSG	8	Sound: the PSG's first channel (its voice 0)
SND_PATCHES	163	Sound: its patches (0-127: General MIDI's programs; 128-162: drums and percussion)
SND_DRUMS	128	Sound: General MIDI's drums (MIDI channel 10's notes)
SND_R_CH	$02	Sound: a command, at a register the chip doesn't have: the channel the next commands are for (0-23)
SND_R_PATCH	$03	Sound: load patch n into the channel (its speakers stay)
SND_R_NOTE	$04	Sound: key on MIDI note n (60: middle C; 69: A, 440 Hz), with the channel's bend
SND_R_OFF	$05	Sound: key off (the release)
SND_R_VOL	$06	Sound: the channel's volume (0-127, as General MIDI's)
SND_R_PAN	$07	Sound: the channel's speakers (SND_PAN_*)
SND_R_BEND	$09	Sound: the channel's bend (signed: 64ths of a semitone; the note playing too)
SND_R_DRUM	$0A	Sound: a General MIDI drum (its patch and pitch), keyed on
SND_R_FREQ_LO	$0B	Sound: a frequency's low byte (Hz), for SND_R_FREQ (only kept)
SND_R_FREQ	$0C	Sound: key on at a frequency, SND_R_FREQ_LO's byte and this its high one (Hz; 0: key off)
SND_R_GLIDE	$0D	Sound: the channel's pitch to note n without a new attack (legato)
SND_R_WAVE	$0E	Sound: a PSG channel's waveform: SND_WAVE_* << 6 | its pulse width (0-63: 63 a square)
SND_PAN_LEFT	1	Sound: SND_R_PAN's speakers
SND_PAN_RIGHT	2	
SND_PAN_BOTH	3	
SND_WAVE_PULSE	0	Sound: SND_R_WAVE's waveforms (the PSG's)
SND_WAVE_SAW	1	
SND_WAVE_TRIANGLE	2	
SND_WAVE_NOISE	3	
r0	$02	The call registers r0-r15: arguments and results

NAME	VALUE	MEANING
r1	$04	
r2	$06	
r3	$08	
r4	$0A	
r5	$0C	
r6	$0E	
r7	$10	
r8	$12	
r9	$14	
r10	$16	
r11	$18	
r12	$1A	
r13	$1C	
r14	$1E	
r15	$20	

Error codes

A call that fails returns with C = 1 and one of these in A. `ERRSTR` gives the message.

CODE	NAME	MESSAGE
$01	E_PERM	not allowed
$02	E_INVAL	invalid argument
$03	E_NOSYS	no such call
$04	E_AGAIN	not yet: try again
$05	E_INTR	interrupted
$06	E_NOMEM	out of memory
$07	E_BUSY	busy
$08	E_RANGE	out of range
$09	E_FAULT	bad address
$0A	E_NAMETOOLONG	name too long
$0B	E_TOOBIG	too big
$20	E_NOENT	not found
$21	E_EXIST	already exists
$22	E_NOTDIR	not a directory
$23	E_ISDIR	is a directory
$24	E_NOTEMPTY	directory not empty
$25	E_BADF	bad file descriptor
$26	E_MFILE	no free file descriptor
$27	E_NFILE	too many open files
$28	E_NOSPC	disk full
$29	E_ROFS	read-only
$2A	E_IO	i/o error
$2B	E_NODEV	no such device
$2C	E_PIPE	broken pipe
$2D	E_NOEXEC	not a program
$2E	E_EOF	end of file
$2F	E_NOTFS	no file system
$30	E_MEDIA	medium error
$40	E_NOTASK	no free task
$41	E_SRCH	no such task

CODE	NAME	MESSAGE
$42	E_CHILD	no such child
$50	E_NSFULL	namespace full
$51	E_NSLOOP	too many binds

CHAPTER 4

HyForth Words

HyForth follows the Forth 2012 standard. A word that the standard defines does what the standard says, and its entry here states that briefly and exactly; where the standard leaves a choice to the system, or calls a case ambiguous, this chapter says what HyForth does. The words that are HyForth's own are described in full.

The stack pictures follow the standard's notation: what a word takes, `--`, what it leaves, the top of the stack on the right. *n* is a signed cell, *u* an unsigned one, *x* any cell, *flag* a truth value, *c-addr u* the address and length of a string, *d* and *ud* double-cell numbers, *xt* an execution token, *ior* an I/O result. `R:` is the return stack and `N:` the numbers library's stack. *"name"* is a word taken from the input after this one.

Each entry says which word set a word belongs to, and where it lives: in the ROM core, which is always there, or in a library. The libraries `coreext`, `exception`, `file` and `tools` are loaded as HyForth starts, and `shell` when it starts as a shell; any other is loaded with `lib` and its name. **Immediate** means the word runs while a definition is being compiled; **compile only**, that it has no meaning outside one.

What the standard leaves to the system

These are the facts the standard requires a system to state, as HyForth has them.

A cell	16 bits: a signed number is −32,768 to 32,767, an unsigned one 0 to 65,535
A double	32 bits, its high cell on top of the stack
A character	8 bits; an address unit is a byte, and nothing needs aligning
True	−1, all bits set. Any value but 0 counts as true
Arithmetic	Two's complement, wrapping silently: `32767 1+` is −32768
Division	Symmetric: `/`, `mod`, `/mod`, `*/` and `*/mod` round the quotient toward zero, and the remainder has the dividend's sign (`-7 2 /` is −3, remainder −1). `fm/mod` is there for floored division, and `sm/rem` for symmetric
Division by zero	The throw code −10
Names	Capitals and small letters are the same: `DUP` is `dup`. The system's own names are in small letters
Numbers	Read in `base`, 2 to 36. A number can say its own base: `$FF` hex, `%1010` binary, `#10` decimal, `'A'` a character. One that ends in `.` is a double
`environment?`	Knows no queries: it answers false to every one
A word that isn't found	At the shell's prompt, the line is taken as a command and given to rc, whose message says it isn't found. In a file being loaded, the throw code −13: the word and a `?`
An empty stack	A word that takes more than the stack holds is the throw code −4, `stack underflow`, found when the line has been run
Errors	The standard's throw codes. An error that nothing catches prints its message, empties both stacks and returns to the prompt. A failure of the system is −512 minus its error code (Appendix G), which is what the file words' *ior*s are
Comments	`\` to the end of the line, and `(` to the next `)`
Code	New definitions are compiled into RAM banks of HyForth's own, so a program is not limited to the 32K of ordinary memory

%

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
%   ( "line" -- )
```

DESCRIPTION

Gives the rest of the line to rc as a command, whatever its first word is.

At HyForth's prompt a line is taken as Forth if its first word is a Forth word or a number, and as a command for rc otherwise. So a program whose name is also a Forth word can't be run just by typing it: `words`, `free` (once the memory library is loaded), or any word you have defined yourself. `%` settles it:

what follows is a command.

The command's exit code is then in `status`. A line that ends in `&` is not waited for, as in rc.

SEE ALSO

`sh`, `status`, `wait`; section 8.14.

EXAMPLE

```
/> : free  ." my own word" ;
/> free
my own word
/> % free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

', [']

Core word set; in the ROM core, always there. `[']` is immediate, and compile only.

STACK

```
'     ( "name" -- xt )
[']   ( "name" -- )        compiling
      ( -- xt )            when the definition runs
```

DESCRIPTION

`'` ("tick") takes the next word of the line, finds it in the dictionary, and leaves its **execution token**: a cell that stands for the word, which `execute` runs. On HyForth a token is the address of the word's code.

`'` does its work when it runs. Inside a definition that would be when the definition runs, taking a name from the line being read then. To put the token of a word named *in* the definition into it, use `[']`: it finds the word as the definition is compiled, and compiles its token as a number.

ERRORS

THROW CODE	MESSAGE	CAUSE
−13	*name* ?	There is no such word

SEE ALSO

`execute`, `compile,`, `>body`, `defer`, `find`, `postpone`.

EXAMPLE

```
/> : square  dup * ;
/> 5 ' square execute .
25
/> : apply  ['] square execute ;
/> 6 apply .
36
/> ' nothere
nothere ?
```

(

Core word set; in the ROM core, always there. Immediate.

STACK

```
(   ( "text<paren>" -- )
```

DESCRIPTION

A comment: everything up to the next) is skipped. Like any word, (must have a space after it. It works at the prompt and inside a definition, where its usual job is the word's stack picture.

SEE ALSO

\, .(.

EXAMPLE

```
/> : square  ( n -- n*n )  dup * ;
/> 4 square ( this is skipped ) .
16
```

*/, */mod

Core word set; in the ROM core, always there.

STACK

```
*/      ( n1 n2 n3 -- n4 )
*/mod   ( n1 n2 n3 -- n4 n5 )
```

DESCRIPTION

Multiplies *n1* by *n2*, and divides the product by *n3*. The product is kept as a 32-bit double, so it does not overflow as * and then / would. */ leaves the quotient; */mod leaves the remainder, and above it the quotient.

This is the way to scale a number by a fraction with whole numbers only: 1000 314 100 */ is 3140, 1000 times 3.14. The division is symmetric, as / is.

The quotient itself must fit in a cell. One that does not is not reported: the result is wrong.

ERRORS

THROW CODE	MESSAGE	CAUSE
−10	`division by zero`	*n3* is 0

SEE ALSO

`m*/`, `/mod`, `um/mod`.

EXAMPLE

```
/> 1000 1000 100 */ .
10000
/> 1000 1000 * 100 / .
169
/> 100 7 3 */mod . .
233 1
```

+, −, *

Core word set; in the ROM core, always there.

STACK

```
+   ( n1 n2 -- n3 )
-   ( n1 n2 -- n3 )
*   ( n1 n2 -- n3 )
```

DESCRIPTION

The sum, the difference *n1* − *n2*, and the product of two cells. They are the same for signed and for unsigned numbers.

A result that does not fit in 16 bits wraps round silently, and nothing reports it: `32767 1 +` is −32768, and `300 300 *` is 24464, the low 16 bits of 90,000. Where a product or a total may be too big for a cell, use the mixed words (`m*`, `um*`, `*/`), the double-number words, or the numbers library, whose integers are of any length.

SEE ALSO

`1+`, `m*`, `um*`, `*/`, `d+`, `n+`.

EXAMPLE

```
/> 3 4 + .
7
/> 3 4 - .
-1
/> 3 4 * .
12
/> 32767 1 + .
-32768
```

+!

Core word set; in the ROM core, always there.

STACK

```
+!   ( n a-addr -- )
```

DESCRIPTION

Adds *n* to the cell stored at the address: it is `dup @ rot + swap !` in one word. A negative *n* takes away.

SEE ALSO

`@`, `!`, `1+`.

EXAMPLE

```
/> variable total  10 total !
/> 5 total +!  -1 total +!
/> total @ .
14
```

, C,

Core word set; in the ROM core, always there.

STACK

```
,    ( x -- )
c,   ( char -- )
```

DESCRIPTION

`,` ("comma") puts a cell into data space at `here`, and moves `here` on by 2. `c,` puts one byte there, the low byte of the cell given, and moves `here` on by 1.

After `create` they build a table of values, in order, as the program is loaded.

SEE ALSO

`create`, `allot`, `here`, `@`.

EXAMPLE

```
/> create primes 2 , 3 , 5 , 7 ,
/> primes 2 cells + @ .
5
/> create msg 3 c, char a c, char b c, char c c,
/> msg count type
abc
```

., u.

Core word set; in the ROM core, always there.

STACK

```
.    ( n -- )
u.   ( u -- )
```

DESCRIPTION

Print the number on top of the stack, in the current `base`, with one space after it, and remove it. `.` ("dot") prints it as a signed number, and `u.` as an unsigned one: the same cell that `.` shows as −5 is 65531 to `u.`. Use `u.` for addresses and for anything that may be above 32,767.

In a base above ten the digits past 9 are capital letters.

SEE ALSO

`.r`, `d.`, `.s`, `?`, `base`, `<#`, `n.`.

EXAMPLE

```
/> 5 . -5 .
5 -5
/> -5 u.
65531
/> 40000 . 40000 u.
-25536 40000
/> 255 hex . decimal
FF
```

."

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
."   ( "text<quote>" -- )
```

DESCRIPTION

Inside a definition, compiles the text that follows, up to the next `"`, so that it is printed when the word runs. One space must follow `."`, as after any word; that space is not part of the text, and everything after it is, spaces too.

It can't be used at the prompt (`."`: `compile only`): there, use `.(`, or `s"` and `type`.

SEE ALSO

`.(`, `s"`, `type`, `abort"`.

EXAMPLE

```
/> : greet  ." Hello, " type ." !" ;
/> s" Ann" greet
Hello, Ann!
```

.(

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate.

STACK

```
.(   ( "text<paren>" -- )
```

DESCRIPTION

Prints the text that follows, up to the next `)`, at once: at the prompt, or, since it is immediate, in the middle of a definition as the definition is compiled. It is for messages from a file as it loads.

SEE ALSO

`."`, `(`, `type`.

EXAMPLE

```
/> .( loading...)
loading...
```

/, mod, /mod

Core word set; in the ROM core, always there.

STACK

```
/      ( n1 n2 -- n3 )
mod    ( n1 n2 -- n3 )
/mod   ( n1 n2 -- n3 n4 )
```

DESCRIPTION

Signed division of *n1* by *n2*. `/` leaves the quotient, `mod` the remainder, and `/mod` both: the remainder *n3*, and above it the quotient *n4*.

HyForth's division is symmetric: the quotient is rounded toward zero, and the remainder has the sign of *n1*. So `-7 2 /` is −3, with the remainder −1. For division that rounds down, use `fm/mod`.

ERRORS

THROW CODE	MESSAGE	CAUSE
−10	`division by zero`	*n2* is 0

SEE ALSO

`*/`, `fm/mod`, `sm/rem`, `um/mod`, `2/`.

EXAMPLE

```
/> 7 2 / .
3
/> -7 2 / .
-3
/> -7 2 mod .
-1
/> 7 2 /mod . .
3 1
/> 1 0 /
division by zero
```

:

Core word set; in the ROM core, always there.

STACK

```
:   ( "name" -- )
```

DESCRIPTION

Starts a definition. `:` takes the next word of the line as the name of a new word, and puts HyForth into compiling: from here to the `;` that ends the definition, the words of the line are not run but compiled into the new word. Immediate words are the exception: they run as they are met, and do the compiling of `if`, `do`, `."` and the like.

The new word cannot be found until its `;`: inside its own definition, its name still means whatever it meant before, so a new version of a word can call the old one. For a word that calls itself, use `recurse`.

A name can be given twice. The later definition is the one found from then on, and nothing reports it; words already compiled keep using the earlier one.

A definition may run over several lines. HyForth compiles to machine code: a call to each word, with the shortest words copied in place.

ERRORS

THROW CODE	MESSAGE	CAUSE
−16	`no name`	Nothing follows `:` on the line
−13	*word* `?`	A word in the definition isn't known, and isn't a number

SEE ALSO

`;`, `:noname`, `immediate`, `recurse`, `see`, `exit`.

EXAMPLE

```
/> : square  dup * ;
/> 7 square .
49
/> : square  square square ;
/> 3 square .
81
```

;

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
;   ( -- )
```

DESCRIPTION

Ends the definition that `:` or `:noname` began: it compiles the return, makes the new word findable, and puts HyForth back into interpreting.

NOTES

HyForth does not check that the definition's control structures are complete. An `if` with no `then`, or a `do` with no `loop`, is not reported at `;`: the word is made, and misbehaves when it runs.

ERRORS

THROW CODE	MESSAGE	CAUSE
−14	`; compile only`	`;` with no definition open

SEE ALSO

`:`, `exit`, `immediate`.

EXAMPLE

```
/> : hello  ." Hello!" ;
/> hello
Hello!
/> ;
;: compile only
```

<#, #, #s, hold, holds, sign, #>

Core word set; in the ROM core. Pictured numeric output: these build the text of a number, one character at a time, from its last digit to its first.

STACK

```
<#      ( -- )
#       ( ud1 -- ud2 )
#s      ( ud1 -- 0 0 )
hold    ( char -- )
sign    ( n -- )
#>      ( xd -- c-addr u )
```

DESCRIPTION

`<#` starts a new picture, in a buffer that belongs to these words. The number being converted is an unsigned double on the stack throughout. `#` divides it by `base`, puts the remainder's digit in front of what the picture holds so far, and leaves the quotient; `#s` does that until the quotient is 0, and at least once, so a 0 gives the one digit `0`. `hold` puts any character in front, and `sign` puts a minus sign there if its argument is negative. `#>` drops the double and returns the finished text.

Since the text is built backwards, the words are written in the reverse of the order the characters will appear: the digits after a decimal point first, then the point, then the rest.

The text is in a transient buffer, which the next picture, and the words that print numbers, use again. Use it or copy it at once.

NOTES

`.`, `u.`, `.r` and `d.` are written with these words. `holds` (*c-addr u* --), in `coreext`, puts a whole string in front.

EXAMPLE

A number of pence as pounds and pence:

```
/> : .money  s>d <# # # [char] . hold #s [char] $ hold #> type ;
/> 12345 .money
$123.45
```

<>, u>, 0<>, 0>

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
<>    ( x1 x2 -- flag )
u>    ( u1 u2 -- flag )
0<>   ( x -- flag )
0>    ( n -- flag )
```

DESCRIPTION

<> is true when two cells differ. u> is true when *u1* is greater than *u2*, as unsigned numbers. 0<> is true when the cell is not zero: it turns any true value into the well-formed flag −1. 0> is true when the cell is greater than zero.

SEE ALSO

=, u<, 0=, 0<.

EXAMPLE

```
/> 1 2 <> . 1 1 <> .
-1 0
/> -1 1 u> .
-1
/> 5 0<> . 0 0<> .
-1 0
/> 5 0> . -5 0> . 0 0> .
-1 0 0
```

=, <, >, u<

Core word set; in the ROM core, always there.

STACK

```
=    ( x1 x2 -- flag )
<    ( n1 n2 -- flag )
>    ( n1 n2 -- flag )
u<   ( u1 u2 -- flag )
```

DESCRIPTION

Compare two cells, and leave true (−1) or false (0). = is true when they are the same. < and > compare them as signed numbers: is *n1* less than *n2*, or greater? u< compares them as unsigned numbers.

The difference matters for anything above 32,767, and for addresses, which are unsigned: −1 is less than 1, but as the unsigned number 65,535 it is not.

There is no <= or >=: use > 0= and < 0=.

SEE ALSO

`<>`, `u>`, `0=`, `0<`, `within`, `d<`, `compare`.

EXAMPLE

```
/> 1 1 = . 1 2 = .
-1 0
/> 1 2 < . 2 1 < .
-1 0
/> -1 1 < . -1 1 u< .
-1 0
```

?

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
?   ( a-addr -- )
```

DESCRIPTION

Prints the cell stored at an address: it is `@ .`. It is for looking at a variable at the prompt.

SEE ALSO

`@`, `.`, `dump`.

EXAMPLE

```
/> variable v  42 v !
/> v ?
42
```

@, !

Core word set; in the ROM core, always there.

STACK

```
@   ( a-addr -- x )
!   ( x a-addr -- )
```

DESCRIPTION

`@` ("fetch") leaves the cell stored at an address, and `!` ("store") stores a cell there. A cell is two bytes, the low byte first, as the 65C02 keeps a word.

An address is any number from 0 to 65,535: it is not checked. The addresses a program owns are those it got from `variable`, `create`, `here` and `allot`, `buffer:` or `allocate`. Anywhere else in the task's 64K can be read safely, and can be written only by a program that knows what is there: the task's own first

pages and HyForth's dictionary are in the same memory (Appendix F).

SEE ALSO

`c@`, `2@`, `+!`, `variable`, `?`.

EXAMPLE

```
/> variable v
/> 42 v !
/> v @ .
42
/> $FFFC @ u.
57344
```

[,]

Core word set; in the ROM core, always there. `[` is immediate.

STACK

```
[   ( -- )
]   ( -- )
```

DESCRIPTION

`[` stops compiling and starts interpreting, in the middle of a definition; `]` goes back to compiling. What is written between them is run then and there, as the definition is compiled. With `literal` this works a value out once, at compile time, where the same words left in the definition would work it out every time the word ran.

SEE ALSO

`literal`, `state`, `immediate`.

EXAMPLE

```
/> : day-minutes  [ 24 60 * ] literal ;
/> day-minutes .
1440
/> see day-minutes
: day-minutes 1440 ;
```

\

Core Extension; in the ROM core, always there. Immediate.

STACK

```
\   ( "text<end of line>" -- )
```

DESCRIPTION

A comment: the rest of the line is skipped. It needs a space after it.

SEE ALSO

`(`, `.(`.

EXAMPLE

```
/> 1 2 + .  \ adds, and prints
3
```

{:

Locals; in the `locals` library (`lib locals`). Immediate. Compile only.

STACK

```
{: arg ... | local ... -- comment :}   ( x ... -- )
```

DESCRIPTION

Gives names to values, for the length of one definition. Between `{:` and `:}` come the names. When the word runs, a value is taken from the stack for each name before the `|`, the last name getting the top of the stack, so the names read in the order the arguments were pushed. The names after a `|`, if there are any, are locals that start with no value set. Anything from `--` to `:}` is a comment, for the results.

In the rest of the definition, a local's name leaves its value, and `to` changes it.

Locals take the place of the `dup`, `swap`, `rot` and `>r` that a word with three or four arguments would otherwise need. They are kept on the return stack, so the rules of `>r` apply to what the word does around them; they can be used inside `do` loops.

SEE ALSO

`to`, `value`, `>r`.

EXAMPLE

```
/> lib locals
/> : add3  {: a b c :}  a b + c + ;
/> 1 2 3 add3 .
6
/> : hypot2  {: x y | sum -- n :}  x x * y y * + to sum  sum ;
/> 3 4 hypot2 .
25
```

0=, 0<

Core word set; in the ROM core, always there.

STACK

```
0=   ( x -- flag )
0<   ( n -- flag )
```

DESCRIPTION

`0=` is true when the cell is zero. As every value but zero counts as true, it is also the logical "not": it turns any true value to false, and false to true.

`0<` is true when the cell is negative, which is to say when its top bit is set.

SEE ALSO

`0<>`, `0>`, `invert`, `=`.

EXAMPLE

```
/> 0 0= . 5 0= .
-1 0
/> -1 0< . 1 0< . 0 0< .
-1 0 0
```

1+, 1−, 2*, 2/

Core word set; in the ROM core, always there.

STACK

```
1+   ( n1 -- n2 )
1-   ( n1 -- n2 )
2*   ( x1 -- x2 )
2/   ( x1 -- x2 )
```

DESCRIPTION

`1+` adds one, and `1-` takes one away. `2*` shifts the cell one bit to the left, which doubles it. `2/` shifts it one bit to the right and keeps the sign bit as it was, which halves it, rounding *down*: `-5 2/` is −3, where `-5 2 /` is −2.

SEE ALSO

`+`, `lshift`, `rshift`, `d2*`, `cell+`.

EXAMPLE

```
/> 5 1+ . 5 1- .
6 4
/> 5 2* . 5 2/ .
10 2
/> -5 2/ .
-3
```

2>r, 2r>, 2r@

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Compile only.

STACK

```
2>r   ( x1 x2 -- )       R: ( -- x1 x2 )
2r>   ( -- x1 x2 )       R: ( x1 x2 -- )
2r@   ( -- x1 x2 )       R: ( x1 x2 -- x1 x2 )
```

DESCRIPTION

A pair of cells moved to the return stack, moved back from it, or copied from it, keeping their order: after `2>r`, *x2* is on top of the return stack, as it was on top of the data stack. `2>r` is `swap >r >r`, not `>r >r`.

They are for use inside a definition, and what is put on the return stack must be taken off again before the definition ends.

2@, 2!

Core word set; in the ROM core, always there.

STACK

```
2@   ( a-addr -- x1 x2 )
2!   ( x1 x2 a-addr -- )
```

DESCRIPTION

Fetch and store a pair of cells: a double number, or a string's address and length. *x2*, the cell on top of the stack, is the one at *a-addr*, and *x1* is in the cell after it. For a double, that puts its high cell first in memory.

SEE ALSO

`@`, `2variable`, `2constant`.

EXAMPLE

```
/> create pair 2 cells allot
/> 100000. pair 2!
/> pair 2@ . .
1 -31072
/> pair @ .
1
```

2constant, 2variable, 2value, 2literal

Double-Number; in the `double` library (`lib double`).

STACK

```
2constant   ( x1 x2 "name" -- )       name: ( -- x1 x2 )
2variable   ( "name" -- )             name: ( -- a-addr )
2value      ( x1 x2 "name" -- )       name: ( -- x1 x2 )
2literal    compiling: ( x1 x2 -- )   running: ( -- x1 x2 )
```

DESCRIPTION

`constant`, `variable`, `value` and `literal` for pairs of cells: a double number, usually. A `2variable`'s two cells are read and written with `2@` and `2!`. A `2value` is changed with `to`. `2literal` is immediate and compile only.

2dup, 2drop, 2swap, 2over

Core word set; in the ROM core.

STACK

```
2dup    ( x1 x2 -- x1 x2 x1 x2 )
2drop   ( x1 x2 -- )
2swap   ( x1 x2 x3 x4 -- x3 x4 x1 x2 )
2over   ( x1 x2 x3 x4 -- x1 x2 x3 x4 x1 x2 )
```

DESCRIPTION

`dup`, `drop`, `swap` and `over` for pairs of cells. A pair is whatever takes two cells: a double number, or a string's address and length.

accept

Core word set; in the ROM core, always there.

STACK

```
accept   ( c-addr +n1 -- +n2 )
```

DESCRIPTION

Reads a line from the keyboard into memory at *c-addr*, and leaves the number of characters it stored. The person types, and can correct what is typed, as at the prompt; ENTER ends the line, and is not stored. At most *+n1* characters are stored: anything typed beyond that is dropped.

SEE ALSO

`key`, `refill`, `pad`, `read-line`, `>number`.

EXAMPLE

Here `Hydra` was typed:

```
/> : ask  ." name? " pad 20 accept  ." Hello, " pad swap type ;
/> ask
name? Hydra
Hello, Hydra
```

again

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate, compile only.

STACK

Compiling: (*dest* --). Running: (--).

DESCRIPTION

Ends a loop that `begin` started, with a jump straight back to the `begin`: `begin ... again` repeats for ever. The way out is `exit`, which leaves the whole definition, or a `throw`.

ahead, cs-pick, cs-roll

Programming-Tools; in the `tools` library, which is loaded as HyForth starts. For writing new control-flow words.

STACK

```
ahead     compiling: ( -- orig )     running: ( -- )
cs-pick   compiling: ( destu ... dest0 u -- destu ... dest0 destu )
cs-roll   compiling: ( origu|destu ... u -- ... origu|destu )
```

DESCRIPTION

While a definition is compiled, the places that jumps come from and go to are kept on a control-flow stack: `if` leaves an *orig*, which `then` resolves, and `begin` leaves a *dest*, which `until` or `again` uses. `ahead` compiles a jump forward with nothing to test, leaving an *orig* for a later `then`. `cs-pick` copies an item of the control-flow stack and `cs-roll` brings one to the top, as `pick` and `roll` do on the data stack.

With them a control structure can be put together from parts: `while` is `if 1 cs-roll`, and `repeat` is `again then`.

align, aligned

Core word set; in the ROM core, always there.

STACK

```
align     ( -- )
aligned   ( addr -- a-addr )
```

DESCRIPTION

On a machine where a cell must begin at an even address, `align` moves `here` up to the next one and `aligned` rounds an address up to one. The 65C02 has no such rule: on HyForth `align` does nothing and `aligned` leaves its address as it is. They are here so that programs written for other Forths load unchanged.

SEE ALSO

`here`, `cells`.

EXAMPLE

```
/> 7 aligned .
7
```

allocate

Memory-Allocation; in the `memory` library (`lib memory`).

STACK

```
allocate   ( u -- a-addr ior )
```

DESCRIPTION

Asks for *u* bytes of memory from the heap, and leaves their address, with an ior of 0. The bytes are not cleared. If there is not that much free, the ior is −59, `allocate failed`, and the address means nothing.

The heap is a different place from data space: what is allocated can be given back, in any order, with `free`, where `allot` only grows. It is for memory whose size is known only as the program runs, or which is needed for a while.

SEE ALSO

`free`, `resize`, `allot`, `buffer:`, `bank-window`.

EXAMPLE

```
/> lib memory
/> 100 allocate throw value m
/> m 100 char A fill  m 5 type
AAAAA
/> m free .
0
/> 40000 allocate nip .
-59
```

allot

Core word set; in the ROM core, always there.

STACK

```
allot   ( n -- )
```

DESCRIPTION

Sets aside *n* bytes of data space, by moving `here` on by *n*. The bytes are not cleared: they hold whatever was there. A negative *n* gives space back.

With `create` it makes a table or a buffer: `create name 100 allot`.

ERRORS

THROW CODE	MESSAGE	CAUSE
−8	`dictionary full`	Fewer than *n* bytes are left

SEE ALSO

`here`, `create`, `buffer:`, `unused`, `cells`, `allocate`.

EXAMPLE

```
/> create buf 16 allot
/> buf 16 char * fill buf 16 type
****************
/> 40000 allot
dictionary full
```

and, or, xor

Core word set; in the ROM core, always there.

STACK

```
and   ( x1 x2 -- x3 )
or    ( x1 x2 -- x3 )
xor   ( x1 x2 -- x3 )
```

DESCRIPTION

The bitwise operations, on all 16 bits at once. Each bit of the result comes from the same bit of the two cells: `and` gives 1 where both are 1, `or` where either is, and `xor` where they differ.

Because true is −1, all bits set, and false is 0, the same three words combine flags: `and` is "both", `or` is "either". This works for well-formed flags only: 1 and 2 are both true, and `1 2 and` is 0. Make a value a flag first, with `0<>`.

SEE ALSO

`invert`, `lshift`, `0<>`, `nand`.

EXAMPLE

```
/> 12 10 and . 12 10 or . 12 10 xor .
8 14 6
/> $1234 $FF and .
52
/> 3 5 < 3 0 > and .
-1
```

argc, arg

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
argc   ( -- n )
arg    ( n -- c-addr u )
```

DESCRIPTION

The arguments HyForth was started with, for a program run as `forth file.fs a b`. `argc` leaves how many there are, the file's name counting as the first: 3 in that case. `arg` leaves argument *n* as a string, number 0 being the file's name; past the last, the string's length is 0.

At the prompt of an interactive HyForth there are none: `argc` is 0.

SEE ALSO

`included`, `getenv`.

EXAMPLE

```
/> lib hydra
/> argc .
0
```

at-xy

Facility; in the `facility` library (`lib facility`).

STACK

```
at-xy   ( u1 u2 -- )
```

DESCRIPTION

Moves the cursor to column *u1* of row *u2*. Both count from 0, and 0 0 is the top left corner of the window. What is printed next appears there.

SEE ALSO

`page`, `form`, `cursor-up`, `clear-line`.

EXAMPLE

A message in the middle of a cleared screen:

```
lib facility
page  30 12 at-xy  ." The Hydra-16"  0 23 at-xy
```

bank!, bank@

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
bank!   ( bank -- )
bank@   ( -- bank )
```

DESCRIPTION

`bank!` chooses which of the task's RAM banks is seen at `bank-window`, and `bank@` leaves the number of the one chosen. A task has 16 banks of 8K for each RAM module fitted. A program asks for the ones it will use with `sys-banks-alloc`, which leaves the number of the first of a run, and gives them back with `sys-banks-free`.

With a bank chosen, the 8K at `bank-window` is ordinary memory, for `@`, `!`, `move` and the rest. This is how a HyForth program keeps far more data than its dictionary has room for.

NOTES

HyForth keeps the code of colon definitions in RAM banks too, at the same window. A definition that uses `bank!` would switch away the very bank it is running from, so one compiled there is the error −21, `unsupported operation`, when it runs. Say `false code-banks` before defining any word that uses `bank!` or `seg-bank!`: such words are then compiled into the dictionary, and work. At the prompt `bank!` can be used freely.

SEE ALSO

`bank-window`, `code-banks`, `seg-bank!`, `allocate`.

EXAMPLE

```
/> lib hydra
/> 2 sys-banks-alloc throw value bk
/> false code-banks
/> : keep  ( n -- )  bk bank!  bank-window ! ;
/> : kept  ( -- n )  bk bank!  bank-window @ ;
/> true code-banks
/> 1234 keep kept .
1234
/> bk 2 sys-banks-free .
0
```

bank-window

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
bank-window   ( -- addr )
```

DESCRIPTION

A constant: `$8000`, the address where the task's chosen RAM bank appears. The 8K from there to `$9FFF` is a window on to one bank at a time, which `bank!` and `seg-bank!` choose.

SEE ALSO

`bank!`, `seg-bank!`, `code-banks`; Appendix F.

base

Core word set; in the ROM core, always there.

STACK

```
base   ( -- a-addr )
```

DESCRIPTION

A variable: the number base, 2 to 36, in which numbers are read and printed. It starts as ten. `decimal` and `hex` set it to 10 and 16, and any other base is stored: `2 base !`.

The base in force applies to what is typed too, the next `base !` included: after `hex`, `10 base !` leaves the base at sixteen. To get back, use `decimal`, or write the number with its own base.

A number can carry its own base, whatever `base` is: `$FF` is hex, `%1010` binary, `#10` decimal, and `'A'` a character's code. A – goes before the mark: `-$10`.

SEE ALSO

`decimal`, `hex`, `.`, `>number`, `set-base`.

EXAMPLE

```
/> 255 hex . decimal
FF
/> 2 base ! 101 decimal .
5
/> $FF . %101 . #10 . -$10 .
255 5 10 -16
```

beep

Facility; in the `facility` library (`lib facility`). A HyForth word, not the standard's.

STACK

```
beep   ( -- )
```

DESCRIPTION

Rings the terminal's bell.

SEE ALSO

`emit`, `snd-note`.

begin, until

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
begin   ( -- )
until   ( x -- )
```

DESCRIPTION

A loop that tests at its end. The words between `begin` and `until` are run; `until` then takes a flag from the stack, and if it is false, 0, goes back to `begin`. When the flag is true the loop is over, and the words after `until` are run. The words in the loop always run at least once.

`begin` also opens the other two loops: `begin` ... `while` ... `repeat`, which tests in the middle, and `begin` ... `again`, which never ends by itself.

SEE ALSO

`while`, `again`, `do`, `leave`, `exit`.

EXAMPLE

```
/> : countdown  begin dup . 1- dup 0= until drop ;
/> 5 countdown
5 4 3 2 1
```

begin-structure, end-structure

Facility; in the `facility` library (`lib facility`).

STACK

```
begin-structure   ( "name" -- struct-sys 0 )
end-structure     ( struct-sys +n -- )
name              ( -- +n )
```

DESCRIPTION

Describe a record: a group of named fields laid out one after another. `begin-structure` starts the description and defines *name*; the field words between add fields; `end-structure` finishes it. Afterward *name* leaves the size of the whole record in bytes, which is what `allot`, `buffer:` or `allocate` needs to make one.

Nothing is set aside by the description itself. It only works out offsets: each field word, used later, adds its field's offset to the address of a record.

SEE ALSO

`+field`, `create`, `allot`.

EXAMPLE

```
/> lib facility
/> begin-structure point  field: p.x  field: p.y  cfield: p.flag  end-structure
/> point .
5
/> create here-i-am  point allot
/> 3 here-i-am p.x !  4 here-i-am p.y !
/> here-i-am p.x @ . here-i-am p.y @ .
3 4
```

bin

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
bin   ( fam1 -- fam2 )
```

DESCRIPTION

In the standard, changes a way of opening into the same way for a binary file. The Hydra's files are all plain bytes, with no difference between text and binary: `bin` leaves the value as it is. It is here so that `r/o bin open-file` in a program from elsewhere works.

SEE ALSO

`r/o`, `open-file`.

bind, mount, unmount, newns

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
bind      ( "[-a|-b] [-c] new old" -- )
mount     ( "[-a|-b] [-c] #x old [spec]" -- )
unmount   ( "[new] old" -- )
newns     ( -- )
```

DESCRIPTION

Change HyForth's own namespace. Each takes the rest of the line as its arguments, written exactly as for rc's commands of the same names, whose entries in Chapter 2 of this book describe them in full: `bind` puts one directory at another's place, or joins it to what is there (`-a` after, `-b` before, `-c` taking new files); `mount` does the same with a device; `unmount` undoes them; `newns` builds the default namespace again.

They are Forth words because a command run by rc would change only the namespace of that command's task. Commands started afterward see the namespace as these leave it.

`newns` gives HyForth's `/ram` a fresh start too. In a HyForth that is a window's first shell, that removes every file in `/ram`: copy what is wanted somewhere else first.

SEE ALSO

`cd`; Chapter 9; `ns`, the command.

EXAMPLE

```
/> mkdir /ram/a /ram/b
/> echo 1 >/ram/a/one
/> bind /ram/a /ram/b
/> ls /ram/b
one
/> unmount /ram/b
/> ls /ram/b
```

bitmap, bitmap-off

The screen; in the `video` library (`lib video`).

STACK

```
bitmap       ( width depth -- )
bitmap-off   ( -- )
```

DESCRIPTION

`bitmap` turns on a bitmap on a Vera X's screen, under the console's text, for the drawing words. *width* is 320 or 640 dots across, giving 240 or 480 down; *depth* is the number of bits for each dot, 1, 2, 4 or 8, which gives 2, 4, 16 or 256 colours. At 640 across only 1 or 2 bits are possible. `bitmap-off` turns it off.

The library needs a Vera X card (the emulator has one with `--vera`). Without one, its words are the error `not found`. A drawing word used with no bitmap on is `invalid argument`.

SEE ALSO

`pen`, `plot`, `clear`; Chapter 16.

EXAMPLE

A square, a filled circle, and the turtle's hexagon:

```
lib video
320 8 bitmap
5 pen  10 10 100 100 box  200 60 30 disc
cs  : hexagon  6 0 do 40 fd 60 rt loop ;  hexagon
bitmap-off
```

bl

Core word set; in the ROM core, always there.

STACK

```
bl   ( -- char )
```

DESCRIPTION

A constant: 32, the code of a space ("blank"). It is what `word` and `fill` are most often given.

SEE ALSO

`space`, `word`, `blank`.

EXAMPLE

```
/> bl .
32
```

black, red, green, yellow, blue, magenta, cyan, white

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
black ... white   ( -- c )
```

DESCRIPTION

The eight colours, as constants, for `color` and `bgcolor`: `black` is 0, `red` 1, `green` 2, `yellow` 3, `blue` 4, `magenta` 5, `cyan` 6 and `white` 7.

SEE ALSO

`color`, `bright`.

EXAMPLE

```
/> lib facility
/> black . red . green . yellow . blue . magenta . cyan . white .
0 1 2 3 4 5 6 7
```

blank

String; in the `string` library (`lib string`).

STACK

```
blank   ( c-addr u -- )
```

DESCRIPTION

Fills *u* bytes of memory with spaces: it is `bl fill`.

SEE ALSO

`fill`, `erase`, `-trailing`.

blk, scr

Block; in the `block` library (`lib block`).

STACK

```
blk   ( -- a-addr )
scr   ( -- a-addr )
```

DESCRIPTION

Two variables. `blk` holds the number of the block being loaded, and is 0 when the source is not a block: the keyboard, a file or a string. `scr` holds the number of the block most recently shown by `list`.

SEE ALSO

`load`, `list`, `source-id`.

block, buffer

Block; in the `block` library (`lib block`).

STACK

```
block    ( u -- a-addr )
buffer   ( u -- a-addr )
```

DESCRIPTION

A **block** is 1,024 bytes of a file, the block file: block *u* is the bytes from *u* × 1,024 on. Blocks are the old Forth way to keep source and data, a screen of 16 lines of 64 characters at a time, with no file names.

`block` leaves the address of a buffer holding block *u*, reading it from the file if it is not in memory already. A block beyond the end of the file reads as spaces. `buffer` leaves a buffer assigned to block *u* without the reading, for a block that is about to be filled anyway.

There are two buffers, so the address is good only until the next `block` or `buffer` but one. After changing what is in a buffer, say `update`, or the change is lost when the buffer is used for another block.

The block file is `blocks.fb` in the directory that is current when a block is first asked for, made if it is not there; `open-blocks` names another.

SEE ALSO

`update`, `load`, `list`, `open-blocks`, `blk`.

EXAMPLE

```
/> lib block
/> s" /ram/work.fb" open-blocks
/> 0 block 1024 bl fill update flush
/> 1 block 1024 bl fill
/> s" .( block one ) 5 6 + ." 1 block swap move update flush
/> 1 load
block one 11
```

>body

Core word set; in the ROM core, always there.

STACK

```
>body   ( xt -- a-addr )
```

DESCRIPTION

Given the execution token of a word made by `create`, leaves the address of its data: the address the word itself leaves when it is used. It is for code that has a word's token, from `'`, and wants its data.

For a word not made by `create` the result means nothing.

SEE ALSO

`create`, `'`, `does>`.

EXAMPLE

```
/> create box 5 ,
/> ' box >body @ .
5
/> ' box >body box = .
-1
```

bright

Facility; in the `facility` library (`lib facility`). A HyForth word, not the standard's.

STACK

```
bright   ( c -- c' )
```

DESCRIPTION

Turns one of the eight colours into its bright version, by adding 8: `red bright` is 9. A colour that is already bright is left as it is.

SEE ALSO

`color`, `red`, `bold`.

EXAMPLE

```
/> lib facility
/> red bright . 9 bright .
9 9
```

buffer:

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
buffer:   ( u "name" -- )
name      ( -- a-addr )
```

DESCRIPTION

Defines a word that owns *u* bytes of data space, and leaves their address when it is used. The bytes are not cleared. It is `create name u allot` in one word.

ERRORS

THROW CODE	MESSAGE	CAUSE
–8	`dictionary full`	Fewer than *u* bytes are left

SEE ALSO

`create`, `allot`, `pad`, `allocate`.

EXAMPLE

```
/> 64 buffer: line
/> s" kept here" line swap move
/> line 9 type
kept here
```

bye

Programming-Tools; in the ROM core, always there.

STACK

```
bye   ( -- )
```

DESCRIPTION

Leaves HyForth, with the exit status 0. Everything defined in this run of it is gone.

When HyForth is the shell of the first window, as it is unless `/lib/shell` says otherwise, leaving it does not leave the window empty: the system starts a fresh shell there.

SEE ALSO

`exits`, `quit`, `marker`.

c"

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate. Compile only.

STACK

```
c"   ( "text<quote>" -- )       compiling
     ( -- c-addr )              when the definition runs
```

DESCRIPTION

Compiles the text up to the next `"` as a counted string: a byte holding its length, and then its characters. When the word runs, the address of the length byte is left. `count` turns that into an address and a length.

Counted strings are the older form, kept for `find` and `word`. New code uses `s"`.

SEE ALSO

`s"`, `count`, `find`, `word`.

EXAMPLE

```
/> : label  c" counted" ;
/> label count type
counted
/> label c@ .
7
```

c@, c!

Core word set; in the ROM core, always there.

STACK

```
c@   ( c-addr -- char )
c!   ( char c-addr -- )
```

DESCRIPTION

`c@` leaves the byte at an address, as a number from 0 to 255: the cell's high byte is zero. `c!` stores the low byte of the cell given, and ignores its high byte.

These are how strings, and any table of bytes, are read and written: a character is one byte.

SEE ALSO

`@`, `count`, `c,`, `fill`.

EXAMPLE

```
/> variable v
/> $1234 v !
/> v c@ . v 1+ c@ .
52 18
/> 65 v c! v c@ emit
A
```

case, of, endof, endcase

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate, compile only.

STACK

```
case      ( -- )
of        ( x1 x2 -- | x1 )
endof     ( -- )
endcase   ( x -- )
```

DESCRIPTION

Chooses among several pieces of code by a value on the stack.

```
case
   1 of ... endof
   2 of ... endof
   ...                  \ any other value: it is still on the stack here
endcase
```

Each `of` compares the value under it with the one given. If they are equal both are dropped, the code up to `endof` runs, and control goes on after `endcase`. If not, the value given is dropped, the selector stays, and the next `of` is tried. Code between the last `endof` and `endcase` runs when nothing matched, with the selector still on the stack; `endcase` then drops it. That code must leave the selector, or something in its place, for `endcase` to drop.

EXAMPLE

```
/> : name  case 1 of ." one" endof 3 of ." three" endof ." other" endcase ;
/> 3 name 7 name
threeother
```

catch

Exception; in the `exception` library, which is loaded as HyForth starts.

STACK

```
catch   ( i*x xt -- j*x 0 | i*x n )
```

DESCRIPTION

Runs the word whose token is given, as `execute` does, and watches for a `throw`.

If the word finishes in the ordinary way, `catch` leaves 0 above whatever the word left.

If a `throw` happens while the word runs, in it or in anything it calls, however deep, the word is abandoned at that point: the return stack is put back as it was at the `catch`, and so is the data stack's *depth*. `catch` then leaves the throw code. The cells that were the word's arguments are there in number, but their values may have been changed: drop them.

So after `catch`, a zero means "it worked, and here are the results", and anything else is the reason it did not.

Every error of the system is a throw, with a code of its own (see `throw`): `catch` is how a program survives division by zero, a file that isn't there, or a word of its own that gives up.

SEE ALSO

`throw`, `abort"`, `execute`, `quit`.

EXAMPLE

```
/> : safe/  ( n1 n2 -- n3 )  ['] / catch if 2drop 0 then ;
/> 10 2 safe/ .
5
/> 10 0 safe/ .
0
/> : risky  10 20 -99 throw ;
/> 1 2 ' risky catch .s
<3> 1 2 -99
```

cd

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
cd   ( "dir" -- )
```

DESCRIPTION

Changes HyForth's current directory to the one named on the rest of the line. It is a Forth word because it has to be: a `cd` run by rc as a command would change only that command's own directory.

The name is written as for rc's `cd`: `'...'` quotes one with spaces in it, and `$name` is a variable's value. With no name, the directory is the one in `$home`, if that is set.

The prompt shows the current directory. Inside a definition, use `set-dir`.

ERRORS

MESSAGE	CAUSE
dir`: not found`	There is no such directory

SEE ALSO

`set-dir`, `get-dir`, `prompt`.

EXAMPLE

```
/> cd /rom/lib
/rom/lib> cd /nothere
/nothere: not found
/rom/lib> cd /
```

cells, cell+, chars, char+

Core word set; in the ROM core, always there.

STACK

```
cells   ( n1 -- n2 )
cell+   ( a-addr1 -- a-addr2 )
chars   ( n1 -- n2 )
char+   ( c-addr1 -- c-addr2 )
```

DESCRIPTION

Sizes, in bytes. `cells` gives the size of *n1* cells, which is twice *n1*, and `cell+` adds the size of one cell, 2, to an address. `chars` gives the size of *n1* characters, which is *n1*, and `char+` adds 1.

Written with these, a program says what it means, and still works on a Forth whose cells are another size: item *i* of a table of cells is at `table i cells +`.

SEE ALSO

`allot`, `@`, `,`.

EXAMPLE

```
/> 1 cells . 3 cells . 5 chars .
2 6 5
/> 100 cell+ . 100 char+ .
102 101
```

char, [char]

Core word set; in the ROM core, always there. `[char]` is immediate, and compile only.

STACK

```
char     ( "name" -- char )
[char]   ( "name" -- )          compiling
         ( -- char )            when the definition runs
```

DESCRIPTION

`char` takes the next word of the line and leaves the code of its first character; the rest of the word is ignored. `[char]` does the same inside a definition: it takes the next word as the definition is compiled, and compiles the code as a number.

A character can also be written as a number in its own right, between single quotes: `'A'` is 65, at the prompt and in a definition alike.

SEE ALSO

`emit`, `bl`, `'`.

EXAMPLE

```
/> char A .
65
/> char Apple .
65
/> : star  [char] * emit ;
/> star
*
/> 'A' .
65
```

circle, disc

The screen; in the `video` library (`lib video`).

STACK

```
circle   ( x y r -- )
disc     ( x y r -- )
```

DESCRIPTION

Draw a circle with its centre at *x*, *y* and radius *r*, in the pen's colour: `circle` its outline, and `disc` filled.

SEE ALSO

`box`, `plot`.

clear

The screen; in the `video` library (`lib video`).

STACK

```
clear   ( -- )
```

DESCRIPTION

Clears the bitmap: every dot becomes colour 0. The console's text over it is not touched.

SEE ALSO

`cs`, `page`, `bitmap`.

clear-line, clear-below

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
clear-line    ( -- )
clear-below   ( -- )
```

DESCRIPTION

`clear-line` clears from the cursor to the end of its line. `clear-below` clears from the cursor to the end of the window. Neither moves the cursor.

SEE ALSO

`page`, `at-xy`.

close-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
close-file   ( fileid -- ior )
```

DESCRIPTION

Closes a file. What was written to it is in it from then on, and the fileid means nothing any more. The ior is 0, or –549, `bad file descriptor`, for a fileid that is not open.

SEE ALSO

`open-file`, `flush-file`.

EXAMPLE

```
/> s" /rom/lib/shell" r/o open-file throw value fd
/> fd close-file .
0
/> fd close-file .
-549
```

cmove, cmove>

String; in the `string` library (`lib string`).

STACK

```
cmove    ( c-addr1 c-addr2 u -- )
cmove>   ( c-addr1 c-addr2 u -- )
```

DESCRIPTION

Copy *u* bytes from the first address to the second, one at a time in a set order: `cmove` from the lowest address up, and `cmove>` from the highest down.

When the two areas don't overlap the order makes no difference. When they do, one order is right and the other spreads a byte over the whole area: to move text toward higher addresses use `cmove>`, and toward lower ones `cmove`. `move` chooses the right order by itself, and is the one to use unless the spreading is what is wanted.

SEE ALSO

`move`, `fill`.

EXAMPLE

```
/> lib string
/> create buf 8 allot
/> s" abcdefgh" buf swap cmove
/> buf buf 1+ 7 cmove>  buf 8 type
aabcdefg
/> buf buf 1+ 7 cmove  buf 8 type
aaaaaaaa
```

code-banks

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
code-banks   ( flag -- )
```

DESCRIPTION

Says where the code of the colon definitions made from now on is to go. With a true flag it goes into RAM banks of HyForth's own, as it does when HyForth starts: that leaves the dictionary for names and data, and lets a program be much bigger than the dictionary. With a false flag it goes into the dictionary, in ordinary memory.

Code in the dictionary is needed for words that change the bank window themselves, with `bank!` or `seg-bank!`. Switch to false, define those words, and switch back.

SEE ALSO

`bank!`, `here`, `unused`.

color, bgcolor

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
color     ( c -- )
bgcolor   ( c -- )
```

DESCRIPTION

Set the colour of the text printed from now on, and of its background. A colour is a number from 0 to 15: the eight named colours, 0 to 7, and their bright versions, 8 to 15, which `bright` gives.

The colours stay set until they are changed or `plain` is used, after the program has ended too.

SEE ALSO

`black`, `bright`, `plain`, `bold`.

EXAMPLE

```
lib facility
yellow color  blue bgcolor  ."  Warning "  plain
```

compare

String; in the `string` library (`lib string`).

STACK

```
compare   ( c-addr1 u1 c-addr2 u2 -- n )
```

DESCRIPTION

Compares two strings, character by character, by their codes. The result is 0 if they are the same, –1 if the first comes before the second, and 1 if it comes after. When one string is the beginning of the other, the shorter comes first.

The comparison is by code, so capitals come before small letters: `"ABC"` is before `"abc"`. `0=` after `compare` is the test for equal strings.

SEE ALSO

`search`, `=`, `ncompare`.

EXAMPLE

```
/> lib string
/> s" abc" s" abc" compare .
0
/> s" abc" s" abd" compare .
-1
/> s" abc" s" ab" compare .
1
```

compile,

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
compile,   ( xt -- )
```

DESCRIPTION

Adds, to the definition being compiled, a call of the word whose token is given: what the compiler itself does with each ordinary word it meets. It is used inside immediate words, which run while another definition is being compiled.

SEE ALSO

`postpone`, `[']`, `immediate`, `literal`.

EXAMPLE

```
/> : dup,  ['] dup compile, ; immediate
/> : square  dup, * ;
/> 6 square .
36
```

[compile]

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate.

STACK

```
[compile]   ( "name" -- )
```

DESCRIPTION

Compiles the next word of the definition as an ordinary call, even if it is immediate. It is the older way to say what `postpone` says for an immediate word, and the standard keeps it only for old programs: write `postpone`.

SEE ALSO

`postpone`, `immediate`.

complex

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
complex   ( N: re im -- z )
```

DESCRIPTION

Makes a complex number from two real ones: its real part, and above it its imaginary part. A complex number can also be typed directly, with digits before its `i`: `1+2i`, `2i`.

SEE ALSO

`complex?`, `n+`, `nsqrt`.

EXAMPLE

```
/> lib numbers
/> 1 s>n 2 s>n complex n.
1+2i
/> 5 s>n 1+2i n+ n.
6+2i
```

count

Core word set; in the ROM core, always there.

STACK

```
count   ( c-addr1 -- c-addr2 u )
```

DESCRIPTION

Turns a counted string into an address and a length. A counted string is a byte holding a length, and then that many characters. `count` leaves the address of the first character, and above it the length.

It is also a way to step through bytes: it leaves the next address, and the byte that was at this one.

SEE ALSO

`type`, `word`, `c"`, `c@`.

EXAMPLE

```
/> create msg 3 c, char a c, char b c, char c c,
/> msg count type
abc
/> msg count . drop
3
```

cr, space, spaces

Core word set; in the ROM core, always there.

STACK

```
cr       ( -- )
space    ( -- )
spaces   ( n -- )
```

DESCRIPTION

`cr` starts a new line. `space` writes one space, and `spaces` writes *n* of them; for an *n* of 0 or less, none.

SEE ALSO

`emit`, `bl`, `.r`, `at-xy`.

EXAMPLE

```
/> 65 emit cr 66 emit
A
B
/> 67 emit 3 spaces 68 emit
C   D
```

create

Core word set; in the ROM core, always there.

STACK

```
create   ( "name" -- )
name     ( -- a-addr )
```

DESCRIPTION

Defines a word that, when it is used, leaves the address of the data space that follows it: the value `here` had just after the word was made. `create` sets nothing aside itself. What follows it, `allot`, `,` or `c,`, builds the word's data, and so a table, a buffer or a record is made.

With `does>`, `create` makes defining words: words that define other words, each with its own data and all with the same behaviour.

SEE ALSO

`does>`, `allot`, `,`, `variable`, `buffer:`, `>body`.

EXAMPLE

```
/> create primes 2 , 3 , 5 , 7 ,
/> : prime  cells primes + @ ;
/> 3 prime .
7
```

cs, home, fd, bk, rt, lt, pu, pd, heading, seth

The screen; in the `video` library (`lib video`).

STACK

```
cs         ( -- )            home       ( -- )
fd         ( n -- )          bk         ( n -- )
rt         ( degrees -- )    lt         ( degrees -- )
pu         ( -- )            pd         ( -- )
heading    ( -- degrees )    seth       ( degrees -- )
```

DESCRIPTION

Turtle graphics, as in the language Logo. The turtle is a point on the bitmap with a heading and a pen. `home` puts it in the middle of the screen, heading up, with its pen down, and `cs` clears the bitmap first. `fd` moves it forward *n* dots along its heading, drawing a line in the pen's colour if the pen is down, and `bk` moves it back. `rt` and `lt` turn it right and left by a number of degrees. `pu` lifts the pen, so that moves draw nothing, and `pd` puts it down.

`heading` leaves the heading, 0 to 359 degrees, 0 being up and 90 to the right; `seth` sets it.

The turtle's place is kept to a sixteenth of a dot, so a figure of many small moves does not drift.

SEE ALSO

`line`, `pen`, `bitmap`.

ctl

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
ctl   ( c-addr1 u1 c-addr2 u2 -- )
```

DESCRIPTION

Writes the second string to the file named by the first, in a single write, and closes it. It is how a command is sent to a device's control file: the same thing as `echo -n text >file` at the shell.

A failure is thrown, and reported with the file's name: the file isn't there, or the driver refused the command.

SEE ALSO

`write-file`, `sh`; Chapter 12.

EXAMPLE

```
/> lib hydra
/> s" /dev/gpio/ctl" s" out 2" ctl
/> s" /dev/gpio/ctl" s" in 2" ctl
/> s" /dev/gpio/ctl" s" fly" ctl
/dev/gpio/ctl: invalid argument
```

cursor-save, cursor-restore, cursor-off, cursor-on

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
cursor-save      ( -- )
cursor-restore   ( -- )
cursor-off       ( -- )
cursor-on        ( -- )
```

DESCRIPTION

`cursor-save` has the terminal remember where the cursor is, and `cursor-restore` puts it back there: a program can write somewhere else on the screen, a status line for instance, and return. `cursor-off` hides the cursor, and `cursor-on` shows it again.

A program that hides the cursor should show it again before it ends: the setting is the window's, and outlasts the program.

SEE ALSO

`at-xy`, `plain`.

cursor-up, cursor-down, cursor-right, cursor-left

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
cursor-up      ( n -- )
cursor-down    ( n -- )
cursor-right   ( n -- )
cursor-left    ( n -- )
```

DESCRIPTION

Move the cursor *n* places from where it is. It stops at the edge of the window.

SEE ALSO

`at-xy`, `cursor-save`.

d+, d-, m+, dnegate, dabs

Double-Number; in the `double` library (`lib double`).

STACK

```
d+        ( d1 d2 -- d3 )
d-        ( d1 d2 -- d3 )
m+        ( d1 n -- d2 )
dnegate   ( d1 -- d2 )
dabs      ( d -- ud )
```

DESCRIPTION

Arithmetic on 32-bit numbers: the sum, the difference, a double plus a single cell, the negative, and the absolute value. Results wrap at 32 bits without an error.

A number typed with a `.` at its end is a double: `100000. 1. d+ d.` prints 100001.

d., d.r

Double-Number; in the `double` library (`lib double`).

STACK

```
d.    ( d -- )
d.r   ( d n -- )
```

DESCRIPTION

`d.` prints a double in `base`, with a minus sign if it is negative and one space after it. `d.r` prints it at the right of a field *n* characters wide, with no space after.

d0=, d0<, d=, d<, du<, dmax, dmin

Double-Number; in the `double` library (`lib double`).

STACK

```
d0=    ( d -- flag )
d0<    ( d -- flag )
d=     ( d1 d2 -- flag )
d<     ( d1 d2 -- flag )
du<    ( ud1 ud2 -- flag )
dmax   ( d1 d2 -- d3 )
dmin   ( d1 d2 -- d3 )
```

DESCRIPTION

The tests and comparisons of the cell words, for doubles: zero, negative, equal, less than (signed, and with `du<` unsigned), and the larger and the smaller of two.

d2, *d2/*, *d>s*, *m/*, 2rot

Double-Number; in the `double` library (`lib double`).

STACK

```
d2*    ( d1 -- d2 )
d2/    ( d1 -- d2 )
d>s    ( d -- n )
m*/    ( d1 n1 +n2 -- d2 )
2rot   ( x1 x2 x3 x4 x5 x6 -- x3 x4 x5 x6 x1 x2 )
```

DESCRIPTION

`d2*` shifts a double left one bit, and `d2/` right one bit, keeping the sign. `d>s` is the double's low cell, which is the same number if it fits in a cell. `m*/` multiplies a double by *n1* and divides by *n2*, which must be positive, keeping a 48-bit product in between so that the scaling is exact. `2rot` is `rot` for pairs.

decimal

Core word set; in the ROM core, always there.

STACK

```
decimal   ( -- )
```

DESCRIPTION

Sets `base` to ten. It is the safe way back from any other base, because it does not depend on how a number would be read.

SEE ALSO

`hex`, `base`.

EXAMPLE

```
/> hex 10 decimal .
16
```

defer, is, defer!, defer@, action-of

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
defer       ( "name" -- )
is          ( xt "name" -- )
defer!      ( xt2 xt1 -- )
defer@      ( xt1 -- xt2 )
action-of   ( "name" -- xt )
```

DESCRIPTION

`defer name` defines a word whose action can be changed. It should be given one before it is run. `' word is name` makes *name* do what *word* does from then on; every definition that already calls *name* follows the change, which is the point of it.

`action-of name` gives the execution token *name* now runs. `defer!` and `defer@` do the same as `is` and `action-of` for a deferred word given by its execution token, *xt1*.

`is` and `action-of` work both at the prompt and inside a definition, where they take the name that follows them in the definition.

EXAMPLE

```
/> defer greet  : hi ." hi " ;  : yo ." yo " ;
/> ' hi is greet greet  ' yo is greet greet
hi yo
```

[defined], [undefined]

Programming-Tools; in the `tools` library, which is loaded as HyForth starts. Immediate.

STACK

```
[defined]     ( "name" -- flag )
[undefined]   ( "name" -- flag )
```

DESCRIPTION

Take the next word of the line, and leave a flag: `[defined]` true if there is a word of that name, and `[undefined]` true if there is not. Neither is an error for a name that isn't known.

With `[if]` they let a file supply a word only where the system lacks it, or load a library only if it is needed.

SEE ALSO

`[if]`, `find`, `lib`.

EXAMPLE

```
/> [defined] dup .
-1
/> [defined] nothere .
0
/> [undefined] 2rot [if] lib double [then]
/> [defined] 2rot .
-1
```

definitions

Search-Order; in the `search` library (`lib search`).

STACK

```
definitions   ( -- )
```

DESCRIPTION

Makes the first word list in the search order the compilation word list too: from now on, new words go into the list that is searched first. `only forth definitions` puts everything back as it starts.

SEE ALSO

`set-current`, `only`, `also`, `order`.

delete-file, rename-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
delete-file   ( c-addr u -- ior )
rename-file   ( c-addr1 u1 c-addr2 u2 -- ior )
```

DESCRIPTION

`delete-file` removes the file named. `rename-file` gives the file named by the first string the name in the second.

A file is renamed where it is: it stays in its directory, and only the last part of the new name is used. To move a file to another directory, copy it and delete the original.

At the prompt, the two strings of `rename-file` can both be typed with `s"`: the two most recent such strings are kept.

SEE ALSO

`create-file`, `file-status`, the commands `rm` and `mv`.

EXAMPLE

```
/> s" /ram/a.txt" w/o create-file throw close-file throw
/> s" /ram/a.txt" s" b.txt" rename-file .
0
/> s" /ram/b.txt" file-status nip .
0
/> s" /ram/b.txt" delete-file .
0
/> s" /ram/b.txt" delete-file .
-544
```

depth

Core word set; in the ROM core, always there.

STACK

```
depth   ( -- +n )
```

DESCRIPTION

Leaves the number of cells that were on the stack before `depth` ran.

SEE ALSO

`.s`, `ndepth`.

EXAMPLE

```
/> depth .
0
/> 10 20 30 depth .
3
```

digits

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
digits   ( -- a-addr )
```

DESCRIPTION

A variable: the number of significant digits the math functions work to when an answer is not exact. It starts as 12, and can be raised as far as 100. It makes no difference to `n+`, `n-`, `n*` and `n/`, which are always exact.

SEE ALSO

`nsqrt`, `npow`, `to-fixed`.

EXAMPLE

```
/> lib numbers
/> 30 digits !
/> 2 s>n nsqrt n.
1.41421356237309504880168872421
/> npi n.
3.14159265358979323846264338328
```

disasm

The disassembler; in the `disasm` library (`lib disasm`).

STACK

```
disasm   ( addr u -- )
```

DESCRIPTION

Disassembles machine code: shows *u* instructions, starting at *addr*, a line each, with the instruction's address, its bytes, and the instruction as the assembler `as` would write it. A call or a jump to a Forth word has the word's name after it.

While the library is loaded, `see` uses it too: a word of the ROM that is written in assembly language is shown as its instructions, between `code` and `end-code`.

The address of a word's code is its execution token, from `'`. For a colon definition whose code is in a code bank, the token's address holds only the call that leads there.

SEE ALSO

`see`, `dump`, `sys`; the commands `dis` and `db`; Chapter 28.

EXAMPLE

```
/> lib disasm
/> see dup
code dup
 A299  CA        dex
 A29A  B5 23     lda $23,x
 A29C  95 22     sta $22,x
 A29E  B5 43     lda $43,x
 A2A0  95 42     sta $42,x
 A2A2  60        rts
end-code
```

disk-ctl, disk-cmd, disk-start, disk-stop, cards

The devices; in the `disk` library, a file of Forth in `/lib/forth` (`lib disk`).

STACK

```
disk-ctl     ( disk -- c-addr u )
disk-cmd     ( c-addr u disk -- )
disk-start   ( disk -- )
disk-stop    ( disk -- )
cards        ( -- mask )
```

DESCRIPTION

The disks (Chapter 14). A disk is named by its letter, as a character: `char 0` to `char f` for the card on that SPI device, `char x` for the ROM disk, `char r` for the RAM disk and `char s` for the shared one.

`disk-ctl` leaves the text of the disk's control file: what it is and its size, and its file system's label and free space. `disk-cmd` sends that file a command, any of those Chapter 14 lists (`check`, `label` ...). `disk-start` starts a card again, as is needed after one is changed, and `disk-stop` stops a RAM disk and gives its memory back.

`cards` leaves a mask of the SD cards that are there: bit *n* is set for one on SPI device *n*.

SEE ALSO

The commands `df`, `fsck` and `label`; Chapter 14.

EXAMPLE

```
/> lib disk
/> char x disk-ctl type
rom 4 MB 8192 blocks
hydrafs label=ROM
partition at block 1024
free 13 KB of 623 KB
/> cards .
1
```

?do

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate. Compile only.

STACK

```
?do   ( n1 n2 -- )
```

DESCRIPTION

`do`, with a check: if the limit and the starting index are equal, the loop's words are not run at all, and the program goes on after `loop`. Otherwise it is `do` exactly.

This is the one to use whenever the count comes from the program and might be 0, such as a length.

SEE ALSO

`do`, `loop`, `leave`.

EXAMPLE

```
/> : stars  0 ?do [char] * emit loop ;
/> 5 stars
*****
/> 0 stars
```

do, loop, +loop

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
do      ( n1 n2 -- )       n1 the limit, n2 the first index
loop    ( -- )
+loop   ( n -- )
```

DESCRIPTION

A counted loop. `do` takes a limit and a starting index. The words between `do` and `loop` are run with the index at its starting value; `loop` then adds 1 to the index, and if it has not reached the limit, goes back for another turn. So `10 0 do ... loop` runs ten times, with the index 0 to 9: the limit itself is never used. Inside the loop, `i` leaves the index.

`+loop` takes its step from the stack each time round, in place of 1. With a positive step the loop ends when the index reaches or passes the limit. With a negative step it counts down, and ends when the index passes *below* the limit: this time the limit itself is used, so `0 10 do i . -3 +loop` shows 10, 7, 4, 1.

The index and the limit are kept on the return stack while the loop runs (see `>r`).

NOTES

`do` always runs its words at least once. If the limit equals the starting index, as when a count turns out to be 0, the loop does not skip: it runs 65,536 times, the index going all the way round. Use `?do` wherever that can happen.

SEE ALSO

`?do`, `i`, `leave`, `unloop`, `begin`.

EXAMPLE

```
/> : count-up  3 0 do i . loop ;
/> count-up
0 1 2
/> : by-threes  10 0 do i . 3 +loop ;
/> by-threes
0 3 6 9
/> : down  0 10 do i . -3 +loop ;
/> down
10 7 4 1
```

does>

Core word set; in the ROM core. Immediate, compile only.

STACK

Running the word that contains it: (--). Running a word it made: (... -- *addr* ...).

DESCRIPTION

`does>` is used inside a defining word, after `create`, and splits the definition in two. What comes before it runs when the defining word is used, and makes a new word; what comes after it is what every word made that way does when it is run, starting with the address of that word's own data on the stack.

```
: constant  create ,  does> @ ;
```

`5 constant five` runs `create ,`: a word `five` is made, with the cell 5 in its data. Running `five` then puts the address of that cell on the stack and runs `@`.

The data address is the one `>body` gives for the word. `does>` changes the most recent word made by `create`.

EXAMPLE

```
/> : counter  create 0 ,  does> dup @ 1+ dup rot ! ;
/> counter hits  hits . hits . hits .
1 2 3
```

drop

Core word set; in the ROM core, always there.

STACK

```
drop   ( x -- )
```

DESCRIPTION

Removes the top cell of the stack. With nothing on the stack it is the error −4, `stack underflow`, reported when the line has been run.

SEE ALSO

`2drop`, `nip`, `dup`.

EXAMPLE

```
/> 1 2 drop .s
<1> 1
/> drop drop
stack underflow
```

dump

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
dump   ( addr u -- )
```

DESCRIPTION

Shows *u* bytes of memory, from *addr*, eight to a line: the address of the line's first byte in hex, the bytes in hex, and then the same bytes as characters, with a dot for any that can't be printed.

SEE ALSO

`?`, `c@`, `type`, the command `xd`.

EXAMPLE

```
/> create msg  char H c, char y c, char d c, char r c, char a c, 0 c, 1 c, 255 c, 16 c,
/> msg 9 dump
3967: 48 79 64 72 61 00 01 FF  Hydra...
396F: 10                       .
```

?dup

Core word set; in the ROM core, always there.

STACK

```
?dup   ( x -- 0 | x x )
```

DESCRIPTION

Copies the top cell if it is not zero, and leaves a zero as it is. It is for a value that is also a flag: `?dup if ... then` runs the part between with the value, and leaves nothing behind when the value was zero, where `dup if ... else drop then` would be needed without it.

SEE ALSO

`dup`, `if`.

EXAMPLE

```
/> 5 ?dup .s
<2> 5 5
/> 2drop 0 ?dup .s
<1> 0
```

dup

Core word set; in the ROM core, always there.

STACK

```
dup   ( x -- x x )
```

DESCRIPTION

Makes a copy of the top cell of the stack, and leaves the copy on top of it. Almost every word uses up what it takes, so `dup` is how a value is kept for a second use.

SEE ALSO

`?dup`, `over`, `2dup`, `drop`.

EXAMPLE

```
/> 7 dup * .
49
/> 1 2 dup .s
<3> 1 2 2
```

ekey, ekey?

Facility; in the `facility` library (`lib facility`).

STACK

```
ekey    ( -- u )
ekey?   ( -- flag )
```

DESCRIPTION

`ekey` waits for a key and leaves a number for it, the special keys included: an arrow or a function key, which sends several characters, comes back as one number. For an ordinary key the number is its character's code; for a special key it is the value of one of the `k-` constants, `k-up`, `k-f1` and the rest, which are 128 and above.

`ekey?` is true if a key is waiting, as `key?` is.

SEE ALSO

`ekey>char`, `ekey>fkey`, `k-up`, `key`.

EXAMPLE

Here the up arrow was pressed:

```
/> lib facility
/> ekey k-up = .
-1
```

ekey>char, ekey>fkey

Facility; in the `facility` library (`lib facility`).

STACK

```
ekey>char   ( u -- u false | char true )
ekey>fkey   ( u -- u false | u true )
```

DESCRIPTION

Sort what `ekey` gave. `ekey>char` leaves true above it if it is an ordinary character, and false if it is a special key. `ekey>fkey` is the other way round: true if it is a special key, one of the `k-` values, and false if it is a character. Either way the number itself stays underneath.

SEE ALSO

`ekey`, `k-up`.

EXAMPLE

Here A and then the up arrow were pressed:

```
/> lib facility
/> : show-key  ekey ekey>fkey if ." special " else ." character " then . ;
/> show-key
character 65
/> show-key
special 128
```

emit

Core word set; in the ROM core, always there.

STACK

```
emit   ( x -- )
```

DESCRIPTION

Writes one character to standard output: the one whose code is *x*. Codes below 32 are the terminal's control characters: 10 moves down a line, 7 rings the bell.

SEE ALSO

`type`, `cr`, `char`, `key`.

EXAMPLE

```
/> 72 emit 105 emit
Hi
/> char A emit
A
```

emit?

Facility; in the `facility` library (`lib facility`).

STACK

```
emit?   ( -- flag )
```

DESCRIPTION

In the standard, true if `emit` would not have to wait. On HyForth output never has to be asked about: the result is always true.

SEE ALSO

`emit`, `key?`.

environment?

Core word set; in the ROM core, always there.

STACK

```
environment?   ( c-addr u -- false | i*x true )
```

DESCRIPTION

In the standard, asks the system about itself by name: the size of a counted string, the largest number, which word sets are there. HyForth knows no queries: the result is false, whatever the string. The facts themselves are in this chapter's introduction.

To find out whether a word is there, use `[defined]`.

SEE ALSO

`[defined]`, `unused`, `libs`.

EXAMPLE

```
/> s" /counted-string" environment? .
0
```

erase

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
erase   ( addr u -- )
```

DESCRIPTION

Sets *u* bytes of memory, from *addr* on, to zero: it is `0 fill`. Memory from `allot` is not cleared, so a table that must start as zeros is erased first.

SEE ALSO

`fill`, `blank`, `allot`.

EXAMPLE

```
/> create buf 8 allot
/> buf 8 char A fill
/> buf 4 erase
/> buf c@ . buf 3 + c@ . buf 4 + c@ .
0 0 65
```

evaluate

Core word set; in the ROM core, always there.

STACK

```
evaluate   ( i*x c-addr u -- j*x )
```

DESCRIPTION

Runs a string as a line of Forth. The string becomes the input for as long as it takes to interpret it, and then the input goes back to what it was: the rest of the line that `evaluate` was on. What the string's words do to the stack is what `evaluate` does to it.

The string is interpreted, or compiled if a definition is open, exactly as typed text would be: it can define words, and a word in it that isn't known is the same error as at the prompt.

SEE ALSO

`included`, `execute`, `source`, `parse`.

EXAMPLE

```
/> s" 2 3 + ." evaluate
5
/> : thirty  s" 10 20 +" evaluate ;
/> thirty .
30
/> s" nothere" evaluate
nothere ?
```

execute

Core word set; in the ROM core, always there.

STACK

```
execute   ( i*x xt -- j*x )
```

DESCRIPTION

Runs the word whose execution token is on top of the stack, with the rest of the stack as the word finds it. A token comes from `'`, `[']`, `:noname`, `find` or `search-wordlist`.

Tokens are numbers, so they can be stored: a table of tokens, indexed and `execute`d, is how a program chooses one of several actions without a chain of `if`s.

NOTES

The token is not checked. A number that is not a word's token sends the processor to that address, and what follows is most likely the end of this HyForth.

SEE ALSO

`'`, `:noname`, `defer`, `compile,`, `catch`.

EXAMPLE

```
/> : hi  ." hi " ;  : bye-bye  ." bye " ;
/> create acts  ' hi ,  ' bye-bye ,
/> : act  cells acts + @ execute ;
/> 0 act 1 act
hi bye
```

exit

Core word set; in the ROM core, always there. Compile only.

STACK

```
exit   ( -- )
```

DESCRIPTION

Returns from the word being run, at once, to the word that called it. The rest of the definition is not run. Inside a `do` loop, `unloop` must come first; inside a `begin` loop, nothing is needed.

What has been put on the return stack with `>r` must be taken off before `exit`, or the return goes astray.

NOTES

Typed at the prompt, where there is no word to return from, `exit` ends HyForth itself.

SEE ALSO

`unloop`, `leave`, `;`, `quit`, `bye`.

EXAMPLE

```
/> : half  dup 1 and if drop ." odd" exit then 2/ . ;
/> 10 half
5
/> 7 half
odd
```

exits

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
exits   ( n -- )
```

DESCRIPTION

Ends HyForth, with *n* as its exit code: what the program that started it sees as its status. `bye` ends it with 0. A script run as a command uses `exits` to say whether it succeeded.

SEE ALSO

`bye`, `status`, `exit`.

+field, field:, cfield:

Facility; in the `facility` library (`lib facility`).

STACK

```
+field    ( n1 n2 "name" -- n3 )
field:    ( n1 "name" -- n2 )
cfield:   ( n1 "name" -- n2 )
name      ( addr1 -- addr2 )
```

DESCRIPTION

Add a field to the structure being described. Each takes the offset reached so far, defines *name* as a word that adds that offset to an address, and leaves the offset of whatever comes next. `+field` adds a field of *n2* bytes: any size, for a string or a record inside a record. `field:` adds a cell, 2 bytes, and `cfield:` a character, 1 byte.

Used later, a field's name turns the address of a record into the address of that field in it, ready for `@`, `!`, `c@` or `move`. The first field's offset is 0.

SEE ALSO

`begin-structure`, `cells`, `@`.

EXAMPLE

```
/> lib facility
/> begin-structure person  16 +field p.name  field: p.age  end-structure
/> person .
18
/> 0 p.name . 0 p.age .
0 16
```

file-position, reposition-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
file-position     ( fileid -- ud ior )
reposition-file   ( ud fileid -- ior )
```

DESCRIPTION

A file's place is where its next read or write will happen, counted in bytes from its start. `file-position` leaves it, and `reposition-file` sets it. The place is a double number, because a file can be longer than 65,535 bytes: write `0.` for the start.

SEE ALSO

`file-size`, `read-file`, `d.`.

EXAMPLE

```
/> s" /rom/lib/shell" r/o open-file throw value fd
/> 5. fd reposition-file .
0
/> pad 5 fd read-file 2drop  pad 5 type
forth
/> fd file-position . . .
0 0 10
/> fd close-file drop
```

file-size, resize-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
file-size     ( fileid -- ud ior )
resize-file   ( ud fileid -- ior )
```

DESCRIPTION

`file-size` leaves the length of an open file in bytes, as a double. `resize-file` sets it: a file made shorter loses what was beyond the new end, and one made longer gains bytes whose contents are not defined.

SEE ALSO

`file-position`, `file-status`, `delete-file`.

EXAMPLE

```
/> s" /ram/g.txt" w/o create-file throw value fd
/> s" 0123456789" fd write-file throw
/> fd file-size . . .
0 0 10
/> 4. fd resize-file .
0
/> fd file-size . . .
0 0 4
/> fd close-file drop
```

file-status

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
file-status   ( c-addr u -- x ior )
```

DESCRIPTION

Asks about a file by name, without opening it. The ior is 0 if the file is there, and –544, `not found`, if it is not: this is the test for whether a file exists. When it is there, *x* is its mode, the permission bits the system keeps for it (438, octal 666, for a file that can be read and written).

SEE ALSO

`open-file`, `file-size`, `open-dir`.

EXAMPLE

```
/> s" /rom/lib/shell" file-status nip .
0
/> s" /rom/nothere" file-status nip .
-544
```

fill

Core word set; in the ROM core, always there.

STACK

```
fill   ( c-addr u char -- )
```

DESCRIPTION

Stores the character in each of *u* bytes of memory, from *c-addr* on. With a *u* of 0 it does nothing.

SEE ALSO

`erase`, `blank`, `move`.

EXAMPLE

```
/> create buf 8 allot
/> buf 8 char - fill
/> buf 3 char * fill
/> buf 8 type
***-----
```

find

Core word set; in the ROM core, always there.

STACK

```
find   ( c-addr -- c-addr 0 | xt 1 | xt -1 )
```

DESCRIPTION

Looks up a word whose name is given as a counted string. If it is not found, the string's address is left with 0 above it. If it is found, its execution token is left, with 1 above it for an immediate word and −1 for an ordinary one.

`find` is the old way to search the dictionary. A counted string is awkward to make outside a definition; `'` at the prompt, `[defined]` in a file, and `search-wordlist`, which takes an address and a length, are easier.

SEE ALSO

`'`, `[defined]`, `search-wordlist`, `c"`.

EXAMPLE

```
/> : name  c" dup" ;
/> name find . ' dup = .
-1 -1
/> : none  c" nothere" ;
/> none find . count type
0 nothere
```

flush-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
flush-file   ( fileid -- ior )
```

DESCRIPTION

Makes sure that everything written to the file so far has reached the disk, without closing it. A program that keeps a file open for a long time, a log for instance, calls it after each thing that must not be lost if the power goes.

SEE ALSO

`close-file`, `write-file`.

form

Facility; in the `facility` library (`lib facility`). A HyForth word, not the standard's.

STACK

```
form   ( -- rows cols )
```

DESCRIPTION

Leaves the size of the window HyForth is running in: its number of rows, and above that its number of columns. A program that draws on the screen asks this, not assuming 24 by 80: a window can be any size, and is as small as the smaller of the terminals it is shown on.

SEE ALSO

`at-xy`, `k-resize`.

EXAMPLE

```
/> lib facility
/> form . .
80 24
```

forth-wordlist

Search-Order; in the `search` library (`lib search`).

STACK

```
forth-wordlist   ( -- wid )
```

DESCRIPTION

Leaves the identifier of the word list that holds the system's words: the ROM core's, those of every library loaded, and, unless a program arranges otherwise, the program's own definitions.

SEE ALSO

`wordlist`, `get-order`, `forth`, `traverse-wordlist`.

free

Memory-Allocation; in the `memory` library (`lib memory`).

STACK

```
free   ( a-addr -- ior )
```

DESCRIPTION

Gives back memory that `allocate` or `resize` handed out. The address must be exactly one that they gave, and it must not be used afterward. The ior is 0, or −60, `free failed`, for an address that is not an allocated block's: freeing the same block twice is that error.

SEE ALSO

`allocate`, `resize`.

get-current, set-current

Search-Order; in the `search` library (`lib search`).

STACK

```
get-current   ( -- wid )
set-current   ( wid -- )
```

DESCRIPTION

The **compilation word list** is the one that new definitions are added to. `get-current` leaves its identifier, and `set-current` makes another word list the one.

Where definitions go and where words are looked for are separate settings. A word defined into a word list that is not in the search order can't be found by name until the list is added to the order.

SEE ALSO

`definitions`, `get-order`, `wordlist`.

get-dir, set-dir

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
get-dir   ( c-addr1 u1 -- c-addr2 u2 )
set-dir   ( c-addr u -- ior )
```

DESCRIPTION

`get-dir` puts the current directory's path into the buffer given, *c-addr1* and its size *u1*, and leaves it as a string: as much of it as fits. `set-dir` makes the directory named the current one, and leaves 0, or the reason it could not.

`set-dir` is the shell's `cd` in a form that works inside a definition. The current directory is where names that don't begin with `/` are looked for.

SEE ALSO

`cd`, `open-dir`, `=mkdir`.

EXAMPLE

```
/> lib hydra
/> s" /rom/lib" set-dir .
0
/rom/lib> pad 80 get-dir type
/rom/lib
/rom/lib> s" /nothere" set-dir .
-544
/rom/lib> s" /" set-dir drop
```

get-order, set-order

Search-Order; in the `search` library (`lib search`).

STACK

```
get-order   ( -- widn ... wid1 n )
set-order   ( widn ... wid1 n -- )
```

DESCRIPTION

The **search order** is the list of word lists that are searched for a name, in order: the first one that has the name wins. `get-order` leaves the identifiers of all of them, the first to be searched on top, and above them their number. `set-order` takes the same picture and makes it the search order. There can be eight word lists in the order.

`-1 set-order` sets the least order a system can work with: the system's word list alone, as `only` does.

To add a word list at the front of the order: `get-order mine swap 1+ set-order`.

NOTES

`0 set-order` empties the search order, and then *no* word can be found, `only` and `set-order` among them: every line is answered with the word and a `?`. There is no way back but to end HyForth, with CTRL-D.

ERRORS

THROW CODE	MESSAGE	CAUSE
–49	`search order full`	More than eight word lists

SEE ALSO

`order`, `only`, `also`, `previous`, `definitions`.

EXAMPLE

```
/> lib search
/> wordlist constant mine
/> mine set-current  : secret 42 ;  definitions
/> get-order mine swap 1+ set-order
/> secret .
42
/> only forth
/> [defined] secret .
0
```

getenv, setenv, unsetenv

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
getenv     ( c-addr1 u1 -- c-addr2 u2 )
setenv     ( c-addr1 u1 c-addr2 u2 -- )
unsetenv   ( c-addr u -- )
```

DESCRIPTION

The environment's variables, which HyForth shares with the commands it runs (section 17.7).

`getenv` leaves the value of the variable named, as a string. A variable that is a list of several words comes with a space between them. If there is no such variable, the length is 0.

`setenv` sets the variable named by the first string to the second, as one word of at most 126 characters. `unsetenv` removes a variable; one that isn't there is not an error.

The commands HyForth starts get a copy of its environment, so a variable set here is `$name` in a command line typed afterward.

SEE ALSO

`sh`, `%`, `arg`.

EXAMPLE

```
/> s" colour" s" red" setenv
/> s" colour" getenv type
red
/> echo my colour is $colour
my colour is red
/> s" colour" unsetenv
/> s" colour" getenv nip .
0
```

gpio, gpio!

The devices; in the `gpio` library, a file of Forth in `/lib/forth` (`lib gpio`).

STACK

```
gpio    ( pin -- level )
gpio!   ( pin level -- )
```

DESCRIPTION

The eight pins of the I/O header J27, numbered 0 to 7 (section 17.2). `gpio` leaves a pin's level now, 0 or

1. `gpio!` makes the pin an output and sets it: low for a *level* of 0, high for anything else.

Every pin is an input when the Hydra starts, and an input with nothing connected reads 1.

The library's words work through the files of `/dev/gpio`, and a failure is thrown, with the word's name: `gpio: not found` for a pin that is not 0 to 7.

SEE ALSO

`gpio-in`, `gpio-port`, `gpio-state`, `ctl`.

EXAMPLE

```
/> lib gpio
/> 2 gpio .
1
/> 2 0 gpio!  2 gpio .
0
/> 2 gpio-in
```

gpio-ca1!, gpio-ca2!, gpio-wait

The devices; in the `gpio` library, a file of Forth in `/lib/forth` (`lib gpio`).

STACK

```
gpio-ca1!   ( rise? -- )
gpio-ca2!   ( n -- )
gpio-wait   ( -- count )
```

DESCRIPTION

The header's two handshake lines. `gpio-ca1!` chooses which change of CA1 is counted: its rises, from 0 to 1, for a true flag, and its falls for a false one. `gpio-wait` waits for the next such change, and leaves the number counted so far: this is how a program waits for a button without watching it.

`gpio-ca2!` sets CA2: 0 makes it an output that is low, 1 an output that is high, and –1 an input.

SEE ALSO

`gpio-state`, `key?`; section 17.2.

gpio-in, gpio-out, gpio-ddr!

The devices; in the `gpio` library, a file of Forth in `/lib/forth` (`lib gpio`).

STACK

```
gpio-in     ( pin -- )
gpio-out    ( pin -- )
gpio-ddr!   ( byte -- )
```

DESCRIPTION

Set the pins' directions. `gpio-in` makes one pin an input, and `gpio-out` an output. `gpio-ddr!` sets all eight at once: bit *n* of the byte is 1 to make pin *n* an output, and 0 for an input.

SEE ALSO

`gpio!`, `gpio-port!`, `gpio-state`.

gpio-port, gpio-port!

The devices; in the `gpio` library, a file of Forth in `/lib/forth` (`lib gpio`).

STACK

```
gpio-port    ( -- byte )
gpio-port!   ( byte -- )
```

DESCRIPTION

All eight pins at once, as one byte: bit *n* is pin *n*. `gpio-port` leaves their levels. `gpio-port!` sets the levels of the pins that are outputs; those that are inputs stay inputs, and their bits are ignored.

SEE ALSO

`gpio`, `gpio-ddr!`.

EXAMPLE

```
/> lib gpio
/> 15 gpio-ddr!  5 gpio-port!
/> gpio-port .
245
/> 0 gpio-ddr!
```

gpio-state

The devices; in the `gpio` library, a file of Forth in `/lib/forth` (`lib gpio`).

STACK

```
gpio-state   ( -- c-addr u )
```

DESCRIPTION

Leaves the text of the pins' control file: a line for each pin, with its number, its direction and its level, then CA1's edge and its count, and CA2's setting. The text is in a buffer of the library's, good until its next word is used.

SEE ALSO

`gpio`, `gpio-ddr!`.

EXAMPLE

```
/> lib gpio
/> gpio-state type
0 in 1
1 in 1
2 in 1
3 in 1
4 in 1
5 in 1
6 in 1
7 in 1
ca1 fall 0
ca2 in
```

here

Core word set; in the ROM core, always there.

STACK

```
here   ( -- addr )
```

DESCRIPTION

Leaves the address of the next free byte of data space: where `,`, `c,` and `allot` will put what they are given, and where the next `create`d word's data will begin.

Data space is the dictionary, in the task's own memory: it holds every word's name, and the data of variables and of `create`d words. It ends at `$7F00`, and `unused` says how much is left. The code of colon definitions is not here: it is compiled into RAM banks of HyForth's own, so a long program doesn't use data space up. Defining any word still moves `here`, by the size of its name.

SEE ALSO

`allot`, `,`, `unused`, `create`, `code-banks`.

EXAMPLE

```
/> here 10 allot here swap - .
10
```

hex

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
hex   ( -- )
```

DESCRIPTION

Sets `base` to sixteen: from then on numbers are read and printed in hexadecimal, until `decimal`. The digits above 9 may be typed as capitals or small letters, and are printed as capitals.

For a single hex number among decimal ones, the `$` mark is simpler, and doesn't change the base.

SEE ALSO

`decimal`, `base`, `dump`.

EXAMPLE

```
/> hex ff . 10 . decimal
FF 10
/> $ff .
255
```

i, j

Core word set; in the ROM core, always there. Compile only.

STACK

```
i   ( -- n )
j   ( -- n )
```

DESCRIPTION

Inside a `do` loop, `i` leaves the loop's index for this turn. Inside a loop that is itself inside another, `j` leaves the index of the outer one.

They read the return stack, where the loops keep their numbers. So they are right only directly in the loop's own words: not in a word the loop calls, and not after a `>r` in the loop that has not been matched by its `r>`. Used outside a loop, `i` leaves whatever is on the return stack.

SEE ALSO

`do`, `?do`, `r@`, `unloop`.

EXAMPLE

```
/> : grid  2 0 do 3 0 do j . i . space loop cr loop ;
/> grid
0 0  0 1  0 2
1 0  1 1  1 2
```

i2c-read, i2c-write, i2c-devices, i2c?, i2c-speed, i2c-reg-size

The devices; in the `i2c` library, a file of Forth in `/lib/forth` (`lib i2c`).

STACK

```
i2c-read       ( addr reg c-addr u -- )
i2c-write      ( addr reg c-addr u -- )
i2c-devices    ( -- )
i2c?           ( addr -- flag )
i2c-speed      ( khz -- )
i2c-reg-size   ( n -- )
```

DESCRIPTION

The I²C bus on the I/O header (section 17.3). *addr* is a chip's address on the bus, `$08` to `$77`.

`i2c-read` reads *u* bytes, 64 at most, from the chip into memory at *c-addr*, and `i2c-write` sends *u* bytes from there. *reg* is the register inside the chip to start from; it is used only when a register size has been set, and is otherwise given as 0.

`i2c-devices` prints the addresses of the chips that answer, in hex, and `i2c?` leaves true if one answers at the address given.

`i2c-speed` sets the bus's speed in kilohertz, 1 to 100. `i2c-reg-size` says how a register's number is sent before the data: 0 not at all, 1 as one byte, 2 as two, high byte first.

A chip that does not answer is the error `i/o error`, thrown with the word's name.

SEE ALSO

`gpio`, `spi`, `ctl`.

EXAMPLE

With nothing on the bus:

```
/> lib i2c
/> $50 i2c? .
0
/> $50 0 pad 2 i2c-read
i2c-read: i/o error
```

if, else, then

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
if     ( x -- )
else   ( -- )
then   ( -- )
```

DESCRIPTION

A choice, inside a definition. `if` takes the top of the stack as a flag. If it is true, which is any value but 0, the words between `if` and `else` are run, and then those after `then`. If it is 0, the words between `else` and `then` are run instead. `else` and its part can be left out: then a false flag skips straight to `then`.

`then` is where the two paths join, not what happens "next": it reads better as "and then, either way". Choices can be put inside one another to any depth, each `if` with its own `then`.

The flag is used up. To test a value and keep it, `dup` it first, or use `?dup`.

NOTES

These can't be used at the prompt, outside a definition: `if: compile only`. For a choice while a file is loading, there is `[if]`.

An `if` with no `then` is not reported when the definition ends, and the word goes wrong when it is run.

SEE ALSO

`case`, `?dup`, `0=`, `[if]`, `begin`.

EXAMPLE

```
/> : sign  dup 0< if ." negative" else 0= if ." zero" else ." positive" then then ;
/> -5 sign
negative
/> 0 sign
zero
/> : warn  100 > if ." too big!" then ;
/> 50 warn 500 warn
too big!
```

[if], [else], [then]

Programming-Tools; in the `tools` library, which is loaded as HyForth starts. Immediate.

STACK

```
[if]     ( flag -- )
[else]   ( -- )
[then]   ( -- )
```

DESCRIPTION

A choice made as the text is read, not when a word runs. `[if]` takes a flag. If it is true, the text goes on being interpreted, and at `[else]` everything is skipped up to `[then]`. If it is false, everything up to `[else]` (or to `[then]`, if there is no `[else]`) is skipped, as if it were not there.

What is skipped is not looked at beyond finding the matching `[else]` and `[then]`, so it may hold definitions, and words the system does not have. In a file the parts may run over many lines, and the three may be nested.

They work at the prompt and in a file. Inside a colon definition, they choose which text is *compiled*.

SEE ALSO

`[defined]`, `if`, `included`.

EXAMPLE

```
/> 1 [if] .( yes) [else] .( no) [then]
yes
/> 0 [if] .( yes) [else] .( no) [then]
no
/> 0 [if] .( skipped) [then] .( after)
after
```

immediate

Core word set; in the ROM core, always there.

STACK

```
immediate   ( -- )
```

DESCRIPTION

Marks the word most recently defined as immediate. An immediate word is run, not compiled, when the compiler meets it inside a definition: so it can do work at compile time, and can compile whatever it chooses into the definition. `if`, `do`, `."` and `;` are immediate words.

It is written straight after the `;` of the word it marks.

SEE ALSO

`postpone`, `[`, `literal`, `compile,`, `[compile]`.

EXAMPLE

```
/> : shout  ." compiling! " ; immediate
/> : quiet  shout ;
compiling!
/> quiet
```

include-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
include-file   ( i*x fileid -- j*x )
```

DESCRIPTION

Loads Forth source from a file that is already open: its lines are interpreted from the file's current place to its end, and the file is then closed. `included`, which takes a name, is the usual way; this one is for a file that the program has opened and looked at first.

SEE ALSO

`included`, `open-file`, `evaluate`.

EXAMPLE

```
/> s" /ram/inc.fs" w/o create-file throw value fd
/> s" .( from the file ) 1 2 + ." fd write-line throw  fd close-file throw
/> s" /ram/inc.fs" r/o open-file throw include-file
from the file 3
```

included, include

File Access; in the ROM core, always there.

STACK

```
included   ( i*x c-addr u -- j*x )
include    ( i*x "name" -- j*x )
```

DESCRIPTION

Load a file of Forth source: each of its lines is interpreted, in turn, as if it had been typed. `included` takes the file's name as a string, and `include` takes it from the line. When the file ends, the input goes back to where it was.

A file can include others, to eight deep. A line of a file can be up to 128 characters long.

An error in the file stops the loading and is reported with the file's name and the number of the line: `/ram/bad.fs:1: nothere ?`. What the file had defined before the error stays defined.

Each loading runs the file again and defines its words again. To load a file only if it has not been loaded already, use `required`, or for a library, `lib`.

ERRORS

THROW CODE	MESSAGE	CAUSE
−512 − *code*	*name*`: not found`, or the system's other messages	The file can't be opened
−258	`sources nested too deeply`	More than eight files deep

SEE ALSO

`required`, `lib`, `evaluate`, `marker`, `include-file`.

EXAMPLE

```
/> s" /ram/t.fs" w/o create-file drop
/> dup s" : loaded 77 ; .( loading) " rot write-line drop close-file drop
/> include /ram/t.fs
loading
/> loaded .
77
/> include /ram/nothere.fs
/ram/nothere.fs: not found
```

int?, fixed?, rational?, complex?

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
int?        ( -- flag ) ( N: x -- )
fixed?      ( -- flag ) ( N: x -- )
rational?   ( -- flag ) ( N: x -- )
complex?    ( -- flag ) ( N: x -- )
```

DESCRIPTION

Remove the top number and leave a flag: is it an integer, a decimal, a fraction, a complex number? The test is of the form the number is in, not its value: 1.0 is a decimal, not an integer, though the two are equal. Arithmetic gives its results in the simplest form, so `4/2` is an integer.

SEE ALSO

`n=`, `to-fixed`, `to-rational`.

EXAMPLE

```
/> lib numbers
/> 1/2 rational? . 1.5 fixed? . 2i complex? .
-1 -1 -1
/> 1.0 int? . 4/2 int? .
0 -1
```

invert

Core word set; in the ROM core, always there.

STACK

```
invert   ( x1 -- x2 )
```

DESCRIPTION

Turns every bit of the cell over. Of a well-formed flag it is "not": `true invert` is false. Of any other number it is not: `5 invert` is −6, which is still true. For "is zero", which is the "not" of any value, use `0=`.

SEE ALSO

`0=`, `negate`, `xor`.

EXAMPLE

```
/> 0 invert . 5 invert .
-1 -6
/> 3 5 < invert .
0
```

ior>text

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
ior>text   ( ior -- c-addr u )
```

DESCRIPTION

Leaves the system's message for an ior, the result that the file words and the `sys-` words leave: for −544, `not found`. For a number that is not a system error's, the standard's throw codes among them, the string is empty.

`throw` prints the same message. `ior>text` is for a program that wants to word its own report.

SEE ALSO

`throw`, `open-file`, `sys-`.

EXAMPLE

```
/> lib hydra
/> -544 ior>text type
not found
/> s" /ram/nothere" r/o open-file nip ior>text type
not found
```

k-resize, k-focus

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
k-resize   ( -- u )
k-focus    ( -- u )
```

DESCRIPTION

Constants, 150 and 151: two more numbers that `ekey` can give, which are not keys but news from the console. `k-resize` comes when the window's size has changed (ask `form` for the new one), and `k-focus` when the window has been given, or has lost, the keyboard.

SEE ALSO

`ekey`, `form`.

k-shift-mask, k-ctrl-mask, k-alt-mask

Facility; in the `facility` library (`lib facility`).

STACK

```
k-shift-mask   ( -- u )
k-ctrl-mask    ( -- u )
k-alt-mask     ( -- u )
```

DESCRIPTION

Constants, one bit each: 256, 512 and 1024. When a special key is pressed with SHIFT, CTRL or ALT held, and the terminal reports it, `ekey`'s number has the matching bit set as well as the key's own value. `and` with a mask tests for the modifier, and `255 and` leaves the key.

SEE ALSO

`ekey`, `k-up`.

k-up, k-down, k-left, k-right, k-home, k-end, k-prior, k-next, k-insert, k-delete, k-f1, k-f12

Facility; in the `facility` library (`lib facility`).

STACK

```
k-up ... k-f12   ( -- u )
```

DESCRIPTION

Constants: the numbers `ekey` gives for the special keys. `k-prior` and `k-next` are PAGE UP and PAGE DOWN. The function keys are `k-f1` to `k-f12`, twelve numbers in a row.

KEY	VALUE	KEY	VALUE
`k-up`	128	`k-end`	133
`k-down`	129	`k-insert`	134
`k-right`	130	`k-delete`	135
`k-left`	131	`k-prior`	136
`k-home`	132	`k-next`	137
`k-f1` … `k-f12`	138 … 149		

A program should compare with the names, not the numbers.

SEE ALSO

`ekey`, `ekey>fkey`, `k-shift-mask`, `k-resize`.

key

Core word set; in the ROM core, always there.

STACK

```
key   ( -- char )
```

DESCRIPTION

Waits for a key, and leaves its character's code. The key is not shown on the screen, and no ENTER is needed: the keyboard is read a key at a time.

Only keys that send one character are whole in one `key`: an arrow or a function key sends several, and each `key` gives the next. For those, the Facility library has `ekey`.

SEE ALSO

`key?`, `ekey`, `accept`, `emit`.

EXAMPLE

Here A was pressed:

```
/> key .
65
```

key?

Facility; in the `facility` library (`lib facility`).

STACK

```
key?   ( -- flag )
```

DESCRIPTION

True if a key has been pressed and is waiting to be read; false if not. It does not wait, and it does not take the key: `key` then reads it at once.

This is what lets a program do something else while it watches the keyboard: a game's loop, or a display that updates until a key is pressed.

The first `key?` puts the keyboard into the mode where each key comes as it is pressed, unshown, and it stays so until the next line is read.

SEE ALSO

`key`, `ekey?`, `ms`.

EXAMPLE

```
lib facility
: spin  begin [char] . emit 100 ms key? until key drop ;
```

leave, unloop

Core word set; in the ROM core, always there. Compile only.

STACK

```
leave    ( -- )
unloop   ( -- )
```

DESCRIPTION

`leave` ends the innermost `do` loop at once: the loop's numbers are thrown away, and the program goes on after `loop`. It is usually inside an `if`.

`unloop` throws away the numbers of the innermost loop and nothing more. It is needed before `exit` inside a loop, because the loop's numbers are on the return stack, on top of the address `exit` must return to. For a loop inside a loop, one `unloop` for each.

SEE ALSO

`do`, `exit`, `i`.

EXAMPLE

```
/> : to-five  10 0 do i 5 = if leave then i . loop ." done" ;
/> to-five
0 1 2 3 4 done
/> : find-five  10 0 do i 5 = if unloop exit then i . loop ." not reached" ;
/> find-five
0 1 2 3 4
```

-lib

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
-lib   ( "name" -- )
```

DESCRIPTION

Hides a loaded library: its words are no longer found, though it stays in memory, and words already compiled that use it go on working. `lib` with the same name brings them back.

This settles a clash, when two libraries have a word of the same name and the other one's is wanted.

SEE ALSO

`lib`, `libs`, `marker`.

EXAMPLE

```
/> lib bits
/> -lib bits
/> libs
forth coreext exception file tools shell (bits)
```

lib

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
lib   ( "name" -- )
```

DESCRIPTION

Makes a library's words available. If the library is not loaded, it is loaded: `lib` looks in `/lib/forth` for *name*`.fl`, a library compiled in advance, and failing that for *name*`.fs`, a file of Forth source. If it is loaded already, nothing is loaded again; and if `-lib` had hidden it, its words are found once more.

So `lib` can be written at the top of every file that needs a library, at no cost.

Most of HyForth is in libraries: the ROM core holds only the Core word set, and each other word set (`double`, `string`, `facility`, `search` ...) and each of the Hydra's own (`hydra`, `numbers`, `sound`, `video`, `gpio` ...) is loaded when it is asked for. Every entry of this chapter names its word's library. A compiled library is small and loads at once.

ERRORS

THROW CODE	MESSAGE	CAUSE
–512 – *code*	*name*`.fs: not found`	There is no library of that name
–259	`not this forth's library`	A `.fl` file made for another version of HyForth

SEE ALSO

`libs`, `-lib`, `required`, `included`.

EXAMPLE

```
/> lib bits
/> lib nothere
nothere.fs: not found
```

library, end-library

Search-Order; in the `search` library (`lib search`). HyForth's own words, not the standard's.

STACK

```
library       ( "name" -- wid )
end-library   ( wid -- )
```

DESCRIPTION

Bracket the definitions of a library that has a word list of its own. `library` makes a new word list, defines *name* as a constant that leaves its identifier, puts the list first in the search order, and makes it the one that definitions go into; it leaves the identifier of the word list that definitions went into before. `end-library` takes that identifier and sends definitions back there. The library's word list stays in the search order, so its words go on being found.

A file written this way keeps its names apart from the system's: they can be searched on their own with `search-wordlist`, and taken out of the order, with `previous` or a `set-order`, without being lost.

SEE ALSO

`wordlist`, `definitions`, `only`, `lib`.

EXAMPLE

```
/> lib search
/> library shapes
/> : square  dup * ;
/> end-library
/> 5 square .
25
/> s" square" shapes search-wordlist nip .
-1
```

libs

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
libs   ( -- )
```

DESCRIPTION

Lists the libraries that are loaded, in the order they were loaded. `forth` is the ROM core itself. A library that `-lib` has hidden is shown in parentheses.

SEE ALSO

`lib`, `-lib`, `words`.

EXAMPLE

```
/> libs
forth coreext exception file tools shell
/> lib bits
/> libs
forth coreext exception file tools shell bits
```

line, box, bar

The screen; in the `video` library (`lib video`).

STACK

```
line   ( x0 y0 x1 y1 -- )
box    ( x0 y0 x1 y1 -- )
bar    ( x0 y0 x1 y1 -- )
```

DESCRIPTION

Draw, in the pen's colour: `line` a straight line from one point to the other; `box` the outline of the rectangle with those two points as opposite corners; `bar` the same rectangle filled.

SEE ALSO

`plot`, `circle`, `fd`.

list

Block; in the `block` library (`lib block`).

STACK

```
list   ( u -- )
```

DESCRIPTION

Shows block *u* as a screen: sixteen numbered lines, 0 to 15, of 64 characters each. It also stores *u* in `scr`.

SEE ALSO

`block`, `load`, `scr`.

literal

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
literal   ( x -- )         compiling
          ( -- x )         when the definition runs
```

DESCRIPTION

Takes a number from the stack as the definition is compiled, and compiles it into the definition, so that the word leaves that number when it runs. The number is usually worked out between `[` and `]`.

SEE ALSO

`[`, `2literal`, `sliteral`, `nliteral`, `[']`.

EXAMPLE

```
/> : five  [ 2 3 + ] literal ;
/> five .
5
```

load, thru

Block; in the `block` library (`lib block`).

STACK

```
load   ( i*x u -- j*x )
thru   ( i*x u1 u2 -- j*x )
```

DESCRIPTION

`load` interprets block *u* as Forth source: its 1,024 characters are read as one long line (a `\` comment ends at the end of its 64-character line). `thru` loads each block from *u1* to *u2* in turn.

Block 0 can't be loaded: by custom it holds a description of the file. `0 load` is the error −35.

SEE ALSO

`block`, `list`, `included`, `blk`.

EXAMPLE

```
/> lib block
/> s" /ram/two.fb" open-blocks
/> 0 block 1024 bl fill update flush
/> 1 block 1024 bl fill  s" .( one ) " 1 block swap move update flush
/> 2 block 1024 bl fill  s" .( two ) " 2 block swap move update flush
/> 1 2 thru
one two
```

lshift, rshift

Core word set; in the ROM core, always there.

STACK

```
lshift   ( x1 u -- x2 )
rshift   ( x1 u -- x2 )
```

DESCRIPTION

Shift the bits of *x1* by *u* places, left or right, filling with zeros. `rshift` is a logical shift: the sign bit is not kept, so `-1 1 rshift` is 32767. A shift of 16 places or more leaves 0.

For a right shift of a signed number by one place, keeping its sign, use `2/`.

SEE ALSO

`2*`, `2/`, `and`, `nlshift`.

EXAMPLE

```
/> 1 4 lshift . 256 4 rshift .
16 16
/> -1 1 rshift .
32767
/> $1234 8 rshift .
18
```

marker

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
marker   ( "name" -- )
name     ( -- )
```

DESCRIPTION

Defines a word that remembers the dictionary as it is at that moment. When that word is run, everything defined after it, and the word itself, is forgotten, and the space is free again.

A program being worked on begins with a marker: running the marker and loading the file again replaces the old version without piling one copy on another.

SEE ALSO

`words`, `included`, `-lib`.

EXAMPLE

```
/> marker fresh
/> : temp  99 ;
/> temp .
99
/> fresh
/> ' temp
temp ?
```

min, max

Core word set; in the ROM core, always there.

STACK

```
min   ( n1 n2 -- n3 )
max   ( n1 n2 -- n3 )
```

DESCRIPTION

The lesser, and the greater, of two signed numbers. `0 max` keeps a number from going below zero, and `limit min` from going above a limit.

The comparison is signed: 65535 is –1, and is less than 1.

SEE ALSO

<, `within`, `dmin`, `dmax`.

EXAMPLE

```
/> 3 9 min . 3 9 max .
3 9
/> -1 1 min .
-1
/> 150 0 max 100 min .
100
```

=mkdir

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
=mkdir   ( c-addr u mode -- ior )
```

DESCRIPTION

Makes a directory. *mode* is the permission bits to give it; 0 leaves them to the system. The ior is 0, or the reason: –545, `already exists`; –544 if the directory it is to go in isn't there.

The name has an = so that `mkdir` typed at the shell's prompt is still the `mkdir` program.

SEE ALSO

`set-dir`, `open-dir`, `delete-file`.

EXAMPLE

```
/> lib hydra
/> s" /ram/newdir" 0 =mkdir .
0
/> s" /ram/newdir" 0 =mkdir .
-545
```

mouse, mouse-wait

The screen; in the `video` library (`lib video`).

STACK

```
mouse        ( -- x y buttons )
mouse-wait   ( -- x y buttons )
```

DESCRIPTION

The mouse of a Vera X. `mouse` leaves where the pointer is now, in the screen's dots, and which buttons are held: 1 for the left, 2 the middle, 4 the right, added together. `mouse-wait` waits until the mouse moves or a button changes, and then leaves the same three.

SEE ALSO

`ekey`, `vsync`; section 16.8.

move

Core word set; in the ROM core, always there.

STACK

```
move   ( addr1 addr2 u -- )
```

DESCRIPTION

Copies *u* bytes from *addr1* to *addr2*. The two parts may overlap, in either direction: the bytes arrive as they were before the copy began, as if they had all been read first.

SEE ALSO

`cmove`, `cmove>`, `fill`.

EXAMPLE

```
/> create buf 8 allot
/> s" abcdefgh" buf swap move
/> buf buf 2 + 6 move
/> buf 8 type
ababcdef
```

ms

Facility; in the `facility` library (`lib facility`).

STACK

```
ms   ( u -- )
```

DESCRIPTION

Waits at least *u* milliseconds. HyForth sleeps for the time, so other tasks have the processor meanwhile. The system's clock ticks 200 times a second, so the wait is to the nearest 5 milliseconds.

CTRL-C ends a wait early.

SEE ALSO

`pause`, `time&date`, `key?`.

EXAMPLE

```
/> lib facility
/> : slowly  3 0 do i . 200 ms loop ;
/> slowly
0 1 2
```

>n

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
>n   ( c-addr u -- flag ) ( N: -- x | )
```

DESCRIPTION

Reads a number from a string. If the whole string is a number, in any of the forms that can be typed, the number goes on the number stack and the flag is true. If it is not, nothing is added, and the flag is false. Spaces around the number are allowed.

This is how a program turns what a person typed, or a line of a file, into a number, and finds out whether it was one.

SEE ALSO

`n>str`, `>number`, `accept`, `s>n`.

EXAMPLE

```
/> lib numbers
/> s" 2/3" >n . n.
-1 2/3
/> s" abc" >n . ndepth .
0 0
```

n+, n−, n*, n/

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
n+   ( N: x y -- z )
n-   ( N: x y -- z )
n*   ( N: x y -- z )
n/   ( N: x y -- z )
```

DESCRIPTION

Add, subtract, multiply and divide the top two numbers of the number stack, exactly. The numbers may be of any kind and any size, and need not be of the same kind: integers, fractions, decimals, complex numbers. `n-` leaves $x - y$, and `n/` leaves x over y.

A result is given in the simplest kind that holds it. `n/` of two integers that don't divide evenly is a fraction in its lowest terms, never a rounded number; a fraction and a decimal make a fraction; a result with no fractional part is an integer.

ERRORS

THROW CODE	MESSAGE	CAUSE
−10	`division by zero`	`n/` by zero
−11	`result out of range`	A number too big for the library: more than 255 bytes, about 600 digits

SEE ALSO

`n/mod`, `npow`, `s>n`, `+`, `d+`.

EXAMPLE

```
/> lib numbers
/> 1/3 1/6 n+ n.
1/2
/> 1.5 2 s>n n* n.
3
/> 7 s>n 2 s>n n/ n.
7/2
/> 12345678901234567890123456789O ndup n* n.
152415787532388367504953515625361987875019051998750190521OO
```

n., n.base

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
n.       ( N: x -- )
n.base   ( c-addr u -- ) ( N: x -- )
```

DESCRIPTION

Print the top number of the number stack, with a space after it, and remove it. `n.` prints in the current base. `n.base` prints in the base named by the string, any of Chapter 18's: `s" x"` hex, `s" b"` binary, `s" o"` octal, a radix such as `s" 16r"`; with a `#` before the radix, the number is printed with that prefix.

SEE ALSO

`n.s`, `n>str`, `nformat`, `set-base`, `..`

EXAMPLE

```
/> lib numbers
/> 2/3 n.
2/3
/> 255 s>n s" x" n.base 255 s>n s" b" n.base
FF 11111111
/> 255 s>n s" #16r" n.base
#16rFF
```

n.s

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
n.s   ( -- )
```

DESCRIPTION

Shows the number stack, and changes nothing: how many numbers are on it, in angle brackets, and then each one, the top last. It is `.s` for the number stack.

SEE ALSO

`ndepth`, `n.`, `.s`.

EXAMPLE

```
/> lib numbers
/> 2/3 1.25 100000000000 n.s
<3> 2/3 1.25 100000000000
```

n/mod

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
n/mod   ( N: x y -- rem quot )
```

DESCRIPTION

Divides one integer by another, and leaves the remainder and, on top, the whole quotient. The division is symmetric, as HyForth's `/mod` is: the quotient is rounded toward zero, and the remainder has the sign of *x*.

ERRORS

THROW CODE	MESSAGE	CAUSE
−24	`invalid numeric argument`	A number that is not an integer
−10	`division by zero`	*y* is zero

SEE ALSO

`n/`, `/mod`, `ngcd`.

EXAMPLE

```
/> lib numbers
/> 10 s>n 3 s>n n/mod n. n.
3 1
/> -7 s>n 2 s>n n/mod n. n.
-3 -1
```

n=, n<, n>, n0=, n0<

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
n=    ( -- flag ) ( N: x y -- )
n<    ( -- flag ) ( N: x y -- )
n>    ( -- flag ) ( N: x y -- )
n0=   ( -- flag ) ( N: x -- )
n0<   ( -- flag ) ( N: x -- )
```

DESCRIPTION

The tests of `=`, `<`, `>`, `0=` and `0<`, for the number stack. Each removes its numbers and leaves a flag on the data stack, where `if` and the rest expect one. Numbers are compared by value: `1/2 0.5 n=` is true.

SEE ALSO

`ncompare`, `int?`, `=`.

EXAMPLE

```
/> lib numbers
/> 1/2 0.5 n= .
-1
/> 1/3 1/2 n< .
-1
/> 0.0 n0= .
-1
```

n>r, nr>

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
n>r   ( i*x +n -- ) ( R: -- i*x +n )
nr>   ( -- i*x +n ) ( R: i*x +n -- )
```

DESCRIPTION

Move several cells to the return stack and back, in one go. `n>r` takes a count and that many cells below it, and puts them all, with the count, on the return stack; `nr>` brings them back as they were, the count on top.

They follow the rules of `>r` and `r>`: a pair in the same definition, not across a loop's boundary.

NOTES

They are for use inside a definition only. Typed at the prompt, `n>r` upsets the interpreter's own return stack, and HyForth stops.

SEE ALSO

`>r`, `2>r`, `save-input`.

EXAMPLE

```
/> : hide-three  1 2 3 3 n>r depth . nr> .s ;
/> hide-three
0 <4> 1 2 3 3
```

n>s, n>d

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
n>s   ( -- n ) ( N: x -- )
n>d   ( -- d ) ( N: x -- )
```

DESCRIPTION

Move a number from the number stack to the data stack, as a signed cell or a signed double. A number with a fractional part loses it: the whole part, toward zero, is what arrives.

ERRORS

THROW CODE	MESSAGE	CAUSE
−11	`result out of range`	The number doesn't fit a cell (`n>s`) or a double (`n>d`)
−24	`invalid numeric argument`	A complex number

SEE ALSO

`s>n`, `truncate`, `d>s`.

EXAMPLE

```
/> lib numbers
/> 7/2 n>s .
3
/> 100000000000 n>s .
result out of range
```

n>str

Numbers; in the `numbers` library (`lib numbers`).

STACK

(-- *c-addr u*) N: (*x* --)

DESCRIPTION

Takes the number on top of the number stack and gives its text, in the base that is set, as `n.` would print it but without the space after it. The text is in a transient buffer: use it or copy it before the next number is converted.

name>string, name>interpret, name>compile

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
name>string      ( nt -- c-addr u )
name>interpret   ( nt -- xt | 0 )
name>compile     ( nt -- x xt )
```

DESCRIPTION

Take a name token, as `traverse-wordlist` gives, apart.

`name>string` leaves the word's name, as a string in the dictionary itself.

`name>interpret` leaves the execution token for the word's behaviour at the prompt.

`name>compile` leaves what is needed to compile the word: a cell *x* and a token *xt*, such that `execute` of *xt*, with *x* under it, does what the compiler does on meeting the word. For an ordinary word that is its own token and `compile,`; for an immediate word, its token and `execute`.

On HyForth a name token is the address of the word's header in the dictionary.

SEE ALSO

`traverse-wordlist`, `find`, `compile,`, `execute`.

nand, nor, nxor, ninvert

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nand      ( N: x y -- z )
nor       ( N: x y -- z )
nxor      ( N: x y -- z )
ninvert   ( N: x -- y )
```

DESCRIPTION

The bitwise operations `and`, `or`, `xor` and `invert`, on integers of any length. (`nand` is the numbers' "and", not "not and".) A negative integer is taken as two's complement, with its sign bit going on for ever: `0 ninvert` is –1.

SEE ALSO

`nlshift`, `nbit?`, `and`.

EXAMPLE

```
/> lib numbers
/> 12 s>n 10 s>n nand n. 12 s>n 10 s>n nor n. 12 s>n 10 s>n nxor n.
8 14 6
/> 0 s>n ninvert n.
-1
```

nbit?

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nbit?   ( u -- flag ) ( N: x -- )
```

DESCRIPTION

Removes an integer from the number stack and leaves a flag: true if its bit *u* is 1. Bit 0 is the lowest. A negative integer has every bit above its top one set.

SEE ALSO

`nand`, `nrshift`, `tbit`.

EXAMPLE

```
/> lib numbers
/> 5 s>n 0 nbit? . 5 s>n 1 nbit? .
-1 0
```

nbytes, nfrom-bytes

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nbytes        ( -- c-addr u ) ( N: x -- )
nfrom-bytes   ( c-addr u -- ) ( N: -- x )
```

DESCRIPTION

A number in the Hydra's stored format: the compact form, of a length that depends on the number, that every language uses to keep a number in a file (hylang's `number-bytes`, HyBASIC's `MKN$`). `nbytes` removes a number and leaves the address and length of its bytes; `nfrom-bytes` takes such bytes and puts the number they hold back on the number stack.

The bytes `nbytes` leaves are in a buffer of the library's own: write or copy them before another number word is used.

SEE ALSO

`n>str`, `>n`, `write-file`.

EXAMPLE

```
/> lib numbers
/> 258 s>n nbytes nip .
3
/> 258 s>n nbytes nfrom-bytes n.
258
```

ncompare

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
ncompare   ( -- n ) ( N: x y -- )
```

DESCRIPTION

Compares the top two numbers of the number stack, removes them, and leaves on the data stack –1 if *x* is less than *y*, 0 if they are equal, and 1 if it is greater. Numbers are compared by value, whatever their kind: 1/2 and 0.5 are equal.

SEE ALSO

`n=`, `n<`, `compare`.

EXAMPLE

```
/> lib numbers
/> 1/2 0.5 ncompare .
0
/> 1/3 0.5 ncompare .
-1
```

ndepth

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
ndepth   ( -- u )
```

DESCRIPTION

Leaves, on the data stack, how many numbers are on the number stack.

The **number stack** is the numbers library's own: it holds the Hydra's numbers (Chapter 18), which are of any length and so can't be cells. With the library loaded, a number typed that a cell can't hold goes there by itself: `2/3`, `1.25`, `#xFF`, `2i`, `10000000000000000000`. A number that fits a cell, such as `42`, still goes on the data stack, and `s>n` moves it across.

The number stack is in a RAM bank of 8K, so it holds a great many small numbers or a few enormous ones. Unlike the data stack, it is not emptied when an error returns HyForth to the prompt: what was on it is still there.

ERRORS

THROW CODE	MESSAGE	CAUSE
–44	`number stack overflow`	No room for another number (any word that adds one)
–45	`number stack underflow`	A word took more numbers than there were

SEE ALSO

`n.s`, `ndrop`, `s>n`, `depth`.

EXAMPLE

```
/> lib numbers
/> 2/3 1.25 ndepth .
2
/> n.s
<2> 2/3 1.25
/> n+
/> n.
23/12
/> n+
number stack underflow
```

ndrop, ndup, nover, nswap, nrot

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
ndrop   N: ( x -- )
ndup    N: ( x -- x x )
nover   N: ( x1 x2 -- x1 x2 x1 )
nswap   N: ( x1 x2 -- x2 x1 )
nrot    N: ( x1 x2 x3 -- x2 x3 x1 )
```

DESCRIPTION

`drop`, `dup`, `over`, `swap` and `rot` for the number stack. A number there is one item whatever its size: an integer of a thousand digits, a fraction, a complex number.

negate, abs

Core word set; in the ROM core, always there.

STACK

```
negate   ( n1 -- n2 )
abs      ( n -- u )
```

DESCRIPTION

`negate` leaves the number with its sign changed: 0 – *n1*. `abs` leaves the number without its sign.

−32768 has no positive counterpart in 16 bits: `negate` and `abs` both leave it as it is, and what `abs` leaves is right as the unsigned number 32,768.

SEE ALSO

`dnegate`, `dabs`, `invert`.

EXAMPLE

```
/> 5 negate . -5 abs .
-5 5
/> -32768 abs u.
32768
```

nfib

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nfib   ( N: n -- f )
```

DESCRIPTION

Replaces a whole number *n*, 0 or more, by the *n*th Fibonacci number: 0, 1, 1, 2, 3, 5 ... , exactly, however long.

ERRORS

THROW CODE	MESSAGE	CAUSE
–24	`invalid numeric argument`	*n* is negative, or not whole

EXAMPLE

```
/> lib numbers
/> 30 s>n nfib n.
832040
/> 100 s>n nfib n.
354224848179261915075
```

nformat

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nformat   ( c-addr u -- ) ( N: x1 ... xn -- )
```

DESCRIPTION

Prints a string in which each `{}` is replaced by a number from the number stack: the first `{}` by the deepest of them, so the numbers appear in the order they were put there. As many numbers are removed as there are places.

A place can name a base, as `{x}` does: that number is printed in it.

SEE ALSO

`n.`, `n.base`, `."`.

EXAMPLE

```
/> lib numbers
/> 1/2 0.25 s" {} and {}" nformat
1/2 and 0.25
/> 255 s>n s" hex {x}" nformat
hex FF
```

nip, tuck

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
nip    ( x1 x2 -- x2 )
tuck   ( x1 x2 -- x2 x1 x2 )
```

DESCRIPTION

`nip` removes the second cell: it is `swap drop`. `tuck` puts a copy of the top cell under the second: it is `swap over`.

SEE ALSO

`drop`, `over`, `swap`.

EXAMPLE

```
/> 1 2 nip .s
<1> 2
/> drop 1 2 tuck .s
<3> 2 1 2
```

nliteral, nconstant, nvariable, nvalue, n@, n!

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nliteral    compiling: N: ( x -- )    running: N: ( -- x )
nconstant   ( "name" -- ) N: ( x -- )           name: N: ( -- x )
nvariable   ( "name" -- )                       name: ( -- addr )
nvalue      ( "name" -- ) N: ( x -- )           name: N: ( -- x )
n@          ( addr -- ) N: ( -- x )
n!          ( addr -- ) N: ( x -- )
```

DESCRIPTION

The number stack's forms of `literal`, `constant`, `variable`, `value`, `@` and `!`. A number written inside a definition is compiled by itself, as a cell's is; `nliteral`, which is immediate and compile only, compiles one that was worked out while compiling. An `nvariable` holds one number of any size, which `n@` copies to the number stack and `n!` stores from it. An `nvalue` is changed with `to`.

EXAMPLE

```
/> lib numbers
/> 1/3 nconstant third  nvariable acc  third third n+ acc n!  acc n@ n.
2/3
/> : big 100000000000 ;  big big n* n.
10000000000000000000000
```

nlshift, nrshift

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nlshift   ( u -- ) ( N: x -- y )
nrshift   ( u -- ) ( N: x -- y )
```

DESCRIPTION

Shift an integer's bits *u* places, the count coming from the data stack. `nlshift` multiplies by 2 to the power *u*, and loses nothing, since the integer simply grows. `nrshift` divides by 2 to the power *u*, rounding down, and keeps the sign.

SEE ALSO

`nand`, `nbit?`, `lshift`.

EXAMPLE

```
/> lib numbers
/> 1 s>n 100 nlshift n.
1267650600228229401496703205376
/> -8 s>n 1 nrshift n.
-4
```

nnegate, nabs, ngcd, npow

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nnegate   ( N: x -- y )
nabs      ( N: x -- y )
ngcd      ( N: x y -- z )
npow      ( N: x y -- z )
```

DESCRIPTION

`nnegate` changes a number's sign. `nabs` leaves a real number without its sign; a complex number is an error. `ngcd` leaves the greatest common divisor of two integers: the largest number that divides both.

`npow` leaves *x* to the power *y*. With a whole *y* the result is exact, whatever *x* is: `2 100` gives all 31 digits, and a negative power gives a fraction. With a fractional *y* the result is exact when the root is, and otherwise a decimal to the precision in `digits`.

ERRORS

THROW CODE	MESSAGE	CAUSE
–24	`invalid numeric argument`	`nabs` of a complex number; `ngcd` of numbers that are not integers
–46	`outside the function's domain`	`npow`: a negative number to a power that isn't whole

SEE ALSO

`n*`, `nsqrt`, `digits`, `negate`.

EXAMPLE

```
/> lib numbers
/> 12 s>n 18 s>n ngcd n.
6
/> 2 s>n 100 s>n npow n.
1267650600228229401496703205376
/> 2 s>n -1 s>n npow n.
1/2
/> 2 s>n 1/2 npow n.
1.41421356237
```

:noname

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
:noname   ( -- xt )
```

DESCRIPTION

Starts a definition with no name. It is compiled as a `:` definition is, up to `;`, and when it is finished its execution token is left on the stack. The token is all there is to run it by: keep it in a constant, a table or a `defer`red word, or `execute` it.

SEE ALSO

`:`, `execute`, `defer`, `'`.

EXAMPLE

```
/> :noname  dup * ;  constant squarer
/> 7 squarer execute .
49
```

note, note-group

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
note          ( task n -- )
note-group    ( group n -- )
```

DESCRIPTION

Send note number *n* to a task, or to every task of a note group. The notes with meanings of their own are 1 (interrupt, which is what CTRL-C sends), 3 (hangup), 4 (alarm) and 9 (kill, which a task cannot refuse); 16 to 31 are for programs to use between themselves.

What a note does is up to the task that gets it. Most programs end. A failure, such as a task that isn't there, is thrown.

SEE ALSO

`on-note`, `spawn`, `wait`; the command `kill`.

EXAMPLE

```
/> lib hydra
/> sleep 30 &
/> s" apid" getenv evaluate 9 note
/> 99 4 note
no such task
```

note-of

Sound; in the `sound` library (`lib sound`).

STACK

```
note-of   ( c-addr u -- n )
```

DESCRIPTION

Leaves the MIDI number of a note given by name: a letter A to G, a `#` for a sharp or a `b` for a flat if there is one, and the octave. `C4` is middle C, 60.

ERRORS

THROW CODE	MESSAGE	CAUSE
−24	`invalid numeric argument`	The string is not a note's name

SEE ALSO

`snd-note`, `tune`.

EXAMPLE

```
/> lib sound
/> s" A4" note-of .
69
/> s" C#4" note-of . s" Db4" note-of .
61 61
```

nrandom, nseed

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nrandom   ( N: n -- r )
nseed     ( u -- )
```

DESCRIPTION

`nrandom` replaces a whole number *n* by a random whole number from 0 to *n* − 1. *n* may be as large as any integer. With an *n* of 0 it gives a random decimal between 0 and 1.

`nseed` starts the generator from a number of the program's choosing. The same seed gives the same series again, which is what testing a game or a simulation needs.

SEE ALSO

`random`, `rand`, `nfib`.

EXAMPLE

```
/> lib numbers
/> 42 nseed 6 s>n nrandom n.
5
/> 42 nseed 6 s>n nrandom n.
5
```

nsqrt, nexp, nlog, nsin, ncos, ntan, natan, npi

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
nsqrt   nexp   nlog   nsin   ncos   ntan   natan   ( N: x -- y )
npi     ( N: -- x )
```

DESCRIPTION

The math functions: the square root, *e* to a power, the natural logarithm, the sine, cosine and tangent of an angle in radians, the angle whose tangent is given, and pi.

Where there is an exact answer it is given: `16 nsqrt` is 4, and the square root of a negative number is an imaginary one. Otherwise the result is a decimal with as many significant digits as the variable `digits` holds, 12 unless a program changes it.

ERRORS

THROW CODE	MESSAGE	CAUSE
−46	`outside the function's domain`	The logarithm of 0
−24	`invalid numeric argument`	A complex number

SEE ALSO

`digits`, `npow`, `to-fixed`.

EXAMPLE

```
/> lib numbers
/> 2 s>n nsqrt n.
1.41421356237
/> 16 s>n nsqrt n. -4 s>n nsqrt n.
4 2i
/> 1 s>n nexp n.
2.71828182846
/> npi n.
3.14159265359
```

>number

Core word set; in the ROM core. Converts the digits at the start of a string, adding them to a double.

STACK

```
>number  ( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
```

DESCRIPTION

ud1 is the number so far: 0 to start a new one. Characters are taken from the left of the string for as long as each is a digit in the current `base`, and for each one *ud* becomes *ud* × `base` + the digit's value. The digits are `0` to `9`, then the letters `A` to `Z` for 10 to 35; small letters count as capitals. A character whose value is not less than `base` is not a digit, and conversion stops at it: in decimal `a` stops it, in hex `g` does.

ud2 is the result, and *c-addr2 u2* is what was not converted: the address of the first character that isn't a digit, and the number of characters from there to the end. *u2* is 0 if the whole string was digits.

Nothing else is understood. A sign, a prefix (`$`, `#` or `%`) and a decimal point all stop the conversion like any other non-digit; an empty string converts nothing and returns *ud1* unchanged. Signs and prefixes are the text interpreter's business: it deals with them itself, and calls this word's code for the digits. A string with no digits is not an error.

NOTES

A value that passes 32 bits wraps: only its low 32 bits are kept, and no error is raised. (The standard calls this an ambiguous condition.) To read a number of any length, use the numbers library (`lib numbers`).

EXAMPLE

`nip` drops the address, leaving the double (its low cell first, as `.s` shows it) and the count of characters not converted:

```
/> 0 0 s" 123abc" >number nip .s
<3> 123 0 3
/> drop 2drop
/> hex 0 0 s" ff zz" >number nip .s decimal
<3> FF 0 3
/> drop 2drop
/> 0 0 s" -5" >number nip .s
<3> 0 0 2
/> drop 2drop
/> 0 0 s" 99999999999" >number nip .s
<3> 2839 -20518 0
```

on-note

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
on-note   ( xt -- )
```

DESCRIPTION

Names the word that is to be given the notes this HyForth receives. The word must take the note's number and leave a flag, (*n -- flag*): true if it has dealt with the note and the program should go on, false if the note should be treated as CTRL-C is, and stop what is running (the throw code –28).

The word is not called at the instant a note arrives, but at the next word the interpreter runs or the next turn of a loop, so it may do anything a word can.

CTRL-C itself and the kill note are not passed to it. `0 on-note` takes the word away again: with none, any note but CTRL-C ends HyForth.

SEE ALSO

`note`, `catch`.

EXAMPLE

```
/> lib hydra
/> : heard  ( n -- flag )  ." note " . true ;
/> ' heard on-note
/> sys-getpid 20 note
note 20
/> 0 on-note
```

only, also, previous, forth, order

Search-Order; in the `search` library (`lib search`).

STACK

All (--).

DESCRIPTION

The search order is the list of word lists in which a name is looked up, first to last. `only` makes it the smallest that will work: the `forth` word list alone. `also` puts a second copy of its first list on the front, ready to be replaced. A word list's name, such as `forth`, replaces the first list with itself. `previous` takes the first list off. `order` prints the order, first list first, and then the current word list, the one new definitions go into.

`also name` is therefore the way to add the word list *name* to the front of the order.

EXAMPLE

```
/> lib search
/> only forth order
forth  current: forth
```

open-blocks

Block; in the `block` library (`lib block`). A HyForth word, not the standard's.

STACK

```
open-blocks   ( c-addr u -- )
```

DESCRIPTION

Chooses the block file: from now on, `block`, `load` and the rest work on the file named. If the file is not there it is made, empty. `flush` first, so that changes to the blocks of the file in use are written to it.

Without `open-blocks` the block file is `blocks.fb` in the current directory.

SEE ALSO

`block`, `flush`.

open-dir, read-dir, close-dir

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
open-dir    ( c-addr u -- dirid ior )
read-dir    ( c-addr u1 dirid -- u2 flag ior )
close-dir   ( dirid -- ior )
```

DESCRIPTION

A directory's names, one at a time. `open-dir` opens the directory named by the string. Each `read-dir` puts the next name into the buffer at *c-addr*, which has room for *u1* characters, and returns its length *u2* and a true *flag*; when there are no more, the flag is false. `close-dir` gives the directory up.

The names come in the directory's own order. An *ior* is 0 for success.

open-file, create-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
open-file     ( c-addr u fam -- fileid ior )
create-file   ( c-addr u fam -- fileid ior )
```

ARGUMENTS

c-addr u

The file's name. One that doesn't start with `/` is looked for from the current directory.

fam

`r/o`, `w/o` or `r/w`.

DESCRIPTION

`open-file` opens a file that exists. `create-file` makes a new, empty file; if there is one of that name already, it is emptied.

Both leave a **fileid**, which is what every other file word takes, and above it an **ior**, an I/O result: 0 if all went well, and the reason if not. The fileid is the system's file descriptor, a small number, and is only good if the ior is 0. The usual way to write a call is with `throw` after it, which does nothing for 0, and reports any failure.

A device's file is opened the same way: `s" /dev/time" r/o open-file`.

A file should be closed when it is finished with: a task can have only so many open at once.

ERRORS

The ior is −512 minus the system's error code (Appendix G), and as a throw code it prints the system's message: −544 is `not found`, −547 `is a directory`, −553 `read-only`.

SEE ALSO

`close-file`, `read-line`, `write-line`, `throw`, `ior>text`, `included`.

EXAMPLE

```
/> s" /ram/f.txt" w/o create-file throw value fd
/> s" line one" fd write-line throw
/> fd close-file throw
/> s" /ram/nothere" r/o open-file . .
-544 0
/> s" /ram/nothere" r/o open-file throw
not found
```

output-of

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
output-of   ( xt -- c-addr u )
```

DESCRIPTION

Runs a Forth word, and leaves what it printed as a string: nothing appears on the screen. It is `sh-out` for HyForth's own output. The string is in the free space above `here`, as `sh-out`'s is, so the word run must not add to the dictionary.

If the word throws an error, the error is thrown again when the output has been put back.

SEE ALSO

`sh-out`, `piped`, `type`.

EXAMPLE

```
/> : say  ." from forth" ;
/> ' say output-of nip .
10
/> ' say output-of type
from forth
```

over

Core word set; in the ROM core, always there.

STACK

```
over   ( x1 x2 -- x1 x2 x1 )
```

DESCRIPTION

Copies the second cell to the top.

SEE ALSO

`dup`, `pick`, `2over`, `tuck`.

EXAMPLE

```
/> 1 2 over .s
<3> 1 2 1
```

pad

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
pad   ( -- c-addr )
```

DESCRIPTION

Leaves the address of a scratch area of 100 bytes, for a program to use as it likes: to build a string, or hold a line.

The area is at a fixed place, not above `here`, so it does not move as words are defined.

SEE ALSO

`buffer:`, `here`, `s"`.

EXAMPLE

```
/> s" Hydra" pad swap move
/> pad 5 type
Hydra
```

page

Facility; in the `facility` library (`lib facility`).

STACK

```
page   ( -- )
```

DESCRIPTION

Clears the window, and puts the cursor at its top left corner.

SEE ALSO

`at-xy`, `clear-below`.

palette!, sprite!, sprite-at, sprite-off, vsync

The screen; in the `video` library (`lib video`).

STACK

```
palette!     ( index rgb -- )
sprite!      ( n c-addr -- )
sprite-at    ( n x y -- )
sprite-off   ( n -- )
vsync        ( -- )
```

DESCRIPTION

`palette!` sets a palette entry to a colour written as `$RGB`: four bits each of red, green and blue. `sprite!` sets sprite *n*'s eight attribute bytes from memory at *c-addr*; `sprite-at` moves a sprite to *x*, *y*; `sprite-off` hides it.

`vsync` waits for the start of the next frame, of which there are about sixty a second. A program that moves things on the screen does so once a frame, after `vsync`, so that the movement is smooth.

SEE ALSO

`vram!`, `pen`, `ms`.

parse, parse-name

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
parse        ( char "text<char>" -- c-addr u )
parse-name   ( "name" -- c-addr u )
```

DESCRIPTION

Take text from the input that follows the word being run: this is how a word reads its own arguments from the line, as `:` and `char` do.

`parse` takes everything up to the next occurrence of the character given, and leaves it as a string; the character itself is used up and is not part of the string. If the character isn't there, the string is the rest of the line. Nothing is skipped first: the space after the word that called `parse` has gone, but any further spaces are in the string.

`parse-name` skips spaces, and takes the next word: everything up to the following space or the end of the line. At the end of the line the string's length is 0.

The string is in the input line itself, and is good only until the line is finished: copy it if it must last.

SEE ALSO

`word`, `source`, `>in`, `char`.

EXAMPLE

```
/> : two-words  parse-name type ." |" parse-name type ." |" ;
/> two-words one   two
one|two|
/> : to-comma  [char] , parse type ." |" ;
/> to-comma a b,
a b|
```

pause

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
pause   ( -- )
```

DESCRIPTION

Gives up the rest of HyForth's turn of the processor, so that any other task that is ready runs now. HyForth goes on at its next turn. A loop that is only waiting for something another task will do should `pause` each time round, or better, wait with `ms`.

SEE ALSO

`ms`, `key?`.

pc?

The devices; in the `pc` library, a file of Forth in `/lib/forth` (`lib pc`).

STACK

```
pc?   ( -- flag )
```

DESCRIPTION

True if the PC tool answers on the serial line, so that the files of `/pc` can be used; false, after a wait of a second, if nothing answers. `/pc`'s files themselves are ordinary files, for the file words.

SEE ALSO

`open-file`, `file-status`; section 9.9.

pen, border

The screen; in the `video` library (`lib video`).

STACK

```
pen      ( colour -- )
border   ( colour -- )
```

DESCRIPTION

`pen` sets the colour that the drawing words draw in from now on: a number of the palette, within what the bitmap's depth allows. `border` sets the colour of the screen's border, outside the picture.

SEE ALSO

`plot`, `palette!`, `bitmap`.

pick

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
pick   ( xu ... x1 x0 u -- xu ... x1 x0 xu )
```

DESCRIPTION

Removes *u*, and copies the cell that is then *u* places down from the top: `0 pick` is `dup`, and `1 pick` is `over`.

NOTES

u is not checked against the depth of the stack. A *u* that reaches below the bottom of the stack gives whatever is in the memory there, and is not an error.

SEE ALSO

`roll`, `dup`, `over`.

EXAMPLE

```
/> 10 20 30 1 pick .
20
/> 0 pick .
30
```

piped

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

(*xt c-addr u* --)

DESCRIPTION

Runs a command line, given as a string, with what the word *xt* prints as its standard input. The word is run with its output going into a pipe, the command reads the pipe, and `piped` waits for the command to end. `' words s" grep file" piped` shows the words with `file` in their names.

plain, bold, dim, underline, blink, reverse

Facility; in the `facility` library (`lib facility`). HyForth's own words, not the standard's.

STACK

```
plain   bold   dim   underline   blink   reverse   ( -- )
```

DESCRIPTION

The styles of the text printed from now on. `bold`, `dim`, `underline`, `blink` and `reverse` (the text's and the background's colours exchanged) each turn a style on, and they add up. `plain` turns every style off, and puts the colours back to the terminal's usual ones.

How each style looks depends on the terminal: not every one shows `dim` or `blink`.

SEE ALSO

`color`, `sgr`, `cursor-off`.

plot

The screen; in the `video` library (`lib video`).

STACK

```
plot   ( x y -- )
```

DESCRIPTION

Draws one dot in the pen's colour. Positions count from 0 0 at the top left corner of the bitmap, *x* across and *y* down. A dot outside the bitmap is not drawn, and is not an error.

SEE ALSO

`line`, `pen`, `bitmap`.

postpone

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
postpone   ( "name" -- )
```

DESCRIPTION

Takes the next word of the definition and compiles it so that its *compiling* action happens later, when the word being defined runs. It is how one immediate word is built from others.

For an immediate word, that means it is compiled as an ordinary call, where it would otherwise have run on the spot. For an ordinary word, it means that when the word being defined runs, it compiles a call of that word into whatever definition is then being compiled.

SEE ALSO

`immediate`, `compile,`, `[compile]`, `literal`.

EXAMPLE

`twice` compiles `dup +` into the definition that uses it:

```
/> : twice  postpone dup postpone + ; immediate
/> : quad  twice twice ;
/> 3 quad .
12
/> see quad
: quad dup + dup + ;
```

prompt, prompt2

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
prompt    ( c-addr u -- )
prompt2   ( c-addr u -- )
```

DESCRIPTION

Set the text of the prompt. `prompt` sets the one shown when HyForth is ready for a line, and `prompt2` the one shown while a definition that was begun on an earlier line is still open. Each takes a string of at most 31 characters, in which these stand for something that changes:

WRITTEN	SHOWS
`%p`	The whole path of the current directory
`%d`	The current directory on its disk
`%v`	The SD card the directory is on, as `0:`; nothing for a directory that is not on a card
`%t`	HyForth's task number
`%w`	Its window's number
`%%`	A `%`

HyForth starts with the prompt `%v%d>` , which shows `/ram>` in the RAM disk and `0:/games>` on a card, and with a tab as the second prompt.

SEE ALSO

`cd`, `:.`

EXAMPLE

```
/> s" [task %t] " prompt
[task 1] s" %v%d> " prompt
/> s" ... " prompt2
/> : unfinished
... 1 2 + ;
/> unfinished .
3
```

quit, abort, abort"

Core word set; in the ROM core.

STACK

```
quit     ( -- )                R: ( i*x -- )
abort    ( i*x -- )            R: ( j*x -- )
abort"   ( i*x flag -- | i*x )    compile only, immediate; "text" follows it
```

DESCRIPTION

`quit` empties the return stack and goes back to reading lines from the keyboard, without touching the data stack and without printing `ok` or anything else. `abort` empties the data stack too: it is `-1 throw`, so a `catch` can stop it.

`abort" text"` is used inside a definition. When it runs, it takes a flag: if the flag is true it prints the text and aborts (`-2 throw`), and if false it does nothing. It is the short way to check an argument.

EXAMPLE

```
/> : half  dup 1 and abort" odd number" 2/ ;
/> 10 half .  7 half .
5 odd number
```

.r, u.r

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
.r    ( n1 n2 -- )
u.r   ( u n -- )
```

DESCRIPTION

Print a number set to the right of a field *n2* (or *n*) characters wide, with spaces before it and none after: `.r` signed, `u.r` unsigned. A number too long for the field is printed in full, and the field is simply overrun. This is how columns of figures are lined up.

SEE ALSO

`.`, `d.r`, `spaces`, `<#`.

EXAMPLE

```
/> : row  dup 4 .r dup dup * 6 .r cr ;
/> 7 row 12 row
   7    49
  12   144
/> 12345 2 .r
12345
```

>r, r>, r@

Core word set; in the ROM core, always there. Compile only.

STACK

```
>r   ( x -- ) ( R: -- x )
r>   ( -- x ) ( R: x -- )
r@   ( -- x ) ( R: x -- x )
```

DESCRIPTION

Move a cell between the data stack and the return stack. `>r` takes the top of the data stack and puts it on the return stack; `r>` brings it back; `r@` copies it without removing it.

The return stack is where a value can be put out of the way while the words that follow work on what is under it. But it is also where each word keeps the address it will return to, and where `do` loops keep their numbers, so its use has rules:

- everything a word puts there, it must take off again before its `;` or an `exit`;
- what is put there outside a loop can't be reached inside it, and what is put there inside a loop must be taken off in the same turn of the loop, before `loop`;
- `i` is wrong while something put there inside the loop is still there.

They work only inside a definition.

SEE ALSO

`2>r`, `n>r`, `i`, `exit`, `{:.`

EXAMPLE

```
/> : sum-and-keep  >r r@ + r> ;
/> 10 5 sum-and-keep . .
5 15
```

r/o, w/o, r/w

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
r/o   ( -- fam )
w/o   ( -- fam )
r/w   ( -- fam )
```

DESCRIPTION

The three ways to open a file, for `open-file` and `create-file`: read only, write only, and read and write. They are the constants 0, 1 and 2, which are the system's own numbers for the three.

SEE ALSO

`open-file`, `create-file`, `bin`.

EXAMPLE

```
/> r/o . w/o . r/w .
0 1 2
```

rand, rand32

Random numbers; in the `random` library (`lib random`).

STACK

```
rand     ( -- u )
rand32   ( -- ud )
```

DESCRIPTION

The next random number from the library's generator: `rand` 16 bits of it, as an unsigned cell, and `rand32` all 32, as an unsigned double.

The generator is seeded from the clock and the tick count as the library loads, so each run gives a different series, unless `rseed` sets it.

SEE ALSO

`random`, `rseed`, `nrandom`.

random

Random numbers; in the `random` library (`lib random`).

STACK

```
random   ( u -- u' )
```

DESCRIPTION

A random number from 0 to *u* – 1: `6 random 1+` is the throw of a die. For a *u* of 0 the result is 0.

SEE ALSO

`rand`, `rseed`, `nrandom`.

EXAMPLE

```
/> lib random
/> 42. rseed  6 random . 6 random .
1 1
```

read-file

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
read-file   ( c-addr u1 fileid -- u2 ior )
```

DESCRIPTION

Reads up to *u1* bytes from the file, from its current place, into memory at *c-addr*, and moves the file's place on by what was read. *u2* is the number of bytes read: less than *u1* when the file has fewer left, and 0 at its end, which is not an error.

The bytes are read as they are, line ends included.

SEE ALSO

`read-line`, `write-file`, `reposition-file`, `file-size`.

EXAMPLE

```
/> s" /rom/lib/shell" r/o open-file throw value fd
/> pad 5 fd read-file . .
0 5
/> pad 5 type
/bin/
/> pad 100 fd read-file . .
0 9
/> pad 100 fd read-file . .
0 0
/> fd close-file drop
```

read-line

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
read-line   ( c-addr u1 fileid -- u2 flag ior )
```

DESCRIPTION

Reads the next line of the file into memory at *c-addr*, without its line end, and leaves its length, *u2*, and a flag: true if there was a line, and false at the end of the file, where *u2* is 0.

At most *u1* characters are stored. A longer line is not lost: it comes in pieces, the rest of it being given by the next `read-line`s. The buffer should have room for two characters more than *u1*, which the standard lets `read-line` use for the line end.

An empty line is a length of 0 with a true flag; the end of the file is 0 with a false one. So a loop reads while the flag is true.

SEE ALSO

`read-file`, `write-line`, `refill`, `accept`.

EXAMPLE

```
/> s" /ram/f.txt" w/o create-file throw value fd
/> s" line one" fd write-line throw  s" two" fd write-line throw  fd close-file throw
/> s" /ram/f.txt" r/o open-file throw to fd
/> pad 80 fd read-line . . .
0 -1 8
/> pad 8 type
line one
/> pad 80 fd read-line . . .
0 -1 3
/> pad 80 fd read-line . . .
0 0 0
/> fd close-file drop
```

recurse

Core word set; in the ROM core. Immediate, compile only.

STACK

Compiling: (--). Running: whatever the definition's own is.

DESCRIPTION

Compiles a call to the definition that is being compiled. A definition's own name can't be used for this, since the name is not findable until the `;`: that rule is what lets a new definition of a name use the old one.

The depth of recursion is limited by the return stack.

EXAMPLE

```
/> : fact  dup 1 > if dup 1- recurse * then ;
/> 5 fact .
120
```

refill, source-id, save-input, restore-input

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
refill          ( -- flag )
source-id       ( -- 0 | -1 | fileid )
save-input      ( -- xn ... x1 n )
restore-input   ( xn ... x1 n -- flag )
```

DESCRIPTION

The input source is where the text interpreter is taking its words from. `source-id` says which: 0 for the keyboard, −1 for a string being `evaluate`d, and a file's id while a file is being loaded.

`refill` reads the next line of the input source into the input buffer and sets `>in` to 0, so that parsing goes on from its start. Its flag is true if there was a line, and false at the end of a file, and always for a string. A word that reads text across several lines, such as a multi-line comment, uses it.

`save-input` puts the input source's present position on the stack as *n* cells and their count, and `restore-input` goes back to it; its flag is true if it could **not**.

replaces, substitute, unescape

String; in the `string` library (`lib string`).

STACK

```
replaces     ( c-addr1 u1 c-addr2 u2 -- )
substitute   ( c-addr1 u1 c-addr2 u2 -- c-addr2 u3 n )
unescape     ( c-addr1 u1 c-addr2 -- c-addr2 u2 )
```

DESCRIPTION

Text substitution, as the standard defines it. `replaces` records that the name *c-addr2 u2* stands for the text *c-addr1 u1*. `substitute` copies the string *c-addr1 u1* into the buffer *c-addr2 u2*, putting in place of each name written between two `%` signs the text recorded for it; `%%` gives one `%`. It returns the result and *n*, the number of substitutions made, or a negative number if the result did not fit.

`unescape` copies a string, doubling every `%` in it, so that `substitute` will pass it through unchanged. Its buffer must have room for twice the string.

required, require

Core word set; in the ROM core.

STACK

```
required   ( i*x c-addr u -- j*x )
require    ( i*x "name" -- j*x )
```

DESCRIPTION

`included` and `include`, once only: the file is loaded if it has not been loaded already, and otherwise nothing happens. `required` takes the file's name as a string and `require` takes it from the input.

A file is known by its name as it was found. One that could not be opened, or that stopped with an error part way, is not counted as loaded, and the next `require` of it tries again.

A name that does not start with `/` is looked for in the current directory and then in `/lib/forth`.

resize

Memory-Allocation; in the `memory` library (`lib memory`).

STACK

```
resize   ( a-addr1 u -- a-addr2 ior )
```

DESCRIPTION

Changes the size of an allocated block to *u* bytes. The block may have to move: the address left is where it is now, and the old address must not be used again. Its contents, up to the smaller of the old and new sizes, are kept.

If it can't be done the ior is –61, `resize failed`, and the block is as it was, at its old address.

SEE ALSO

`allocate`, `free`.

EXAMPLE

```
/> lib memory
/> 100 allocate throw value m
/> m 100 char A fill
/> m 200 resize throw to m
/> m 5 type
AAAAA
/> m free drop
```

roll

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
roll   ( xu xu-1 ... x0 u -- xu-1 ... x0 xu )
```

DESCRIPTION

Removes *u*, takes the cell that is then *u* places down out of the stack, and puts it on top; the cells that were above it each move down one. `1 roll` is `swap`, `2 roll` is `rot`, and `0 roll` does nothing.

SEE ALSO

`pick`, `rot`.

EXAMPLE

```
/> 1 2 3 2 roll .s
<3> 2 3 1
```

rot

Core word set; in the ROM core, always there.

STACK

```
rot   ( x1 x2 x3 -- x2 x3 x1 )
```

DESCRIPTION

Brings the third cell to the top, moving the other two down. `rot rot` turns the three the other way: it puts the top cell under the other two.

SEE ALSO

`roll`, `swap`, `2rot`.

EXAMPLE

```
/> 1 2 3 rot .s
<3> 2 3 1
/> rot rot .s
<3> 1 2 3
```

rseed

Random numbers; in the `random` library (`lib random`).

STACK

```
rseed   ( ud -- )
```

DESCRIPTION

Starts the generator from a number of the program's choosing, a double. The same seed gives the same series again, for testing. (A seed of 0 is taken as 1.)

SEE ALSO

`rand`, `random`.

EXAMPLE

```
/> lib random
/> 42. rseed rand u.
17704
/> 42. rseed rand u.
17704
```

run

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
run   ( c-addr u -- status )
```

DESCRIPTION

Runs a program, given as a string holding its name and its arguments with spaces between them, waits for it, and leaves its exit code. The line is split at its spaces and nothing more: there is no quoting, no pattern and no redirection. A name with no `/` in it is looked for in `/bin`.

ERRORS

THROW CODE	MESSAGE	CAUSE
−544	`not found`	There is no such program

SEE ALSO

`sh`, `spawn`, `wait`.

EXAMPLE

```
/> lib hydra
/> s" echo one two" run .
one two
0
/> s" nothere" run .
not found
```

.s

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
.s   ( -- )
```

DESCRIPTION

Shows the whole data stack, and changes nothing: the number of cells in angle brackets, and then each cell as a signed number in the current base, the top of the stack last.

It is the most useful word there is for finding out what a line or a definition really leaves behind.

SEE ALSO

`depth`, `.`, `n.s`, `dump`.

EXAMPLE

```
/> 1 2 3 .s
<3> 1 2 3
/> 2drop drop .s
<0>
```

s"

Core word set; in the ROM core, always there. Immediate.

STACK

```
s"   ( "text<quote>" -- )       compiling
     ( -- c-addr u )            when the definition runs, or at the prompt
```

DESCRIPTION

A string. The text that follows, up to the next `"`, becomes a string, and its address and length are left on the stack. One space follows `s"` and is not part of the string.

Inside a definition the text is compiled into the word, and stays as long as the word does.

At the prompt the text is put in a buffer of HyForth's own, and its address is good only for a while. There are two such buffers, used in turn, so the last two strings typed are both intact, which is what a word that takes two strings (such as `rename-file`) needs; a third overwrites the first. Each holds 100 characters. A string that must last is copied somewhere of the program's own.

There is no way to put a `"` in the text: for that, and for control characters, use `s\"`.

SEE ALSO

`s\"`, `c"`, `."`, `type`, `sliteral`, `pad`.

EXAMPLE

```
/> s" hello" type
hello
/> s" hello" nip .
5
/> : name  s" Hydra-16" ;
/> name type
Hydra-16
```

s>d

Core word set; in the ROM core, always there.

STACK

```
s>d   ( n -- d )
```

DESCRIPTION

Makes a signed cell a signed double of the same value: a cell of 0 is put above it for a number that is 0 or more, and a cell of –1, all ones, for a negative one.

For an unsigned cell, the high cell is always 0: put a `0` on the stack.

SEE ALSO

`d>s`, `m*`, `um/mod`.

EXAMPLE

```
/> 3 s>d . .
0 3
/> -3 s>d . .
-1 -3
```

s>n, d>n

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
s>n   ( n -- ) ( N: -- x )
d>n   ( d -- ) ( N: -- x )
```

DESCRIPTION

Move a number from the data stack to the number stack: `s>n` a signed cell, and `d>n` a signed double. This is how a small number, which HyForth reads as a cell, joins the big ones: `2 s>n`.

SEE ALSO

`n>s`, `>n`, `s>d`.

EXAMPLE

```
/> lib numbers
/> 100 s>n 10 s>n npow n.
10000000000000000000
/> 70000. d>n n.
70000
```

s"

Core Extension; in the `coreext` library, which is loaded as HyForth starts. Immediate.

STACK

```
s\"   ( "text<quote>" -- )      compiling
      ( -- c-addr u )           when the definition runs, or at the prompt
```

DESCRIPTION

`s"`, with escapes: in the text, a backslash and the character after it stand for a character that can't be typed.

WRITTEN	IS	WRITTEN	IS
`\n`	A new line	`\"` and `\q`	A double quote
`\t`	A tab	`\\`	A backslash
`\r`	A carriage return (13)	`\e`	Escape (27)
`\l`	A line feed (10)	`\a`	The bell (7)
`\m`	A carriage return and a line feed	`\b`	A backspace (8)
`\z`	A zero byte	`\f`, `\v`	A form feed (12), a vertical tab (11)
`\x` and two hex digits	The character with that code		

SEE ALSO

`s"`, `emit`, `type`.

EXAMPLE

```
/> s\" one\ntwo" type
one
two
/> s\" a \"quoted\" word" type
a "quoted" word
```

sbit, cbit

Bits; in the `bits` library (`lib bits`).

STACK

```
sbit   ( n b -- n' )
cbit   ( n b -- n' )
```

DESCRIPTION

`sbit` leaves *n* with its bit *b* set to 1, and `cbit` with it cleared to 0. The other bits are as they were.

SEE ALSO

`tbit`, `or`, `and`, `lshift`.

EXAMPLE

```
/> lib bits
/> 0 3 sbit .
8
/> 15 0 cbit .
14
```

search

String; in the `string` library (`lib string`).

STACK

```
search   ( c-addr1 u1 c-addr2 u2 -- c-addr3 u3 flag )
```

DESCRIPTION

Looks for the second string inside the first. If it is found, the flag is true, and the string left is the rest of the first string from the place of the match on. If it is not found, the flag is false, and the first string is left as it was.

An empty second string is found at the start.

SEE ALSO

`compare`, `/string`.

EXAMPLE

```
/> lib string
/> s" hello world" s" o w" search . type
-1 o world
/> s" hello world" s" xyz" search . type
0 hello world
```

search-wordlist

Search-Order; in the `search` library (`lib search`).

STACK

```
search-wordlist   ( c-addr u wid -- 0 | xt 1 | xt -1 )
```

DESCRIPTION

Looks for a name, given as a string, in one word list, whether or not that list is in the search order. If it is not there the result is 0. If it is, the word's execution token is left, with 1 above it for an immediate word and –1 for any other.

Capitals and small letters are the same in a name.

SEE ALSO

`find`, `'`, `wordlist`, `traverse-wordlist`.

EXAMPLE

```
/> lib search
/> s" dup" forth-wordlist search-wordlist . drop
-1
/> s" if" forth-wordlist search-wordlist . drop
1
/> s" nothere" forth-wordlist search-wordlist .
0
```

see

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
see   ( "name" -- )
```

DESCRIPTION

Shows how a word is made. For a colon definition, the source is rebuilt from the compiled code: the words it calls, by name, with its numbers and strings. A constant is shown as its value, and a word made by `create` or `variable` as `create`.

The rebuilding is of the code, not of what was typed: comments are gone, a value worked out between `[` and `]` is a plain number, and the branches of `if` and `begin` appear as `?branch` and `branch` with the addresses they jump to. `do` loops and strings come back as they were written. `immediate` follows the `;` of an immediate word.

A word of the ROM that is written in assembly language has no Forth inside it. With the `disasm` library loaded (`lib disasm`), `see` shows its machine code; without it, just its name.

ERRORS

THROW CODE	MESSAGE	CAUSE
−13	*name* `?`	There is no such word

SEE ALSO

`words`, `disasm`, `dump`, `'`.

EXAMPLE

```
/> : square  ( n -- n*n )  dup * ;
/> see square
: square dup * ;
/> : count-up  10 0 do i . loop ;
/> see count-up
: count-up 10 0 do i . loop ;
/> : pick-one  if 1 else 2 then ;
/> see pick-one
: pick-one ?branch $804B 1 branch $8054 2 ;
/> 5 constant five  see five
: five 5 ;
```

seg-bank!

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
seg-bank!   ( seg n -- ior )
```

DESCRIPTION

Puts bank *n* of a shared segment at `bank-window`: the bank is then ordinary memory there, as one of the task's own is after `bank!`. The segment must be one this task is attached to: `sys-seg-create` makes one, and attaches its maker, and `sys-seg-attach` attaches another task that has its number.

Like `bank!`, it can't be used in a definition compiled into the code banks: see `code-banks`.

SEE ALSO

`bank!`, `bank-window`, `code-banks`; section 17.9.

send

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
send   ( "n line" -- )
```

DESCRIPTION

Types a line into another window. The first word of the rest of the line is a window's number, and what follows it is given to that window's shell as if it had been typed there, with ENTER after it. A line can be 126 characters at most.

With it one window can drive another: start a long job there, or a program whose output should be kept apart.

SEE ALSO

`spawn`; Chapter 10; the command `new-window`.

set-base, get-base

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
set-base   ( c-addr u -- )
get-base   ( -- c-addr u )
```

DESCRIPTION

The base in which numbers are read and printed, by name. `set-base` takes any of Chapter 18's base strings, which go far beyond what the variable `base` can say: `s" x"` and `s" b"`, but also bases with digits of their own, and balanced ones. `get-base` leaves the name of the base in force, `d` for decimal.

Cells and the number stack's numbers share the one setting. Setting `base`, or saying `hex` or `decimal`, sets it for both.

ERRORS

THROW CODE	MESSAGE	CAUSE
−24	`invalid numeric argument`	The string names no base

SEE ALSO

`base`, `n.base`, `hex`; sections 18.3 to 18.9.

EXAMPLE

```
/> lib numbers
/> get-base type
d
/> s" x" set-base  255 s>n n.  decimal
255
/> 255 s>n s" x" set-base n. decimal
FF
```

set-date, rtc

The devices; in the `clock` library, a file of Forth in `/lib/forth` (`lib clock`).

STACK

```
set-date   ( c-addr u -- )
rtc        ( -- c-addr u )
```

DESCRIPTION

`set-date` sets the clock from a string in the form `2026-10-13 09:41:00`; with a clock chip fitted, the chip is set too. `rtc` leaves the text that says what the system found for a clock chip as it started: `running`, `stopped` (it needs setting) or `none`, with `battery low` added if its battery is running down.

Reading the clock needs no library: `time&date`, in the Facility library, gives it.

SEE ALSO

`time&date`; section 17.5.

EXAMPLE

```
/> lib clock
/> rtc type
none
/> s" nonsense" set-date
set-date: invalid argument
```

sgr

Facility; in the `facility` library (`lib facility`). A HyForth word, not the standard's.

STACK

```
sgr   ( n -- )
```

DESCRIPTION

Sends the terminal any one of its text attributes by number: the sequence `ESC [` *n* `m`. The words for the styles and the colours are built on it; `sgr` reaches the attributes that have no word (section 13.7 lists the ones the Hydra's console knows).

SEE ALSO

`plain`, `color`.

sh

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
sh   ( c-addr u -- status )
```

DESCRIPTION

Runs a command line, given as a string, with the shell rc (`rc -c`), waits for it to finish, and leaves its exit status: 0 for success. Pipes, patterns, redirection and variables in the line are all rc's to deal with.

This is the way to run a command from inside a definition, where a line can't simply be typed. For a single program with plain arguments, `run` does without the shell.

SEE ALSO

`run`, `sh-out`, `%`, `status`.

EXAMPLE

```
/> lib hydra
/> s" echo hi | wc -c" sh .
      3
0
/> s" ls /nothere" sh .
ls: /nothere: not found
1
```

sh-out

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
sh-out   ( c-addr u -- c-addr2 u2 )
```

DESCRIPTION

Runs a command line with rc, as `sh` does, and leaves what it wrote to standard output as a string. Its exit code is in `status`. What it wrote to standard error still goes to the screen.

The string is in the free space above `here`, and holds as much of the output as fits there. It is good only until something is added to the dictionary: use it, or copy it, before defining anything.

SEE ALSO

`sh`, `output-of`, `piped`, `status`.

EXAMPLE

```
/> s" echo hi" sh-out type
hi
/> s" ls /rom/lib/shell" sh-out nip .
15
```

sliteral

String; in the `string` library (`lib string`). Immediate. Compile only.

STACK

```
sliteral   ( c-addr u -- )      compiling
           ( -- c-addr2 u )     when the definition runs
```

DESCRIPTION

Takes a string from the stack as a definition is compiled, and compiles a copy of it into the definition, so that the word leaves the copy's address and length when it runs. It is `literal` for a string, and what `s"` uses inside a definition.

SEE ALSO

`literal`, `s"`, `2literal`.

EXAMPLE

```
/> lib string
/> : motto  [ s" compiled in" ] sliteral ;
/> motto type
compiled in
```

snd-bend, snd-glide

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-bend    ( ch n -- )
snd-glide   ( ch n -- )
```

DESCRIPTION

`snd-bend` bends the pitch of a channel's note by *n* 64ths of a semitone, up for a positive number and down for a negative one. `snd-glide` moves the sounding note to note *n* without starting it again: the pitch changes, and the note's attack is not repeated.

SEE ALSO

`snd-note`, `snd-freq`.

snd-claim, snd-release, snd-claim-psg, snd-release-psg

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-claim         ( mask -- )
snd-release       ( mask -- )
snd-claim-psg     ( mask -- )
snd-release-psg   ( mask -- )
```

DESCRIPTION

Claim sound channels, so that no other program's sounds reach them, and give them back. The mask has a bit for each channel: bit *n* is channel *n* of the YM2151's eight, and for the `-psg` pair, voice *n* of the Vera X's PSG, which is channel 8 + *n*.

A claim lasts until it is released or HyForth ends. While HyForth has a channel, the `play` program can't use it: `snd-mml`, `snd-chord` and `snd-play` want it released first.

SEE ALSO

`snd-note`, `snd-reset`; Chapter 15.

snd-drum

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-drum   ( ch n -- )
```

DESCRIPTION

Plays a drum on a channel: *n* is a General MIDI drum number, 35 being a kick drum and 38 a snare.

SEE ALSO

`snd-note`, `tune`.

snd-freq

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-freq   ( ch hz -- )
```

DESCRIPTION

Starts a note at a frequency given in hertz, for a pitch that is not one of the scale's notes. A frequency of 0 ends it.

SEE ALSO

`snd-note`, `snd-bend`.

snd-level, snd-volume, snd-pan

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-level    ( ch v -- )
snd-volume   ( v -- )
snd-pan      ( ch pan -- )
```

DESCRIPTION

`snd-level` sets one channel's volume, 0 to 127. `snd-volume` sets the master volume for all sound, 0 to 200, where 100 is "as written". `snd-pan` chooses a channel's speakers: 1 the left, 2 the right, 3 both.

SEE ALSO

`snd-note`, `snd-reset`.

snd-mml, snd-chord, snd-play

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-mml     ( ch c-addr u -- status )
snd-chord   ( ch c-addr u -- status )
snd-play    ( c-addr u times -- status )
```

DESCRIPTION

Play music with the `play` program, and leave its exit status: 0 if all went well.

`snd-mml` plays a line of MML, the score language of section 15.6, on a channel. `snd-chord` plays the notes of such a line together, as a chord, on that channel and the ones after it. `snd-play` plays a song file, a ZSM, the number of times given; 0 plays its loop until CTRL-C.

Each waits until the music is over. `play` is another program, and needs the channels free: release any that HyForth has claimed first.

SEE ALSO

`tune`, `snd-claim`; the command `play`.

EXAMPLE

```
/> lib sound
/> 0 s" cde" snd-mml .
0
/> s" /nothere.zsm" 1 snd-play .
play: /nothere.zsm: not found
1
```

snd-note, snd-off

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-note   ( ch n -- )
snd-off    ( ch -- )
```

DESCRIPTION

`snd-note` starts a note on a channel: *n* is a MIDI note number, 60 being middle C and each semitone one more. `snd-off` ends the channel's note, which then dies away as its instrument does.

A note sounds until it is ended, so a note of a given length is `snd-note`, a wait, and `snd-off`.

SEE ALSO

`note-of`, `snd-patch`, `snd-freq`, `tune`, `ms`.

EXAMPLE

```
/> lib sound
/> lib facility
/> 0 5 snd-patch
/> 0 60 snd-note  500 ms  0 snd-off
```

snd-patch

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-patch   ( ch p -- )
```

DESCRIPTION

Loads an instrument into a channel: patch *p*, one of the 163 that the sound driver holds, 0 to 162 (Chapter 15). The notes played on the channel afterward have that instrument's sound.

A channel is 0 to 7 for the YM2151's voices, and 8 to 23 for the PSG's, with a Vera X.

SEE ALSO

`snd-note`, `snd-wave`, `snd-level`.

snd-regs

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-regs   ( c-addr -- )
```

DESCRIPTION

Copies the YM2151's 256 registers, as they were last written, into memory at *c-addr*, which must have room for 256 bytes. The chip itself can't be read: these are the driver's record of what has been sent to it.

SEE ALSO

`snd-reg`, `dump`.

EXAMPLE

```
/> lib sound
/> create regs 256 allot
/> 32 199 snd-reg  regs snd-regs  regs 32 + c@ .
199
```

snd-reset

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-reset   ( -- )
```

DESCRIPTION

Silences the sound chips and clears every setting: patches, volumes, bends. The channels a program has claimed stay claimed.

SEE ALSO

`snd-off`, `snd-volume`, `snd-release`.

snd-sens, snd-lfo, snd-noise, snd-reg

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-sens    ( ch pms ams -- )
snd-lfo     ( rate pmd amd wave -- )
snd-noise   ( n -- )
snd-reg     ( reg val -- )
```

DESCRIPTION

The sound driver's less common commands (section 15.3), each the driver's command of the same name with its numbers in the same order. `snd-sens` sets how strongly the low-frequency oscillator bends a channel's pitch (*pms*) and its loudness (*ams*). `snd-lfo` sets the oscillator: its rate, how deeply it works on pitch and on loudness, and its wave. `snd-noise` sets the noise generator's pitch. `snd-reg` writes a value straight to one of the FM chip's registers.

snd-wave

Sound; in the `sound` library (`lib sound`).

STACK

```
snd-wave   ( ch w width -- )
```

DESCRIPTION

Sets the waveform of a PSG channel, 8 to 23: *w* is 0 for a pulse, 1 a sawtooth, 2 a triangle, 3 noise. *width* is a pulse's width, 0 to 63, where 63 is a square wave.

SEE ALSO

`snd-patch`, `snd-note`.

spawn

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
spawn   ( c-addr u -- task )
```

DESCRIPTION

Starts a program, given as a string of its name and arguments as `run` takes them, and does not wait for it: the task's number is left, and both go on at once. The program has a note group of its own, and its task number is also put in the environment variable `apid`, as rc does for a command started with `&`.

`wait` waits for it to end.

SEE ALSO

`wait`, `run`, `note`, `status`.

EXAMPLE

```
/> s" sleep 1" spawn wait status .
0
```

spi, spi-read, spi-mode

The devices; in the `spi` library, a file of Forth in `/lib/forth` (`lib spi`).

STACK

```
spi        ( dev c-addr u -- )
spi-read   ( dev c-addr u -- )
spi-mode   ( dev mode -- )
```

DESCRIPTION

The SPI devices, 0 to 15 (section 17.4): 0 to 7 are the board's headers, and 8 to 15 the slots' cards.

`spi` is one transaction: the device is selected, the *u* bytes at *c-addr* are sent, and the bytes the device sent back meanwhile are put in their place, in the same memory. `spi-read` clocks *u* bytes in, sending `$FF` for each. `spi-mode` sets the clock's mode for the device, 0 or 3.

A device with nothing connected gives `$FF` for every byte. One that is an SD card the system is using can't be opened: `busy`.

SEE ALSO

`i2c-read`, `gpio`.

EXAMPLE

With nothing connected to device 3:

```
/> lib spi
/> create tx  1 c, 2 c, 3 c,
/> 3 tx 3 spi
/> tx c@ . tx 1+ c@ . tx 2 + c@ .
255 255 255
```

state

Core word set; in the ROM core, always there.

STACK

```
state   ( -- a-addr )
```

DESCRIPTION

Leaves the address of a cell that is zero while HyForth is interpreting and not zero while it is compiling a definition. An immediate word reads it to do one thing in a definition and another at the prompt.

A program must not store into it: `[` and `]`, `:` and `;` are what change it.

SEE ALSO

`[`, `]`, `immediate`.

EXAMPLE

```
/> state @ .
0
/> : compiling?  state @ . ; immediate
/> : test  compiling? ;
-1
```

status

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
status   ( -- n )
```

DESCRIPTION

Leaves the exit code of the last command that was run and waited for: a command line typed at the prompt, a `%` line, or a task that `wait` waited for. 0 means it succeeded.

SEE ALSO

`%`, `wait`, `sh`, `exits`.

EXAMPLE

```
/> ls /nothere
ls: /nothere: not found
/> status .
1
/> echo fine
fine
/> status .
0
```

/string

String; in the `string` library (`lib string`).

STACK

```
/string   ( c-addr1 u1 n -- c-addr2 u2 )
```

DESCRIPTION

Cuts *n* characters off the front of a string: the address goes up by *n*, and the length down by *n*. It is how a program steps along a string.

n is not checked. One greater than the length leaves a negative length, and a negative *n* moves the start back.

SEE ALSO

`-trailing`, `search`, `count`.

EXAMPLE

```
/> lib string
/> s" hello world" 6 /string type
world
```

swap

Core word set; in the ROM core, always there.

STACK

```
swap   ( x1 x2 -- x2 x1 )
```

DESCRIPTION

Exchanges the top two cells.

SEE ALSO

`over`, `rot`, `2swap`, `nip`, `tuck`.

EXAMPLE

```
/> 1 2 swap .s
<2> 2 1
/> 10 3 swap - .
-7
```

synonym

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
synonym   ( "newname" "oldname" -- )
```

DESCRIPTION

Defines a new name for a word that exists. The new word behaves exactly as the old one does, at the prompt and when it is compiled, and is immediate if the old one is.

ERRORS

THROW CODE	MESSAGE	CAUSE
–13	*oldname* `?`	There is no such word

SEE ALSO

`defer`, `:`, `'`.

EXAMPLE

```
/> synonym plus +
/> 2 3 plus .
5
```

sys

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
sys   ( addr a x y -- a x y p )
```

DESCRIPTION

Calls machine code. The 65C02's A, X and Y registers are loaded from the low bytes of *a*, *x* and *y*, and the code at *addr* is called as a subroutine. When it returns, with `RTS`, the three registers as it left them are on the stack, and above them the processor's flags, *p*: the carry is bit 0.

The code can be bytes laid down with `c,`, a routine loaded from a file, or an entry of the system's. HyForth's own data stack is kept in the zero page from `$22` up, indexed by X: code that uses zero page must keep clear of it. (The system's calls are easier to make with the `sys-` words.)

SEE ALSO

`sys-`, `c,`, `disasm`; Chapter 26.

EXAMPLE

The three bytes are `LDA #$2A` and `RTS`:

```
/> lib hydra
/> create code  $A9 c, $2A c, $60 c,
/> code 0 0 0 sys drop 2drop .
42
```

sys-

The Hydra; in the `hydra` library (`lib hydra`). A family of words, one for each system call.

STACK

(*inputs* -- *outputs* [*ior*])

DESCRIPTION

For each system call that a program can make there is a word named `sys-` and the call's name in small letters, with - for _: `sys-getpid`, `sys-open`, `sys-banks-alloc`. It takes the call's inputs from the stack in the order Chapter 3 lists them, the first deepest; makes the call; and leaves its outputs in the same order. If the call can fail, an *ior* is left on top: 0 for success, and otherwise −512 minus the system's error code.

An input that is an address is given as an address. A path is the address of a string that ends with a zero byte: `>z` (*c-addr u -- z-addr*) makes one from an ordinary string.

EXAMPLE

```
/> lib hydra
/> sys-getpid .
1
```

task-args, task-cwd, task-env, task-ns, task-regs, task-mem, task-ram

The devices; in the `proc` library, a file of Forth in `/lib/forth` (`lib proc`).

STACK

```
task-args   ( task -- c-addr u )
task-cwd    ( task -- c-addr u )
task-env    ( task -- c-addr u )
task-ns     ( task -- c-addr u )
task-regs   ( task -- c-addr u )
task-mem    ( task addr c-addr u -- )
task-ram    ( task bank offset c-addr u -- )
```

DESCRIPTION

A look inside any task, through its files in `/proc` (section 17.8). The first five leave text, in a buffer of the library's, 512 characters at most: the task's arguments; its current directory; its environment, a line for each variable; its namespace, as the `bind` and `mount` lines that made it; and the processor's registers as they were when it last stopped running.

`task-mem` reads *u* bytes of the task's memory, from *addr* as the task sees its 64K, into memory at *c-addr*; `task-ram` reads from one of its RAM banks. The memory and registers of the kernel's task and of the drivers can't be read: `not allowed`.

SEE ALSO

`sys-getpid`, `note`; the commands `ps` and `db`.

EXAMPLE

```
/> lib proc
/> 1 task-args type
-1
/> 1 task-cwd type
/
/> 9 task-args type
task-args: not found
```

tbit

Bits; in the `bits` library (`lib bits`).

STACK

```
tbit   ( n b -- n flag )
```

DESCRIPTION

Tests one bit of a cell: the flag is true if bit *b* of *n* is 1. Bit 0 is the lowest and bit 15 the highest. *n* stays on the stack, under the flag, so several bits can be tested in turn.

Only the low four bits of *b* count: bit 16 is bit 0.

SEE ALSO

`sbit`, `and`, `nbit?`.

EXAMPLE

```
/> lib bits
/> 5 0 tbit . drop
-1
/> 5 1 tbit . drop
0
```

text

The screen; in the `video` library (`lib video`).

STACK

```
text   ( x y c-addr u -- )
```

DESCRIPTION

Draws a string on the bitmap, in the pen's colour and the console's 8 by 8 font, with the top left of its first character at *x*, *y*. At most 40 characters are drawn.

Text drawn this way is part of the picture: unlike what `type` prints, it does not scroll.

SEE ALSO

`type`, `at-xy`, `pen`.

throw

Exception; in the `exception` library, which is loaded as HyForth starts.

STACK

```
throw   ( k*x n -- k*x | i*x n )
```

DESCRIPTION

With 0 on the stack, `throw` removes it and does nothing more. With any other number, it abandons what is running and returns to the most recent `catch` that is still waiting, which leaves that number as its result. So a word can write `throw` after anything that leaves an error code, zero meaning "none": the file words leave such a code, and `open-file throw` is the usual way to use them.

If no `catch` is waiting, the throw goes to HyForth itself: its message is printed, both stacks are emptied, and the prompt returns.

The codes. A program may use any positive number for errors of its own. The negative ones are the system's:

CODE	MESSAGE	MEANING
−1	(none)	`abort`
−2	The text given	`abort"`
−3	`stack overflow`	More than 32 cells on the data stack
−4	`stack underflow`	A word took more than the stack held
−8	`dictionary full`	No data space left
−10	`division by zero`	
−11	`result out of range`	
−13	The word, and `?`	A word that isn't known
−14	The word, and `compile only`	A word that works only in a definition, used at the prompt
−16	`no name`	A defining word with no name after it
−17	`number too long`	Pictured numeric output with too many digits
−21	`unsupported operation`	
−22	`control structure mismatch`	
−24	`invalid numeric argument`	
−28	`interrupt`	CTRL-C
−44, −45	`number stack overflow`, `underflow`	The numbers library's stack
−46	`outside the function's domain`	The numbers library: the logarithm of 0, and the like
−49, −50	`search order full`, `empty`	
−59, −60, −61	`allocate failed`, `free failed`, `resize failed`	The memory library
−258	`sources nested too deeply`	Files that include files, too deep
−259	`not this forth's library`	A library file made for another HyForth
−512 − *code*	The system's message	A system call failed: *code* is its error code (Appendix G). −544 is `not found`

Any other number is reported as `error` and the number.

SEE ALSO

`catch`, `abort"`, `ior>text`, `quit`.

EXAMPLE

```
/> : check  ( n -- )  100 > if 1 throw then ;
/> 50 ' check catch .
0
/> 500 ' check catch . drop
1
/> -10 throw
division by zero
/> 99 throw
error 99
/> -544 throw
not found
```

time&date

Facility; in the `facility` library (`lib facility`).

STACK

```
time&date   ( -- +n1 +n2 +n3 +n4 +n5 +n6 )
```

DESCRIPTION

Leaves the date and time now, as six numbers: the seconds (0 to 59), the minutes, the hours (0 to 23), the day of the month, the month (1 to 12) and, on top, the year in full.

On a Hydra with no clock chip, whose clock has not been set, the time counts from the start of 1 January 2000 when it is switched on (section 17.5).

SEE ALSO

`ms`; the command `date`.

EXAMPLE

```
/> lib facility
/> time&date . . . . . .
2026 10 13 9 41 0
```

to-fixed

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
to-fixed   ( u -- ) ( N: x -- y )
```

DESCRIPTION

Makes the top number a decimal with at most *u* places after the point. Digits beyond that are cut off, not rounded: two thirds to two places is 0.66. This is where a fraction is turned into something to print.

SEE ALSO

`to-rational`, `nround`, `n.`.

EXAMPLE

```
/> lib numbers
/> 1 s>n 3 s>n n/ 4 to-fixed n.
0.3333
/> 2/3 2 to-fixed n.
0.66
```

to-rational, rational.n, rational.d

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
to-rational   ( N: x -- y )
rational.n    ( N: x -- n )
rational.d    ( N: x -- n )
```

DESCRIPTION

`to-rational` makes a decimal the exact fraction it equals, in its lowest terms. `rational.n` leaves the top of that fraction, the numerator, and `rational.d` the bottom, the denominator; an integer's denominator is

1.

SEE ALSO

`to-fixed`, `n/`, `ngcd`.

EXAMPLE

```
/> lib numbers
/> 0.25 to-rational n.
1/4
/> 3/4 rational.n n. 3/4 rational.d n.
3 4
```

-trailing

String; in the `string` library (`lib string`).

STACK

```
-trailing   ( c-addr u1 -- c-addr u2 )
```

DESCRIPTION

Shortens a string so that it does not end in spaces: the address stays, and the length becomes that of the string without them. A string of nothing but spaces becomes one of length 0.

SEE ALSO

`/string`, `blank`.

EXAMPLE

```
/> lib string
/> s" hi   " -trailing . drop
2
```

traverse-wordlist

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
traverse-wordlist   ( i*x xt wid -- j*x )
```

DESCRIPTION

Calls a word once for each name in a word list, newest first. The word, given by its token *xt*, is called with the name's **name token** on the stack, above whatever it left from the call before, and must leave a flag: true to go on to the next name, false to stop there.

A name token, *nt*, stands for one entry in the dictionary. The words below take it apart.

`forth-wordlist`, in the `search` library, gives the word list that holds the system's words.

SEE ALSO

`name>string`, `words`, `forth-wordlist`, `search-wordlist`.

EXAMPLE

```
/> lib search
/> : count-it  ( n nt -- n' flag )  drop 1+ true ;
/> : first-three  ( n nt -- n' flag )  name>string type space  1+ dup 3 < ;
/> 0 ' first-three forth-wordlist traverse-wordlist drop
first-three count-it end-library
```

true, false

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
true    ( -- true )
false   ( -- false )
```

DESCRIPTION

The two flags, as constants: `true` is −1, a cell with every bit set, and `false` is 0.

SEE ALSO

`0=`, `and`, `if`.

EXAMPLE

```
/> true . false .
-1 0
```

truncate, nfloor, nround

Numbers; in the `numbers` library (`lib numbers`).

STACK

```
truncate   ( N: x -- n )
nfloor     ( N: x -- n )
nround     ( N: x -- n )
```

DESCRIPTION

Make a real number whole. `truncate` drops what follows the point, which rounds toward zero. `nfloor` rounds down, toward minus infinity. `nround` rounds to the nearest whole number, and a number exactly halfway between two goes to the even one: 0.5 to 0, 1.5 and 2.5 to 2, 3.5 to 4.

SEE ALSO

`to-fixed`, `n>s`, `n/mod`.

EXAMPLE

```
/> lib numbers
/> 2.7 truncate n. -2.7 truncate n.
2 -2
/> -2.7 nfloor n.
-3
/> 2.5 nround n. 3.5 nround n.
2 4
```

tune

Sound; in the `sound` library (`lib sound`).

STACK

```
tune   ( c-addr u ch tempo -- )
```

DESCRIPTION

Plays a tune on a channel, and returns when it is over. The string is a list of notes and their lengths, with spaces between: a note's name as `note-of` takes it, or – for a rest, and then its length in beats, 1 to 255. *tempo* is the number of beats in a minute.

Each note is started on its beat and ended just before the next, in time with the system's tick.

SEE ALSO

`note-of`, `snd-mml`, `snd-patch`.

EXAMPLE

```
/> lib sound
/> 0 5 snd-patch
/> s" C4 1 E4 1 - 1 G4 2" 0 240 tune
```

type

Core word set; in the ROM core, always there.

STACK

```
type   ( c-addr u -- )
```

DESCRIPTION

Writes a string to standard output: *u* characters, starting at the address. With a *u* of 0, nothing.

A string in Forth is an address and a length, on the stack as two cells: `s"` leaves one in that form, and so do `count`, `parse` and the file words.

SEE ALSO

`s"`, `count`, `emit`, `."`, `dump`.

EXAMPLE

```
/> s" hello" type
hello
/> s" hello" drop 4 type
hell
```

um, m

Core word set; in the ROM core, always there.

STACK

```
um*    ( u1 u2 -- ud )
m*     ( n1 n2 -- d )
```

DESCRIPTION

Multiply two cells and leave the whole 32-bit product, which cannot overflow. `um*` takes the cells as unsigned and leaves an unsigned double; `m*` takes them as signed and leaves a signed double.

SEE ALSO

`*`, `um/mod`, `*/`, `d.`, `m*/`.

EXAMPLE

```
/> lib double
/> 40000 3 um* d.
120000
/> -2 3 m* d.
-6
/> 300 300 m* d.
90000
```

(`d.`, which prints a double, is in the `double` library.)

um/mod, fm/mod, sm/rem

Core word set; in the ROM core, always there.

STACK

```
um/mod    ( ud u1 -- u2 u3 )
fm/mod    ( d n1 -- n2 n3 )
sm/rem    ( d n1 -- n2 n3 )
```

DESCRIPTION

Divide a double by a cell, and leave the remainder, and above it the quotient, each a cell.

WORD	THE NUMBERS	THE QUOTIENT	THE REMAINDER
`um/mod`	Unsigned		
`fm/mod`	Signed	Rounded down, toward minus infinity ("floored")	Has the divisor's sign, or is 0
`sm/rem`	Signed	Rounded toward zero ("symmetric")	Has the dividend's sign, or is 0

For numbers of the same sign the two signed words agree. They differ when the signs differ: −7 by 2 is −4 remainder 1 floored, and −3 remainder −1 symmetric. HyForth's `/` and `mod` are symmetric; `fm/mod` is the one for a remainder that is never negative with a positive divisor.

The quotient must fit in a cell: one that does not is not reported.

ERRORS

THROW CODE	MESSAGE	CAUSE
−10	`division by zero`	The divisor is 0

SEE ALSO

`/mod`, `um*`, `m*`, `s>d`.

EXAMPLE

```
/> 100000. 7 um/mod . .
14285 5
/> -7. 2 fm/mod . .
-4 1
/> -7. 2 sm/rem . .
-3 -1
```

unused

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
unused   ( -- u )
```

DESCRIPTION

Leaves the number of bytes of data space still free: from `here` to the dictionary's end.

That is the room for names and data. The code of colon definitions has its own room, in RAM banks, which `code-banks` reports.

SEE ALSO

`here`, `allot`, `code-banks`.

EXAMPLE

The number depends on what has been loaded:

```
/> unused u.
17785
```

update, save-buffers, flush, empty-buffers

Block; in the `block` library (`lib block`).

STACK

```
update          ( -- )
save-buffers    ( -- )
flush           ( -- )
empty-buffers   ( -- )
```

DESCRIPTION

`update` marks the block most recently given by `block` or `buffer` as changed, so that it will be written back. `save-buffers` writes every changed block to the block file. `flush` does the same and then lets the buffers go. `empty-buffers` lets them go *without* writing: changes not yet saved are thrown away.

Changed blocks are also written when HyForth ends.

NOTES

A block file grows a block at a time. A block can be written only if every block before it is in the file: in a new file, write block 0 first, then 1, and so on, with a `flush` after each, since several changed blocks are not always written in their order. Writing block 4 to a file of three blocks is the error `end of file`.

SEE ALSO

`block`, `open-blocks`.

value, to

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
value   ( x "name" -- )
name    ( -- x )
to      ( x "name" -- )
```

DESCRIPTION

`value` takes a number from the stack and defines a word that leaves that number when it is used, as a constant does. `to` changes it: `20 to name` stores 20 as the value's new number. `to` works at the prompt and inside a definition.

A value reads like a constant and can be changed like a variable, without a `@` at every use.

NOTES

`to` is only for a name that `value` made (or `2value`, `nvalue`, or a local of `{:`). It does not check: `to` followed by a constant's or a variable's name is not reported, and stores the number somewhere it does not belong.

SEE ALSO

`variable`, `constant`, `2value`, `{:`.

EXAMPLE

```
/> 10 value limit
/> limit .
10
/> 20 to limit  limit .
20
/> : raise  limit 5 + to limit ;
/> raise limit .
25
```

variable, constant

Core word set; in the ROM core, always there.

STACK

```
variable   ( "name" -- )
constant   ( x "name" -- )
name       ( -- a-addr )       a variable
name       ( -- x )            a constant
```

DESCRIPTION

`variable` defines a word that owns one cell of data space, which starts as 0. When the word is used it leaves the address of the cell, for `@` and `!`.

`constant` takes a value from the stack and defines a word that leaves that value when it is used.

A constant's value is not meant to change. For a value that changes now and then and is read often, `value` is neater than a variable.

SEE ALSO

`value`, `create`, `2variable`, `2constant`, `@`, `!`.

EXAMPLE

```
/> variable score
/> score @ .
0
/> 100 score ! score @ .
100
/> 60 constant minute
/> minute 5 * .
300
```

vpeek, vpoke, vram@, vram!

The screen; in the `video` library (`lib video`).

STACK

```
vpeek   ( addr bank -- c )
vpoke   ( addr bank c -- )
vram@   ( addr bank c-addr u -- )
vram!   ( addr bank c-addr u -- )
```

DESCRIPTION

The VERA's own memory, the video RAM: 128K, so an address is 17 bits, given as a cell and a *bank* of 0 or

1. `vpeek` leaves the byte there and `vpoke` stores one. `vram@` reads *u* bytes from it into memory at *c-addr*, and `vram!` writes *u* bytes to it.

This is the chip's raw memory: the bitmap, the tiles and the sprites' pictures are all in it (Chapter 16 gives the layout).

SEE ALSO

`palette!`, `sprite!`, `bitmap`.

wait

The shell; in the `shell` library, which is loaded when HyForth starts as a shell.

STACK

```
wait   ( task -- )
```

DESCRIPTION

Waits until a task that this HyForth started has ended: one from `spawn`, or from a command line that ended in `&`. Its exit code is then in `status`.

SEE ALSO

`spawn`, `status`.

while, repeat

Core word set; in the ROM core, always there. Immediate. Compile only.

STACK

```
while    ( x -- )
repeat   ( -- )
```

DESCRIPTION

A loop that tests before its work: `begin` *test* `while` *work* `repeat`. The words of the test are run, and `while` takes a flag from the stack. If it is true, the work is done and `repeat` goes back to `begin` for the test again. If it is false, the loop is over, and the words after `repeat` are run.

Unlike `begin` ... `until`, the work may be done no times at all, when the test fails the first time.

SEE ALSO

`begin`, `until`, `again`, `do`.

EXAMPLE

```
/> : countdown  begin dup while dup . 1- repeat drop ;
/> 5 countdown
5 4 3 2 1
/> 0 countdown
```

window, windows, new-window, new-group, show-window, window-label, window-status, window-ctl

The devices; in the `cons` library, a file of Forth in `/lib/forth` (`lib cons`).

STACK

```
window          ( -- n )
windows         ( -- c-addr u )
show-window     ( n -- )
new-window      ( c-addr u -- )
new-group       ( c-addr u -- )
window-label    ( c-addr u -- )
window-status   ( c-addr u -- )
window-ctl      ( c-addr u -- )
```

DESCRIPTION

The console's windows (Chapter 10). `window` leaves the number of the window HyForth is in. `windows` leaves the text that lists them all, a line each, with a `*` on the one that is shown. `show-window` shows window *n*.

`new-window` makes a window, shows it, and runs the command line given in it; an empty string gives it a shell. `new-group` does the same with the window in a note group of its own.

`window-label` sets this window's title (an empty string puts back its program's name), `window-status` the text of its status line, and `window-ctl` writes any line to its control file.

SEE ALSO

`send`, `form`, `k-resize`; the command `new-window`.

EXAMPLE

```
/> lib cons
/> window .
0
/> windows type
0 0 80 24 *
```

within

Core Extension; in the `coreext` library, which is loaded as HyForth starts.

STACK

```
within   ( n1 n2 n3 -- flag )
```

DESCRIPTION

True when *n1* is in the range that starts at *n2* and stops just before *n3*: *n2* ≤ *n1* < *n3*. The lower limit is in the range, and the upper one is not.

The three may all be signed numbers or all unsigned ones, and the test is right for either. It is worked out as a single unsigned comparison, (*n1* – *n2*) `u<` (*n3* – *n2*), so if the limits are given in the wrong order the result is not simply false: it is true for the numbers *outside* the range between them.

SEE ALSO

`<`, `u<`, `min`, `max`.

EXAMPLE

```
/> 5 1 10 within . 1 1 10 within . 10 1 10 within .
-1 -1 0
/> -1 -5 5 within .
-1
```

word, source, >in

Core word set; in the ROM core.

STACK

```
word     ( char "<chars>ccc<char>" -- c-addr )
source   ( -- c-addr u )
>in      ( -- a-addr )
```

DESCRIPTION

The text interpreter's own tools, for words that read what follows them in the input.

`source` gives the address and length of the input buffer: the line, or the string, being interpreted. `>in` is a variable holding how far into it the interpreter has got, in characters; storing into it moves the place the next word will be read from.

`word` skips any of *char* at the place, reads up to the next *char* or the end of the line, and returns the address of a **counted string**: a length byte, then the characters. The string is in a transient buffer. `bl word` reads the next name.

`word` is the older tool. `parse-name` and `parse`, which return an address and a length within the input buffer and copy nothing, are simpler to use.

wordlist

Search-Order; in the `search` library (`lib search`).

STACK

```
wordlist   ( -- wid )
```

DESCRIPTION

Makes a new, empty word list, and leaves its identifier. A **word list** is a set of names of its own: the same name can be in two word lists and mean two things. The system's words are all in one, the one `forth-wordlist` gives.

A new word list is not searched, and takes no definitions, until a program says so: `set-current` makes it the one that new words go into, and `set-order` (or `also` and a `set-order`) puts it in the search order. The identifier is the only handle on it, so it is kept in a constant.

For the usual case, a named word list that holds a library's private words, `library` does all of this in one step.

SEE ALSO

`set-current`, `set-order`, `search-wordlist`, `library`, `order`.

EXAMPLE

```
/> lib search
/> wordlist constant mine
/> mine set-current
/> : secret  42 ;
/> definitions
/> s" secret" mine search-wordlist . execute .
-1 42
/> s" secret" forth-wordlist search-wordlist .
0
```

words

Programming-Tools; in the `tools` library, which is loaded as HyForth starts.

STACK

```
words   ( -- )
```

DESCRIPTION

Lists the words of the first word list in the search order, the newest first: for each, its execution token in hex, three marks, and its name, as many to a line as the screen's width allows.

The marks say what kind of word it is: an `l` for a word that is a number compiled in place (a constant such as `true`), an `i` for an immediate word, and then `a` for a word written in assembly language or `f` for one written in Forth.

The list is long: the ROM core and the libraries loaded at the start have about 270 words. Words defined since are at the top.

SEE ALSO

`see`, `libs`, `order`, `traverse-wordlist`, `[defined]`.

EXAMPLE

The first lines, after two words were defined:

```
/> : square dup * ;  : cube dup square * ;
/> words
 3974 --f cube            3967 --f square          3756 --a unsetenv
 36E7 --a setenv          3671 --a getenv          365F --a wait
 3641 -ia exit            361D --a exits           3609 --a status
...
```

write-file, write-line

File Access; in the `file` library, which is loaded as HyForth starts.

STACK

```
write-file   ( c-addr u fileid -- ior )
write-line   ( c-addr u fileid -- ior )
```

DESCRIPTION

Write a string to a file, at its current place, and move the place on. `write-file` writes the *u* bytes and nothing else. `write-line` writes them and then a line end, a line feed.

The file must have been opened `w/o` or `r/w`: writing to one opened `r/o` is the ior –549.

SEE ALSO

`read-file`, `read-line`, `flush-file`, `create-file`, `type`.

EXAMPLE

```
/> s" /ram/f.txt" w/o create-file throw value fd
/> s" one" fd write-line .
0
/> s" two" fd write-file .
0
/> fd close-file drop
/> cat /ram/f.txt
one
two/>
```

>z, zcount

The Hydra; in the `hydra` library (`lib hydra`).

STACK

```
>z       ( c-addr u -- z-addr )
zcount   ( z-addr -- c-addr u )
```

DESCRIPTION

The system's calls take a name as a **zero-terminated string**: its characters, and then a byte of 0. Forth's strings are an address and a length. These two convert between them.

`>z` copies a string, puts a zero after it, and leaves the copy's address, ready for a `sys-` word. The copy is in one of two buffers, used in turn, so two names can be held at once, for a call that takes two; a third `>z` overwrites the first. A string can be 127 characters at most.

`zcount` goes the other way: given the address of a zero-terminated string, it leaves that address and the number of characters before the zero.

SEE ALSO

`sys-`, `count`, `s"`.

EXAMPLE

```
/> lib hydra
/> s" /rom/lib/shell" >z 0 sys-open throw value fd
/> fd sys-close .
0
/> s" hello" >z zcount type
hello
```

CHAPTER 5

hylang Forms

hylang is a Lisp. A program is a series of expressions, and each is evaluated to a value.

Evaluation. A number, a string, a character or an atom is its own value. A symbol's value is what it is bound to, and an unbound one is an error. An expression in parentheses, an **S-expression**, is a call: its first element is evaluated to find a function, the rest are evaluated left to right as its arguments, and the function is applied to them. An expression in braces, a **Q-expression**, is data: a list, which is not evaluated until something runs it. `[a b]` is short for `(list a b)`.

Special forms are the exception to "the arguments are evaluated first". Each receives its arguments as they were written and decides for itself which to evaluate, and when: `if` evaluates one of its two branches, `def` doesn't evaluate the names it is given. Their entries say exactly what each one does with each argument. Everything else in this chapter is a function, and its arguments are values.

Truth. `NIL` is false, and so is the empty list, which is the same thing. Everything else is true, including 0 and the empty string. `T` is the usual value for true. The two may be written in small letters too, `t` and `nil`, and for that reason neither can be the name of a variable or of a function's formal: `(fn {t} ...)` is an error.

Scopes. A name is looked up in the current scope, then in the one that encloses it, and so on out to the global scope. A call of a function has a scope of its own, enclosed by the scope the function was made in, not by the caller's. A function can therefore go on using the names that were in force where it was made, after that code has finished: a closure.

Functions and their arguments. A function called with fewer arguments than it has formals is not an error. The result is another function, which takes the rest: `((add 1) 5)` is `(add 1 5)`. Called with more than it has, it is an error, unless the function uses `&_`, which is the list of the extra ones. A function made with no formals refers to its arguments as `&1`, `&2` and so on.

The built-in functions follow the same rule. `(cons 1)` is not an error but a function, waiting for its list, and so is a built-in function called with no arguments at all: `(join)` is `join` itself. One given more arguments than it takes is an error that says so: `'len' takes 1 argument, not 2`. And a call whose first item is not a function is the error `S-Expression starts with incorrect type. Got Number, Expected Function`. The entries do not repeat these for every function.

Errors. An error is a value, printed as `Error:` and a message. A function that is given an error as an argument does not run: the error is its result. So an error travels outward through every expression that contains it until `try` catches it or it reaches the prompt. `(error "text")` makes one.

Numbers are those of Chapter 18: integers of any length, exact fractions, decimals, complex numbers.

Each entry says what kind of thing it describes: a special form, a function, a function of the library, which is written in hylang itself and loaded as hylang starts, or one of the Hydra's libraries, which a program loads with `use` (the entries named "gpio library", "snd library" and so on).

+, −, *, /

Functions: arithmetic. + also joins values to a string.

SYNTAX

```
(+ x ...)
(- x ...)
(* x ...)
(/ x ...)
(+ s x ...)
```

ARGUMENTS

x

Numbers of any kind: integers, fractions, decimals, complex numbers. There may be any number of them.

s

A string. When +'s first argument is a string, the rest may be any values.

RESULT

With two or more numbers, the sum, the difference, the product or the quotient, worked from the left: `(- 10 3 2)` is 10 - 3 - 2. With one number, + and * give it back, – gives its negative, and / gives its reciprocal, 1 over it. `(+)` is 0.

DESCRIPTION

The arithmetic is exact (Chapter 18). An integer has as many digits as it needs. A division that does not come out whole gives a fraction in its lowest terms, never a rounded decimal, and decimals add and multiply without error: `(+ 0.1 0.2)` is exactly 0.3. A result is given in the simplest kind that holds it: a fraction and a decimal make a fraction, and a result with no fractional part is an integer, so `(* 1.5 2)` is 3.

Joining to a string. If +'s first argument is a string, every other argument is turned to text, as `print` would show it, and joined on: the result is a string. This is the usual way to build a message. Only the *first* argument decides: a number first and a string later is an error.

ERRORS

MESSAGE	CAUSE
`Division by zero.`	`/` with a divisor of 0
`'+' adds numbers, or joins values to a string`	`+` with a first argument that is a number, and a later one that is not
`All parameters to operator '-' must be numbers.`	`-`, `*` or `/` (each with its own sign in the message) given anything but numbers

NOTES

An imaginary number is written with digits before its `i`: `2i`, `1+1i`. `1+i` is read as a symbol, and is an error unless something has that name.

SEE ALSO

`1+`, `sum`, `product`, `div`, `mod`, `pow`, `format`, `str-join`.

EXAMPLE

```
hylang> (+ 1 2 3)
=> 6
hylang> (- 5)
=> -5
hylang> (/ 1 3)
=> 1/3
hylang> (/ 6 4)
=> 3/2
hylang> (+ 0.1 0.2)
=> 0.3
hylang> (* 99999999999 99999999999)
=> 9999999998800000000001
hylang> (* 2i 2i)
=> -4
hylang> (+ "n = " 1/2 " and " {1 2})
=> "n = 1/2 and {1 2}"
hylang> (+ 1 "a")
=> Error: '+' adds numbers, or joins values to a string
```

<, >, <=, >=, cmp

Functions: compare two values.

SYNTAX

```
(< x y)
(> x y)
(<= x y)
(>= x y)
(cmp x y)
```

ARGUMENTS

x*, *y

Any two values.

RESULT

<, >, <= and >= give `T` or `NIL`: is *x* before *y*, after it, not after it, not before it? `cmp` gives -1 if *x* comes before *y*, 0 if they are equal, and 1 if it comes after.

DESCRIPTION

Any two values can be compared: there is no error for values of different kinds. The order is by kind first: numbers, then characters, strings, atoms, symbols, Q-expressions, S-expressions, `T`, functions, hashes, buffers, streams and errors. Within a kind:

KIND	ORDER
Numbers	By value, whatever their form
Strings	Character by character, by character code: capitals come before small letters
Atoms, symbols	By name
Lists	Item by item. A list that is the start of another comes first

This is the order `sort`, `min` and `max` use.

Each takes exactly two arguments: `(< 1 2 3)` is an error. To test a range, use `and`, or the library's `between?`.

SEE ALSO

==, `sort`, `min`, `<=>`, `between?`.

EXAMPLE

```
hylang> (< 1 2)
=> T
hylang> (<= 1 1)
=> T
hylang> (< 1/3 0.5)
=> T
hylang> (< "apple" "pear")
=> T
hylang> (cmp 2 1)
=> 1
hylang> (cmp {1 2} {1 3})
=> -1
hylang> (< 1 "a")
=> T
```

<=>

A special form: chooses by the sign of a number.

SYNTAX

```
(<=> n {test value} ...)
```

ARGUMENTS

n

An expression whose value is a number.

test

One of =, <>, <, >, <=, >=.

value

An expression.

DESCRIPTION

n is evaluated, and compared with 0. The cases are taken in order, and the first whose test is true of that comparison has its *value* evaluated and returned: `{< x}` if *n* is negative, `{= x}` if it is zero, `{>= x}` if it is zero or positive, and so on. If no case's test is true, the result is `NIL`.

It is the three-way branch for the result of a subtraction or of a comparison function.

EXAMPLE

```
hylang> (<=> -5 {< "negative"} {= "zero"} {> "positive"})
=> "negative"
hylang> (<=> 0 {< "negative"} {>= "not negative"})
=> "not negative"
```

==, eq, neq

Functions: are two values equal, or not?

SYNTAX

```
(== x y)
(eq x y)
(neq x y)
```

ARGUMENTS

x, y

Any two values.

RESULT

== and `eq`, which are the same function, give `T` if the values are equal and `NIL` if not. `neq` gives the opposite. The library adds two more names for `neq`: <> and `!=`.

DESCRIPTION

Two values of different kinds are never equal: the number 1 is not the string `"1"`. Within a kind:

KIND	EQUAL WHEN
Numbers	They have the same value, whatever their form: 1, 1.0 and 2/2 are equal, and so are 1/2 and 0.5
Strings, characters	They have the same characters. Capitals and small letters differ
Atoms, symbols	They have the same name
Lists	They have the same length, and each item equals the other's. `NIL` equals `{}`
Functions	They have the same formals and the same body

Each takes exactly two arguments.

SEE ALSO

`cmp`, `<`, `elem?`.

EXAMPLE

```
hylang> (== 1 1.0)
=> T
hylang> (== 1/2 0.5)
=> T
hylang> (== "a" "A")
=> NIL
hylang> (== {1 2} {1 2})
=> T
hylang> (== 1 "1")
=> NIL
hylang> (neq 1 2)
=> T
```

??

A function of the library, written in hylang: the first of its arguments that is not `NIL`.

SYNTAX

```
(?? a ...)
```

ARGUMENTS

a

Any values: at least one.

RESULT

The first argument that is not `NIL`. If all are `NIL`, `NIL`.

DESCRIPTION

`??` supplies a value where one may be missing: `(?? (env "editor") "edit")` is the variable's value if there is one, and `"edit"` if not. Only `NIL` (and the empty list) counts as missing: 0 and the empty string are values, and are returned.

It is a function, so all its arguments are evaluated before it chooses. Where a later argument must not be evaluated unless it is needed, use `or`, which stops at the first true value.

SEE ALSO

`or`, `if`, `nil?`.

EXAMPLE

```
hylang> (?? NIL NIL 3 4)
=> 3
hylang> (?? 0 1)
=> 0
hylang> (?? (env "nothere") "a default")
=> "a default"
```

=0, =1, >0, <0, >=0, <=0, !=0, !=1

Functions of the library: comparisons with 0 and with 1, each of one argument.

SYNTAX

```
(=0 n)   (=1 n)   (!=0 n)   (!=1 n)
(>0 n)   (<0 n)   (>=0 n)   (<=0 n)
```

ARGUMENTS

n

A number.

RESULT

`T` or `NIL`: is *n* equal to 0, equal to 1, not 0, not 1, above 0, below 0, 0 or more, 0 or less?

DESCRIPTION

Each is the comparison its name spells, with the 0 or 1 already given. They save writing a function where one is wanted as an argument: `(filter >0 l)` keeps the positive numbers of a list. `=0` and `=1` are other names for `zero?` and `one?`.

They are plain comparisons, and give no error for a value that is not a number: such a value is ordered by its kind, after every number, so `(>0 "a")` is `T`.

SEE ALSO

`zero?`, `one?`, `pos?`, `neg?`, `<`.

EXAMPLE

```
hylang> (=0 0)
=> T
hylang> (>0 -5)
=> NIL
hylang> (filter >0 {3 -1 0 7})
=> {3 7}
hylang> (count =0 {0 1 0})
=> 2
```

1+, 1-, abs

Functions: a number plus one; minus one; and without its sign.

SYNTAX

```
(1+ x)
(1- x)
(abs x)
```

ARGUMENTS

x

A number.

RESULT

`1+` gives *x* + 1 and `1-` gives *x* - 1, for any kind of number. `abs` gives *x* if it is 0 or more, and its negative if it is below 0.

ERRORS

MESSAGE	CAUSE
`Cannot perform 'abs' on non-number`	`abs` of something that is not a number
`All parameters to operator '-' must be numbers.`	`1-` of something that is not a number

NOTES

`1+` is `+` with a 1, so `(1+ "a")` is not an error: it is the string `"a1"`.

`abs` does not give a complex number's size. It treats one as it treats any number, by its order: a complex number whose real part is negative comes back with both its signs changed, and any other comes back as it is.

EXAMPLE

```
hylang> (1+ 5)
=> 6
hylang> (1- 5)
=> 4
hylang> (1+ 1/2)
=> 3/2
hylang> (abs -5)
=> 5
hylang> (abs -2.5)
=> 2.5
```

alpha?, digit?, space?, upper?, lower?

Functions: tests of a character's kind.

SYNTAX

```
(alpha? c)
(digit? c)
(space? c)
(upper? c)
(lower? c)
```

ARGUMENTS

c

A character, or a string.

RESULT

`T` or `NIL`:

FUNCTION	`T` FOR
`alpha?`	A letter, A to Z or a to z
`digit?`	A digit, 0 to 9
`space?`	A space, a tab, a line feed, a carriage return or a form feed
`upper?`	A capital letter
`lower?`	A small letter

For a string, the test must pass for every character of it: `(digit? "12")` is `T` and `(alpha? "a1")` is `NIL`. An empty string gives `NIL`.

ERRORS

MESSAGE	CAUSE
`'alpha?' expects a String`	*c* is neither a character nor a string (each function with its own name)

EXAMPLE

```
hylang> (alpha? \a)
=> T
hylang> (digit? \a)
=> NIL
hylang> (digit? "12")
=> T
hylang> (upper? "Ab")
=> NIL
hylang> (filter digit? (str-chars "a1b22"))
=> {\1 \2 \2}
```

and, or

Special forms: the logical operators, which stop as soon as the answer is known.

SYNTAX

```
(and x ...)
(or x ...)
```

DESCRIPTION

`and` evaluates its arguments from the left. At the first whose value is `NIL` it stops, and returns `NIL`; the rest are not evaluated. If none is, it returns the last one's value. With no arguments it returns `T`.

`or` evaluates its arguments from the left. At the first whose value is not `NIL` it stops, and returns that value. If all are `NIL`, or there are none, it returns `NIL`.

Because they return a value and not just `T`, they also serve as "the first of these that exists" (`or`) and "this, provided that" (`and`).

EXAMPLE

```
hylang> (and 1 NIL (print "not reached"))
=> NIL
hylang> (or NIL 2 (print "not reached"))
=> 2
hylang> (list (and) (or))
=> {T NIL}
```

any?, all?

Functions: does a test pass for any item of a list, or for every item?

SYNTAX

```
(any? f l)
(all? f l)
```

ARGUMENTS

f

A function of one argument: the test.

l

A list.

RESULT

`any?` gives `T` if *f* returns a true value for at least one item, and `NIL` if not. `all?` gives `T` if *f* returns a true value for every item, and `NIL` if not. The result is always `T` or `NIL`, whatever *f* itself returned.

For an empty list `any?` is `NIL` (there is no item that passes) and `all?` is `T` (there is none that fails).

DESCRIPTION

The items are tested in order, and the testing stops as soon as the answer is known: at the first item that passes for `any?`, and at the first that fails for `all?`. *f* is not called for the items after it.

ERRORS

MESSAGE	CAUSE
`'any?' expects a function and a list`	*l* is not a list (and `'all?'` for `all?`)

SEE ALSO

`find`, `count`, `filter`, `elem?`.

EXAMPLE

```
hylang> (any? odd? {2 4 5})
=> T
hylang> (any? odd? {2 4})
=> NIL
hylang> (all? odd? {1 3})
=> T
hylang> (all? odd? {})
=> T
```

banks, bank-alloc, bank-free, bank-read, bank-write

Functions: the task's RAM banks: how many it has, taking some, giving them back, and reading and writing them.

SYNTAX

```
(banks)
(bank-alloc n)
(bank-free bank n)
(bank-read bank offset n)
(bank-write bank offset bytes)
```

ARGUMENTS

n

For `bank-alloc` and `bank-free`, a number of banks, 1 or more. For `bank-read`, a number of bytes.

bank

A bank's number: the one `bank-alloc` gave, or one of those after it in the same run.

offset

A position in the bank, 0 to 8,191.

bytes

A list of whole numbers from 0 to 255.

RESULT

`banks` gives the number of RAM banks the task has in all: 16 for each RAM module fitted, each bank 8K.

`bank-alloc` takes *n* banks that are next to one another and gives the number of the first: the others are that number plus 1, plus 2 and so on. The banks are the task's until it gives them back or ends.

`bank-free` gives *n* banks back, from *bank* on, and gives `NIL`.

`bank-read` gives a list of *n* bytes from the bank, starting at *offset*. `bank-write` stores the bytes in the bank from *offset* on, and gives `NIL`. The bytes read or written must all be inside the one bank: *offset* plus the count can't pass 8,192.

DESCRIPTION

A task has far more RAM in its banks than the 64K it can see at once (384K, with three modules). These functions let a hylang program keep data there, outside hylang's own memory: a large table, a picture, a file's contents. hylang reaches the bank only for the moment of the read or the write, so nothing a program keeps in a bank is disturbed by hylang, or disturbs it.

hylang keeps some of the task's banks for its own values. `bank-alloc` gives only banks that are free.

ERRORS

MESSAGE	CAUSE
`'bank-read' was given a number out of its range`	The offset is past the bank's end, or the bytes would run past it
`'bank-write' expects a list of bytes (0-255)`	*bytes* is not such a list: a string or a buffer must be made a list first, with `bytes`
n`: invalid argument`	`bank-alloc` of no banks; `bank-free` of banks that aren't taken
`'bank-alloc' was given a number out of its range`	More banks than a task can have

SEE ALSO

`free`, `buffer`, `seg-create`, `peek`.

EXAMPLE

The bank's number depends on what hylang has taken already:

```
hylang> (banks)
=> 48
hylang> (def {bk} (bank-alloc 2))
=> NIL
hylang> bk
=> 7
hylang> (bank-write bk 0 {1 2 3})
=> NIL
hylang> (bank-read bk 0 4)
=> {1 2 3 0}
hylang> (bank-free bk 2)
=> NIL
```

base

A function: the base that numbers are read and written in.

SYNTAX

```
(base)
(base b)
```

ARGUMENTS

b

A base, as a string: one of Chapter 18's base names, such as `"d"` (decimal), `"x"` (hex), `"b"` (binary) or `"o"` (octal), or a radix such as `"#16r"`.

RESULT

`(base)` gives the current base, as a string. `(base b)` sets it, and gives the base that was in force before, so that a program can put it back.

DESCRIPTION

The base setting belongs to this run of hylang, and starts as decimal. While it is something else, every number without a prefix of its own is *read* in that base, and every number is *written* in it. That includes the numbers in the lines typed afterward: in base `"x"`, `10` is sixteen, and in base `"b"` a `5` is not a number at all.

A number written with a prefix (`#xFF`, `#b101`) is read in its own base whatever the setting.

ERRORS

MESSAGE	CAUSE
`'base' expects a string: a base ("x", "#16r", "c" ...)`	*b* is not a string. `(base 16)` is this error: write `(base "x")`
`'base': Invalid base specifier zz`	The string names no base

SEE ALSO

`hex`, `bin`, `to-str`, `val`; sections 18.3 to 18.9.

EXAMPLE

```
hylang> (base "x")
=> "d"
hylang> (+ 9 1)
=> A
hylang> (+ 8 8)
=> 10
hylang> (base "d")
=> "x"
hylang> (base)
=> "d"
```

between?, ><?, not-between?

Functions of the library, written in hylang: is a value in a range?

SYNTAX

```
(between? lo hi x)
(><? lo hi x)
(not-between? lo hi x)
```

ARGUMENTS

lo, hi

The two ends of the range. They may be given in either order.

x

The value to test.

RESULT

`between?` gives `T` if *x* is at one of the ends or anywhere between them, and `NIL` if it is outside. `><?` is another name for it. `not-between?` gives the opposite, and has another name too, `!><?`.

DESCRIPTION

The comparison is <='s, so it works on anything that has an order: numbers, strings, characters.

Three more go through a list: `(all-between? lo hi l)` is `T` when every item of the list *l* is in the range, `(all-not-between? lo hi l)` when none is, and `(not-all-between? lo hi l)` when at least one is outside.

SEE ALSO

<, `and`, `min`, `max`.

EXAMPLE

```
hylang> (between? 1 10 5)
=> T
hylang> (between? 1 10 10)
=> T
hylang> (between? 10 1 5)
=> T
hylang> (between? 1 10 11)
=> NIL
hylang> (between? "a" "m" "c")
=> T
hylang> (all-between? 0 9 {3 5 7})
=> T
```

bind, mount, unmount, ns

Functions: change, and list, the task's namespace.

SYNTAX

```
(bind new old [:before | :after] [:create])
(mount dev old [spec] [:before | :after] [:create])
(unmount old [new])
(ns)
```

ARGUMENTS

new
The directory, or file, to make appear at *old*.

old
The place in the tree that is changed. It must be there already.

dev
A device's name, such as `"#f"`, with *spec* choosing which of the device's trees: `"x"` is the ROM disk.

`:before, :after`
Make a union: *old* shows what it showed before and *new*'s contents too, with *new* looked in first (`:before`) or last (`:after`). With neither, *new* replaces what *old* showed.

`:create`
In a union, a file created at *old* is made in this member.

RESULT

`bind`, `mount` and `unmount` give `NIL`.

`unmount` undoes what was bound or mounted at *old*: all of it, or with *new*, only that member of a union.

`ns` gives the namespace as a list of hashes, one for each bind and mount, in the kernel's order: `:old`, the place; `:new`, what is there, as a device's path such as `"#fx/lib"`; and `:create`, `T`, on a member that takes creates.

DESCRIPTION

These are the system's `BIND`, `MOUNT` and `UNMOUNT` calls, and work as rc's `bind`, `mount` and `unmount` commands do (Chapter 9 of the manual explains namespaces, and Chapter 2 of this book has the commands' entries).

The namespace is this task's. Programs that hylang starts afterward see the same tree; the shell that started hylang does not see the changes, and they are gone when hylang ends. `(newns)` puts the default namespace back.

In a union with no member that takes creates, a new file is made in the directory that was there first.

ERRORS

MESSAGE	CAUSE
path`: not found`	*new* or *old* isn't there; or `unmount` of a place where nothing is bound
`'bind' expects :before, :after or :create`	Another atom was given
dev`: no such device`	`mount` of a device that isn't there

SEE ALSO

`newns`, `ls`, `cd`; the commands `bind`, `mount`, `unmount` and `ns` (Chapter 2 of this book).

EXAMPLE

```
hylang> (mkdir "/ram/a")
=> NIL
hylang> (mkdir "/ram/b")
=> NIL
hylang> (write-file "/ram/a/one" "1")
=> NIL
hylang> (write-file "/ram/b/two" "2")
=> NIL
hylang> (bind "/ram/a" "/ram/b" :after)
=> NIL
hylang> (ls "/ram/b")
=> {"one" "two"}
hylang> (unmount "/ram/b")
=> NIL
hylang> (ls "/ram/b")
=> {"two"}
hylang> (bind "/ram/a" "/ram/b")
=> NIL
hylang> (ls "/ram/b")
=> {"one"}
```

bit-and, bit-or, bit-xor, bit-not

Functions: the bitwise operations.

SYNTAX

```
(bit-and n ...)
(bit-or n ...)
(bit-xor n ...)
(bit-not n)
```

ARGUMENTS

n

An integer. `bit-and`, `bit-or` and `bit-xor` take any number of them.

RESULT

An integer. Each bit of the result is worked out from the same bit of the arguments: `bit-and` gives a 1 where all have a 1, `bit-or` where any has, and `bit-xor` where an odd number have. `bit-not` turns every bit over.

DESCRIPTION

Integers have no fixed width, so a negative number is treated as two's complement with its sign bit going on for ever: -1 is all ones. That is why `(bit-not 0)` is -1 and `(bit-not 5)` is -6, and why `(bit-and -1 255)` is 255. To keep a result to one byte or one word, `bit-and` it with 255 or 65535.

ERRORS

MESSAGE	CAUSE
`'bit-and''s argument must be an integer`	An argument is a fraction, a decimal, or not a number (each function with its own name)

SEE ALSO

`shl`, `shr`, `bit?`, `lo`, `hi`, `hex`, `bin`; section 18.10.

EXAMPLE

```
hylang> (bit-and 12 10)
=> 8
hylang> (bit-or 12 10)
=> 14
hylang> (bit-xor 12 10)
=> 6
hylang> (bit-not 5)
=> -6
hylang> (bin (bit-and #b1100 #b1010))
=> "1000"
```

buffer

A function: makes a buffer, a block of bytes that can be changed in place.

SYNTAX

```
(buffer n [fill])
(buffer s)
(buffer l)
(buffer b)
```

ARGUMENTS

n

The buffer's size in bytes: a whole number from 0 to 8,184.

fill

The value every byte starts with: 0 to 255, or a character. Without it, 0.

s

A string: the buffer holds its bytes.

l

A list of bytes, each a whole number from 0 to 255 or a character.

b

Another buffer: the result is a copy of it.

RESULT

A buffer. It prints as `<buffer>` and its bytes.

DESCRIPTION

A **buffer** is a row of bytes of a fixed length. Unlike a string or a list, it is changed where it is: `buffer-put`, `buffer-fill`, `buffer-copy` and `read-buffer` alter the buffer they are given, and every name bound to that buffer sees the change. That makes it the value to use for binary data, for a file read a block at a time, and for a table that is updated many times, where making a new list for each change would be slow.

A buffer has a length (`len`), is equal to another buffer with the same bytes (`==`), and can be called with an index, `(b i)`, to read a byte. `bytes` gives its bytes as a list and `from-bytes` as a string. It is not a list: `map`, `nth` and the other list functions do not take one.

ERRORS

MESSAGE	CAUSE
`'buffer': -1 bytes is outside 0 to 8184`	*n* is negative, or more than a buffer can hold
`'buffer': a size must be a whole number, not 1.5`	The argument is not a size, a string, a list or a buffer
`'buffer': 256 isn't a byte (0-255)`	*fill*, or an item of the list, is not a byte

SEE ALSO

`buffer-get`, `buffer-fill`, `read-buffer`, `write-bytes`, `bytes`, `from-bytes`.

EXAMPLE

```
hylang> (buffer 4)
=> <buffer>{0 0 0 0}
hylang> (buffer 4 255)
=> <buffer>{255 255 255 255}
hylang> (buffer "Hi")
=> <buffer>{72 105}
hylang> (buffer {1 2 3})
=> <buffer>{1 2 3}
hylang> (len (buffer 100))
=> 100
hylang> (from-bytes (buffer {72 105}))
=> "Hi"
```

buffer-cmp

A function of the library, written in hylang: compares part of a buffer with a string, or with another buffer.

SYNTAX

```
(buffer-cmp b at x [i [n]])
```

ARGUMENTS

b

A buffer.

at

Where in *b* the comparison starts.

x

What to compare with: a string, or a buffer.

i

Where in *x* to start. Without it, 0.

n

How many bytes of *x* to compare. Without it, all of them from *i* on.

RESULT

-1, 0 or 1, as `cmp` gives for two strings: is the part of *b* before, the same as, or after the part of *x*?

DESCRIPTION

At most as many bytes of *b* are looked at as *x* has. So the result is 0 when *b*, from *at*, *begins* with *x*: this is the test for "is this text here in the buffer?", such as a file's signature at its start. If *b* ends before *x* does, and they agree as far as *b* goes, the result is -1.

ERRORS

MESSAGE	CAUSE
`'buffer-cmp' expects a buffer, not 5`	*b* is not a buffer
`'buffer-cmp' expects a buffer or a string, not 5`	*x* is neither
`'buffer-cmp': index 5 is outside the buffer`	*at* or *i* is negative, or past the end

SEE ALSO

`cmp`, `==`, `index-of`.

EXAMPLE

```
hylang> (def {b} (buffer "HYX2 and more"))
=> NIL
hylang> (buffer-cmp b 0 "HYX2")
=> 0
hylang> (buffer-cmp b 0 "HYX3")
=> -1
hylang> (buffer-cmp b 5 "and")
=> 0
```

buffer-fill, buffer-copy

Functions: set many bytes of a buffer to one value; and copy bytes from one buffer to another.

SYNTAX

```
(buffer-fill b byte [i [n]])
(buffer-copy to at from [i [n]])
```

ARGUMENTS

b

The buffer to fill.

byte

The value to fill with: 0 to 255, or a character.

to, at

The buffer to copy into, and the position in it where the first byte goes.

from

The buffer to copy from.

i

The position of the first byte to fill, or to copy from *from*. Without it, 0.

n

How many bytes. Without it, all the bytes from *i* to the end.

RESULT

`NIL`. The buffer *b*, or *to*, is changed in place.

DESCRIPTION

A count that goes past the end of a buffer is not an error: the fill stops at the end of *b*, and the copy stops when *from* runs out or *to* is full, whichever comes first.

to and *from* may be the same buffer, and the two parts may overlap: the bytes are copied as if they were all read first.

Both arguments of `buffer-copy` must be buffers. To put a string's or a list's bytes in a buffer, make a buffer of them first: `(buffer-copy b 0 (buffer "xy"))`.

ERRORS

MESSAGE	CAUSE
`'buffer-fill': index 5 is outside the buffer`	*i*, or *at*, is negative or past the end
`'buffer-fill': 300 isn't a byte (0-255)`	*byte* is not a byte
`'buffer-copy' expects a buffer, not xy`	*to* or *from* is not a buffer

SEE ALSO

`buffer`, `buffer-put`, `read-buffer`.

EXAMPLE

```
hylang> (def {b} (buffer 6))
=> NIL
hylang> (buffer-fill b 7)
=> NIL
hylang> (buffer-fill b 1 4)
=> NIL
hylang> b
=> <buffer>{7 7 7 7 1 1}
hylang> (buffer-copy b 1 (buffer "abcdef") 2 3)
=> NIL
hylang> b
=> <buffer>{7 99 100 101 1 1}
```

buffer-get, buffer-put

Functions: read one byte of a buffer; and set one.

SYNTAX

```
(buffer-get b i)
(b i)
(buffer-put b i byte)
```

ARGUMENTS

b

A buffer.

i

The byte's position, from 0 to one less than the buffer's length.

byte

The new value: a whole number from 0 to 255, or a character, whose code is stored.

RESULT

`buffer-get` gives the byte at *i*, a number from 0 to 255. Calling the buffer itself, `(b i)`, does the same.

`buffer-put` stores *byte* at *i*, in the buffer itself, and gives the byte that was there before.

ERRORS

MESSAGE	CAUSE
`'buffer-get': index 4 is outside the buffer`	*i* is negative, or the buffer's length or more (and `'buffer-put'`)
`'buffer-get': an index must be a whole number, not 1.5`	*i* is not a whole number
`'buffer-put': 256 isn't a byte (0-255)`	*byte* is not a byte
`'buffer-get' expects a buffer, not {1}`	*b* is not a buffer

SEE ALSO

`buffer`, `buffer-fill`, `peek`, `poke`.

EXAMPLE

```
hylang> (def {b} (buffer 4))
=> NIL
hylang> (buffer-put b 0 65)
=> 0
hylang> (buffer-put b 0 66)
=> 65
hylang> (b 0)
=> 66
hylang> (buffer-get b 1)
=> 0
hylang> (buffer-get b 4)
=> Error: 'buffer-get': index 4 is outside the buffer
```

buffer?

A function: is a value a buffer?

SYNTAX

```
(buffer? x)
```

ARGUMENTS

x

Any value.

RESULT

`T` if *x* is a buffer, and `NIL` if it is anything else: a list of bytes is not a buffer, and nor is a string.

SEE ALSO

`buffer`, `type-of`.

EXAMPLE

```
hylang> (buffer? (buffer 4))
=> T
hylang> (buffer? {1 2})
=> NIL
```

bytes, from-bytes

Functions: a string's bytes as a list of numbers; and a string from such a list.

SYNTAX

```
(bytes s)
(from-bytes l)
```

ARGUMENTS

s

A string.

l

A list of whole numbers from 0 to 255.

RESULT

`bytes` gives a list of the character codes of *s*, one number for each byte, in order: `NIL` for the empty string. `from-bytes` gives the string made of those bytes: the empty string for an empty list.

ERRORS

MESSAGE	CAUSE
`'bytes' expects a String`	*s* is not a string
`256 isn't a byte`	An item of *l* is not a whole number from 0 to 255

SEE ALSO

`char-code`, `code-char`, `str-chars`, `buffer`.

EXAMPLE

```
hylang> (bytes "AB")
=> {65 66}
hylang> (from-bytes {72 105})
=> "Hi"
hylang> (from-bytes (map 1+ (bytes "HAL")))
=> "IBM"
```

cd, cwd

Functions: set, and ask, the current directory.

SYNTAX

```
(cd [path])
(cwd)
```

ARGUMENTS

path

A directory. Without it, `/`.

RESULT

`cd` gives `NIL`, and makes *path* the current directory: the one that names not starting with `/` are looked for in. `cwd` gives the current directory's full path, as a string.

DESCRIPTION

The current directory is this task's. Programs that hylang starts afterward begin in it; the shell that started hylang is not affected.

ERRORS

MESSAGE	CAUSE
path`: not found`	There is no such directory
`'cd' expects a path`	*path* is not a string

EXAMPLE

```
hylang> (cd "/rom/lib")
=> NIL
hylang> (cwd)
=> "/rom/lib"
hylang> (exists? "shell")
=> T
hylang> (cd)
=> NIL
hylang> (cwd)
=> "/"
```

char-code, code-char

Functions: a character's code; and the character with a code.

SYNTAX

```
(char-code c)
(code-char n)
```

ARGUMENTS

c

A character, or a string of exactly one character.

n

A whole number from 0 to 255.

RESULT

`char-code` gives the character's code, 0 to 255 (Appendix E of the manual lists them). `code-char` gives the character. The prompt shows a character with a backslash, and shows the ones that can't be printed by name: `\space`, `\lf` for code 10, `\null` for 0.

ERRORS

MESSAGE	CAUSE
`'char-code' expects one character`	*c* is a string that is empty, or longer than one character
`'char-code' expects a String`	*c* is neither a string nor a character
`'code-char': 256 isn't a character's code (0-255)`	*n* is out of range
`'code-char': a number must be a whole number, not 1.5`	*n* is not a whole number

SEE ALSO

`bytes`, `from-bytes`, `alpha?`.

EXAMPLE

```
hylang> (char-code \a)
=> 97
hylang> (char-code "A")
=> 65
hylang> (code-char 65)
=> \A
hylang> (code-char 32)
=> \space
```

clock library

A library, loaded with `(use "clock")`: setting the clock, and the clock chip.

FUNCTIONS

FORM	WHAT IT DOES
`(set-date text)`	Sets the date and time, from a string in the form `"2026-10-13 09:41:00"`. With a clock chip fitted, the chip is set too. `NIL`
`(rtc)`	What the system found for a clock chip as it started: a list holding `:running`, `:stopped` (the chip is there and needs setting) or `:none`, and `:battery-low` as well if its battery is running down

DESCRIPTION

Reading the clock needs no library: `time`, `date` and `date-parts` are always there. This one writes `/dev/time` and reads `/dev/rtc` (section 17.5).

ERRORS

MESSAGE	CAUSE
`invalid argument`	The text is not a date and time in that form

SEE ALSO

`time`, `date`, `seconds-of`; section 17.5.

EXAMPLE

```
hylang> (use "clock")
=> NIL
hylang> (rtc)
=> {:none}
hylang> (set-date "2026-10-13 09:41:00")
=> NIL
hylang> (substring (date) 0 16)
=> "2026-10-13 09:41"
hylang> (set-date "nonsense")
=> Error: invalid argument
```

close

A function: closes a stream.

SYNTAX

```
(close s)
```

ARGUMENTS

s

A stream.

RESULT

`NIL`. The file is closed, and what was written to it is safely in it.

DESCRIPTION

Closing a stream that is already closed does nothing, and is not an error. Reading or writing a closed stream is one: `Cannot read from this stream`, `Cannot write this stream`.

ERRORS

MESSAGE	CAUSE
`'close' expects a stream`	*s* is not a stream

SEE ALSO

`open`.

EXAMPLE

```
hylang> (def {s} (open "/rom/lib/shell"))
=> NIL
hylang> (close s)
=> NIL
hylang> (read-line s)
=> Error: Cannot read from this stream
```

complex

A function: a complex number from its two parts.

SYNTAX

```
(complex re im)
```

ARGUMENTS

re

The real part: a real number.

im

The imaginary part: a real number.

RESULT

The number *re* + *im*i. Each part keeps its own kind, and the two need not match. If *im* is 0 the result is the real number *re*.

ERRORS

MESSAGE	CAUSE
`One or more parameters passed to 'complex' is not a Number`	A part is not a number

EXAMPLE

```
hylang> (complex 1 2)
=> 1+2i
hylang> (complex 1/2 0.5)
=> 1/2+0.5i
hylang> (complex 1 0)
=> 1
```

cond, case

Functions of the library, written in hylang: choose one of several results.

SYNTAX

```
(cond {test value} ...)
(case x {match result} ...)
```

ARGUMENTS

{test value}

A Q-expression of two expressions. Neither is evaluated until `cond` reaches it.

x

A value to look for.

{match result}

A Q-expression of two expressions.

RESULT

`cond` evaluates each *test* in turn, and at the first that is true it evaluates that pair's *value* and gives it. No later pair is looked at. A last pair with the test `T` is the "otherwise".

`case` evaluates each *match* in turn, and at the first that equals *x*, as == compares, it evaluates that pair's *result* and gives it.

DESCRIPTION

Both are a chain of `if`s written flat. For a choice on the sign of a number, the special form `<=>` is shorter.

Neither has a silent way out: if no test is true, or nothing matches, the result is an error. Give `cond` a last `{T ...}`. `case` has no "anything else" pair, so check the value first, or catch the error with `try`.

ERRORS

MESSAGE	CAUSE
`Selection not found`	`cond`: no test was true
`Case not found`	`case`: nothing equalled *x*

SEE ALSO

`if`, `<=>`, `and`, `or`, `try`.

EXAMPLE

```
hylang> (def {n} 5)
=> NIL
hylang> (cond {(< n 0) "negative"} {(== n 0) "zero"} {T "positive"})
=> "positive"
hylang> (case n {1 "one"} {5 "five"} {9 "nine"})
=> "five"
hylang> (case n {1 "one"})
=> Error: Case not found
hylang> (try (case n {1 "one"}) "many")
=> "many"
```

cons

A function: a list with one more item at its front.

SYNTAX

```
(cons x l)
```

ARGUMENTS

x

Any value: the new first item.

l

A list. `NIL`, the empty list, is one.

RESULT

A list whose first item is *x* and whose remaining items are *l*'s. *l* itself is not changed.

DESCRIPTION

x goes in as one item, whatever it is: a list given as *x* becomes a list inside the result. To put two lists end to end, use `join`.

ERRORS

MESSAGE	CAUSE
`'cons' expects a list second`	*l* is not a list

SEE ALSO

`join`, `list`.

EXAMPLE

```
hylang> (cons 1 {2 3})
=> {1 2 3}
hylang> (cons {1} {2 3})
=> {{1} 2 3}
hylang> (cons 1 {})
=> {1}
hylang> (cons 1 2)
=> Error: 'cons' expects a list second
```

cons library

A library, loaded with `(use "cons")`: the console's windows.

FUNCTIONS

FORM	WHAT IT DOES
`(window)`	The number of the window hylang is running in
`(window-size)`	That window's size: a list, the columns and the rows
`(windows)`	The numbers of all the windows there are: a list
`(shown-window)`	The number of the window that is shown now
`(show-window n)`	Shows window *n*. `NIL`
`(new-window [line])`	Makes a window and shows it, running the command line in it; with none, a shell. It is the `new-window` command, and gives its exit code
`(new-group [line])`	The same, in a note group of its own (`new-window -g`)
`(window-label [s])`	With a string, sets this window's title (`""` gives it its program's name again), `NIL`. With none, gives the title
`(window-status s)`	Sets the text of this window's status line. `NIL`
`(window-ctl s)`	Writes any line to the window's control file, `/dev/wctl`. `NIL`
`(snarf)`, `(snarf! s)`	The console's cut buffer: its text; and set it
`(raw-on)`, `(raw-off)`	Turns raw keys on: each key comes as it is pressed, and is not shown. And off again
`(beep)`	Rings the bell

DESCRIPTION

The library works through the console's files in `/dev` (Chapters 10 and 13). The cut buffer is shared by all the windows: what one program puts there, CTRL-] Y pastes into another.

`raw-on` lasts until `raw-off`: while it is on, the prompt's own line editing is off too, so a program must turn it off before it ends. For single keys, `key` is simpler, and looks after this itself.

SEE ALSO

`key`, the `screen` library; the command `new-window` (Chapter 2 of this book); Chapter 10.

EXAMPLE

```
hylang> (use "cons")
=> NIL
hylang> (window)
=> 0
hylang> (window-size)
=> {80 24}
hylang> (windows)
=> {0}
hylang> (snarf! "cut text")
=> NIL
hylang> (snarf)
=> "cut text"
```

def

A special form: binds names in the global scope.

SYNTAX

```
(def {name ...} value ...)
(def name value)
```

ARGUMENTS

name

Symbols, in a Q-expression so that they are not evaluated. One name can be written without the braces.

value

Expressions, as many as there are names. Each is evaluated.

RESULT

`NIL`.

DESCRIPTION

Each name is bound to the value in the same position. The binding is made in the global scope wherever `def` is used: inside a function, inside a `let`, it still defines a global name. A name that is bound already is bound again to the new value.

To bind a name in the scope the code is in, use `set`. To change a name where it already is, use `set!`.

ERRORS

MESSAGE	CAUSE
`'def' passed too many arguments or symbols. Expected` *n*`, got` *m*	The names and the values don't pair up

SEE ALSO

`set`, `set!`, `let`, `fun`.

EXAMPLE

```
hylang> (def {a b} 1 2)
=> NIL
hylang> (fun {inside} {def {made-inside} 5})
=> NIL
hylang> (inside)
=> NIL
hylang> (list a b made-inside)
=> {1 2 5}
hylang> (def {a} 1 2)
=> Error: 'def' passed too many arguments or symbols.  Expected 1, got 2
```

defined?, expr?

Special forms: tests on a name and on an expression.

SYNTAX

```
(defined? {name})
(expr? x)
```

DESCRIPTION

`defined?` returns `T` if the name is bound, in the current scope or any that encloses it, and `NIL` if it is not. The name is written in braces, so that it is not looked up, which would be an error for exactly the names the test is for.

`expr?` returns `T` if its argument is something that can be evaluated.

dev library

A library, loaded with `(use "dev")`: the few functions that the device libraries are built from.

FUNCTIONS

FORM	WHAT IT DOES
`(dev.read path [n])`	One read of the file: a string of at most *n* bytes, or 256
`(dev.lines path)`	One read, as a list of its lines, without the empty ones
`(dev.num path)`	One read, as a number: the text is trimmed and given to `val`
`(dev.write path data)`	One write to the file, of a string or a list of bytes. `NIL`
`(dev.with path mode f)`	Opens the file (*mode* 0 to read, 1 to write, 2 for both), calls `(f fd)` with its file descriptor, closes the file, and gives *f*'s result
`(dev.bytes s)`	A string's bytes, as a list
`(dev.hex n w)`	A number in hex, *w* digits, in small letters: the way devices are named in their directories

DESCRIPTION

A device's file is not like a disk's: each read is one request to the driver (a pin's level, a transaction's bytes, a control file's text), and each write is one command. `read-file` may make several reads and `write-file` several writes; these make exactly one, which is what a device wants. `dev.with` is for anything longer, such as a seek and then a read, with the `sys-` functions.

Whatever goes wrong, the file is closed before the error is passed on.

These are the pieces to use for a device that has no library of its own, and the libraries themselves, in `/lib/hylang`, are short examples of their use.

SEE ALSO

`sys`, `read-file`, `write-file`; Chapter 12.

EXAMPLE

```
hylang> (use "dev")
=> NIL
hylang> (dev.read "/dev/rtc")
=> "none\n"
hylang> (dev.lines "/dev/serctl")
=> {"b9600" "x1" "pc 0 req, 0 cut short, 0 unanswered, 0 damaged, 0 nak, 0 gone" "line 0
overruns, 0 framing errors"}
hylang> (dev.write "/dev/gpio/ctl" "out 2")
=> NIL
hylang> (dev.num "/dev/gpio/2")
=> 1
hylang> (dev.write "/dev/gpio/ctl" "in 2")
=> NIL
```

dice library

A library, loaded with `(use "dice")`: dice for games.

FUNCTIONS

FORM	WHAT IT DOES
`(die sides)`	One throw of a die with that many sides: 1 to *sides*
`(die+ sides mod)`	A throw, with *mod* added
`(d3)`, `(d4)`, `(d6)`, `(d8)`, `(d10)`, `(d12)`, `(d20)`, `(d24)`, `(d30)`, `(d60)`, `(d100)`	A throw of the die named
`(d6+ mod)`, and the same for each of those	A throw of it, with *mod* added
`(dice n sides)`, `(dice+ n sides mod)`	The total of *n* throws; and with *mod* added
`(roll n d)`	A list of *n* throws: *d* is a Q-expression, run for each, as `repeat`'s is. `(roll 4 {d6})`
`(sumrolls l)`	Throws every die that the list describes, and gives the total. An item is a number of sides, or a list: `{sides}`, `{n sides}` or `{n sides mod}`
`(best-of-rolls a b d)`, `(worst-of-rolls a b d)`	The best, or the worst, *a* of *b* throws of *d*: a list
`(oe-die sides)`, `(oe-dice n sides)`, and each with `+` and a *mod*	Open-ended throws: a throw of the highest face is thrown again and added
`(special-die sides cases)`	A throw looked up with `case`: *cases* is a list of `{face result}` pairs
`(dnd-attr-rolls [n])`	Scores for a role-playing character: a list of *n* sets (one, if *n* isn't given), each set the best six of eight scores, greatest first, and each score the total of the best three of four six-sided dice

DESCRIPTION

Every throw comes from `random`, so no two runs give the same numbers.

SEE ALSO

`random`, `repeat`, `best-of`, `sum`.

EXAMPLE

The numbers differ from run to run:

```
hylang> (use "dice")
=> NIL
hylang> (d6)
=> 4
hylang> (dice 3 6)
=> 11
hylang> (roll 4 {d6})
=> {2 6 3 5}
hylang> (sum (best-of-rolls 3 4 {d6}))
=> 14
hylang> (sumrolls {6 {2 8} {1 4 3}})
=> 17
hylang> (dnd-attr-rolls)
=> {{18 16 16 14 13 12}}
```

digits

A function: the precision of the math functions.

SYNTAX

```
(digits)
(digits n)
```

ARGUMENTS

n

A whole number from 1 to 100.

RESULT

`(digits)` gives the current precision. `(digits n)` sets it, and gives the one that was in force before.

DESCRIPTION

`sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `atan`, `pi` and `pow` give an exact answer when there is one, and otherwise a decimal of this many significant digits. The setting starts as 12. It has no effect on `+`, `-`, `*` and `/`, which are always exact.

ERRORS

MESSAGE	CAUSE
`'digits' takes a whole number from 1 to 100`	*n* is out of range, or not a whole number

EXAMPLE

```
hylang> (sqrt 2)
=> 1.41421356237
hylang> (digits 30)
=> 12
hylang> (sqrt 2)
=> 1.41421356237309504880168872421
hylang> (digits 12)
=> 30
```

disk library

A library, loaded with `(use "disk")`: the disks.

FUNCTIONS

FORM	WHAT IT DOES
`(disks)`	The disks that have been started: a list of hashes, each with `:disk`, its letter, a string; `:kind`, an atom such as `:sdhc`, `:rom`, `:ram` or `:sram`; `:blocks`, its size in 512-byte blocks; and `:label`, if it holds a file system
`(df)`	Each disk's room: a list of hashes with `:disk`, `:label`, and `:free` and `:size` in kilobytes
`(cards)`	The SPI devices that have an SD card: a list of numbers. Asking starts each card that is there
`(disk-start d)`	Starts a card again, as is needed after one is changed. `NIL`
`(disk-stop d)`	Stops a RAM disk, and gives its memory back. `NIL`

ARGUMENTS

d

A disk: its letter, as a string (`"0"` to `"f"` for the card on that SPI device, `"x"` the ROM disk, `"r"` the RAM disk, `"s"` the shared RAM disk); or, for a card, its SPI device's number.

DESCRIPTION

The library works through the files of `/dev/sd` (Chapter 14). `df` gives a program the numbers that the `df` command prints.

ERRORS

MESSAGE	CAUSE
path`: not found`	There is no such disk

SEE ALSO

Chapter 14; the commands `df`, `fsck`, `mkfs` and `label` (Chapter 2 of this book).

EXAMPLE

```
hylang> (use "disk")
=> NIL
hylang> (map (fn {d} {d :disk}) (disks))
=> {"0" "r" "s" "x"}
hylang> (map (fn {d} {list (d :label) (d :size)}) (df))
=> {{"HYDRA-16" 16380} {"RAM" 255} {"SRAM" 511} {"ROM" 623}}
hylang> (cards)
=> {0}
```

div, mod, divmod, fdiv

Functions of the library, written in hylang: division with a whole quotient and a remainder; and division to a decimal.

SYNTAX

```
(div a b)
(mod a b)
(divmod a b)
(fdiv a b [places])
```

ARGUMENTS

a, b

Real numbers.

places

For `fdiv`: how many digits to keep after the point. Without it, 10.

RESULT

`div` gives the quotient rounded down to a whole number: the `floor` of *a* / *b*. `mod` gives what is left over: *a* - *b* × `(div a b)`. `divmod` gives both, as a list of two.

`fdiv` gives *a* / *b* as a decimal of at most *places* places, cut off, not rounded: it is `to-fixed` of the quotient.

`%` is another name for `mod`.

DESCRIPTION

Because the quotient is rounded *down*, not toward zero, the remainder always has the sign of *b*: `(mod -7 2)` is 1, and `(mod 7 -2)` is -1. With a positive *b*, `mod` is therefore always from 0 up to, but not including, *b*, which is what is wanted for wrapping a position round a table or an angle round a circle.

The numbers need not be whole: `(mod 7.5 2)` is 1.5.

ERRORS

MESSAGE	CAUSE
`Cannot perform 'div' on non-numbers`	An argument is not a number (each function with its own name)
`Division by zero.`	*b* is 0

SEE ALSO

`/`, `floor`, `to-fixed`, `odd?`.

EXAMPLE

```
hylang> (div 7 2)
=> 3
hylang> (mod 7 2)
=> 1
hylang> (div -7 2)
=> -4
hylang> (mod -7 2)
=> 1
hylang> (divmod 7 2)
=> {3 1}
hylang> (fdiv 1 3 4)
=> 0.3333
```

do

A special form: evaluates expressions in order.

SYNTAX

```
(do x ...)
```

DESCRIPTION

Each expression is evaluated in turn, and the value of the last is returned; the others are evaluated for what they do. With no expressions the result is `NIL`. It is the way to put several expressions where one is expected, such as a branch of an `if`.

dotimes

A special form: repeats a body a number of times.

SYNTAX

```
(dotimes {i n} body ...)
(dotimes n f)
```

DESCRIPTION

In the first form, *n* is evaluated once, and the body is evaluated *n* times with the symbol *i* bound to 0, then 1, up to *n* − 1. In the second form, the function *f* is called *n* times with those numbers as its argument.

The result is `NIL`.

EXAMPLE

```
hylang> (dotimes {k 3} (print k))
0
1
2
=> NIL
```

each

A special form: does something with every item of a list, or every character of a string.

SYNTAX

```
(each {x list} body ...)
(each f list)
```

DESCRIPTION

In the first form, *list* is evaluated once, and the body is evaluated once for each of its items, in order, with the symbol *x* bound to the item. In the second form, the function *f* is called with each item.

A string in place of the list gives its characters, one at a time.

The result is `NIL`: `each` is for what the body does. To collect results, use `map`.

EXAMPLE

```
hylang> (each {x {1 2 3}} (print (* x x)))
1
4
9
=> NIL
hylang> (each print "ab")
a
b
=> NIL
```

elem?, in?

Functions: is a value one of a list's items?

SYNTAX

```
(elem? x l)
(in? x l)
```

ARGUMENTS

x

Any value.

l

A list.

RESULT

`T` if an item of *l* is equal to *x*, and `NIL` if none is. The two names are the same function.

DESCRIPTION

Items are compared as == compares: numbers by value, so 1.0 is found in a list holding 1; strings exactly, capitals and small letters being different; lists item by item.

Only the list's own items are looked at, not the items of lists inside it.

ERRORS

MESSAGE	CAUSE
`'elem?' expects a list`	*l* is not a list. To look for a character or a piece of text in a string, use `index-of` or `contains?`

SEE ALSO

`find`, `index-of`, `any?`.

EXAMPLE

```
hylang> (elem? 2 {1 2 3})
=> T
hylang> (elem? 5 {1 2 3})
=> NIL
hylang> (elem? 1.0 {1 2})
=> T
hylang> (elem? {1} {{1} {2}})
=> T
hylang> (elem? "A" {"a"})
=> NIL
```

env, setenv, unsetenv

Functions: read, set and remove an environment variable.

SYNTAX

```
(env [name])
(setenv name value)
(unsetenv name)
$name
```

ARGUMENTS

name

The variable's name, a string.

value

A string; a list of strings, for a variable of several words; or a number, which is turned to text.

RESULT

`(env name)` gives the variable's value: a string, or a list of strings if it holds several words, as rc's `$path` does. If there is no such variable, `NIL`. `$name` is another way to write `(env "name")`.

`(env)`, with no name, gives a hash of all the variables, with their names, as strings, for keys.

`setenv` and `unsetenv` give `NIL`. Removing a variable that isn't there is not an error.

DESCRIPTION

The environment is this task's own set of variables (section 17.7). hylang starts with a copy of the environment of the shell that started it, and the programs it starts get a copy of its own: so `setenv` is how a setting is passed down to a program run with `run` or `sh`. Nothing set here goes back up to the shell.

Environment values are text. A number put in comes back as a string: use `val` to make it a number again.

ERRORS

MESSAGE	CAUSE
`'setenv' expects a path`	*name* is not a string

SEE ALSO

`sh`, `run`, `val`.

EXAMPLE

```
hylang> (env "path")
=> {"." "/bin"}
hylang> (setenv "colour" "red")
=> NIL
hylang> $colour
=> "red"
hylang> (sh "echo my colour is $colour")
my colour is red
=> 0
hylang> (unsetenv "colour")
=> NIL
hylang> (env "colour")
=> NIL
```

error

A function: makes an error.

SYNTAX

```
(error message [code])
```

ARGUMENTS

message

The error's message: a string, or any value, which is turned to text as `print` shows it.

code

An atom that names the kind of error, for the program that catches it. Without it the error has no code.

RESULT

An error value. It prints as `Error:` and the message.

DESCRIPTION

An error is a value, but one that most functions will not accept: a function given an error as an argument does not run, and the error is its result. So the error passes outward through every enclosing call until it is caught, or reaches the prompt, which prints it. `error` is how a program reports that it cannot go on; `try` is how another part of the program catches the report.

The few forms that do accept an error are `try`; `def`, `set` and `set!`, so that an error can be kept in a variable; `error?`, `type-of`, `error-message` and `error-code`; and `print`.

The errors that the system's calls give have codes too. Each is the name of the system's error code (Chapter 3 of this book lists them) in small letters, without its `E_`: `:noent` for `E_NOENT`, a file that is not found.

ERRORS

MESSAGE	CAUSE
`An error's code must be an atom`	*code* is not an atom

SEE ALSO

`try`, `error-message`, `error?`.

EXAMPLE

```
hylang> (error "out of cheese")
=> Error: out of cheese
hylang> (+ 1 (error "inside"))
=> Error: inside
hylang> (fun {half n} {if (odd? n) (error (format "{} is odd" n) :odd) (/ n 2)})
=> NIL
hylang> (half 10)
=> 5
hylang> (half 7)
=> Error: 7 is odd
hylang> (try (half 7) (list &err &code))
=> {"7 is odd" :odd}
```

error-message, error-code

Functions: the message and the code of an error.

SYNTAX

```
(error-message e)
(error-code e)
```

ARGUMENTS

e

An error, usually one kept in a variable, or the value of an expression that may fail.

RESULT

`error-message` gives the message, as a string, without the `Error:` that is printed before it. `error-code` gives the code, an atom, or `NIL` for an error that has none. For a value that is not an error, both give `NIL`.

DESCRIPTION

These two are given errors without passing them on, so `(error-code (open path))` is a way to ask why a call failed without `try`. For an error from the system the code is the system's name for it: `:noent` (not found), `:exist` (already exists), `:isdir` (is a directory), `:rofs` (read-only), and so on.

SEE ALSO

`error`, `try`, `error?`, `errstr`.

EXAMPLE

```
hylang> (def {e} (error "out of cheese" :cheese))
=> NIL
hylang> (error-message e)
=> "out of cheese"
hylang> (error-code e)
=> :cheese
hylang> (error-code (open "/nothere"))
=> :noent
hylang> (error-message (open "/nothere"))
=> "/nothere: not found"
hylang> (error-code 5)
=> NIL
```

eval

A special form: evaluates a list as code.

SYNTAX

```
(eval x)
```

DESCRIPTION

The argument is evaluated in the ordinary way, and if the result is a list, that list is then evaluated as an S-expression: its first item as the function and the rest as its arguments. A Q-expression is thus "code not yet run", and `eval` runs it. A result that is not a list is returned as it is.

EXAMPLE

```
hylang> (def {later} {+ 1 2})
=> NIL
hylang> (eval later)
=> 3
hylang> (eval (list * 6 7))
=> 42
```

exists?, dir?, file?

Functions: is a path there, and is it a directory or a file?

SYNTAX

```
(exists? path)
(dir? path)
(file? path)
```

ARGUMENTS

path

A string.

RESULT

`T` or `NIL`. `exists?` is `T` if the path names anything; `dir?` if it names a directory; `file?` if it names something that is not a directory. For a path that names nothing, all three are `NIL`: none of them gives an error for a missing file.

SEE ALSO

`stat`, `ls`, `open`.

EXAMPLE

```
hylang> (exists? "/rom/lib/shell")
=> T
hylang> (file? "/rom/lib/shell")
=> T
hylang> (dir? "/rom/lib")
=> T
hylang> (exists? "/rom/nothere")
=> NIL
```

exit

A special form: leaves hylang.

SYNTAX

```
(exit [n])
exit
```

DESCRIPTION

hylang ends, with the exit code *n*, or 0 if none is given. At the prompt the bare word `exit` does the same. The exit code is what the shell sees as the command's status.

fexpr

A special form: makes a function that receives its arguments unevaluated.

SYNTAX

```
(fexpr {formals} {body})
```

DESCRIPTION

A function made by `fexpr` is called like any other, but its arguments are not evaluated first: each formal is bound to the expression that was written in that place, as data. The body can look at the expressions, and evaluate them, or not, with `eval`, as many times as it likes.

This is how a program adds control structures of its own: a form that evaluates an argument twice, or only under some condition, can't be an ordinary function, because by the time an ordinary function runs its arguments have all been evaluated once already.

EXAMPLE

```
hylang> (def {twice} (fexpr {x} {do (eval x) (eval x)}))
=> NIL
hylang> (twice (print "hi"))
hi
hi
=> NIL
```

fib

A function: a Fibonacci number.

SYNTAX

```
(fib n)
```

ARGUMENTS

n

A whole number, 0 or more.

RESULT

The *n*th number of the series 0, 1, 1, 2, 3, 5, 8 ..., in which each is the sum of the two before it: `(fib 0)` is 0 and `(fib 1)` is 1. The result is exact however long it is.

ERRORS

MESSAGE	CAUSE
`'fib' parameter 'n' cannot be negative`	*n* is below 0
`'fib': n must be a whole number, not 1.5`	*n* is not a whole number

EXAMPLE

```
hylang> (fib 10)
=> 55
hylang> (fib 100)
=> 354224848179261915075
```

filter

A function: the items of a list that pass a test.

SYNTAX

```
(filter f l)
```

ARGUMENTS

f

A function of one argument: the test.

l

A list.

RESULT

A list of the items for which *f* returned a true value (anything but `NIL`), in their original order. If none passes, `NIL`.

DESCRIPTION

f is called with each item in turn. If a call's result is an error, `filter` stops, and the error is its result.

ERRORS

MESSAGE	CAUSE
`'filter' expects a function and a list`	*l* is not a list
`S-Expression starts with incorrect type. Got Number, Expected Function.`	*f* is not a function

SEE ALSO

`map`, `find`, `count`, `remove-all`.

EXAMPLE

```
hylang> (filter odd? {1 2 3 4 5})
=> {1 3 5}
hylang> (filter (fn {x} {> x 2}) {1 2 3 4})
=> {3 4}
hylang> (filter odd? {2 4})
=> NIL
```

find, count

Functions: the first item of a list that passes a test; and how many pass.

SYNTAX

```
(find f l)
(count f l)
```

ARGUMENTS

f

A function of one argument: the test.

l

A list.

RESULT

`find` gives the first item for which *f* returns a true value, and `NIL` if there is none. `count` gives the number of items for which it does: 0 if there are none.

DESCRIPTION

`find` stops at the first item that passes, and *f* is not called for the items after it. `count` tests every item.

`find` gives the item, not its position. Its `NIL` cannot tell "nothing was found" from "the item found was `NIL`": where that matters, use `any?`.

ERRORS

MESSAGE	CAUSE
`'count' expects a function and a list`	*l* is not a list (and `'find'` for `find`)
`S-Expression starts with incorrect type. Got Number, Expected Function.`	*f* is not a function

SEE ALSO

`filter`, `any?`, `elem?`.

EXAMPLE

```
hylang> (find odd? {2 3 4 5})
=> 3
hylang> (find odd? {2 4})
=> NIL
hylang> (count odd? {1 2 3})
=> 2
```

first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tenth

Functions of the library: an item of a list, by its place.

SYNTAX

```
(first l)
(second l)
...
(tenth l)
```

ARGUMENTS

l

A list long enough to have the item.

RESULT

The item: `first` is `(nth 0 l)`, `second` is `(nth 1 l)`, and so on to `tenth`, which is `(nth 9 l)`. The item is given as it is in the list, not evaluated.

`first`, `second` and `third` are the same as `fst`, `snd` and `thd`.

ERRORS

MESSAGE	CAUSE
`'nth': the list has no such item`	The list is too short

SEE ALSO

`nth`, `fst`, `last`.

EXAMPLE

```
hylang> (first {1 2 3})
=> 1
hylang> (third {1 2 3})
=> 3
hylang> (fourth {1 2 3})
=> Error: 'nth': the list has no such item
```

flip, comp, unpack, apply, curry, pack, uncurry

Functions of the library, written in hylang: functions that work on functions.

SYNTAX

```
(flip f a b)
(comp f g x)
(unpack f l)
(pack f x ...)
```

ARGUMENTS

f*, *g
Functions.

a*, *b*, *x
Values.

l
A list of arguments.

RESULT

FUNCTION	RESULT
`flip`	`(f b a)`: *f* called with its two arguments the other way round
`comp`	`(f (g x))`: *g*, and then *f* of its result
`unpack`	*f* called with the items of *l* as its arguments: `(unpack + {1 2 3})` is `(+ 1 2 3)`. `apply` and `curry` are other names for it, and `.(f l)` is another way to write the call
`pack`	*f* called with one argument, the list of all the values that follow it: `(pack len 1 2 3)` is `(len {1 2 3})`. `uncurry` is another name for it

DESCRIPTION

Each is most useful with fewer arguments than it takes, because the result is then a new function (see `fn`): `(comp 1+ square)` is a function that squares a number and adds one, and `(flip - 1)` is one that subtracts 1 from its argument.

`unpack` is how a function that takes separate arguments is given a list that a program has built.

ERRORS

MESSAGE	CAUSE
`'cons' expects a list second`	`unpack`'s second argument is not a list

SEE ALSO

`fn`, `map`, `eval`.

EXAMPLE

```
hylang> (flip - 1 10)
=> 9
hylang> (comp 1+ square 3)
=> 10
hylang> (def {inc-sq} (comp 1+ square))
=> NIL
hylang> (map inc-sq {1 2 3})
=> {2 5 10}
hylang> (unpack + {1 2 3})
=> 6
hylang> (apply max {3 9 2})
=> 9
hylang> (pack len 1 2 3)
=> 3
```

floor, ceil, round, trunc

Functions of the library, written in hylang: a number made whole.

SYNTAX

```
(floor a)
(ceil a)
(round a)
(trunc a)
```

ARGUMENTS

a

A real number.

RESULT

An integer:

FUNCTION	RESULT	2.7	-2.7	2.5	-2.5
`floor`	The nearest whole number that is not above *a*	2	-3	2	-3
`ceil`	The nearest whole number that is not below *a*	3	-2	3	-2
`round`	The nearest whole number; a half goes away from zero	3	-3	3	-3
`trunc`	The whole part: toward zero. Another name for `truncate`	2	-2	2	-2

A number that is already whole is returned as it is.

ERRORS

MESSAGE	CAUSE
`'truncate' expects a real number`	*a* is complex, or not a number (all four are built on `truncate`)

SEE ALSO

`truncate`, `to-fixed`, `div`.

EXAMPLE

```
hylang> (floor -2.7)
=> -3
hylang> (ceil 2.1)
=> 3
hylang> (round 2.5)
=> 3
hylang> (round -2.5)
=> -3
hylang> (round 7/2)
=> 4
```

fn

A special form: makes a function.

SYNTAX

```
(fn {formals} {body})
```

ARGUMENTS

formals

The symbols the arguments will be bound to, in order. They can be none.

body

A Q-expression: the expression that the function evaluates.

RESULT

A function. It has no name; `def` or `fun` gives it one.

DESCRIPTION

Nothing in the body is evaluated when the function is made. When the function is called, a new scope is made, enclosed by the scope `fn` was used in; the formals are bound there to the arguments' values; and the body is evaluated there. Its value is the call's.

The body is one expression. For several, wrap them in `do`.

Called with fewer arguments than formals, the function returns a function of the remaining formals, with the ones given already bound. Called with more, it is an error, unless the body refers to `&_`, which is bound to the list of the arguments beyond the formals. With no formals at all, the arguments are `&1`, `&2` and so on.

A function prints as `<function>` and its definition.

A function with no formals needs nothing more before it can run, and the prompt treats it so: when the value of a line typed there is such a function, the prompt calls it, and shows what it returns. So the name of a function of no formals, typed alone, runs it, with or without parentheses. Inside a larger expression the function is an ordinary value: `(list f)` is a list holding the function.

ERRORS

MESSAGE	CAUSE
`The function takes` *n* `arguments, not` *m*	Too many arguments, in a function that doesn't use `&_`

SEE ALSO

`fun`, `fexpr`, `let`, `def`.

EXAMPLE

```
hylang> (def {add} (fn {x y} {+ x y}))
=> NIL
hylang> (list (add 1 2) ((add 1) 5))
=> {3 6}
hylang> (add 1 2 3)
=> Error: The function takes 2 arguments, not 3
hylang> ((fn {x} {list x &_}) 1 2 3)
=> {1 {2 3}}
hylang> ((fn {} {* &1 &2}) 6 7)
=> 42
```

foldl, foldr

Functions: combine all the items of a list into one value.

SYNTAX

```
(foldl f init l)
(foldr f init l)
```

ARGUMENTS

f

A function of two arguments.

init

The value to start from. It is the result for an empty list.

l

A list.

RESULT

The value that is left when every item has been combined.

DESCRIPTION

`foldl` works from the left. It calls *f* with the value so far and the next item, `(f acc x)`, and the result is the value so far for the next call: for a list `{a b c}` the result is `(f (f (f init a) b) c)`.

`foldr` works from the right, and calls *f* with the item first, `(f x acc)`: the result is `(f a (f b (f c init)))`.

For a function such as + the two give the same answer. For one such as – or `cons`, where the order of the arguments matters, they do not.

ERRORS

MESSAGE	CAUSE
`'foldl' expects a function, a value and a list`	*l* is not a list (and `'foldr'` for `foldr`)

An error from *f* is the result.

SEE ALSO

`sum`, `product`, `map`.

EXAMPLE

```
hylang> (foldl + 0 {1 2 3})
=> 6
hylang> (foldl - 0 {1 2 3})
=> -6
hylang> (foldr - 0 {1 2 3})
=> 2
hylang> (foldr cons {} {1 2 3})
=> {1 2 3}
hylang> (foldl max 0 {3 9 2})
=> 9
```

format

A function: a string with values set into it.

SYNTAX

```
(format text x ...)
~(text x ...)
```

ARGUMENTS

text

A string. Each `{}` in it is a place for a value.

x

The values: one for each place, in order.

RESULT

The text, with each `{}` replaced by the next value, as `print` shows it: a string without its quotes, a number in the current base.

DESCRIPTION

A pair of braces with a base name between them, such as `{x}` or `{b}`, is a place for a number, which is written in that base whatever the base setting is. The names are Chapter 18's, and a radix such as `{#16r}` is one too. The braces cannot hold anything else: there are no widths and no positions. To set a value to a width, pad it first with `str-pad-left`.

The count must match: a value for every place, and no values over.

`~(text x ...)` is another way to write the call.

ERRORS

MESSAGE	CAUSE
`'format' has too few arguments for its {}s`	There are more places than values
`'format' has more arguments than {}s`	There are more values than places
`'format's {x} is a base, for a number`	The value for a base's place is not a number
`Invalid base specifier :5`	The braces hold something that is not a base
`'format' expects a String`	*text* is not a string

SEE ALSO

+, `str-join`, `to-str`, `print`.

EXAMPLE

```
hylang> (format "{} + {} = {}" 1 2 3)
=> "1 + 2 = 3"
hylang> (format "{} is {x} in hex" 255 255)
=> "255 is FF in hex"
hylang> (format "{} and {}" 1)
=> Error: 'format' has too few arguments for its {}s
```

from

A function: a hash as a list.

SYNTAX

```
(from# h)
```

ARGUMENTS

h

A hash.

RESULT

A list of the hash's entries, each `{key value tag ...}`, in the order they were put, followed by the hash's own tags. A value that is a hash is given as such a list too. The list is in the form `hash-create` takes, so `(to# (from# h))` makes a hash like *h*.

DESCRIPTION

Every entry is in the list, private ones too, with their tags: `from#` is how a hash is examined, saved or sent somewhere.

ERRORS

MESSAGE	CAUSE
`First parameter to 'from#' must be a hash`	*h* is not a hash

SEE ALSO

`hash-keys`, `hash-values`, `save`.

EXAMPLE

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (from# pet)
=> {{:name "Rex"} {:legs 4}}
hylang> (map fst (from# pet))
=> {:name :legs}
```

fst, snd, thd, last, nth

Functions: one item of a list.

SYNTAX

```
(fst l)
(snd l)
(thd l)
(last l)
(nth n l)
```

ARGUMENTS

l

A list.

n

Which item, counting from 0: a whole number from 0 to one less than the list's length.

RESULT

The item itself: `fst` the first, `snd` the second, `thd` the third, `last` the last, `nth` item *n*.

DESCRIPTION

The item is given as it is in the list, and is not evaluated. An item of a Q-expression that was written as code is still code: `(fst {(+ 1 2) x})` is the expression `(+ 1 2)`, not 3.

The library names ten more, `first` to `tenth`, which are `nth` with its number already given.

ERRORS

MESSAGE	CAUSE
`'nth': the list has no such item`	The list is too short, or *n* is negative. `fst`, `snd`, `thd` and `last` give this message too, with `nth`'s name
`'nth' expects a list`	*l* is not a list (a string is not one)
`'nth': an index must be a whole number, not 1.5`	*n* is not a whole number

SEE ALSO

`item-at`, `head`, `end`, `first`.

EXAMPLE

```
hylang> (fst {1 2 3})
=> 1
hylang> (last {1 2 3})
=> 3
hylang> (nth 2 {1 2 3})
=> 3
hylang> (fst {(+ 1 2) x})
=> (+ 1 2)
hylang> (nth 3 {1 2 3})
=> Error: 'nth': the list has no such item
```

fun

A special form: defines a named function.

SYNTAX

```
(fun {name formal ...} {body})
```

DESCRIPTION

`(fun {name a b} {body})` is `(def {name} (fn {a b} {body}))`: the first symbol in the braces is the function's name and the rest are its formals. The name is global, wherever `fun` is used. The result is `NIL`.

A function can call itself by its name. A call that is the last thing a function does takes no more stack than a loop would, so recursion of that shape can go on as long as it likes.

EXAMPLE

```
hylang> (fun {count-down n} {if (> n 0) (count-down (- n 1)) "done"})
=> NIL
hylang> (count-down 2000)
=> "done"
```

gensym

A function: a new symbol that no program has written.

SYNTAX

```
(gensym [prefix])
```

ARGUMENTS

prefix

A string to begin the name with. Without it, `g`.

RESULT

A symbol named by the prefix, two underscores and a number that goes up by one each time `gensym` is called: `g__1`, `g__2`, `tmp__3`. The same name is not given twice in a run of hylang.

DESCRIPTION

A `fexpr`, or any code that builds code, uses `gensym` for the names it needs, so that they cannot be the same as a name in the code it is given.

ERRORS

MESSAGE	CAUSE
`'gensym' expects a String`	*prefix* is not a string

EXAMPLE

The numbers depend on how many have been made:

```
hylang> (gensym)
=> g__1
hylang> (gensym "tmp")
=> tmp__2
```

gpio library

A library, loaded with `(use "gpio")`: the eight pins of the I/O header J27, and its two handshake lines.

FUNCTIONS

FORM	WHAT IT DOES
`(gpio pin)`	The pin's level now: 0 or 1
`(gpio! pin level)`	Makes the pin an output and sets it: low for a *level* of 0, high for anything else. `NIL`
`(gpio-in pin)`, `(gpio-out pin)`	Makes the pin an input, or an output. `NIL`
`(gpio-port)`	All eight pins' levels as one byte: bit *n* is pin *n*
`(gpio-port! byte)`	Sets all the outputs at once from a byte. Pins that are inputs stay inputs. `NIL`
`(gpio-ddr! byte)`	Sets all eight directions at once: bit *n* is 1 for an output, 0 for an input. `NIL`
`(gpio-ca1! edge)`	Chooses the change of CA1 that is counted: `:rise` (0 to 1) or `:fall` (1 to 0). `NIL`
`(gpio-ca2! v)`	CA2: 0 makes it an output that is low, 1 an output that is high, and `:in` an input. `NIL`
`(gpio-wait)`	Waits for CA1's next counted change, and gives the number of changes counted so far
`(gpio-state)`	The state of everything, as a hash

ARGUMENTS

pin

0 to 7: PA0 to PA7 of the VIA, on J27's pins 3 to 10.

DESCRIPTION

The library works through the files of `/dev/gpio` (section 17.2 has the pins, the cautions about what may be connected to them, and the files themselves). Every pin is an input when the Hydra starts, and an input with nothing connected reads 1.

Pins 0 and 1 are also the I²C bus: while a program is using the bus, they are not the pins' to set.

`gpio-state` gives a hash with a key for each pin, 0 to 7, whose value is a list of its direction and its level, such as `{:out 1}`; `:ca1`, a list of the edge counted and the count, such as `{:fall 0}`; and `:ca2`, its level, or `:in`.

`gpio-wait` does not return until the line changes: CTRL-C ends the wait.

ERRORS

MESSAGE	CAUSE
`/dev/gpio/8: not found`	A pin that is not 0 to 7, given to `gpio` or `gpio!`
`invalid argument`	A pin that is not 0 to 7, given to `gpio-in` or `gpio-out`; a byte that is not 0 to 255
`'gpio-ca1!' expects :rise or :fall`	Another value for the edge

SEE ALSO

Section 17.2; the `dev` library; `sleep`.

EXAMPLE

```
hylang> (use "gpio")
=> NIL
hylang> (gpio 2)
=> 1
hylang> (gpio! 2 0)
=> NIL
hylang> (gpio 2)
=> 0
hylang> ((gpio-state) 2)
=> {:out 0}
hylang> (gpio-in 2)
=> NIL
hylang> (dotimes {i 3} (do (gpio! 3 0) (sleep 0.1) (gpio! 3 1) (sleep 0.1)))
=> NIL
hylang> (gpio-in 3)
=> NIL
```

hash-add-tag, hash-tag?

Functions: mark a hash, or one of its entries, with a tag; and test for one.

SYNTAX

```
(hash-add-tag h tag ...)
(hash-add-tag h {key tag} ...)
(hash-tag? h tag)
(hash-tag? h {key tag})
```

ARGUMENTS

h

A hash.

tag

An atom: a tag for the hash itself.

{key tag}

A key and an atom: a tag for that key's entry.

RESULT

`hash-add-tag` gives `T` if the tag was added, and `NIL` if it was already there. With several tags, a list of those answers. `hash-tag?` gives `T` if the hash, or the entry, has the tag, and `NIL` if not.

DESCRIPTION

A **tag** is a label a program puts on a hash or on an entry, to mean whatever the program wants: a kind of record, a field to be saved, a unit. Tags are no part of the values. They show after an entry's value, and after the last entry for the hash's own, when the hash is printed or listed with `from#`, and they can be

given in the same places when a hash is made.

Four tags are the system's, and change what can be done with a hash: `hash-lock` and the functions beside it put them on.

ERRORS

MESSAGE	CAUSE
`Failed to add tag: invalid parameters "notatom"`	A tag is not an atom, or the list is not a key and one tag
`hash-add-tag failed: entry not found`	The key is not in the hash

SEE ALSO

`hash-lock`, `hash-locked?`, `from#`.

EXAMPLE

```
hylang> (def {h} #({{:a 1} {:b 2}}))
=> NIL
hylang> (hash-add-tag h :big)
=> T
hylang> (hash-add-tag h {:a :metres})
=> T
hylang> h
=> <hash>{{:a 1 :metres} {:b 2} :big}
hylang> (hash-tag? h :big)
=> T
hylang> (hash-tag? h {:b :metres})
=> NIL
```

hash-call

A function: call a method of a hash. Calling the hash with the key and arguments does the same.

SYNTAX

```
(hash-call h key x ...)
(h key x ...)
```

ARGUMENTS

h

A hash.

key

The key of an entry whose value is a function.

x

The function's arguments.

RESULT

What the function returns.

DESCRIPTION

A function stored in a hash is a **method**. When it is called through the hash, the symbol `&0` is bound, inside it, to the hash, so the method can read and change the hash's other entries: `(&0 :n)`, `(hash-put &0 {:n 1})`. A hash with data and methods is an object, and `hash-clone` makes more like it.

Through `&0` a method reaches every entry, private ones too. That is what a private entry is: one that only the hash's methods can use.

The short form `(h key x ...)` is the usual way to write the call. With no arguments after the key it still calls the method: `(counter :inc)`. To get the function itself, without calling it, use `hash-get`.

Given fewer arguments than it has formals, a method, like any function, returns a function of the rest.

ERRORS

MESSAGE	CAUSE
`Second parameter to 'hash-call' must be a key to a member function`	The key isn't there, or its value isn't a function
`:v isn't a method: its value isn't a function`	The short form, with arguments after a key whose value isn't a function
`First parameter to 'hash-call' must be a hash`	*h* is not a hash

SEE ALSO

`hash-get`, `hash-clone`, `hash-make-private`, `fn`.

EXAMPLE

```
hylang> (def {counter} #({{:n 0} {:inc (fn {} {hash-put &0 {:n (1+ (&0 :n))}})} {:plus
(fn {x} {+ x (&0 :n)})}}))
=> NIL
hylang> (counter :inc)
=> 0
hylang> (counter :inc)
=> 1
hylang> (counter :n)
=> 2
hylang> (counter :plus 10)
=> 12
hylang> (hash-call counter :plus 10)
=> 12
```

hash-clone

A function: a copy of a hash.

SYNTAX

```
(hash-clone h [override ...])
```

ARGUMENTS

h

A hash.

override

An entry, `{key value tag ...}`, to put in the copy: it replaces the entry of that key, or is added. An atom on its own is a tag for the copy. Unlike `hash-create`'s, an override's value is taken as it is written, not evaluated.

RESULT

A new hash with the same entries and tags, and then the overrides. The two are separate from then on: a change to one is not seen in the other. A value that is itself a hash is copied too, so the copy shares nothing with the original.

DESCRIPTION

This is how one object is made from another: clone a hash that has the methods and the usual values, giving the entries that differ.

ERRORS

MESSAGE	CAUSE
`First parameter to 'hash-clone' must be a hash`	*h* is not a hash

SEE ALSO

`hash-create`, `hash-call`.

EXAMPLE

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (def {twin} (hash-clone pet {:name "Max"}))
=> NIL
hylang> (twin :name)
=> "Max"
hylang> (pet :name)
=> "Rex"
hylang> (hash-put twin {:legs 3})
=> 4
hylang> (pet :legs)
=> 4
```

hash-create, to

Functions: make a hash.

SYNTAX

```
(hash-create [entries])
(to# entries)
#(entries)
```

ARGUMENTS

entries

A list of entries. Each entry is a list: a key, a value, and after them any tags, `{key value tag ...}`. Without the argument, the hash is empty.

RESULT

A hash holding the entries, in the order given. It prints as `<hash>` and its entries.

DESCRIPTION

A **hash** is a table of values found by key. A key is an atom, a string, or an integer (a decimal with a whole value, such as 3.0, is the integer). A value is anything, another hash or a function too.

Each entry's value is evaluated as the hash is made, so `{:legs (+ 2 2)}` stores 4; its key and its tags are taken as written. If a key is given twice, the later entry replaces the earlier.

An item of the list that is an atom on its own, not an entry, is a tag on the hash itself (see `hash-add-tag`).

`to#` is the same function under a shorter name, and `#(entries)` is another way to write the call.

A hash is small and searched from its first entry to its last: it is meant for records and objects, tens of entries, not for thousands.

ERRORS

MESSAGE	CAUSE
`A hash's entries are a list`	*entries* is not a list
`A hash's entry is {key value tag...}, not 1`	An item of the list is not an entry: usually one pair of braces is missing
`A hash key must be an atom, a string or an integer, not 1.5`	A key of another kind

SEE ALSO

`hash-put`, `hash-get`, `from#`, `hash-clone`.

EXAMPLE

```
hylang> (def {pet} (hash-create {{:name "Rex"} {:legs (+ 2 2)}}))
=> NIL
hylang> pet
=> <hash>{{:name "Rex"} {:legs 4}}
hylang> (len pet)
=> 2
hylang> #({{1 "one"} {"two" 2}})
=> <hash>{{1 "one"} {"two" 2}}
hylang> (hash-create {:a 1})
=> Error: A hash's entry is {key value tag...}, not 1
```

hash-get

A function: the value at a key of a hash. Calling the hash itself does the same.

SYNTAX

```
(hash-get h key)
(h key)
```

ARGUMENTS

h

A hash.

key

An atom, a string or an integer.

RESULT

The value stored at the key, or `NIL` if the hash has no such key. Since a stored value can itself be `NIL`, use `hash-key?` to tell the two apart.

DESCRIPTION

A hash can stand where a function does: `(h key)` looks the key up. The two forms differ only when the value is a function. `hash-get` gives the function as a value. `(h key)` calls it, as a method, with `&0` bound to the hash (see `hash-call`).

`(h)`, with no key, is the hash itself. `hash-get` of `NIL` is `NIL`, so a lookup in a hash that was never made does not stop a program.

ERRORS

MESSAGE	CAUSE
`First parameter to 'hash-get' must be a hash`	*h* is neither a hash nor `NIL`
`hash-get error: cannot access private hash entry`	The entry is private (see `hash-make-private`): only the hash's own methods can read it

SEE ALSO

`hash-put`, `hash-key?`, `hash-call`.

EXAMPLE

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (pet :name)
=> "Rex"
hylang> (hash-get pet :legs)
=> 4
hylang> (pet :wings)
=> NIL
```

hash-key?, hash-keys, hash-values

Functions: is a key in a hash? And all its keys, and all its values.

SYNTAX

```
(hash-key? h key)
(hash-keys h)
(hash-values h)
```

ARGUMENTS

h

A hash.

key

An atom, a string or an integer.

RESULT

`hash-key?` gives `T` if the hash has an entry with that key, whatever its value, and `NIL` if not. `hash-keys` gives a list of the keys and `hash-values` a list of the values, both in the order the entries were put, so the two lists match item for item. For an empty hash each is `NIL`.

Private entries are listed with the rest.

ERRORS

MESSAGE	CAUSE
`First parameter to 'hash-keys' must be a hash`	*h* is not a hash (each function with its own name)

SEE ALSO

`from#`, `hash-get`, `len`.

EXAMPLE

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (hash-key? pet :name)
=> T
hylang> (hash-key? pet :wings)
=> NIL
hylang> (hash-keys pet)
=> {:name :legs}
hylang> (hash-values pet)
=> {"Rex" 4}
```

hash-lock, hash-make-const, hash-make-private, hash-make-not-nil

Functions: protect a hash, or some of its entries.

SYNTAX

```
(hash-lock h [key ...])
(hash-make-const h [key ...])
(hash-make-private h [key ...])
(hash-make-not-nil h [key ...])
```

ARGUMENTS

h

A hash.

key

The keys of the entries to protect. With none, the protection is put on the hash as a whole.

RESULT

`T` if the protection was added, `NIL` if it was already there; a list of those for several keys.

DESCRIPTION

Each puts one of the system's four tags on the hash or on the entries named, and the tag changes what `hash-put`, `hash-get` and `hash-remove` will do:

FUNCTION	TAG	ON THE HASH	ON AN ENTRY
`hash-lock`	`:__locked`	No entry can be added or removed. The values of the entries it has can still be changed	
`hash-make-const`	`:__read-only`	Nothing can be changed, added or removed	The entry can't be changed or removed
`hash-make-private`	`:__private`		The entry can be read and changed only by the hash's own methods, through `&0`
`hash-make-not-nil`	`:__not_nil`		The entry can't be given the value `NIL` (it can still be removed)

A protection, once put on, stays: there is no function that takes one off. A copy made with `from#` and `to#` carries the tags with it.

A private entry is hidden from `hash-get` and from `hash-put`, but not from `hash-keys`, `hash-values` and `from#`, which list it with the rest: the protection is against accidents, not against a determined program.

The same tags can be written in an entry as a hash is made: `{:a 1 :__read-only}`.

ERRORS

MESSAGE	CAUSE
`hash-add-tag failed: entry not found`	A key is not in the hash
`First parameter to 'hash-lock' must be a hash`	*h* is not a hash (each function with its own name)

The errors that a protection causes are listed under `hash-put`, `hash-get` and `hash-remove`.

SEE ALSO

`hash-locked?`, `hash-add-tag`, `hash-call`.

EXAMPLE

```
hylang> (def {acct} #({{:owner "Ann"} {:pin 1234} {:check (fn {n} {== n (&0 :pin)})}}))
=> NIL
hylang> (hash-make-private acct :pin)
=> T
hylang> (acct :pin)
=> Error: hash-get error: cannot access private hash entry
hylang> (acct :check 1234)
=> T
hylang> (hash-make-const acct :owner)
=> T
hylang> (hash-put acct {:owner "Bob"})
=> Error: hash-put error: cannot modify read-only hash entry
hylang> (hash-lock acct)
=> T
hylang> (hash-put acct {:extra 1})
=> Error: hash-put error: cannot add new entries to a locked hash
```

hash-locked?, hash-private?, hash-const?

Functions: is a hash, or one of its entries, protected?

SYNTAX

```
(hash-locked? h [key])
(hash-private? h [key])
(hash-const? h [key])
```

ARGUMENTS

h

A hash.

key

An entry's key. Without it, the question is about the hash as a whole.

RESULT

`T` if the hash, or the entry, has the protection that `hash-lock`, `hash-make-private` or `hash-make-const` puts on, and `NIL` if not.

The hash and its entries are separate: `(hash-const? h)` is `NIL` for a hash that has constant entries but was not itself made constant.

ERRORS

MESSAGE	CAUSE
`Key :v not found when looking up tag for hash entry`	The key is not in the hash

SEE ALSO

`hash-lock`, `hash-tag?`.

EXAMPLE

```
hylang> (def {k} #({{:a 1} {:b 2}}))
=> NIL
hylang> (hash-make-const k :a)
=> T
hylang> (hash-const? k :a)
=> T
hylang> (hash-const? k :b)
=> NIL
hylang> (hash-const? k)
=> NIL
hylang> (hash-locked? k)
=> NIL
```

hash-put

A function: add entries to a hash, or change them.

SYNTAX

```
(hash-put h {key value} ...)
```

ARGUMENTS

h

A hash.

{key value}

An entry, as `hash-create` takes: the value is evaluated, and tags may follow it. There may be several entries.

RESULT

The value the key had before, or `NIL` if the key is new. With several entries, a list of those values, one for each.

DESCRIPTION

The hash itself is changed: every name bound to it sees the change. An entry whose key is already there takes the old one's place in the order; a new key goes at the end.

Each entry needs its own braces. `(hash-put h :a 1)` is not an entry, and gives errors for `:a` and for 1.

ERRORS

MESSAGE	CAUSE
`A hash's entry is {key value tag...}, not :a`	An argument is not an entry
`hash-put error: cannot add new entries to a locked hash`	The hash is locked, and the key is new (see `hash-lock`)
`hash-put error: cannot modify read-only hash`	The hash is constant (see `hash-make-const`)
`hash-put error: cannot modify read-only hash entry`	That entry is constant
`hash-put error: cannot access private hash entry`	The entry is private, and this is not one of the hash's methods
`Cannot set non-nillable field :v to NIL.`	The entry must not be `NIL` (see `hash-make-not-nil`)

SEE ALSO

`hash-get`, `hash-remove`, `hash-create`.

EXAMPLE

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (hash-put pet {:legs 3})
=> 4
hylang> (hash-put pet {:colour "brown"} {:age 2})
=> {NIL NIL}
hylang> (hash-keys pet)
=> {:name :legs :colour :age}
```

hash-remove

A function: take an entry out of a hash.

SYNTAX

```
(hash-remove h key)
```

ARGUMENTS

h

A hash.

key

The entry's key.

RESULT

The value the entry had. If there was no such key, `NIL`, and nothing is changed.

ERRORS

MESSAGE	CAUSE
`hash-remove error: cannot remove an entry from a locked hash`	The hash is locked, or the entry is constant

SEE ALSO

`hash-put`, `hash-lock`.

EXAMPLE

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (hash-remove pet :legs)
=> 4
hylang> (hash-remove pet :legs)
=> NIL
hylang> (hash-keys pet)
=> {:name}
```

head, tail

Functions: the first item of a list, as a list; and the rest of the list.

SYNTAX

```
(head l)
(tail l)
```

ARGUMENTS

l

A list with at least one item.

RESULT

`head` gives a list of one item, the first. `tail` gives the list without its first item; for a list of one item that is `NIL`.

DESCRIPTION

`head` keeps the item inside a list so that it can be run: `(eval (head l))` evaluates the first item of a list of code. For the item itself, use `fst`.

Neither works on a string: for that there are `substring` and `char-at`.

ERRORS

MESSAGE	CAUSE
`'head' passed an empty list`	*l* is `NIL` (and `'tail'` for `tail`)
`'head' expects a QExpr`	*l* is not a list

SEE ALSO

`fst`, `init`, `take`, `drop`.

EXAMPLE

```
hylang> (head {1 2 3})
=> {1}
hylang> (tail {1 2 3})
=> {2 3}
hylang> (tail {1})
=> NIL
hylang> (eval (head {(+ 1 2) 3}))
=> 3
hylang> (head {})
=> Error: 'head' passed an empty list
```

hex, bin

Functions: an integer's digits in hexadecimal, or in binary, as a string.

SYNTAX

```
(hex n [width])
(bin n [width])
```

ARGUMENTS

n

An integer.

width

The least number of digits: the string is filled out with zeros on the left to this length. A number that needs more digits has them all.

RESULT

A string of the digits, with no prefix. Hex digits above 9 are capitals. A negative number is a - and the digits of its size: `(hex -1)` is `"-1"`, not a row of F's. For the byte or word a negative number would be in memory, `bit-and` it first: `(hex (bit-and -1 255))` is `"FF"`.

DESCRIPTION

These give text, whatever the base setting is. To have every number written in another base, set `base`; to turn one number to text in any base, use `to-str` with a base.

ERRORS

MESSAGE	CAUSE
`'hex''s number must be an integer`	*n* is not an integer (and `'bin'`)
`A width must be an integer`	*width* is not an integer

SEE ALSO

`to-str`, `base`, `val`.

EXAMPLE

```
hylang> (hex 255)
=> "FF"
hylang> (hex 255 4)
=> "00FF"
hylang> (bin 5)
=> "101"
hylang> (bin 5 8)
=> "00000101"
hylang> (hex (bit-and -1 255))
=> "FF"
```

hold

A special form: evaluates expressions without letting other tasks run in between.

SYNTAX

```
(hold body ...)
```

DESCRIPTION

The body's expressions are evaluated in order, as `do` evaluates them, and the last one's value is returned. While they run, HydraOS does not switch to another task: the program keeps the processor. It is for the few moments when two tasks share something, a segment of memory or a device, and must not see it half changed.

Every other task waits while the body runs, so the body should be short, and must not wait for anything itself.

i2c library

A library, loaded with `(use "i2c")`: the I²C bus on the I/O header.

FUNCTIONS

FORM	WHAT IT DOES
`(i2c-devices)`	A list of the addresses of the chips that answer on the bus
`(i2c-read addr n [reg])`	Reads *n* bytes, 64 at most, from the chip at *addr*: a list. With *reg*, from that register on
`(i2c-write addr bytes [reg])`	Sends bytes, a list or a string's, to the chip. With *reg*, to that register. `NIL`
`(i2c-speed khz)`	Sets the bus's speed, in kilohertz: 1 to 100. `NIL`
`(i2c-reg-size n)`	Says how a register's number is sent: 0, not at all (plain bytes); 1, as one byte; 2, as two, high byte first. `NIL`

ARGUMENTS

addr

The chip's address on the bus, a number: `#x08` to `#x77`.

reg

A register inside the chip: used only after `(i2c-reg-size 1)` or `2`, and 0 if it isn't given.

DESCRIPTION

The library works through the files of `/dev/i2c` (section 17.3). Each read or write is one transaction on the bus. With a register size set, the register's number is sent first, and a read follows it with a repeated start, which is what most chips with registers expect.

The settings are the bus's, not the program's: they stay as they are left.

ERRORS

MESSAGE	CAUSE
`i/o error`	The chip did not answer: nothing is at that address
`invalid argument`	A speed or a register size out of range

SEE ALSO

Section 17.3; the `gpio` and `dev` libraries.

EXAMPLE

With nothing on the bus:

```
hylang> (use "i2c")
=> NIL
hylang> (i2c-devices)
=> NIL
hylang> (i2c-read #x50 2)
=> Error: i/o error
hylang> (i2c-speed 40)
=> NIL
```

if

A special form: evaluates one of two expressions.

SYNTAX

```
(if test then [else])
```

DESCRIPTION

test is evaluated. If its value is anything but `NIL`, *then* is evaluated and its value returned, and *else* is not looked at. If it is `NIL`, *else* is evaluated and returned; with no *else*, the result is `NIL`.

0 and the empty string are not `NIL`, and count as true. The empty list is `NIL`.

Each branch is one expression. For several, use `do`. A branch written in braces is not run: a Q-expression's value is itself, so `(if T {print "x"})` returns the list `{print "x"}` and prints nothing.

EXAMPLE

```
hylang> (if 0 "zero counts as true" "zero counts as false")
=> "zero counts as true"
hylang> (if {} "an empty list is true" "an empty list is NIL")
=> "an empty list is NIL"
hylang> (if NIL 1)
=> NIL
```

index-of, last-index-of

Functions: where a piece of text is in a string, or an item in a list.

SYNTAX

```
(index-of s sub)
(last-index-of s sub)
```

ARGUMENTS

s

A string, or a list.

sub

What to look for. In a string: a string or a character. In a list: any value, which is compared with each item as == compares.

RESULT

The position, from 0, of the first place (`index-of`) or the last place (`last-index-of`) where *sub* is found. If it isn't found, the result is -1. An empty *sub* is found at 0.

DESCRIPTION

The result -1 is a number, and so is true: test it with `>=0`, or `(< i 0)`, not with `if` alone. The library's `contains?`, `starts-with?` and `ends-with?` do that for strings.

ERRORS

MESSAGE	CAUSE
`'index-of' expects a String and a String`	*s* is a string and *sub* is neither a string nor a character

SEE ALSO

`contains?`, `substring`, `elem?`, `find`.

EXAMPLE

```
hylang> (index-of "hello" "l")
=> 2
hylang> (last-index-of "hello" "l")
=> 3
hylang> (index-of "hello" "z")
=> -1
hylang> (index-of {"a" "b"} "b")
=> 1
hylang> (substring "a.b.c" (1+ (last-index-of "a.b.c" ".")))
=> "c"
```

init, end

Functions: a list without its last item; and its last item, as a list.

SYNTAX

```
(init l)
(end l)
```

ARGUMENTS

1

A list with at least one item.

RESULT

`init` gives the list without its last item (`NIL` for a list of one). `end` gives a list of one item, the last. They are `tail` and `head` seen from the other end.

ERRORS

MESSAGE	CAUSE
`'init' passed an empty list`	*l* is `NIL` (and `'end'` for `end`)
`'init' expects a QExpr`	*l* is not a list

SEE ALSO

`head`, `tail`, `last`.

EXAMPLE

```
hylang> (init {1 2 3})
=> {1 2}
hylang> (end {1 2 3})
=> {3}
hylang> (init {1})
=> NIL
```

item-at

A function: one item of a list, by its position.

SYNTAX

```
(item-at l i)
```

ARGUMENTS

l

A list.

i

The item's position, from 0.

RESULT

Item *i*, as it is in the list, not evaluated.

DESCRIPTION

`item-at` is `nth` with its arguments the other way round, and with messages that name the position.

ERRORS

MESSAGE	CAUSE
`'item-at': the list has no item 2`	*i* is the list's length or more, or negative
`First 'item-at' parameter must be a QExpr`	*l* is not a list

SEE ALSO

`nth`, `subset`.

EXAMPLE

```
hylang> (item-at {a (+ 1 2) c} 1)
=> (+ 1 2)
hylang> (item-at {a b} 2)
=> Error: 'item-at': the list has no item 2
```

join

A function: lists put end to end.

SYNTAX

```
(join l ...)
```

ARGUMENTS

1

A list. There may be any number of them, and an empty one adds nothing.

RESULT

One list, of the first list's items, then the second's, and so on.

DESCRIPTION

Only the outer lists are opened: a list that is an item of one of them is still a list in the result.

`join` is for lists. Strings are joined with `+`, or with `str-join`.

ERRORS

MESSAGE	CAUSE
`Invalid parameter passed to 'join'. Expected QExpr.`	An argument is not a list

SEE ALSO

`cons`, `list`, `flatten`.

EXAMPLE

```
hylang> (join {1 2} {3} {} {4 5})
=> {1 2 3 4 5}
hylang> (join {1} {{2}})
=> {1 {2}}
hylang> (join {1} 2)
=> Error: Invalid parameter passed to 'join'.  Expected QExpr.
```

key, key?

Functions: the next key pressed, without waiting for a line; and whether one is waiting.

SYNTAX

```
(key)
(key?)
```

RESULT

`key` waits for a key and gives it: a character for an ordinary key, and an atom for one of the terminal's named keys: `:up`, `:down`, `:left`, `:right`, `:home`, `:end`, `:ins`, `:del`, `:pgup`, `:pgdn`, and `:f1` to `:f12`. ENTER is the character `\cr`. At the end of the input it gives `NIL`.

`key?` gives `T` if a key has been pressed and not yet read, and `NIL` if not. It does not wait.

DESCRIPTION

Keys are read raw: as they are pressed, without being shown on the screen, and without the line editing that `read-line` and the prompt give. The keyboard goes back to lines when the next line is read.

Together they make the loop of a game or an editor: `(if (key?) (key))` takes a key when there is one, and lets the program go on when there isn't.

CTRL-C pressed while a program waits in `key` is the error `interrupted`.

SEE ALSO

`read-line`, `on-note`, `sleep`.

EXAMPLE

Here x and then the up arrow were pressed:

```
hylang> (key)
=> \x
hylang> (key)
=> :up
```

len

A function: the length of a list, a string, a hash or a buffer.

SYNTAX

```
(len x)
```

ARGUMENTS

x

A list, a string, a hash or a buffer.

RESULT

A whole number: a list's items (a list inside it counts as one), a string's characters, a hash's entries, a buffer's bytes. The empty list and the empty string have length 0.

ERRORS

MESSAGE	CAUSE
`'len' requires parameter of type string, list, hash or buffer`	*x* is a number, an atom, or anything else that has no length

EXAMPLE

```
hylang> (len {1 2 3})
=> 3
hylang> (len {a {b c} d})
=> 3
hylang> (len "hello")
=> 5
hylang> (len {})
=> 0
hylang> (len 5)
=> Error: 'len' requires parameter of type string, list, hash or buffer
```

let

A special form: evaluates expressions in a new scope, with names bound in it first.

SYNTAX

```
(let {{name [value]} ...} body ...)
(let body)
```

ARGUMENTS

name
A symbol.

value
An expression: what the name is bound to. If it is left out, the name is bound to `NIL`.

body
An expression. It can also be written as a Q-expression, `{print x}`, which is then run as code.

RESULT

The value of the last *body* expression.

DESCRIPTION

1. A new scope is made, enclosed by the scope the `let` is in.
2. The bindings are taken in order. For each, *value* is evaluated **in the new scope**, and *name* is bound to the result there. A value can therefore use the names bound before it in the same `let`; a name bound twice keeps its last value.
3. The *body* expressions are evaluated in order, in the new scope. The values of all but the last are discarded.
4. The new scope is dropped, unless something made in it still refers to it: a function made by `fn` in the body keeps the scope, and can go on using its names after the `let` has returned (a closure).

A name bound by `let` hides one of the same name outside it for as long as the body runs, and the outer one is unchanged afterwards.

Inside the body, `set` binds or changes a name in the `let`'s own scope; `set!` changes a name in whichever scope it was made, which may be outside; and `def` always binds globally.

The last body expression is evaluated in tail position: a function whose body ends in a `let` that ends in a call to itself runs in constant stack, however many times it repeats.

With one argument only, that argument is the body: `(let body)` evaluates it in a new, empty scope. This means `(let {{a 1}})` does not bind anything; its one argument is run as code, and the result is the list `{a 1}`.

ERRORS

MESSAGE	CAUSE
`A 'let' binding is {name value}`	A binding that isn't a list, whose first element isn't a symbol, or that has more than two elements
`'let' supplied too few parameters`	No arguments at all

An error from evaluating a *value* or a *body* expression ends the `let` at once, and is its result.

SEE ALSO

`def`, `set`, `set!`, `fn`, `do`.

EXAMPLE

```
hylang> (let {{x 2} {y (+ x 1)}} (list x y))
=> {2 3}
hylang> (def {x} 10)
=> NIL
hylang> (let {{x 1}} (set {x} 5) x)
=> 5
hylang> x
=> 10
hylang> (def {add5} (let {{n 5}} (fn {a} {+ a n})))
=> NIL
hylang> (add5 1)
=> 6
hylang> n
=> Error: Unbound Symbol 'n'
hylang> (let {a 1} a)
=> Error: A 'let' binding is {name value}
```

list

A function: makes a list of its arguments' values.

SYNTAX

```
(list x ...)
[x ...]
```

ARGUMENTS

x

Any value. There may be any number of them.

RESULT

A list of the values, in order. With no arguments, `NIL`, which is the empty list.

DESCRIPTION

`list` is how a list is made from values that have to be worked out. A Q-expression, `{a b}`, is a list of what was written, with nothing evaluated; `(list a b)` is a list of what `a` and `b` evaluate to. Square brackets are another way to write the call: `[a b]` is read as `(list a b)`.

An argument that is itself a list becomes one item of the result: `list` nests, where `join` flattens.

SEE ALSO

`cons`, `join`, `range`.

EXAMPLE

```
hylang> (list 1 (+ 1 1) "three")
=> {1 2 "three"}
hylang> [1 (+ 1 1)]
=> {1 2}
hylang> (list (+ 1 2) {+ 1 2})
=> {3 {+ 1 2}}
hylang> (list)
=> NIL
```

lo, hi, word

Functions: the two bytes of a 16-bit word; and a word from two bytes.

SYNTAX

```
(lo n)
(hi n)
(word lo hi)
```

ARGUMENTS

n

An integer.

lo, hi

Integers: the low byte and the high byte.

RESULT

`lo` gives the low byte of *n*, bits 0 to 7, and `hi` the next byte, bits 8 to 15: each a number from 0 to 255. Bits above 15 are ignored, and a negative number is taken as two's complement, so `(lo -1)` is 255.

`word` puts two bytes together: *lo* + 256 × *hi*. It is the opposite of `lo` and `hi`, and is how an address read from memory a byte at a time is made whole (the 65C02 keeps the low byte first).

ERRORS

MESSAGE	CAUSE
`'lo''s number must be an integer`	*n* is not an integer (and `'hi'`)
`A low byte must be an integer`	`word`'s *lo* is not an integer

SEE ALSO

`peek`, `peek-word`, `bit-and`, `shr`.

EXAMPLE

```
hylang> (lo #x1234)
=> 52
hylang> (hi #x1234)
=> 18
hylang> (hex (word #x34 #x12))
=> "1234"
hylang> (lo -1)
=> 255
```

load

A special form: evaluates the expressions in files.

SYNTAX

```
(load path ...)
```

DESCRIPTION

Each file is read and its expressions are evaluated in order. It is how a program is split across files, and how a library is brought in.

A file that can't be read is an error, with the system's message and the file's name.

EXAMPLE

```
hylang> (load "/ram/nothere.hl")
=> Error: /ram/nothere.hl: not found
```

ls, dir, stat, glob

Functions: the names in a directory; their details; one file's details; and the paths that match a pattern.

SYNTAX

```
(ls [path])
(dir [path])
(stat path)
(glob pattern)
```

ARGUMENTS

path

A directory, or a file. For `ls` and `dir`, the current directory if it isn't given.

pattern

A path in which `*` stands for any run of characters, `?` for any one character, and `[...]` for any one of the characters in the brackets (`a-z` is a range, and `~` first means any character *not* listed). Each part of the path between slashes can be a pattern.

RESULT

`ls` gives a list of the names in the directory, as strings, sorted. Given a file, it gives a list of that one name.

`stat` gives a hash of the file's details:

KEY	VALUE
`:name`	Its name, without its directory
`:length`	Its length in bytes (0 for a directory)
`:dir`	`T`, for a directory. A file has no such entry
`:mtime`	When it was last changed, in seconds since the start of the year 2000, as `time` counts
`:mode`	Its permission bits, as a number (438 is octal 666: it can be read and written)
`:qid`	The file system's own number for the file
`:dev`	The letter of the device it is on, a character

`dir` gives a list of such hashes, one for each name `ls` would give.

`glob` gives a sorted list of the paths that match, each written as the pattern was (with the directory it gave), and `NIL` if none does.

ERRORS

MESSAGE	CAUSE
path`: not found`	There is no such file or directory

SEE ALSO

`exists?`, `date`, `cd`.

EXAMPLE

```
hylang> (mkdir "/ram/sub")
=> NIL
hylang> (write-file "/ram/sub/f1" "1")
=> NIL
hylang> (write-file "/ram/sub/f2.hl" "22")
=> NIL
hylang> (ls "/ram/sub")
=> {"f1" "f2.hl"}
hylang> (glob "/ram/sub/*.hl")
=> {"/ram/sub/f2.hl"}
hylang> ((stat "/ram/sub/f2.hl") :length)
=> 2
hylang> ((stat "/ram/sub") :dir)
=> T
hylang> (map (fn {d} {d :name}) (dir "/ram/sub"))
=> {"f1" "f2.hl"}
```

map

A function: a list of a function's results, one for each item of a list.

SYNTAX

```
(map f l)
```

ARGUMENTS

f

A function of one argument.

l

A list.

RESULT

A list as long as *l*: *f* of its first item, *f* of its second, and so on. For `NIL` the result is `NIL`.

DESCRIPTION

f is called with each item in turn, from the first. If a call's result is an error, `map` stops there, and the error is `map`'s result.

`map` takes exactly one list. To walk two lists together, `zip` them first.

ERRORS

MESSAGE	CAUSE
`'map' expects a function and a list`	*l* is not a list (a string is not one)
`S-Expression starts with incorrect type. Got Number, Expected Function.`	*f* is not a function
`'map' takes 2 arguments, not 3`	More than one list was given

SEE ALSO

`each`, `filter`, `foldl`, `zip`.

EXAMPLE

```
hylang> (map 1+ {1 2 3})
=> {2 3 4}
hylang> (map (fn {x} {* x x}) {1 2 3})
=> {1 4 9}
hylang> (map len {"a" "bb" {1 2 3}})
=> {1 2 3}
hylang> (map (fn {x} {/ 1 x}) {1 0 2})
=> Error: Division by zero.
```

`math.pi, math.e, math.phi, math.gamma`

Constants of the library: four numbers, to many places.

SYNTAX

```
math.pi
math.e
math.phi
math.gamma
```

RESULT

Each is a variable holding a decimal:

NAME	THE NUMBER	PLACES
`math.pi`	Pi, the ratio of a circle's circumference to its diameter: 3.14159...	125
`math.e`	The base of the natural logarithms: 2.71828...	50
`math.phi`	The golden ratio: 1.61803...	50
`math.gamma`	Euler's constant: 0.57721...	20

DESCRIPTION

These are ordinary decimals, so arithmetic with them is exact arithmetic on the digits given, and a result carries every one of them: `(* 2 math.pi)` has 125 places. That is seldom what a program wants. `(pi)` gives pi to the precision `digits` has set, 12 places to begin with, and `to-fixed` cuts any number to a length.

SEE ALSO

`pi`, `digits`, `to-fixed`, `exp`.

EXAMPLE

```
hylang> math.e
=> 2.71828182845904523536028747135266249775724709369995
hylang> (to-fixed math.pi 5)
=> 3.14159
hylang> (to-fixed (* 2 math.pi) 8)
=> 6.2831853
```

min, max

Functions: the least of their arguments; and the greatest.

SYNTAX

```
(min x ...)
(max x ...)
```

ARGUMENTS

x

Any values: at least one.

RESULT

The argument that comes first in order (`min`) or last (`max`).

DESCRIPTION

The arguments are compared as < and > compare, which is `cmp`'s order: numbers by value whatever their form, strings character by character, and values of different kinds by kind, numbers before strings before atoms. So the arguments need not all be numbers.

These compare their *arguments*. Given one list, they have one argument, and return it: `(min {3 1 2})` is `{3 1 2}`. For the least item of a list, use the library's `minimum` and `maximum`.

SEE ALSO

`minimum`, `maximum`, `sort`, `cmp`.

EXAMPLE

```
hylang> (min 3 1 2)
=> 1
hylang> (max 3 1 2)
=> 3
hylang> (min 1/2 0.3)
=> 0.3
hylang> (min "b" "a")
=> "a"
hylang> (min {3 1 2})
=> {3 1 2}
```

minimum, maximum, sort-desc, best-of, worst-of

Functions of the library, written in hylang: the least and the greatest item of a list; and its items in falling order.

SYNTAX

```
(minimum l)
(maximum l)
(sort-desc l)
(best-of n l)
(worst-of n l)
```

ARGUMENTS

l

A list.

n

A count.

RESULT

`minimum` and `maximum` give the least and the greatest item of the list, in `cmp`'s order: they are `min` and `max` given the list's items as their arguments.

`sort-desc` gives the list sorted with the greatest first. `best-of` gives the *n* greatest items, greatest first, and `worst-of` the *n* least, least first; if the list has fewer than *n*, all of them. `top` and `bottom` are other names for `best-of` and `worst-of`.

NOTES

Each takes one list. `(minimum 3 1)` is an error: use `min` for separate arguments.

`minimum` or `maximum` of an empty list is not an error and not `NIL`: it is the function `min`, or `max`, waiting for an argument. Test for an empty list first.

SEE ALSO

`min`, `max`, `sort`, `take`, `remove-min`.

EXAMPLE

```
hylang> (minimum {3 1 2})
=> 1
hylang> (maximum {3 1 2})
=> 3
hylang> (sort-desc {3 1 2})
=> {3 2 1}
hylang> (best-of 2 {3 1 4 2})
=> {4 3}
hylang> (worst-of 2 {3 1 4 2})
=> {1 2}
```

mkdir, remove, rename, copy-file

Functions: make a directory; remove a file or a directory; rename a file; copy a file.

SYNTAX

```
(mkdir path)
(remove path)
(rename path name)
(copy-file from to)
```

ARGUMENTS

path

A string: what to make, remove or rename.

name

The new name.

from, to

The file to copy, and the file to make.

RESULT

Each gives `NIL` when it has done its work.

DESCRIPTION

`mkdir` makes one directory. The directory it is to be in must be there already: it doesn't make a path of several.

`remove` takes away a file, or a directory that is empty.

`rename` changes a file's name, in the directory it is in. It does not move a file: only the last part of *name* is used, and any directory written before it is ignored. To move a file to another directory, copy it and remove the original.

`copy-file` makes *to*, or empties it if it is there, and writes *from*'s bytes into it. *to* must be a file's name, not a directory's.

ERRORS

MESSAGE	CAUSE
path: `already exists`	`mkdir` of something that is there
path: `not found`	The file isn't there; or, for `mkdir`, the directory it is to go in
path: `directory not empty`	`remove` of a directory that still holds something
path: `is a directory`	`copy-file` to a directory

SEE ALSO

`ls`, `exists?`, `write-file`.

EXAMPLE

```
hylang> (mkdir "/ram/t1")
=> NIL
hylang> (write-file "/ram/t1/a" "hello")
=> NIL
hylang> (rename "/ram/t1/a" "b")
=> NIL
hylang> (copy-file "/ram/t1/b" "/ram/t1/c")
=> NIL
hylang> (ls "/ram/t1")
=> {"b" "c"}
hylang> (remove "/ram/t1")
=> Error: /ram/t1: directory not empty
hylang> (remove "/ram/t1/b")
=> NIL
hylang> (remove "/ram/t1/c")
=> NIL
hylang> (remove "/ram/t1")
=> NIL
```

neg?, pos?, zero?, one?, not

Functions: tests of a number's sign and value; and the opposite of a truth value.

SYNTAX

```
(neg? x)
(pos? x)
(zero? x)
(one? x)
(not x)
```

ARGUMENTS

x

Any value.

RESULT

`T` or `NIL`:

FUNCTION	`T` WHEN
`neg?`	*x* is a number below 0
`pos?`	*x* is a number above 0
`zero?`	*x* is equal to 0 (0.0 is)
`one?`	*x* is equal to 1
`not`	*x* is `NIL`, or the empty list, which is the same

DESCRIPTION

The four number tests are comparisons with 0 or 1, and since any two values can be compared, they give no error for something that is not a number: `(zero? "a")` is simply `NIL`. The library's `positive?` and `negative?` are the strict ones, which give an error for a non-number.

`not` follows hylang's rule of truth: only `NIL` is false. `(not 0)` and `(not "")` are `NIL`, because 0 and the empty string are true.

SEE ALSO

`positive?`, `negative?`, `odd?`, `even?`, `=0`, `and`, `or`.

EXAMPLE

```
hylang> (neg? -1)
=> T
hylang> (pos? 0)
=> NIL
hylang> (zero? 0.0)
=> T
hylang> (not NIL)
=> T
hylang> (not 0)
=> NIL
```

newns

A special form: rebuilds the namespace.

SYNTAX

```
(newns)
```

DESCRIPTION

hylang's namespace is thrown away and the default one built again from `/lib/namespace`, as a login shell does when it starts and as rc's `newns` does. Every `bind` and `mount` the program has made is undone.

The default namespace gives the task a `/ram` of its own, an area of the RAM disk named for its task number, and that area starts empty. A hylang started from a shell had been using the shell's `/ram`: after `(newns)` those files are still the shell's, but they are no longer at hylang's `/ram`. A hylang that is itself a window's first shell already had its own area, and `(newns)` removes the files in it.

note, note-group, on-note

Functions: send a note to a task, or to a group of tasks; and choose what hylang does when one arrives.

SYNTAX

```
(note task n)
(note-group group n)
(on-note f)
```

ARGUMENTS

task

A task's number.

group

A note group's number, as `task-info`'s `:group` gives it.

n

The note: one of the atoms `:interrupt`, `:kill`, `:hangup`, `:alarm` and `:brk`, or a note's number, 1 to 31. The numbers 16 to 31 are for programs' own purposes.

f

A function of one argument, or `NIL`.

RESULT

Each gives `NIL`.

DESCRIPTION

A **note** is a short message from one task to another, or from the system: CTRL-C is the note `:interrupt`, sent to every task in the window's group. `note` sends one to a task, and `note-group` to every task of a group, hylang itself included if it is in it.

What a note does to the task that gets it is that task's choice. Left alone, hylang treats `:interrupt` as the error `interrupted`, which stops what it is doing and returns to the prompt, and ends when any other note arrives, as most programs do.

`on-note` changes that. From then on, each note that arrives is passed to *f*, as an atom for the named notes and a number for the others. If *f* returns a true value, the note has been dealt with and the program goes on. If it returns `NIL`, the note has its usual effect. `(on-note NIL)` goes back to the usual effect for all of them.

f is not called at the instant the note arrives, but at hylang's next step after it, so it can do anything a function can. The `:kill` note can't be caught: it always ends the task.

A task ended by a note has the exit code 128 plus the note's number: 130 for `:interrupt`, 137 for `:kill`.

ERRORS

MESSAGE	CAUSE
task`: no such task`	There is no such task
`'note' expects a note: :interrupt, :kill, :hangup, :alarm, :brk, or 1-31`	*n* is not a note
`'on-note' expects a function or NIL`	*f* is neither

SEE ALSO

`kill`, `wait`, `sleep`, `key`; the commands `kill` and `slay` (Chapter 2 of this book).

EXAMPLE

```
hylang> (def {got} {})
=> NIL
hylang> (on-note (fn {n} {do (set! {got} (cons n got)) T}))
=> NIL
hylang> (note (pid) :alarm)
=> NIL
hylang> (note (pid) 20)
=> NIL
hylang> got
=> {20 :alarm}
hylang> (def {k} (spawn "sleep" "30"))
=> NIL
hylang> (note k :interrupt)
=> NIL
hylang> (wait k)
=> 130
```

num?, int?, fixed?, rational?, complex?

Functions: is a value a number, and of which kind?

SYNTAX

```
(num? x)
(int? x)
(fixed? x)
(rational? x)
(complex? x)
```

ARGUMENTS

x

Any value.

RESULT

`T` or `NIL`:

FUNCTION	`T` WHEN *X* IS
`num?`	A number of any kind
`int?`	An integer
`fixed?`	A decimal, such as 1.5. 1.0 is one, though its value is whole
`rational?`	A fraction, such as 1/2
`complex?`	A complex number, with an imaginary part that isn't 0

DESCRIPTION

The four kinds go by the form the number is in, not by its value: 1.0 is `fixed?` and not `int?`, and 0.5 is not `rational?`, though it equals 1/2. Arithmetic gives each result in the simplest form that holds it (see `+`), so `(/ 4 2)` is an integer and `(/ 1 4)` is a fraction, and a complex number whose imaginary part comes to 0 is real.

Nothing here is an error: a value that is not a number gives `NIL`.

SEE ALSO

`type-of`, `zero?`, `positive?`, `to-fixed`, `to-rational`.

EXAMPLE

```
hylang> (num? 42)
=> T
hylang> (num? "42")
=> NIL
hylang> (int? 1.0)
=> NIL
hylang> (fixed? 1.0)
=> T
hylang> (rational? (/ 1 4))
=> T
hylang> (complex? 2i)
=> T
hylang> (filter num? {1 "a" 2/3 :b})
=> {1 2/3}
```

number-bytes, bytes-number

Functions: a number as the bytes it is stored in; and back.

SYNTAX

```
(number-bytes n)
(bytes-number b)
```

ARGUMENTS

n

A number of any kind.

b

A list of bytes (whole numbers from 0 to 255), or a buffer, holding one number in the stored format.

RESULT

`number-bytes` gives a list of bytes: the number in the Hydra's compact stored format, which every language shares (HyBASIC's `MKN$` and `CVN` write and read the same bytes). `bytes-number` gives the number back, exactly, in the kind it had.

DESCRIPTION

These are for keeping numbers in a file or a buffer, or passing them to a program in another language, without turning them to text. The format has a different length for different numbers: the integers from -64 to 63 take one byte, and a long integer takes as many as it needs.

ERRORS

MESSAGE	CAUSE
`'bytes-number': not a number in the stored format`	The bytes are not a whole stored number

SEE ALSO

`bytes`, `write-bytes`, `buffer`.

EXAMPLE

```
hylang> (number-bytes 0)
=> {0}
hylang> (number-bytes 258)
=> {129 2 1}
hylang> (bytes-number (number-bytes 258))
=> 258
hylang> (bytes-number (number-bytes 1/2))
=> 1/2
```

odd?, even?, positive?, negative?

Functions of the library, written in hylang: tests of a number, which give an error for anything else.

SYNTAX

```
(odd? n)
(even? n)
(positive? a)
(negative? a)
```

ARGUMENTS

n, a

A number.

RESULT

`T` or `NIL`. `odd?` is `T` when *n* leaves 1 on division by 2, and `even?` when it leaves 0; a number that is not whole is neither. `positive?` is `T` when *a* is above 0, and `negative?` when it is below: 0 is neither.

DESCRIPTION

`positive?` and `negative?` are `pos?` and `neg?` with a check: those two answer `NIL` for a value that is not a number, and these give an error.

ERRORS

MESSAGE	CAUSE
`Cannot perform 'odd?' on non-number`	`odd?` or `even?` (with its own name) of something that is not a number
`Only numbers can be positive`	`positive?` of something that is not a number (and `negative` for `negative?`)

SEE ALSO

`pos?`, `neg?`, `zero?`, `mod`.

EXAMPLE

```
hylang> (odd? 3)
=> T
hylang> (even? 3)
=> NIL
hylang> (odd? -3)
=> T
hylang> (filter even? (range 10))
=> {0 2 4 6 8}
hylang> (positive? 0)
=> NIL
```

open

A function: opens a file, giving a stream.

SYNTAX

```
(open path [mode])
```

ARGUMENTS

path

The file's name, a string. A name that doesn't start with `/` is looked for from the current directory.

mode

An atom: how the file will be used. Without it, `:read`.

MODE	THE FILE	THE STREAM
`:read`	Must be there	Is read from its start
`:write`	Is made, or emptied if it is there	Is written from its start
`:append`	Is made if it isn't there, and nothing in it is lost	Is written at its end
`:update`	Must be there. Nothing is emptied	Is read and written in place, from its start

RESULT

A stream: a value that stands for the open file and its current place. It prints as `<stream>`.

DESCRIPTION

A stream is passed to the reading functions (`read-line`, `read-byte`, `read-all`, `read-bytes`, `read-buffer`), the writing functions (`print-to`, `write-to`, `write-bytes`), `seek` and `tell`, and then to `close`. Each read or write moves the stream's place on by what was read or written.

Any file can be opened, a device's too: `(open "/dev/time")` is a stream on the clock.

A program should close what it opens: a task can have only so many files open at once. For a whole file at a time, `read-file`, `read-lines` and `write-file` open and close it themselves.

ERRORS

MESSAGE	CAUSE
path`: not found`	`:read` or `:update` of a file that isn't there, or a directory in the path that isn't
path`: is a directory`	The path names a directory. Use `ls` or `dir`
`'open' mode must be :read, :write, :append or :update, not :bogus`	Another atom was given as the mode
`'open' mode must be :read, :write, :append or :update`	The mode is not an atom
`'open' expects a String`	*path* is not a string

Other failures give the system's message after the path (Chapter 3 of this book lists them), and its code as the error's.

SEE ALSO

`close`, `read-line`, `print-to`, `seek`, `read-file`, `write-file`, `exists?`.

EXAMPLE

```
hylang> (def {w} (open "/ram/d.txt" :write))
=> NIL
hylang> (print-to w "hello" 42)
=> NIL
hylang> (close w)
=> NIL
hylang> (def {s} (open "/ram/d.txt"))
=> NIL
hylang> (read-line s)
=> "hello 42"
hylang> (close s)
=> NIL
hylang> (open "/ram/nothere")
=> Error: /ram/nothere: not found
```

output-of

A special form: collects what expressions print.

SYNTAX

```
(output-of x ...)
```

DESCRIPTION

The expressions are evaluated in order, with standard output going into a string in place of the screen. The string is the result; the expressions' own values are discarded. Line ends are in the string as they were printed.

EXAMPLE

```
hylang> (output-of (print "a") (print "b"))
=> "a\nb\n"
```

pc library

A library, loaded with `(use "pc")`: is the PC's file server there?

FUNCTIONS

FORM	WHAT IT DOES
`(pc?)`	`T` if the PC tool answers on the serial line, so that `/pc` can be used. If nothing answers within a second, `NIL`

DESCRIPTION

`/pc` is a folder, or a disk image, on the PC at the other end of the serial cable, served by the PC tool (section 9.9). Its files are ordinary files: `read-file`, `ls`, `open` and the rest work on them, and need no library. This one function lets a program find out, before it tries, whether the tool is running.

SEE ALSO

`exists?`, `ls`; section 9.9.

peek, poke, peek-word, poke-word

Functions: read and write the task's memory.

SYNTAX

```
(peek addr)
(poke addr byte)
(peek-word addr)
(poke-word addr n)
```

ARGUMENTS

addr

An address, 0 to 65,535 (`#xFFFF`).

byte

A whole number from 0 to 255.

n

A whole number from 0 to 65,535.

RESULT

`peek` gives the byte at the address, and `peek-word` the 16-bit word there: the byte at *addr* is its low half and the byte at *addr* + 1 its high half, as the 65C02 keeps a word. `poke` and `poke-word` store a byte, or a word in the same order, and give `NIL`.

DESCRIPTION

The memory is this task's own 64K, as it is while hylang runs: the task's RAM from `$0000` to `$7FFF`, the RAM bank it has chosen at `$8000`, a bank of hylang's own code at `$A000`, and the kernel's ROM and the I/O ports from `$E000` up (Appendix F). Other tasks' memory can't be reached.

Reading is always safe. Writing is not: hylang itself lives in this memory, with its stack, its variables and its values, and a `poke` into any of them is likely to stop it. `poke` is for the places a program knows to be its own, such as a block it has set aside, and for the hardware's registers. For a block of bytes to work in, a buffer, or a RAM bank from `bank-alloc`, is the safe choice.

ERRORS

MESSAGE	CAUSE
`'peek' was given a number out of its range`	The address, or the value, is too big (each function with its own name)
`'peek' expects a whole number`	An argument is negative, or not a whole number

SEE ALSO

`buffer`, `bank-read`, `lo`, `hi`, `word`, `hex`.

EXAMPLE

```
hylang> (hex (peek-word #xFFFC) 4)
=> "E000"
hylang> (hex (peek #xFFFC) 2)
=> "00"
hylang> (hex (peek #xFFFD) 2)
=> "E0"
```

(`$FFFC` holds the address the 65C02 starts from when it is reset.)

pid

A function: this task's number.

SYNTAX

```
(pid)
```

RESULT

The number of the task hylang is running in, 1 to 15. It is the number `ps` shows for it, and the name of its directory in `/proc`.

SEE ALSO

`task-info`, `ps`, `note`.

EXAMPLE

```
hylang> (pid)
=> 2
hylang> ((task-info (pid)) :name)
=> "hylang"
```

platform, hydra?

Functions: which system hylang is running on.

SYNTAX

```
(platform)
(hydra?)
```

RESULT

On the Hydra-16, `platform` gives the atom `:hydra`, and `hydra?` gives `T`.

DESCRIPTION

hylang is the Hydra's version of a language that also runs on a PC (danlang). A program meant for both asks these before it uses anything that only the Hydra has: its devices, its RAM banks, its system calls.

EXAMPLE

```
hylang> (platform)
=> :hydra
hylang> (hydra?)
=> T
```

print

A function: prints values, and ends the line.

SYNTAX

```
(print x ...)
```

ARGUMENTS

x

Any values. There may be none.

RESULT

`NIL`. The values are written to standard output with one space between them, and then a new line. With no arguments, only the new line.

DESCRIPTION

Each value is written as text meant for a person. A string is written as its characters, without quotes, and a character as itself. A number is written in the current base. A list is written in braces, and the strings and characters *inside* a list keep their quotes and backslashes, so that the items can be told apart.

`print` accepts an error, and prints it as the prompt would.

To write values as they would be typed, so that they can be read back, use `save`. To write without the new line or the spaces, use `write`. To write to a file, use `print-to`.

SEE ALSO

`write`, `save`, `print-to`, `format`, `output-of`.

EXAMPLE

```
hylang> (print "one" 2 {3})
one 2 {3}
=> NIL
hylang> (print "a" \b :c 1/2)
a b :c 1/2
=> NIL
hylang> (print {1 "two" \3})
{1 "two" \3}
=> NIL
```

print-to, write-to

Functions: `print` and `write`, to a stream.

SYNTAX

```
(print-to s x ...)
(write-to s x ...)
```

ARGUMENTS

s

A stream open for writing: `:write`, `:append` or `:update`.

x

Any values.

RESULT

`NIL`. `print-to` writes what `print` would show: the values with spaces between, then a line feed. `write-to` writes what `write` would: the values with nothing between or after them.

ERRORS

MESSAGE	CAUSE
`'print-to' expects a stream first`	*s* is not a stream (and `'write-to'`)
`Cannot write this stream`	The stream was opened with `:read`, or has been closed

SEE ALSO

`print`, `write`, `write-bytes`, `write-file`, `open`.

EXAMPLE

```
hylang> (def {w} (open "/ram/d.txt" :write))
=> NIL
hylang> (print-to w "hello" 42)
=> NIL
hylang> (write-to w "no" "nl")
=> NIL
hylang> (close w)
=> NIL
hylang> (read-file "/ram/d.txt")
=> "hello 42\nnonl"
```

proc library

A library, loaded with `(use "proc")`: a look inside any task.

FUNCTIONS

FORM	WHAT IT DOES
`(task-args task)`	Its program's arguments: a list of strings
`(task-cwd task)`	Its current directory: a string
`(task-env task)`	Its environment: a hash, each variable's name, a string, and its value, a string (several words with spaces between)
`(task-ns task)`	Its namespace: a list of strings, the `bind` and `mount` lines that made it
`(task-regs task)`	The processor's registers as they were when the task last stopped running: a hash of numbers, `:pc`, `:a`, `:x`, `:y`, `:s`, `:p`, `:w`, `:u`, `:ram` and `:rom`
`(task-mem task addr n)`	*n* bytes of the task's memory from *addr*, as the task sees its 64K: a list
`(task-ram task bank offset n)`	*n* bytes of one of the task's RAM banks, from *offset*: a list

ARGUMENTS

task

A task's number.

DESCRIPTION

The library reads the files of `/proc` (section 17.8). With `ps` and `task-info`, which are always there, it is what a program needs to watch another: the debugger `db` reads the same files.

The registers and the memory of the kernel's task and of the drivers can't be read.

ERRORS

MESSAGE	CAUSE
path`: not found`	There is no such task
`not allowed`	`task-regs`, `task-mem` or `task-ram` of the kernel's task, or a driver's

SEE ALSO

`ps`, `task-info`, `pid`, `peek`; section 17.8.

EXAMPLE

```
hylang> (use "proc")
=> NIL
hylang> (task-args 1)
=> {"-l"}
hylang> (task-cwd (pid))
=> "/"
hylang> ((task-env (pid)) "prog")
=> "/bin/hylang"
hylang> (fst (task-ns (pid)))
=> "bind '#/' /"
hylang> (hex ((task-regs 1) :pc) 4)
=> "EAA0"
```

qexpr?, sexpr?, function?, error?

Functions: is a value a list, an unevaluated call, a function, or an error?

SYNTAX

```
(qexpr? x)
(sexpr? x)
(function? x)
(error? x)
```

ARGUMENTS

x

Any value.

RESULT

`T` or `NIL`:

FUNCTION	`T` WHEN *X* IS
`qexpr?`	A list, the empty one too
`sexpr?`	An S-expression that has not been evaluated: a call, as data
`function?`	A function: one made with `fn` or `fun`, a built-in one, or a special form
`error?`	An error

DESCRIPTION

`error?` can be given an error: it answers `T`, where another function would not run, and would pass the error on. With `try` it is one of the two ways to deal with an error.

An S-expression is only seen as one when it comes out of a list unevaluated, or from `read`.

There is a test for buffers too, `buffer?`.

SEE ALSO

`type-of`, `try`, `error`, `eval`.

EXAMPLE

```
hylang> (qexpr? {1})
=> T
hylang> (qexpr? "a")
=> NIL
hylang> (sexpr? (fst {(+ 1 2)}))
=> T
hylang> (function? len)
=> T
hylang> (error? (error "x"))
=> T
hylang> (error? 5)
=> NIL
```

random

A function: a random whole number.

SYNTAX

```
(random n)
```

ARGUMENTS

n

A whole number, 1 or more. It may be as large as any integer.

RESULT

A whole number from 0 to *n* - 1, chosen at random. `(random 6)` is one of 0 to 5: add 1 for the throw of a die.

ERRORS

MESSAGE	CAUSE
`'random' expects a number above 0`	*n* is 0 or negative
`'random' expects an integer`	*n* is not a whole number

EXAMPLE

The answers differ from run to run:

```
hylang> (random 6)
=> 2
hylang> (1+ (random 6))
=> 4
```

range

A function: a list of evenly spaced numbers.

SYNTAX

```
(range to)
(range from to [step])
```

ARGUMENTS

from

The first number. With one argument, 0.

to

Where to stop. *to* itself is never in the list.

step

The difference between one number and the next: 1 if it isn't given. It may be negative, to count down, and it may be a fraction or a decimal, as *from* and *to* may be.

RESULT

A list of *from*, *from* + *step*, and so on, for as long as the number is before *to*: less than it for a positive step, greater for a negative one. If the first number is already at or past *to*, the result is `NIL`.

DESCRIPTION

`(range n)` is the numbers 0 to *n* - 1: *n* of them, which are also the positions of a list of *n* items. The arithmetic is exact, so a fractional step never drifts.

ERRORS

MESSAGE	CAUSE
`'range' step cannot be 0`	*step* is 0
`'range' expects Numbers`	An argument is not a number
`'range' takes at most 3 arguments, not 4`	Too many arguments

SEE ALSO

`dotimes`, `map`.

EXAMPLE

```
hylang> (range 5)
=> {0 1 2 3 4}
hylang> (range 2 6)
=> {2 3 4 5}
hylang> (range 5 0 -2)
=> {5 3 1}
hylang> (range 0 1 1/4)
=> {0 1/4 1/2 3/4}
hylang> (range 5 0)
=> NIL
```

rational.n, rational.d

Functions: the top and the bottom of a fraction.

SYNTAX

```
(rational.n x)
(rational.d x)
```

ARGUMENTS

x

A real number.

RESULT

`rational.n` gives the numerator and `rational.d` the denominator of *x* written as a fraction in its lowest terms. The sign is the numerator's. An integer's denominator is 1, and a decimal is turned to a fraction first: 0.75 gives 3 and 4.

ERRORS

MESSAGE	CAUSE
`'rational.n' expects a real number`	*x* is complex, or not a number (and `'rational.d'`)

EXAMPLE

```
hylang> (rational.n 3/4)
=> 3
hylang> (rational.d 3/4)
=> 4
hylang> (rational.d 5)
=> 1
hylang> (rational.n 0.75)
=> 3
```

read

A function: the expressions that a string of text holds.

SYNTAX

```
(read text)
```

ARGUMENTS

text

A string of hylang source.

RESULT

A list of the expressions in the text, in order, as the reader makes them and with nothing evaluated: a call written in parentheses is an S-expression in the list. Empty text gives `NIL`.

DESCRIPTION

`read` is the reader on its own: the first half of what `load` and the prompt do. `eval` is the second half. Given the list, `eval` runs the expressions in turn and gives the last one's value; given one item of it, `eval` runs that one.

ERRORS

MESSAGE	CAUSE
`'read': missing )`	The text ends inside an expression (the message names what is missing)
`'read' expects a String`	*text* is not a string

SEE ALSO

`eval`, `load`, `val`, `repr`.

EXAMPLE

```
hylang> (read "(+ 1 2)")
=> {(+ 1 2)}
hylang> (read "1 2 3")
=> {1 2 3}
hylang> (eval (read "(+ 1 2)"))
=> 3
hylang> (read "(+ 1")
=> Error: 'read': missing )
```

read-buffer

A function: reads bytes from a stream into a buffer.

SYNTAX

```
(read-buffer s b [at [n]])
```

ARGUMENTS

s

A stream open for reading.

b

The buffer to read into.

at

Where in the buffer the first byte goes. Without it, 0.

n

The most bytes to read. Without it, as many as fit from *at* to the end of the buffer.

RESULT

The number of bytes read, which is less than was asked for when the file has fewer left; and `NIL` at the end of the file. The bytes are in the buffer from *at* on, and the rest of the buffer is as it was.

DESCRIPTION

This is the way to read a file a block at a time without making a new value for each block: one buffer is used over and over, and the loop ends when the result is `NIL`. `write-bytes` writes a buffer, or a part of one, back to a stream.

ERRORS

MESSAGE	CAUSE
`'read-buffer' expects a stream`	*s* is not a stream
`Cannot read from this stream`	The stream is not open for reading

SEE ALSO

`read-bytes`, `write-bytes`, `open`, `buffer`.

EXAMPLE

```
hylang> (write-file "/ram/bf" "0123456789")
=> NIL
hylang> (def {s} (open "/ram/bf"))
=> NIL
hylang> (def {r} (buffer 4))
=> NIL
hylang> (read-buffer s r)
=> 4
hylang> r
=> <buffer>{48 49 50 51}
hylang> (read-buffer s r)
=> 4
hylang> (read-buffer s r)
=> 2
hylang> (read-buffer s r)
=> NIL
hylang> (close s)
=> NIL
```

read-bytes, write-bytes

Functions: read and write a stream's bytes as numbers.

SYNTAX

```
(read-bytes s n)
(write-bytes s bytes)
(write-bytes s buffer [i [n]])
```

ARGUMENTS

s

A stream, open for reading (`read-bytes`) or for writing (`write-bytes`).

n

How many bytes: a whole number, 0 or more.

bytes

A list of whole numbers from 0 to 255.

buffer

A buffer. With *i*, its bytes from position *i* on are written; with *n* too, only *n* of them.

RESULT

`read-bytes` gives a list of the next *n* bytes. Near the end of the file the list is shorter than was asked for, and at the end it is `NIL`.

`write-bytes` gives `NIL`. The bytes are written as they are, with nothing added: this is how a file that is not text is made.

ERRORS

MESSAGE	CAUSE
`A count must be an integer`	*n* is negative, or not a whole number
`'write-bytes' expects a list of bytes`	The second argument is neither a list nor a buffer. For a string's bytes, use `write-to`, or `(bytes s)`
`256 isn't a byte`	An item of the list is not a whole number from 0 to 255
`'write-bytes' takes a part (i n) only of a buffer`	*i* was given with a list
`Cannot write this stream`	The stream is not open for writing

SEE ALSO

`read-byte`, `read-buffer`, `bytes`, `buffer`, `number-bytes`.

EXAMPLE

```
hylang> (def {w} (open "/ram/data" :write))
=> NIL
hylang> (write-bytes w {72 105 33})
=> NIL
hylang> (close w)
=> NIL
hylang> (def {s} (open "/ram/data"))
=> NIL
hylang> (read-bytes s 2)
=> {72 105}
hylang> (read-bytes s 100)
=> {33}
hylang> (read-bytes s 100)
=> NIL
hylang> (close s)
=> NIL
```

read-file, read-lines

Functions: a whole file, as one string or as a list of lines.

SYNTAX

```
(read-file path)
(read-lines path)
```

ARGUMENTS

path

The file's name, a string.

RESULT

`read-file` gives the file's contents as one string, line ends and all. `read-lines` gives a list of its lines, each a string without its line end; a carriage return before a line feed is dropped, and a last line with no line end still counts. An empty file is `""`, and `NIL`.

DESCRIPTION

The file is opened, read and closed in the one call. The whole of it must fit in hylang's memory: for a large file, `open` it and read it a line at a time.

Devices are files, so these read them too: `(read-file "/dev/time")` is the date and time.

ERRORS

MESSAGE	CAUSE
path`: not found`	There is no such file (and the system's other messages, for its other errors)
`'read-file' expects a path`	*path* is not a string

SEE ALSO

`write-file`, `open`, `read-line`, `str-split`, `load`.

EXAMPLE

```
hylang> (write-file "/ram/a.txt" "line one\nline two\n")
=> NIL
hylang> (read-file "/ram/a.txt")
=> "line one\nline two\n"
hylang> (read-lines "/ram/a.txt")
=> {"line one" "line two"}
hylang> (read-lines "/rom/lib/shell")
=> {"/bin/forth -l"}
hylang> (read-file "/ram/nothere")
=> Error: /ram/nothere: not found
```

read-line, read-byte, read-all

Functions: read from the keyboard, or from a stream.

SYNTAX

```
(read-line [s])
(read-byte [s])
(read-all [s])
```

ARGUMENTS

s

A stream open for reading (see `open`). Without it, standard input: the keyboard, unless hylang's input was redirected.

RESULT

FUNCTION	RESULT	AT THE END
`read-line`	The next line, as a string, without the line end that closed it	`NIL`
`read-byte`	The next byte, as a number from 0 to 255	`NIL`
`read-all`	Everything from the current place to the end, as one string	`""`

DESCRIPTION

A line ends at a line feed, and a carriage return before it is dropped. The last line of a file is given whether or not it ends with one.

Because `read-line` gives `NIL` only at the end, and an empty line is `""`, which is true, a loop can read until `NIL`. From the keyboard, the end is CTRL-D at the start of a line.

The keyboard gives its characters a line at a time, when ENTER is pressed, so `read-byte` from the keyboard waits for a whole line and then gives its bytes one by one, the line feed last. For keys one at a time as they are pressed, use `key`.

ERRORS

MESSAGE	CAUSE
`Cannot read from this stream`	The stream was opened for writing only, or has been closed

SEE ALSO

`open`, `read-file`, `read-lines`, `read-bytes`, `key`.

EXAMPLE

```
hylang> (write-file "/ram/a.txt" "line one\nline two\n")
=> NIL
hylang> (def {s} (open "/ram/a.txt"))
=> NIL
hylang> (read-line s)
=> "line one"
hylang> (read-byte s)
=> 108
hylang> (read-all s)
=> "ine two\n"
hylang> (read-line s)
=> NIL
hylang> (close s)
=> NIL
```

remove-fst, remove-all, remove-min, remove-max

Functions of the library, written in hylang: a list without some of its items.

SYNTAX

```
(remove-fst x l)
(remove-all x l)
(remove-min l)
(remove-max l)
```

ARGUMENTS

x

A value to look for. It is a value, not a test: items are compared with it as == compares.

l

A list.

RESULT

A new list. `remove-fst` leaves out the first item equal to *x*, and `remove-all` every item equal to it; if there is none, the list is returned as it was. `remove-min` leaves out the least item, and `remove-max` the greatest: one of them, the first, if several are equal.

To remove the items that pass a test, use `filter` with the opposite test.

SEE ALSO

`filter`, `distinct`, `minimum`, `drop`.

EXAMPLE

```
hylang> (remove-fst 2 {1 2 3 2})
=> {1 3 2}
hylang> (remove-all 2 {1 2 3 2})
=> {1 3}
hylang> (remove-min {3 1 2 1})
=> {3 2 1}
hylang> (remove-max {3 1 4 4})
=> {3 1 4}
```

repeat, times

Functions of the library, written in hylang: a list made by evaluating something several times.

SYNTAX

```
(repeat n d)
(times n d)
```

ARGUMENTS

n

How many items: a whole number, 0 or more.

d

What each item is to be. It is evaluated afresh for each item, as `eval` evaluates: a Q-expression is run as code, and anything else is itself.

RESULT

A list of *n* items; `NIL` for 0. `times` is another name for `repeat`.

DESCRIPTION

Because *d* is evaluated each time, a Q-expression that calls `random`, or reads a line, gives a different item each time: `(repeat 3 {random 6})` is three throws of a die. A number or a string simply appears *n* times.

To have a list itself as each item, wrap it once more: `(repeat 2 {{1 2}})`.

SEE ALSO

`range`, `dotimes`, `map`, `str-repeat`.

EXAMPLE

```
hylang> (repeat 3 7)
=> {7 7 7}
hylang> (repeat 2 {+ 1 2})
=> {3 3}
hylang> (repeat 2 {{1 2}})
=> {{1 2} {1 2}}
hylang> (len (repeat 5 {random 6}))
=> 5
```

reverse

A function: a list, or a string, backwards.

SYNTAX

```
(reverse x)
```

ARGUMENTS

x

A list or a string.

RESULT

A list of the same items in the opposite order, or a string of the same characters in the opposite order. Only the outer list is reversed: a list inside it keeps its own order.

ERRORS

MESSAGE	CAUSE
`'reverse' expects a QExpr or a String`	*x* is neither

EXAMPLE

```
hylang> (reverse {1 2 3})
=> {3 2 1}
hylang> (reverse {1 {2 3} 4})
=> {4 {2 3} 1}
hylang> (reverse "hello")
=> "olleh"
```

run, spawn, wait, kill

Functions: run a program and wait for it; start one without waiting; wait for one that was started; end one.

SYNTAX

```
(run prog x ...)
(spawn prog x ...)
(wait task)
(kill task)
```

ARGUMENTS

prog

The program: a path, or a bare name, which is looked for in `/bin`.

x

The program's arguments. Each value is turned to text as `print` shows it, and is one argument, whatever spaces it holds.

task

A task's number, as `spawn` gave it.

RESULT

`run` starts the program, waits until it ends, and gives its exit code: 0 for success, by the system's custom, and something else for a failure.

`spawn` starts the program and returns at once with its task number. The program runs alongside hylang, with the same standard input and output, so what it prints appears on the same screen.

`wait` waits until the task ends, and gives its exit code. Each task can be waited for once, and only by the program that started it.

`kill` sends the task the kill note, which ends it, and gives `NIL`. The exit code that `wait` then gives is 137 (128 + 9, the kill note's number).

DESCRIPTION

The program is started directly, with no shell: no pattern is expanded and nothing is redirected. For a command line with pipes, patterns or redirection, use `sh`.

A program that reports its own failure still ran, so `run` gives its exit code. `run` gives an *error* only when the program could not be started.

ERRORS

MESSAGE	CAUSE
prog`: not found`	There is no such program
`'run' expects a path`	*prog* is not a string
`A task must be an integer`	*task* is not a whole number
task`: no such child`	`wait` for a task that this program did not start, or has already waited for
task`: no such task`	`kill` of a task that isn't there

SEE ALSO

`sh`, `sh-out`, `note`, `pid`, `ps`.

EXAMPLE

```
hylang> (run "echo" "hello" "there")
hello there
=> 0
hylang> (run "ls" "/nothere")
ls: /nothere: not found
=> 1
hylang> (run "nothere")
=> Error: nothere: not found
hylang> (def {tk} (spawn "sleep" "60"))
=> NIL
hylang> (kill tk)
=> NIL
hylang> (wait tk)
=> 137
```

save

A function: writes a value as it would be typed, to the screen or to a file.

SYNTAX

```
(save x)
(save x path [mode])
```

ARGUMENTS

x

Any value.

path

A file to write to. Without it, the value is printed.

mode

What to do if the file is already there: `:overwrite`, to replace its contents, or `:append`, to add to them.

RESULT

`NIL`.

DESCRIPTION

The value is written in the form that reads back as the same value: a string with its quotes and escapes, a character with its backslash, a hash as `#(...)`, and a new line after it. This is the opposite of `read` and `load`: a file of saved values is a file of hylang source, and `load` gives the value of the last one in it.

For a file that does not exist, no mode is needed. For one that does, a mode must be given, so that a file is never replaced by accident.

ERRORS

MESSAGE	CAUSE
`File exists but neither ':append' nor ':overwrite' were specified`	The file is there, and no mode was given
The system's message, after the path	The file could not be made or written

SEE ALSO

`print`, `repr`, `load`, `read`, `write-file`.

EXAMPLE

```
hylang> (save "a string")
"a string"
=> NIL
hylang> (save {1 "two" \3 :four 5/6})
{1 "two" \3 :four 5/6}
=> NIL
hylang> (save {1 2} "/ram/saved.hl")
=> NIL
hylang> (save {3 4} "/ram/saved.hl" :append)
=> NIL
hylang> (read-file "/ram/saved.hl")
=> "{1 2}\n{3 4}\n"
hylang> (load "/ram/saved.hl")
=> {3 4}
```

screen library

A library, loaded with `(use "screen")`: the terminal's cursor, colours and clearing.

FUNCTIONS

FORM	WHAT IT DOES
`(cls)`	Clears the screen, and puts the cursor at its top left
`(at col row)`	Moves the cursor: columns and rows count from 1, from the top left
`(clear-line)`	Clears from the cursor to the end of its line
`(color fg [bg])`	Sets the colour of the text written from now on, and its background's if given: each one of the atoms `:black`, `:red`, `:green`, `:yellow`, `:blue`, `:magenta`, `:cyan`, `:white`
`(bold)`, `(plain)`	Bold text; and back to plain text in the usual colours
`(cursor-off)`, `(cursor-on)`	Hides the cursor; shows it

DESCRIPTION

Each writes one of the terminal's control sequences to standard output (section 13.7 lists them), so they work on the Hydra's console, in any of its windows, and on a PC's terminal alike. They act on the window hylang is writing to.

What they set stays set after the program ends: a program that changes the colour or hides the cursor should finish with `(plain)` and `(cursor-on)`.

SEE ALSO

`write`, `key`, the `cons` library; section 13.7.

EXAMPLE

A title in the middle of a cleared screen:

```
(use "screen")
(cls)
(at 30 12)
(color :yellow :blue)
(write " The Hydra-16 ")
(plain)
(at 1 24)
```

seek, tell

Functions: set, and ask, a stream's place in its file.

SYNTAX

```
(seek s n)
(tell s)
```

ARGUMENTS

s

A stream.

n

The place to move to: a number of bytes from the start of the file, from 0 to the file's length.

RESULT

Both give the stream's place, as a number of bytes from the start: for `seek`, the place it has just set. The next read or write happens there.

DESCRIPTION

A place is counted in bytes, the first being 0. `(seek s 0)` goes back to the start. To find a file's length, ask `stat`.

With a stream opened `:update`, `seek` is how a part of a file is changed in place: writing there replaces the bytes that were there, and does not move the ones after.

ERRORS

MESSAGE	CAUSE
`Cannot seek past end of stream`	*n* is more than the file's length
`Cannot seek before beginning of stream`	*n* is negative
`'seek' expects a stream`	*s* is not a stream

SEE ALSO

`open`, `stat`, `read-bytes`.

EXAMPLE

```
hylang> (write-file "/ram/d.txt" "hello 42\n")
=> NIL
hylang> (def {u} (open "/ram/d.txt" :update))
=> NIL
hylang> (seek u 6)
=> 6
hylang> (read-line u)
=> "42"
hylang> (tell u)
=> 9
hylang> (seek u 0)
=> 0
hylang> (write-to u "J")
=> NIL
hylang> (close u)
=> NIL
hylang> (read-file "/ram/d.txt")
=> "Jello 42\n"
```

seg-create, seg-attach, seg-detach, seg-read, seg-write

Functions: shared memory segments, which several tasks can use together.

SYNTAX

```
(seg-create banks)
(seg-attach seg)
(seg-detach seg)
(seg-read seg bank offset n)
(seg-write seg bank offset bytes)
```

ARGUMENTS

banks

The segment's size, in 8K banks: 1 to 128.

seg

A segment's number, as `seg-create` gave it.

bank
: Which of the segment's banks, the first being 0.

offset
: A position in that bank, 0 to 8,191.

n
: A number of bytes.

bytes
: A list of whole numbers from 0 to 255.

RESULT

`seg-create` makes a segment of the shared RAM and gives its number. `seg-attach` and `seg-detach` join the task to a segment and take it away again, and give `NIL`. `seg-read` gives a list of *n* bytes from one of the segment's banks, and `seg-write` stores bytes there and gives `NIL`. As with a task's own banks, a read or a write must stay inside one bank.

DESCRIPTION

A **segment** is a piece of the Hydra's shared RAM (section 17.9). Unlike a task's own banks, a segment can be attached by any task that knows its number, so it is how two programs share a large amount of data without copying it through a pipe: one creates the segment and passes its number, as an argument or in a file, and both attach it. A segment lasts while any task has it attached, or while it has a name in `/dev/seg`.

A task must be attached to a segment to read or write it. The task that creates one is attached to it already; any other attaches it with `seg-attach`.

ERRORS

MESSAGE	CAUSE
seg`: invalid argument`	A read or a write of a segment that isn't attached
seg`: out of range`	*bank* is not one of the segment's banks
`'seg-read' was given a number out of its range`	The bytes would run past the bank's end; or a number is too big for what it is
banks`: invalid argument`	`seg-create` of no banks

SEE ALSO

`bank-alloc`, `free`; section 17.9.

EXAMPLE

The segment's number depends on what is already in use:

```
hylang> (def {sg} (seg-create 1))
=> NIL
hylang> sg
=> 14
hylang> (seg-attach sg)
=> NIL
hylang> (seg-write sg 0 0 {1 2 3})
=> NIL
hylang> (seg-read sg 0 0 3)
=> {1 2 3}
hylang> (seg-detach sg)
=> NIL
```

set

A special form: binds names in the current scope.

SYNTAX

```
(set {name ...} value ...)
```

DESCRIPTION

As `def`, but the binding is made in the scope the `set` is in: inside a function or a `let`, the name is local to it, and a global of the same name is hidden, not changed. At the prompt, where the current scope is the global one, `set` and `def` are the same.

If the name is already bound in the current scope, it is bound again to the new value. A name bound in an enclosing scope is not touched: `set` makes a new one that hides it. To change the outer one, use `set!`.

The result is `NIL`.

EXAMPLE

```
hylang> (def {a} 5)
=> NIL
hylang> (fun {local} {do (set {a} 99) a})
=> NIL
hylang> (list (local) a)
=> {99 5}
```

set!

A special form: changes names where they are already bound.

SYNTAX

```
(set! {name ...} value ...)
```

DESCRIPTION

As `def`, but each name must be bound already, and it is changed in whichever scope holds it: the nearest one, looking outward from the current scope. This is how a function changes a variable that belongs to the code around it, or a global.

The result is `NIL`.

ERRORS

MESSAGE	CAUSE
`Unbound Symbol '`*name*`'`	The name is not bound in any scope

EXAMPLE

```
hylang> (def {total} 0)
=> NIL
hylang> (fun {add-to-total n} {set! {total} (+ total n)})
=> NIL
hylang> (do (add-to-total 5) (add-to-total 7) total)
=> 12
hylang> (set! {nosuch} 1)
=> Error: Unbound Symbol 'nosuch'
```

sh, sh-out

Functions: run a command line with the shell; and the same, giving what it printed.

SYNTAX

```
(sh line [input])
(sh-out line [input])
```

ARGUMENTS

line

A command line, as it would be typed at rc's prompt: pipes, patterns, redirection and variables all work.

input

Text to give the command as its standard input. Without it, the command reads what hylang itself would: the keyboard.

RESULT

`sh` runs the line (with `rc -c`) and gives its exit status, a number: 0 for success. What the command prints goes to the screen.

`sh-out` runs it the same way but collects what it writes to standard output, and gives that as a string, line ends included. What it writes to standard error still goes to the screen, and its exit status is not given.

DESCRIPTION

These are the bridge to everything else on the Hydra: any program, and any pipeline of them, can do part of a hylang program's work. `str-split` with `"\n"`, or `read-lines` on a file the command wrote, turns the output into a list.

The command gets a copy of hylang's environment, so a variable set with `setenv` is `$name` in the line.

SEE ALSO

`run`, `output-of`, `setenv`, `str-split`.

EXAMPLE

```
hylang> (sh "echo hi | wc -c")
      3
=> 0
hylang> (sh "ls /nothere")
ls: /nothere: not found
=> 1
hylang> (sh-out "echo hi")
=> "hi\n"
hylang> (sh-out "cat" "from input")
=> "from input"
hylang> (str-split (str-trim (sh-out "ls /rom/lib | head -3")) "\n")
=> {"as/" "basic/" "edit/"}
```

shl, shr, bit?

Functions: shift an integer's bits left or right; and test one bit.

SYNTAX

```
(shl n k)
(shr n k)
(bit? n k)
```

ARGUMENTS

n

An integer.

k

For the shifts, how many places: an integer. For `bit?`, which bit, bit 0 being the lowest.

RESULT

`shl` gives *n* shifted left *k* places, which is *n* times 2 to the power *k*; nothing is lost off the top, because integers grow. `shr` gives *n* shifted right *k* places, which is *n* divided by 2 to the power *k*, rounded down (toward minus infinity, so the sign is kept). A negative *k* shifts the other way.

`bit?` gives `T` if bit *k* of *n* is 1, and `NIL` if it is 0. A negative number is two's complement, with all the bits above its top one set.

ERRORS

MESSAGE	CAUSE
`'shr''s count must be an integer`	*k* is not an integer (each function with its own name)

EXAMPLE

```
hylang> (shl 1 4)
=> 16
hylang> (shr 256 4)
=> 16
hylang> (shr -8 1)
=> -4
hylang> (shl 1 100)
=> 1267650600228229401496703205376
hylang> (bit? 5 0)
=> T
hylang> (bit? 5 1)
=> NIL
```

sleep, sleep-until, yield

Functions: wait for a time; wait until a tick; and give other tasks a turn.

SYNTAX

```
(sleep s)
(sleep-until tick)
(yield)
```

ARGUMENTS

s

A number of seconds, 0 or more. It may be a fraction or a decimal: `(sleep 1/10)`.

tick

A value of the tick count, as `ticks` gives it, at most 32,767 ticks ahead.

RESULT

Each gives `NIL`, when the wait is over. While hylang sleeps it uses none of the processor's time, and the other tasks run.

`sleep-until` waits until the tick count reaches *tick*. A loop that adds a fixed step to a tick value and sleeps until it keeps an even beat, however long each turn of the loop takes: `sleep`, which waits from when it is called, would drift.

`yield` does not wait: it ends hylang's turn at once, so that any other task that is ready runs before hylang goes on.

DESCRIPTION

A note that arrives during a sleep ends it early. CTRL-C does: the sleep's result is then the error `interrupted`.

ERRORS

MESSAGE	CAUSE
`'sleep' expects a number of seconds`	*s* is negative, or not a number

SEE ALSO

`ticks`, `clock`, `on-note`, `key?`.

EXAMPLE

```
hylang> (sleep 1/2)
=> NIL
hylang> (dotimes {i 3} (do (write i " ") (sleep 0.1)))
0 1 2 => NIL
```

snd library

A library, loaded with `(use "snd")`: notes and sounds from the YM2151 and, with a Vera X, its PSG.

FUNCTIONS

FORM	WHAT IT DOES
`(snd-claim ch ...)`, `(snd-release ch ...)`	Claims channels, so that no other program's sounds reach them; and gives them back
`(snd-patch ch n)`	Loads an instrument into the channel: one of the 163 patches, 0 to 162
`(snd-note ch note)`	Starts a note: a MIDI note number, 60 being middle C
`(snd-off ch)`	Ends the channel's note (it dies away as its instrument does)
`(snd-level ch v)`	The channel's own volume: 0 to 127
`(snd-pan ch pan)`	Which speakers: `:left`, `:right` or `:both`
`(snd-bend ch n)`	Bends the pitch, by *n* 64ths of a semitone, up or down
`(snd-drum ch n)`	Plays a drum: a General MIDI drum number (35 is a kick)
`(snd-freq ch hz)`	Starts a note at a frequency, in hertz; 0 ends it
`(snd-glide ch note)`	Moves the sounding note to another pitch without starting it again
`(snd-wave ch w [width])`	For a PSG channel: its waveform, `:pulse`, `:saw`, `:triangle` or `:noise`, and a pulse's width, 0 to 63
`(snd-volume v)`	The master volume for all sound: 0 to 200, 100 being as written
`(snd-reset)`	Silences the chip and clears every setting
`(snd-lfo rate pmd amd wave)`	The YM2151's low-frequency oscillator: its rate, its depths of pitch and of loudness (0 to 127), and its wave (0 saw, 1 square, 2 triangle, 3 noise)
`(snd-sens ch pms ams)`	How much the oscillator moves a channel: its pitch (0 to 7) and its loudness (0 to 3)
`(snd-noise n)`	Noise on channel 7, at frequency 0 to 31; a negative number turns it off
`(snd-reg reg value ...)`	Writes the chip's own registers: a register and its value, in pairs
`(snd-regs)`	The chip's 256 registers as they were last written: a buffer
`(note-of name)`	A note's MIDI number from its name: `"C4"` is 60, `"C#4"` and `"Db4"` 61, `"A4"` 69
`(tune notes [ch [tempo]])`	Plays a list of notes, each `{note beats}`, in time. A note is a name, a number, or `NIL` for a rest. The channel is 0 and the tempo 120 beats a minute if they aren't given
`(play path [times])`	Plays a song file (ZSM) with the `play` program: its exit code
`(snd-mml ch mml)`, `(snd-chord ch notes)`	Plays a line of the score language (MML) on a channel, or its notes together as a chord, with the `play` program: its exit code

ARGUMENTS

ch

A channel: 0 to 7 are the YM2151's eight voices, and 8 to 23, with a Vera X, the PSG's sixteen.

DESCRIPTION

The library works through the sound driver's files, `/dev/snd` and `/dev/sndctl` (Chapter 15 describes the driver, its channels, the patches and MML). Each function gives `NIL` unless the table says otherwise.

A note sounds until it is ended: `snd-note`, a `sleep`, and `snd-off` make one note of a chosen length. `tune` does that for a whole list, keeping time by the system's tick.

A channel a program has claimed is its own until it is released or the program ends. `play`, `snd-mml` and `snd-chord` run another program, which needs the channel free: release it first, or they report `channels busy`.

ERRORS

MESSAGE	CAUSE
`'snd-pan' expects :left, :right or :both`	Another value for the speakers
`'note-of' expects a note: C4, C#4, Db4 ...`	A name that is not a note's

SEE ALSO

Chapter 15; section 22.6; the command `play` (Chapter 2 of this book).

EXAMPLE

```
hylang> (use "snd")
=> NIL
hylang> (note-of "A4")
=> 69
hylang> (snd-patch 0 5)
=> NIL
hylang> (snd-note 0 (note-of "C4"))
=> NIL
hylang> (sleep 1/2)
=> NIL
hylang> (snd-off 0)
=> NIL
hylang> (tune {{"C4" 1} {"E4" 1} {NIL 1} {"G4" 2}} 0 240)
=> NIL
```

sort

A function: a list in order.

SYNTAX

```
(sort l [less])
```

ARGUMENTS

1

A list.

less

A function of two items, which returns a true value when the first must come before the second. Without it, the items are put in `cmp`'s order.

RESULT

A new list of the same items in order. *l* is not changed.

DESCRIPTION

Without *less* the order is that of `cmp`: by kind first (numbers before characters, then strings, atoms, symbols and lists), then by value. Numbers are compared by value whatever their form, strings character by character, and lists item by item, a list that is the start of another coming first. So a list of mixed kinds sorts without an error.

The sort is a merge sort, and it is stable: items that *less* does not put in order keep the order they had. To sort records by one field, give a *less* that compares that field.

ERRORS

MESSAGE	CAUSE
`'sort' expects a QExpr`	*l* is not a list (a string is not one)
`Second 'sort' parameter must be a function`	*less* is not a function

SEE ALSO

`cmp`, `min`, `max`, `sort-desc`.

EXAMPLE

```
hylang> (sort {3 1 2})
=> {1 2 3}
hylang> (sort {3 1 2} >)
=> {3 2 1}
hylang> (sort {"pear" "apple" "fig"})
=> {"apple" "fig" "pear"}
hylang> (sort {3 "a" 1})
=> {1 3 "a"}
hylang> (sort {{2 "b"} {1 "a"}} (fn {a b} {< (fst a) (fst b)}))
=> {{1 "a"} {2 "b"}}
```

spi library

A library, loaded with `(use "spi")`: the sixteen SPI devices.

FUNCTIONS

FORM	WHAT IT DOES
`(spi d bytes)`	One transaction: the device is selected, the bytes (a list, or a string's) are sent, and the device is let go. The result is a list of the bytes the device sent back meanwhile, one for each byte sent
`(spi-read d n)`	Clocks in *n* bytes, sending `$FF` for each: a list
`(spi-mode d mode)`	The clock's mode for the device: 0 (the clock rests low) or 3 (it rests high). `NIL`

ARGUMENTS

d

The device, 0 to 15: 0 to 7 are the board's headers J18 to J25, and 8 to 15 the slots' cards.

DESCRIPTION

The library works through the files of `/dev/spi` (section 17.4). A device with nothing connected sends back `$FF`, 255, for every byte.

One program at a time can use a device, and a device that is an SD card the system has started can't be used this way at all: the error is `busy`.

ERRORS

MESSAGE	CAUSE
`invalid argument`	A mode that is not 0 or 3
`busy`	Another program has the device, or it is a card's
path`: not found`	A device that is not 0 to 15

SEE ALSO

Section 17.4; section 14.2; the `dev` library.

EXAMPLE

With nothing connected to device 3:

```
hylang> (use "spi")
=> NIL
hylang> (spi-mode 3 0)
=> NIL
hylang> (spi 3 {1 2 3})
=> {255 255 255}
hylang> (spi-read 3 4)
=> {255 255 255 255}
```

split, zip, flatten, distinct, avg

Functions of the library, written in hylang: five more ways to work on lists.

SYNTAX

```
(split n l)
(zip a b)
(flatten l)
(distinct l)
(avg l)
```

ARGUMENTS

l, a, b

Lists.

n

A count, 0 or more.

RESULT

FUNCTION	RESULT
`split`	A list of two lists: the first *n* items of *l*, and the rest. It is `(list (take n l) (drop n l))`
`zip`	A list of pairs: the first items of *a* and *b* together, then the second, and so on. It is as long as the shorter of the two
`flatten`	One list of all the items, with every list inside *l* opened, however deep
`distinct`	The list without its repeats: each value appears once, where it first appeared
`avg`	The average of a list of numbers: their sum over their count, exactly

DESCRIPTION

These are written in hylang, and call themselves once for each item, so they are for lists of modest length: tens or hundreds of items, not thousands.

`zip` is how two lists are walked together: `(map f (zip a b))` calls *f* with a pair from each place.

ERRORS

MESSAGE	CAUSE
`Division by zero.`	`avg` of an empty list

SEE ALSO

`take`, `drop`, `join`, `map`, `sum`, `sort`.

EXAMPLE

```
hylang> (split 2 {1 2 3 4 5})
=> {{1 2} {3 4 5}}
hylang> (zip {1 2 3} {"a" "b" "c"})
=> {{1 "a"} {2 "b"} {3 "c"}}
hylang> (flatten {1 {2 {3 4}} 5})
=> {1 2 3 4 5}
hylang> (distinct {1 2 1 3 2})
=> {1 2 3}
hylang> (avg {1 2 3 4})
=> 5/2
```

sqrt, exp, log, sin, cos, tan, atan, pi, pow

Functions: square root, exponential, natural logarithm, the trigonometric functions, pi, and powers.

SYNTAX

```
(sqrt x)
(exp x)
(log x)
(sin x)
(cos x)
(tan x)
(atan x)
(pi)
(pow x y)
```

ARGUMENTS

x

A real number. For `sin`, `cos` and `tan` it is an angle in radians. For `pow` with a whole *y* it may be any number, complex too.

y

The power: any real number.

RESULT

FUNCTION	RESULT
`sqrt`	The square root. Of a negative number, an imaginary one: `(sqrt -4)` is `2i`
`exp`	*e* to the power *x*
`log`	The natural logarithm (base *e*). Of a negative number, a complex one
`sin`, `cos`, `tan`	The sine, cosine and tangent of the angle *x*
`atan`	The angle, in radians, whose tangent is *x*
`pi`	Pi, to the precision in force. It is a function, called with no arguments
`pow`	*x* to the power *y*

DESCRIPTION

Where the answer is exact, it is given exactly: `(sqrt 16)` is 4, `(sqrt 1/4)` is 1/2, `(sqrt 2.25)` is 1.5, and `pow` with a whole power is exact for any *x*, so `(pow 2 100)` has all its digits and `(pow 2 -1)` is 1/2. A power that is a fraction is exact when the root is, as in `(pow 4 1/2)`.

Otherwise the answer is a decimal with the number of significant digits that `digits` has set, 12 to begin with. Such an answer is rounded, so results that should be 0 can come out as a very small number: `(sin (pi))` is not exactly 0.

Each function takes exactly the arguments shown: `log` has no second argument for a base (divide two logarithms), and `atan` has no two-argument form.

ERRORS

MESSAGE	CAUSE
`'sqrt' expects a number`	The argument is not a number (each function with its own name)
`'sqrt' expects a real number`	The argument is complex
`'log': the logarithm of 0`	`(log 0)`
`'pow': a negative number to a power that isn't whole`	A negative *x* with a fractional *y*
`Division by zero.`	0 to a negative power

SEE ALSO

`digits`, `square`, `cube`, `math.pi`.

EXAMPLE

```
hylang> (sqrt 16)
=> 4
hylang> (sqrt 2)
=> 1.41421356237
hylang> (sqrt -4)
=> 2i
hylang> (pow 2 10)
=> 1024
hylang> (pow 2 1/2)
=> 1.41421356237
hylang> (exp 1)
=> 2.71828182846
hylang> (log 10)
=> 2.30258509299
hylang> (* 4 (atan 1))
=> 3.141592653588
hylang> (pi)
=> 3.14159265359
```

square, cube, recip, inverse

Functions of the library, written in hylang: a number's square, its cube, and its reciprocal.

SYNTAX

```
(square n)
(cube n)
(recip n)
(inverse n)
```

ARGUMENTS

n

A number.

RESULT

`square` gives $n \times n$, `cube` gives $n \times n \times n$, and `recip` gives $1 / n$, each exactly. `inverse` is another name for `recip`.

ERRORS

MESSAGE	CAUSE
`Division by zero.`	`recip` of 0
`All parameters to operator '*' must be numbers.`	`square` or `cube` of something that is not a number

SEE ALSO

`pow`, `sqrt`, `/`, `fdiv`.

EXAMPLE

```
hylang> (square 5)
=> 25
hylang> (square 1/2)
=> 1/4
hylang> (cube 3)
=> 27
hylang> (recip 4)
=> 1/4
```

starts-with?, ends-with?, contains?

Functions of the library, written in hylang: is a piece of text at the start of a string, at its end, or anywhere in it?

SYNTAX

```
(starts-with? s sub)
(ends-with? s sub)
(contains? s sub)
```

ARGUMENTS

s

A string.

sub

The text to look for: a string.

RESULT

`T` or `NIL`. An empty *sub* is found everywhere: all three give `T` for it.

DESCRIPTION

`starts-with?` and `contains?` are `index-of` with its answer tested, so capitals and small letters are different, and a character may be given in place of a one-character string (`ends-with?` needs a string). `contains?` also takes a list for *s*, and then says whether *sub* is one of its items, as `elem?` does with its arguments the other way round.

ERRORS

MESSAGE	CAUSE
`'index-of' expects a String and a String`	*s* is a string, and *sub* is neither a string nor a character; or *s* is not a string or a list

SEE ALSO

`index-of`, `substring`, `elem?`.

EXAMPLE

```
hylang> (starts-with? "hello" "he")
=> T
hylang> (ends-with? "hello" "lo")
=> T
hylang> (contains? "hello" "ell")
=> T
hylang> (contains? "hello" "z")
=> NIL
hylang> (filter (fn {f} {ends-with? f ".hl"}) {"a.hl" "b.txt" "c.hl"})
=> {"a.hl" "c.hl"}
```

str-pad-left, str-pad-right

Functions: a string filled out to a length.

SYNTAX

```
(str-pad-left s n [c])
(str-pad-right s n [c])
```

ARGUMENTS

s

A string. A number must be made one first, with `to-str`.

n

The length wanted: a whole number, 0 or more.

c

What to fill with: a character, or a string of one character. Without it, a space.

RESULT

A string of at least *n* characters. `str-pad-left` puts the filling before *s*, which sets it to the right of a column, as numbers are set; `str-pad-right` puts it after. A string already *n* long or longer is returned whole: it is never cut.

ERRORS

MESSAGE	CAUSE
`'str-pad-left' expects a String`	*s* is not a string (and `'str-pad-right'`)
`'str-pad-left' pads with one character`	*c* is a string of more than one character
`'str-pad-left' length cannot be negative`	*n* is below 0
`'str-pad-left': a number must be a whole number, not 1.5`	*n* is not a whole number

SEE ALSO

`format`, `to-str`, `hex`.

EXAMPLE

```
hylang> (str-pad-left (to-str 42) 6)
=> "    42"
hylang> (str-pad-left "7" 3 "0")
=> "007"
hylang> (str-pad-right "ab" 5 "*")
=> "ab***"
hylang> (str-pad-left "1234" 3)
=> "1234"
```

str-replace, str-repeat

Functions: a string with every copy of some text replaced; and a string repeated.

SYNTAX

```
(str-replace s old new)
(str-repeat s n)
```

ARGUMENTS

s

A string. For `str-repeat` it may be a character.

old

The text to look for: a string that is not empty, or a character.

new

The text to put in its place: a string, which may be empty, or a character.

n

How many times: a whole number, 0 or more.

RESULT

`str-replace` gives *s* with every place where *old* is found replaced by *new*. The string is gone through once, from the left, and what has been put in is not looked at again: replacing `"a"` by `"aa"` in `"aaa"` gives six a's and stops. If *old* is not found, the result is *s*.

`str-repeat` gives *n* copies of *s*, one after another: the empty string for 0.

ERRORS

MESSAGE	CAUSE
`'str-replace' cannot replace an empty String`	*old* is `""`
`'str-replace' expects a String`	An argument is not a string or a character
`'str-repeat' count cannot be negative`	*n* is below 0
`'str-repeat': a number must be a whole number, not 1.5`	*n* is not a whole number

EXAMPLE

```
hylang> (str-replace "a-b-c" "-" "+")
=> "a+b+c"
hylang> (str-replace "abc" "b" "")
=> "ac"
hylang> (str-replace "aaa" "a" "aa")
=> "aaaaaa"
hylang> (str-repeat "ab" 3)
=> "ababab"
```

str-split, str-join, str-chars

Functions: a string cut into a list of strings; a list joined into a string; and a string's characters.

SYNTAX

```
(str-split s [sep])
(str-join l [sep])
(str-chars s)
```

ARGUMENTS

s

A string.

sep

For `str-split`, the text to cut at: a string, or a list of strings, any of which cuts. Without it, a space.

For `str-join`, the text to put between the items: without it, nothing.

l

A list. Its items need not be strings: each is turned to text as `print` shows it.

RESULT

`str-split` gives a list of the pieces of *s* between the separators, without the separators. Two separators together have an empty string between them, and the list always has at least one item: a string with no separator in it gives a list of itself.

`str-join` gives one string: the items, with *sep* between each two. An empty list gives the empty string.

`str-chars` gives a list of the string's characters (`NIL` for the empty string). They are characters, not strings of one character.

ERRORS

MESSAGE	CAUSE
`First 'str-split' parameter must be a string`	*s* is not a string
`Second 'str-split' parameter must be a list of strings or a single string`	*sep* is something else: a character is not accepted
`'str-join' expects a QExpr`	*l* is not a list
`'str-chars' expects a String`	*s* is not a string

SEE ALSO

`read-lines`, `format`, `bytes`, `+`.

EXAMPLE

```
hylang> (str-split "one two three")
=> {"one" "two" "three"}
hylang> (str-split "a,b,,c" ",")
=> {"a" "b" "" "c"}
hylang> (str-split "a,b;c" {"," ";"})
=> {"a" "b" "c"}
hylang> (str-join {"a" "b" "c"} ", ")
=> "a, b, c"
hylang> (str-join {1 2 3} "-")
=> "1-2-3"
hylang> (str-chars "abc")
=> {\a \b \c}
hylang> (str-join (reverse (str-split "a b c")) " ")
=> "c b a"
```

str-upper, str-lower, str-trim

Functions: a string in capitals; in small letters; and without the blank space at its ends.

SYNTAX

```
(str-upper s)
(str-lower s)
(str-trim s)
```

ARGUMENTS

s

A string. `str-upper` and `str-lower` also take a character, and give a character.

RESULT

A new string. `str-upper` and `str-lower` change the letters A to Z and leave every other character as it is. `str-trim` removes the spaces, tabs and line ends from the start and the end, and leaves those in the middle.

ERRORS

MESSAGE	CAUSE
`'str-upper' expects a String`	*s* is neither a string nor a character (each function with its own name)

EXAMPLE

```
hylang> (str-upper "Hello 1")
=> "HELLO 1"
hylang> (str-lower "Hello 1")
=> "hello 1"
hylang> (str-trim "  hi  ")
=> "hi"
hylang> (str-upper \a)
=> \A
```

string?, char?, atom?, symbol?

Functions: is a value a string, a character, an atom, or a symbol?

SYNTAX

```
(string? x)
(char? x)
(atom? x)
(symbol? x)
```

ARGUMENTS

x

Any value.

RESULT

`T` or `NIL`:

FUNCTION	`T` WHEN *X* IS
`string?`	A string. A character is not a string, even a string of one character
`char?`	A character, such as `\a`
`atom?`	An atom, such as `:name`
`symbol?`	A symbol that has not been evaluated

DESCRIPTION

A symbol written as an argument is evaluated before the function sees it, so `symbol?` is only `T` for a symbol taken out of a list, made by `to-sym` or `gensym`, or read by `read`.

SEE ALSO

`type-of`, `to-sym`, `to-atom`, `alpha?`.

EXAMPLE

```
hylang> (string? "a")
=> T
hylang> (string? \a)
=> NIL
hylang> (char? \a)
=> T
hylang> (atom? :a)
=> T
hylang> (symbol? (fst {a}))
=> T
hylang> (symbol? :a)
=> NIL
```

subset

A function: part of a list.

SYNTAX

```
(subset l i [n])
```

ARGUMENTS

l

A list.

i

Where the part starts, from 0. It may be the list's length, which gives `NIL`.

n

How many items. Without it, or if it is more than there are, all the items from *i* to the end.

RESULT

A new list of the items.

ERRORS

MESSAGE	CAUSE
`'subset': the list has no item 6`	*i* is more than the list's length, or negative
`First 'subset' parameter must be a QExpr`	*l* is not a list. For part of a string, use `substring`

SEE ALSO

`take`, `drop`, `substring`.

EXAMPLE

```
hylang> (subset {1 2 3 4 5} 1 3)
=> {2 3 4}
hylang> (subset {1 2 3 4 5} 3)
=> {4 5}
hylang> (subset {1 2 3 4 5} 2 99)
=> {3 4 5}
hylang> (subset {1 2 3 4 5} 5)
=> NIL
```

substring, char-at

Functions: part of a string; and one character of it.

SYNTAX

```
(substring s i [n])
(char-at s i)
```

ARGUMENTS

s

A string.

i

A position in the string, from 0. For `substring` it may be the string's length, which gives the empty string; for `char-at` it must be less.

n

How many characters. Without it, or if it is more than there are, all the characters from *i* to the end.

RESULT

`substring` gives a string. `char-at` gives a character, which is a different kind of value from a string of one character: `(char-at "abc" 0)` is `\a`, and is not equal to `"a"`. `to-str`, or `+` after a string, makes a character a string.

ERRORS

MESSAGE	CAUSE
`'substring': index 6 is outside the string`	*i* is negative, or past the end (and `'char-at'`)
`'substring': a count can't be negative`	*n* is below 0
`'substring': an index must be a whole number, not 1.5`	*i* or *n* is not a whole number
`First 'substring' parameter must be a String`	*s* is not a string. For part of a list, use `subset`
`'char-at' expects a String`	`char-at`'s *s* is not a string

SEE ALSO

`index-of`, `str-chars`, `subset`, `nth`.

EXAMPLE

```
hylang> (substring "hello" 1 3)
=> "ell"
hylang> (substring "hello" 1)
=> "ello"
hylang> (substring "hello" 5)
=> ""
hylang> (char-at "hello" 1)
=> \e
hylang> (substring "hello" 6)
=> Error: 'substring': index 6 is outside the string
```

sum, product

Functions: all the numbers of a list added together; and multiplied together.

SYNTAX

```
(sum l)
(product l)
```

ARGUMENTS

l

A list of numbers.

RESULT

The total, or the product, exact as all of hylang's arithmetic is. For an empty list `sum` is 0 and `product` is 1.

ERRORS

MESSAGE	CAUSE
`'sum' expects a list`	*l* is not a list (and `'product'` for `product`)
`'+' adds numbers, or joins values to a string`	`sum`: an item is not a number
`All parameters to operator '*' must be numbers.`	`product`: an item is not a number

SEE ALSO

`+`, `*`, `foldl`, `avg`.

EXAMPLE

```
hylang> (sum {1 2 3})
=> 6
hylang> (sum {1/2 1/3})
=> 5/6
hylang> (product {2 3 4})
=> 24
hylang> (sum {})
=> 0
```

sys

A function: makes a system call. Each call also has a function of its own, named `sys-` and the call's name.

SYNTAX

```
(sys name x ...)
(sys-name x ...)
```

ARGUMENTS

name

The call's name in small letters, with `-` where the call has `_`: an atom, a symbol or a string. `OPEN` is `:open`, and `SLEEP_UNTIL` is `:sleep-until`.

x

The call's inputs, in the order its entry in Chapter 3 of this book gives them. A string is passed as the address of its text.

RESULT

The call's output: a number, or a list of them if the call gives several. A call with no output gives `NIL`. If the call fails, the result is an error, with the system's message, and its code as the error's code.

DESCRIPTION

`(sys :getpid)` and `(sys-getpid)` are the same call, written two ways. The first form takes the name as a value, so it can be chosen as the program runs; the second reads like any other function.

These reach everything the kernel offers, at the level a program in assembly language works at: file descriptors are numbers, and nothing is closed for you. Most programs never need them, because the functions of this chapter (`open`, `read-line`, `spawn`, `sleep` and the rest) are built on the same calls and look after the details. They are for the calls that have no function of their own.

"From the languages", in each call's entry in Chapter 3 of this book, says whether a call can be made this way.

ERRORS

MESSAGE	CAUSE
`'sys' has no call :bogus`	There is no call of that name
`'sys-open' takes at least 1 argument, not 0`	Too few inputs for the call
The system's message	The call failed

SEE ALSO

Chapter 3 of this book; `error-code`, `errstr`.

EXAMPLE

```
hylang> (sys-getpid)
=> 2
hylang> (sys :getppid)
=> 1
hylang> (def {fd} (sys-open "/rom/lib/shell" 0))
=> NIL
hylang> fd
=> 3
hylang> (sys-close fd)
=> NIL
hylang> (error-code (sys-open "/nothere" 0))
=> :noent
```

sysinfo, mods, ps, task-info, errstr, free

Functions: facts about the system: the machine, the ROM's modules, the tasks, an error's message, and the memory.

SYNTAX

```
(sysinfo)
(mods)
(ps)
(task-info task)
(errstr code)
(free)
```

ARGUMENTS

task

A task's number, 0 to 15.

code

A system error code, as a number, 0 to 255.

RESULT

FUNCTION	RESULT
`sysinfo`	A hash: `:abi`, the version of the system's calls; `:ram-modules`, how many RAM modules are fitted; `:free-tasks`, a list of the task numbers not in use
`mods`	A list of hashes, one for each module in the paged ROM: `:name`, `:type` (`:program`, `:driver` or `:library`), `:bank` (its first ROM bank) and `:banks` (how many it has)
`ps`	A list of hashes, one for each task in use, as `task-info` gives
`task-info`	A hash for one task: `:task`, its number; `:name`; `:state`; `:parent`, the task that started it (no entry if none did); `:cpu`, the processor time it has had, in ticks; `:group`, its note group; `:args`, its arguments, a list of strings (no entry if it has none)
`errstr`	The system's message for the error code, a string
`free`	A hash of memory sizes, in kilobytes: `:ram`, this task's own RAM banks; `:shared`, the shared RAM that segments can use; `:used` and `:free`, how much of that is and isn't in segments; `:segments`, how many segments there are

A task's `:state` is one of `:ready`, `:wait` (for a child to end), `:call` (in a driver's call), `:idle` (a driver with nothing to do), `:new`, `:sleep`, `:blocked` and `:event`.

DESCRIPTION

These give, as values a program can use, what the commands `mods`, `ps` and `free` print. Being lists of hashes, they are searched and sorted with the list functions.

ERRORS

MESSAGE	CAUSE
task: `no such task`	`task-info` of a task number that isn't in use
`'task-info' was given a number out of its range`	*task* is not 0 to 15
`'errstr' expects an error's code`	*code* is not 0 to 255

SEE ALSO

`pid`, `banks`, `error-code`; the commands `ps`, `mods` and `free` (Chapter 2 of this book).

EXAMPLE

```
hylang> ((sysinfo) :ram-modules)
=> 3
hylang> (errstr 32)
=> "not found"
hylang> ((task-info 0) :name)
=> "kernel"
hylang> (map (fn {p} {p :name}) (ps))
=> {"kernel" "forth" "hylang" "snd" "kdev" "storage" "cons"}
hylang> (map (fn {m} {m :name}) (filter (fn {m} {== (m :type) :driver}) (mods)))
=> {"cons" "storage" "kdev" "snd" "vid"}
hylang> (free)
=> <hash>{{:ram 384} {:shared 1024} {:used 512} {:free 512} {:segments 1}}
```

t?, nil?

Functions: is a value `T`, or `NIL`?

SYNTAX

```
(t? x)
(nil? x)
```

ARGUMENTS

x

Any value.

RESULT

`t?` gives `T` only for `T` itself: other values that count as true, such as 1, are not `T`.

`nil?` gives `T` for `NIL` and for the empty list, which are the same value. 0 and the empty string are not `NIL`. `nil?` does the same as `not`.

SEE ALSO

`not`, `and`, `or`, `if`.

EXAMPLE

```
hylang> (t? T)
=> T
hylang> (t? 1)
=> NIL
hylang> (nil? {})
=> T
hylang> (nil? 0)
=> NIL
hylang> (nil? "")
=> NIL
```

take, drop

Functions: the first items of a list; and the list without them.

SYNTAX

```
(take n l)
(drop n l)
```

ARGUMENTS

n

A count: a whole number, 0 or more.

l

A list.

RESULT

`take` gives a list of the first *n* items, and `drop` the list without them. A count greater than the list's length is not an error: `take` then gives the whole list, and `drop` gives `NIL`.

ERRORS

MESSAGE	CAUSE
`'take': a count can't be negative`	*n* is less than 0
`'take': a count must be a whole number, not 1.5`	*n* is not a whole number
`'take' expects a list`	*l* is not a list

`drop`'s messages are the same, with its own name.

SEE ALSO

`subset`, `head`, `tail`, `split`.

EXAMPLE

```
hylang> (take 2 {1 2 3})
=> {1 2}
hylang> (drop 2 {1 2 3})
=> {3}
hylang> (take 5 {1 2 3})
=> {1 2 3}
hylang> (drop 5 {1 2 3})
=> NIL
```

ticks, tick-rate, clock

Functions: the system's tick count, how many ticks there are in a second, and the time since hylang started.

SYNTAX

```
(ticks)
(tick-rate)
(clock)
```

RESULT

`ticks` gives the system's tick count: a number from 0 to 32,767, which goes up by one at each tick of the system's timer and starts again from 0 when it passes 32,767.

`tick-rate` gives the number of ticks in a second: 200.

`clock` gives the number of seconds since this hylang started, as a decimal, to the nearest tick (a two hundredth of a second).

DESCRIPTION

For timing a piece of a program, take `(clock)` before and after, and subtract.

`ticks` is the raw count, which a few system calls and `sleep-until` use. It wraps every 164 seconds, so the difference of two readings must be taken with that in mind: `clock` does that work, and is the one to use for anything longer than a moment.

SEE ALSO

`time`, `sleep`, `sleep-until`.

EXAMPLE

The numbers depend on the moment:

```
hylang> (tick-rate)
=> 200
hylang> (def {t0} (clock))
=> NIL
hylang> (fib 5000)
=>
3878968454388325633701916308325905312082127714646245106160597214895550139044037097010822
916462210669
...
hylang> (- (clock) t0)
=> 1.385
```

time, date, date-parts, seconds-of

Functions: the clock, and conversions between a count of seconds and a date.

SYNTAX

```
(time)
(date [t])
(date-parts [t])
(seconds-of year month day [hour minute second])
```

ARGUMENTS

t

A time, as a number of seconds since the start of 1 January 2000. Without it, now.

year, month, day

A date: the year 2000 or later, the month 1 to 12, the day 1 to the month's last.

hour, minute, second

A time of day. Without them, midnight.

RESULT

FUNCTION	RESULT
`time`	The clock, now: the number of seconds since 2000-01-01 00:00:00
`date`	The time as a string, `"2026-10-13 09:41:00"`: the form `/dev/time` uses
`date-parts`	The time as a hash: `:year`, `:month`, `:day`, `:hour`, `:minute`, `:second`, and `:weekday`, which is 0 for Sunday to 6 for Saturday
`seconds-of`	The number of seconds since 2000 of the date and time given

DESCRIPTION

A time is a plain number, so times are compared with < and subtracted with -: the difference is in seconds. A file's `:mtime`, from `stat`, is a time of the same kind.

The Hydra's clock starts at the beginning of 2000 each time it is switched on, unless it has a clock chip or its time has been set (section 17.5): so on a board without one, `(time)` is the number of seconds since it was started.

ERRORS

MESSAGE	CAUSE
year`: invalid argument`	`seconds-of` given a date that doesn't exist, or a year before 2000
`a time must be an integer`	*t* is not a whole number

SEE ALSO

`ticks`, `clock`, `sleep`, `stat`.

EXAMPLE

```
hylang> (date 0)
=> "2000-01-01 00:00:00"
hylang> (seconds-of 2026 10 13 9 41 0)
=> 845199660
hylang> (date 845199660)
=> "2026-10-13 09:41:00"
hylang> ((date-parts 845199660) :weekday)
=> 2
hylang> (- (seconds-of 2026 12 25) (seconds-of 2026 10 13))
=> 6307200
```

to-fixed, to-rational

Functions: a number as a decimal; and as a fraction.

SYNTAX

```
(to-fixed x [places])
(to-rational x)
```

ARGUMENTS

x

A real number.

places

How many digits to keep after the point: 0 to 10,000. Without it, 10.

RESULT

`to-fixed` gives *x* as a decimal with at most *places* digits after the point. Digits beyond that are cut off, not rounded: `(to-fixed 2/3 2)` is 0.66. A number that needs fewer places is not padded with zeros, and a whole number stays an integer.

`to-rational` gives *x* as a fraction in its lowest terms, exactly. A whole number stays an integer.

DESCRIPTION

A fraction such as 1/3 has no exact decimal, so `to-fixed` is where exactness is given up, and it is the usual last step before a number is shown to a person. `to-rational` loses nothing.

ERRORS

MESSAGE	CAUSE
`'to-fixed' expects a real number`	*x* is complex, or not a number (and `'to-rational'` for `to-rational`)
`'to-fixed': -1 places is outside 0 to 10000`	*places* is out of range

SEE ALSO

`round`, `fdiv`, `to-str`, `format`.

EXAMPLE

```
hylang> (to-fixed 1/3)
=> 0.3333333333
hylang> (to-fixed 2/3 2)
=> 0.66
hylang> (to-fixed 1.23456 2)
=> 1.23
hylang> (to-rational 0.25)
=> 1/4
hylang> (to-rational 1.5)
=> 3/2
```

to-str, repr

Functions: a value as text.

SYNTAX

```
(to-str x [base])
(repr x)
```

ARGUMENTS

x

Any value.

base

For a number: the base to write it in, as a string (`"x"`, `"b"`, a radix). Without it, the current base.

RESULT

A string.

`to-str` gives the text that `print` would show: a string is itself, a character is the one character, and a number is its digits.

`repr` gives the text as the prompt shows it, which is the text that reads back as the same value: a string has its quotes, and its special characters are written as escapes; a character has its backslash.

For a number, a list, an atom, `T` or `NIL` the two are the same. A string inside a list is shown with its quotes by both.

ERRORS

MESSAGE	CAUSE
`Second 'to-str' parameter must be a String`	*base* is not a string: write `"x"`, not 16

SEE ALSO

`val`, `read`, `format`, `hex`, `save`.

EXAMPLE

```
hylang> (to-str 42)
=> "42"
hylang> (to-str 255 "x")
=> "FF"
hylang> (to-str "a")
=> "a"
hylang> (repr "a")
=> "\"a\""
hylang> (to-str {1 "a"})
=> "{1 \"a\"}"
hylang> (len (to-str 1/3))
=> 3
```

to-sym, to-atom

Functions: a symbol, or an atom, with a given name.

SYNTAX

```
(to-sym x)
(to-atom x)
```

ARGUMENTS

x

For `to-sym`, a string or an atom. For `to-atom`, a string, a symbol or a number.

RESULT

`to-sym` gives the symbol whose name is the string's text, or the atom's name without its colon. `to-atom` gives the atom of that name.

DESCRIPTION

A symbol made this way is a value like any other until it is evaluated; `eval` then looks it up, as it would a name written in the program. This is how a program reaches a function, or a variable, whose name it has only as text.

ERRORS

MESSAGE	CAUSE
`Only atoms and strings can be converted into symbols`	`to-sym` of anything else
`Only symbols, strings and numbers can be converted to atoms`	`to-atom` of anything else

SEE ALSO

`gensym`, `eval`, `def`.

EXAMPLE

```
hylang> (to-sym "abc")
=> abc
hylang> (to-atom "abc")
=> :abc
hylang> ((eval (to-sym "len")) "hello")
=> 5
```

truncate

A function: the whole part of a number.

SYNTAX

```
(truncate x)
```

ARGUMENTS

x

A real number: an integer, a fraction or a decimal.

RESULT

An integer: *x* with whatever follows the point taken away. The result is the nearest whole number toward zero, so a negative number moves up: `(truncate -2.7)` is -2.

ERRORS

MESSAGE	CAUSE
`'truncate' expects a real number`	*x* is complex, or not a number

SEE ALSO

`floor`, `ceil` and `round` in the library, which round down, up and to the nearest; `trunc`, which is another name for `truncate`.

EXAMPLE

```
hylang> (truncate 2.7)
=> 2
hylang> (truncate -2.7)
=> -2
hylang> (truncate 7/2)
=> 3
```

try

A special form: evaluates an expression, and catches an error.

SYNTAX

```
(try x [handler])
```

DESCRIPTION

x is evaluated. If its value is not an error, that value is the result, and *handler* is not looked at.

If its value is an error, *handler* is evaluated and its value is the result; with no handler, the result is `NIL`. While the handler is evaluated, `&err` is bound to the error's message, as a string, and `&code` to the error's code, an atom: the system's name for the error if it came from the system (`:noent`), or the code the program gave to `error`. It is `NIL` for an error that has none.

EXAMPLE

```
hylang> (try (/ 1 0) (list "caught" &err &code))
=> {"caught" "Division by zero." NIL}
hylang> (try (+ 1 2) "not used")
=> 3
hylang> (try (error "mine"))
=> NIL
```

type-of

A function: the kind of a value.

SYNTAX

```
(type-of x)
```

ARGUMENTS

x

Any value.

RESULT

An atom:

ATOM	THE VALUE IS
`:number`	A number of any kind: an integer, a fraction, a decimal or a complex number
`:string`	A string
`:char`	A character
`:atom`	An atom
`:symbol`	A symbol that has not been evaluated
`:list`	A Q-expression: a list. `NIL`, the empty list, is one
`:sexpr`	An S-expression that has not been evaluated: a call, as data
`:t`	`T`
`:function`	A function: one made with `fn` or `fun`, a built-in one, or a special form
`:hash`	A hash
`:buffer`	A buffer
`:stream`	A stream
`:error`	An error

DESCRIPTION

`type-of` is one of the few functions that can be given an error without the error becoming its result: it answers `:error`. A symbol or an S-expression is only seen as one when it comes out of a list unevaluated, as `(fst {a})` does; written as an argument, it is evaluated first, and `type-of` sees its value.

To tell the kinds of number apart, use `int?`, `fixed?`, `rational?` and `complex?`.

SEE ALSO

`num?` and the other tests; `==`.

EXAMPLE

```
hylang> (type-of 42)
=> :number
hylang> (type-of "42")
=> :string
hylang> (type-of {4 2})
=> :list
hylang> (type-of NIL)
=> :list
hylang> (type-of len)
=> :function
hylang> (type-of (fst {a}))
=> :symbol
hylang> (type-of (error "x"))
=> :error
```

use

A function of the library, written in hylang: loads a library, once.

SYNTAX

```
(use name)
```

ARGUMENTS

name

The library's name, a string: the name of a file of hylang source, as `load` takes it. A bare name such as `"gpio"` is the file `gpio.hl` in `/lib/hylang`.

RESULT

`NIL`.

DESCRIPTION

The first time a name is used, the file is loaded: its expressions are evaluated, which defines the library's functions. The name is then remembered, in the list `used`, and a later `use` of the same name does nothing. So every file that needs a library can say so, and the library is still loaded only once.

`load` always loads: use it for a file of your own that you are changing.

The Hydra's libraries, each with an entry of its own in this chapter:

NAME	WHAT IT IS FOR
`"gpio"`, `"i2c"`, `"spi"`	The I/O header's pins, the I²C bus, the SPI devices
`"clock"`, `"disk"`, `"proc"`, `"pc"`, `"dev"`	Setting the clock; the disks; the tasks; the PC's files; devices' control files
`"cons"`, `"screen"`	The console's windows; the terminal's cursor and colours
`"snd"`, `"video"`	Sound; the Vera X's graphics
`"dice"`	Dice, for games

ERRORS

MESSAGE	CAUSE
name`: not found`	There is no such file

SEE ALSO

`load`, `def`, `fun`.

EXAMPLE

```
hylang> (use "dice")
=> NIL
hylang> (use "dice")
=> NIL
hylang> used
=> {"dice"}
hylang> (use "nothere")
=> Error: nothere: not found
```

val

A function: the number that a string spells.

SYNTAX

```
(val s)
```

ARGUMENTS

s

A string holding one number, written as hylang itself reads numbers. Spaces before and after it are allowed.

RESULT

The number. Any form that can be typed can be read: an integer, a fraction, a decimal, a complex number, or a number with a base prefix. A number with no prefix is read in the current base.

DESCRIPTION

`val` is how text from a person, or from a file, becomes a number. The whole string must be the number: text after it is an error, not ignored, so a program can tell a bad entry from a good one with `try`.

ERRORS

MESSAGE	CAUSE
`'val': "abc" isn't a number`	The string is not a number, is empty, or has something after the number
`'val' expects a String`	*s* is not a string

SEE ALSO

`to-str`, `read`, `base`.

EXAMPLE

```
hylang> (val "42")
=> 42
hylang> (val "2/3")
=> 2/3
hylang> (val " 12 ")
=> 12
hylang> (val "#xFF")
=> 255
hylang> (val "12abc")
=> Error: 'val': "12abc" isn't a number
hylang> (try (val "abc") 0)
=> 0
```

video library

A library, loaded with `(use "video")`: drawing on a Vera X's screen, turtle graphics, and the chip's memory, sprites and mouse.

FUNCTIONS

The bitmap, and drawing on it.

FORM	WHAT IT DOES
`(bitmap w d)`	Turns on a bitmap under the console's text: *w* is 320 or 640 dots across, and *d* the bits for each dot, 1, 2, 4 or 8 (640 across allows only 1 or 2)
`(bitmap-off)`	Turns it off
`(pen c)`	The colour drawn in from now on: a palette number
`(plot x y)`	One dot
`(line x0 y0 x1 y1)`	A line
`(box x0 y0 x1 y1)`, `(bar x0 y0 x1 y1)`	A rectangle's outline; a filled rectangle
`(circle x y r)`, `(disc x y r)`	A circle's outline; a filled circle
`(text x y s)`	Text in the console's 8 × 8 font, 40 characters at most
`(clear)`	Clears the bitmap
`(border c)`	The colour of the screen's border

The turtle.

FORM	WHAT IT DOES
`(cs)`, `(home)`	Clears the bitmap and sends the turtle home; sends it home only: the middle of the screen, facing up
`(fd n)`, `(bk n)`	Forward, or back, *n* dots, drawing a line if the pen is down
`(rt d)`, `(lt d)`	Turns right, or left, *d* degrees
`(pu)`, `(pd)`	Pen up (moves draw nothing); pen down
`(heading)`, `(seth d)`	The turtle's heading in degrees, 0 being up and 90 right; and set it

The chip.

FORM	WHAT IT DOES
`(vpeek addr)`, `(vpoke addr v)`	A byte of the VERA's memory, at a 17-bit address; and set one
`(vram@ addr n)`, `(vram! addr bytes)`	*n* bytes of it, as a list; and a list of bytes written to it
`(palette! index rgb)`	Sets a palette entry to a colour, `#xRGB`: four bits each of red, green and blue
`(sprite! n bytes)`	Sets sprite *n*'s eight attribute bytes
`(sprite-at n x y)`, `(sprite-off n)`	Moves a sprite; hides it
`(vsync)`	Waits for the next frame
`(mouse)`, `(mouse-wait)`	The mouse now: a list of *x*, *y* and the buttons (1 left, 2 middle, 4 right, added together). And the same after waiting for its next change

DESCRIPTION

The library works through the video driver's files in `/dev/vid` (Chapter 16), and needs a Vera X card; the emulator has one when it is started with `--vera`. Each function gives `NIL` unless it says otherwise.

The bitmap lies under the console's text, so the prompt, and whatever a program prints, stay on the screen over the drawing. Positions are in dots, from the top left corner; a dot outside the bitmap is not drawn, and is not an error. The turtle keeps its place exactly, as a fraction, so a figure of many small steps closes where it began.

The names are the same as those of HyForth's video library, and in both it is the driver that does the drawing.

ERRORS

MESSAGE	CAUSE
`invalid argument`	Drawing with no bitmap on; a width or depth that `bitmap` doesn't have
`busy`	Another program has claimed the chip
`/dev/vid/ctl: not found`	There is no Vera X

SEE ALSO

Chapter 16; section 22.7; the `screen` and `cons` libraries.

EXAMPLE

A square, a filled circle, and a turtle's hexagon:

```
hylang> (use "video")
=> NIL
hylang> (bitmap 320 8)
=> NIL
hylang> (pen 5)
=> NIL
hylang> (box 10 10 100 100)
=> NIL
hylang> (disc 200 60 30)
=> NIL
hylang> (cs)
=> NIL
hylang> (dotimes {i 6} (do (fd 40) (rt 60)))
=> NIL
hylang> (heading)
=> 0
hylang> (bitmap-off)
=> NIL
```

`while`

A special form: repeats a body while a test holds.

SYNTAX

```
(while test body ...)
```

DESCRIPTION

test is evaluated. If its value is `NIL`, the loop ends. Otherwise the body's expressions are evaluated in order, and the test is evaluated again. The test is made before each pass, the first included, so the body may not run at all.

The result is `NIL`. The body runs in the scope the `while` is in, which is how it changes the names its test looks at.

EXAMPLE

```
hylang> (let {{i 0}} (while (< i 3) (set {i} (+ i 1))) i)
=> 3
```

write

A function: prints values, with nothing added.

SYNTAX

```
(write x ...)
```

ARGUMENTS

x

Any values.

RESULT

`NIL`. The values are written to standard output as `print` writes each, but with nothing between them and no new line after them.

DESCRIPTION

`write` is for building a line a piece at a time, and for prompts that the answer should follow on the same line. A new line is written when the program asks for one, with `"\n"` in a string or with `(print)`.

SEE ALSO

`print`, `write-to`, `format`.

EXAMPLE

```
hylang> (write "no new line")
no new line=> NIL
hylang> (write 1 2)
12=> NIL
hylang> (do (write "a" "b") (print))
ab
=> NIL
```

write-file, append-file

Functions: write values to a file, replacing it or adding to it.

SYNTAX

```
(write-file path x ...)
(append-file path x ...)
```

ARGUMENTS

path

The file's name, a string.

x

Any values. Each is written as `print` shows it, with nothing between them and no line end added: put `"\n"` where a line should end.

RESULT

`NIL`. `write-file` makes the file, or empties it if it is there, and writes the values. `append-file` makes it if it isn't there, and writes the values after what it holds. With no values, `write-file` leaves an empty file.

DESCRIPTION

The file is opened, written and closed in the one call. Devices are files, so this is also how a command is sent to a device's control file (Chapter 12 of the manual).

ERRORS

MESSAGE	CAUSE
path`: not found`	A directory in the path isn't there (and the system's other messages, for its other errors)

SEE ALSO

`read-file`, `save`, `open`, `print-to`.

EXAMPLE

```
hylang> (write-file "/ram/b.txt" "x = " 42 "\n")
=> NIL
hylang> (append-file "/ram/b.txt" "y = " {1 2} "\n")
=> NIL
hylang> (read-file "/ram/b.txt")
=> "x = 42\ny = {1 2}\n"
```

xor, <>, !=

Functions of the library: exclusive or; and two more names for "not equal".

SYNTAX

```
(xor a b)
(<> a b)
(!= a b)
```

ARGUMENTS

a, b

Any two values.

RESULT

`xor` gives `T` when exactly one of *a* and *b* is true (not `NIL`), and `NIL` when both are or neither is. Both arguments are always evaluated: it cannot stop early as `and` and `or` do.

`<>` and `!=` are other names for `neq`: `T` when the two values are not equal, as `==` compares.

SEE ALSO

`and`, `or`, `not`, `==`, `bit-xor`.

EXAMPLE

```
hylang> (xor T NIL)
=> T
hylang> (xor T T)
=> NIL
hylang> (<> 1 2)
=> T
hylang> (!= "a" "a")
=> NIL
```